Indira Gandhi National Open University

A Project on

Bug Tracker System

MCSP-232

Submitted to the School of Computer and Information Sciences,

IGNOU In the partial fulfillment of the requirements for the degree of

Masters of Computer Applications (**MCA_NEW**)

PROJECT GUIDE

Deepak Singh Kathait

SUBMITTED BY

Rohit Singh

E.N. : 2401012750

Indira Gandhi National Open University

Maidan Garhi

New Delhi - 110068.

Synopsis Approval Page

Certificate of Originality

Guide Resume

Guide I'd Proof

SYNOPSIS

(BUG TRACKER SYSTEM)

TABLE OF CONTENTS

1. TITLE OF THE PROJECT

This project titled **BUG TRACKER SYSTEM** is a web-based application designed to automate and streamline the complete lifecycle of software bug management. The system efficiently manages bugs through seven distinct phases, namely Bug Reporting, Bug Confirmation, Bug Analysis, Maintenance, Final Testing, Deployment, and Closure.

It helps development and testing teams to track, monitor, and resolve bugs systematically, thereby improving software quality, reducing development time, and ensuring better coordination among team members.

2. INTRODUCTION OF THE PROJECT

The Bug Tracker System is a comprehensive web-based application designed to revolutionize bug management processes in software development environments. This system provides a centralized, automated platform for managing the complete bug lifecycle through seven distinct phases ensuring systematic quality assurance at each stage of bug resolution.

2.1 Background

In contemporary software development environments, efficient bug tracking and management have become critical factors in ensuring software quality, timely project delivery, and effective team collaboration. Traditional bug management approaches involving manual processes, email communications, and spreadsheet-based tracking systems suffer from significant limitations including poor coordination, lack of accountability, inadequate workflow management, and insufficient audit trails.

2.2 Problem Statement

Current bug management practices in many software development organizations rely heavily on manual processes, fragmented tools, and inconsistent methodologies. These approaches create significant challenges:

1. **Lack of Structured Workflow Management:** Bugs may skip essential quality assurance steps without enforced workflows
2. **Poor Coordination and Communication**: Information silos where team members work with incomplete or outdated information
3. **Inadequate Quality Assurance:** Without mandatory testing phases, ensuring proper bug resolution becomes challenging
4. **Insufficient Accountability:** Lack of comprehensive audit trails makes it difficult to track actions and measure performance
5. **Limited Scalability:** Manual processes don't scale well with increasing bug volumes and team sizes

2.3 Proposed Solution

The Bug Tracker System addresses these challenges through a comprehensive, automated solution that implements systematic workflow management, enforces quality standards, and provides complete visibility into the bug resolution process.

Key Features:

1. **Seven-Phase Workflow Architecture:**
   - Phase I: Bug Report - Initial submission with comprehensive details
   - Phase II: Bug Confirmation - Tester verification and validation

- Phase III: Bug Analysis - Root cause identification by developer
- Phase IV: Maintenance - Fix implementation and documentation
- Phase V: Final Testing - Resolution validation and approval
- Phase VI: Deployment - Production deployment documentation
- Phase VII: Closure - Final resolution summary and lessons learned

2. Role-Based Access Control: Four distinct user roles (Bug Raiser, Tester, Developer, Administrator) with appropriate permissions

3. Tool-Based Organization: Bugs organized by tools/applications with pre-assigned testers and developers for automatic routing

4. Comprehensive Logging: Complete audit trail of all actions with timestamps and user information

5. Configurable System Settings: Dynamic configuration of stages, status types, priorities, and technology stacks

2.4 Scope

The system addresses critical challenges in software development by providing transparency, accountability, and efficiency in bug management processes while reducing resolution time through automated workflows and systematic quality assurance procedures.

3. OBJECTIVE OF THE PROJECT

3.1 Primary Objectives

1. Automate Bug Lifecycle Management
   - Implement a seven-phase automated workflow
   - Ensure systematic progression of bugs from reporting to closure
   - Built-in quality gates at each phase

2. Implement Role-Based Access Control
   - Design comprehensive user management system
   - Role-based permissions (Bug Raiser, Tester, Developer, Administrator)
   - Users can only perform actions appropriate to their roles

3. Ensure Quality Assurance
   - Establish mandatory testing and validation phases
   - Prevent bugs from being marked as resolved without proper verification
   - Include initial testing after confirmation and final testing before deployment

4. Provide Tool-Based Organization
   - Implement tool management system
   - Categorize bugs by application/module
   - Pre-assigned testers and developers for automatic routing

5. Maintain Comprehensive Audit Trails
   - Log all bug-related activities
   - Track creation, status changes, phase progressions, and user actions
   - Complete timestamp and user information for accountability

3.2 Secondary Objectives

1. Enhance Team Collaboration: Facilitate better communication between stakeholders through centralized information sharing and automated notifications
2. Reduce Resolution Time: Minimize bug resolution time through automated routing and clear responsibility assignment
3. Support Multiple Technology Stacks: Manage bugs across diverse platforms (Scripts, Web Apps, Android, iOS)
4. Enable Data-Driven Decision Making: Comprehensive reporting capabilities to identify bottlenecks and measure team performance
5. Ensure System Configurability: Allow administrators to customize system settings without code modifications
6. Implement Secure Data Management: Ensure data integrity and security through proper authentication and authorization.

4. PROJECT CATEGORY

The Bug Tracker System falls under the **Web Application Development** category with strong elements of **Database Management Systems** (**DBMS**) and **Software Engineering**.

4.1 Primary Category

Web Application Development:
- Full-stack web application using MERN technology stack
- React.js for frontend development with single-page application (SPA) architecture
- Node.js with Express.js for backend services
- MongoDB for data persistence
- RESTful API architecture

4.2 Secondary Categories

Database Management:
- Document-oriented database (MongoDB)
- Complex schemas with embedded documents and references between collections
- Indexing strategies for performance optimization
- Data validation and referential integrity

Software Engineering Principles:
- Modular architecture design and separation of concerns
- Comprehensive requirements analysis and systematic design using UML diagrams
- Structured implementation methodology and rigorous testing strategies

Workflow Management:
- Sophisticated workflow automation with seven distinct phases
- Automated routing based on user roles and validation checkpoints at each phase transition

4.3 Project Complexity

This multi-faceted project requires integration of diverse technical skills including frontend development (React.js), backend programming (Node.js, Express.js), database design (MongoDB), API development (RESTful services), security implementation (JWT, bcrypt), and user experience design (UI/UX). The project demonstrates comprehensive understanding of full-stack development, making it an ideal project for MCA degree requirements.

5. DRAWBACK OF THE EXISTING SYSTEM

5.1 Traditional Bug Tracking Systems

1. Jira Software:
- Complex interface with steep learning curve
- Expensive licensing for growing teams
- Over-engineered for small to medium projects
- Lacks structured quality assurance workflow with no enforced testing phases

3. GitHub Issues:
- Basic bug tracking capabilities only
- Cannot enforce structured workflows
- Minimal reporting and analytics capabilities
- Tightly coupled with GitHub ecosystem

5.2 Common Drawbacks of Manual/Spreadsheet-Based Systems

1. Communication Gaps: Manual tracking creates information silos with email chains difficult to track and important details getting lost

2. Lack of Structured Workflows: No enforced quality assurance steps, bugs skip essential validation, inconsistent handling procedures

3. Inadequate Documentation: Missing knowledge management with no standardized documentation format, resolution steps not captured

4. Poor Accountability: No comprehensive audit trails, difficult to identify bottlenecks and measure team performance

5. Scalability Limitations: Cannot handle growing bug volumes, manual coordination becomes overwhelming, performance degrades with scale

6. Quality Assurance Challenges: No mandatory testing checkpoints, bugs marked resolved without proper testing, leading to production issues

5.3 Key Missing Features in Existing Systems

1. Enforced seven-phase workflow with mandatory multi-phase quality assurance
2. Tool-based organization with automatic routing based on tool/application assignment
3. Phase-specific documentation requirements at each phase
4. Comprehensive audit trail with system-wide action logging across all entities
5. Dynamic configuration without requiring code changes for workflow modifications
6. Integrated testing phases including confirmation testing, final testing, and deployment validation.

6. H/W & S/W REQUIREMENT

6.1 Hardware Requirements

Development Environment
- Processor: Intel Core i5 or higher
- RAM: 8GB minimum, 16GB recommended
- Storage: 256GB SSD minimum
- Network: Broadband internet connection
- Display: 1920x1080 resolution or higher

Production Server
- Processor: Multi-core CPU (4+ cores recommended)
- RAM: 16GB minimum, 32GB recommended
- Storage: 500GB SSD with RAID for reliability
- Network: High-speed internet (100+ Mbps)
- Backup: Regular automated backup system

6.2 Software Requirements

Operating System
- Development: Windows 10/11, macOS 11+, or Ubuntu 20.04+
- Production Server: Ubuntu Server 20.04 LTS or higher
- Client: Windows 7+, macOS 10.12+, Linux, Android 6+, iOS 12+

Development Tools
- Code Editor: Visual Studio Code
- Version Control: Git
- API Testing: Postman
- Database GUI: MongoDB Compass
- Code Quality: ESLint, Prettier

Runtime Environment
- Backend: Node.js v16+ LTS
- Database: MongoDB v5.0+
- Process Manager: PM2 for production

Frontend Technologies
- UI Framework: React.js v18.x
- Routing: React Router v6.x
- HTTP Client: Axios v1.x
- Form Management: React Hook Form v7.x
- Styling: Tailwind CSS v3.x or Bootstrap v5.x
- Icons: React Icons / Lucide React

Backend Technologies

- Framework: Express.js v4.x
- ODM: Mongoose v6.x
- Authentication: jsonwebtoken v9.x
- Password Hashing: bcrypt v5.x
- File Upload: Cloudinary
- Email: nodemailer v6.x

Browser Requirements

Supported Browsers:

- Google Chrome 90+
- Mozilla Firefox 88+
- Safari 14+
- Microsoft Edge 90+

Package Managers

- npm or yarn for dependency management

Cloud/Deployment (Optional)

- Cloud Platforms: AWS, Google Cloud, Azure, or DigitalOcean
- Database Hosting: MongoDB Atlas (managed MongoDB)
- File Storage: Cloud storage for attachments

7. SOFTWARE DESIGN

* **System Architecture**

The Bug Tracker System follows a three-tier client-server architecture:

Presentation Layer (Client-Side):

- Built with React.js providing component-based UI
- Responsive design for multi-device support
- Handles user interactions and client-side validation
- Communicates with backend via REST API

Application Layer (Server-Side):

- Express.js framework handling HTTP requests
- Implements business logic for bug lifecycle
- Enforces security through authentication/authorization
- Validates data integrity and business rules
- Provides RESTful API endpoints
- Manages notifications and activity logging

Data Layer:

- MongoDB document database for flexible storage
- Mongoose ODM for schema definition and validation
- Supports complex nested documents
- Implements indexes for performance optimization
- Handles file storage for attachments

* **Database Design**

Core Collections:

1. Bugs Collection: Stores complete bug lifecycle data with seven embedded phase objects, user references for assignments, and audit trail information
2. Users Collection: User authentication credentials, role-based permissions, contact information, and activity tracking

3. Tools Collection: Tool/application configuration, tester and developer assignments, technology stack information, SOP and release notes

4. Configuration Collection: System-wide settings including stages, status types, priorities, and technology stack list

5. Logs Collection: Comprehensive action logging, user activity tracking, and bug history

* **Security Design**

Authentication:
- JWT-based stateless authentication
- Bcrypt password hashing (10 salt rounds)
- Secure token generation and validation

Authorization:
- Role-based access control (RBAC)
- Middleware verification on protected routes
- Resource-level permissions

Data Protection:
- Input validation and sanitization
- CORS configuration and rate limiting

7.1. DFD (DATA FLOW DIAGRAM)

7.1.1 Level 0 - Context Diagram

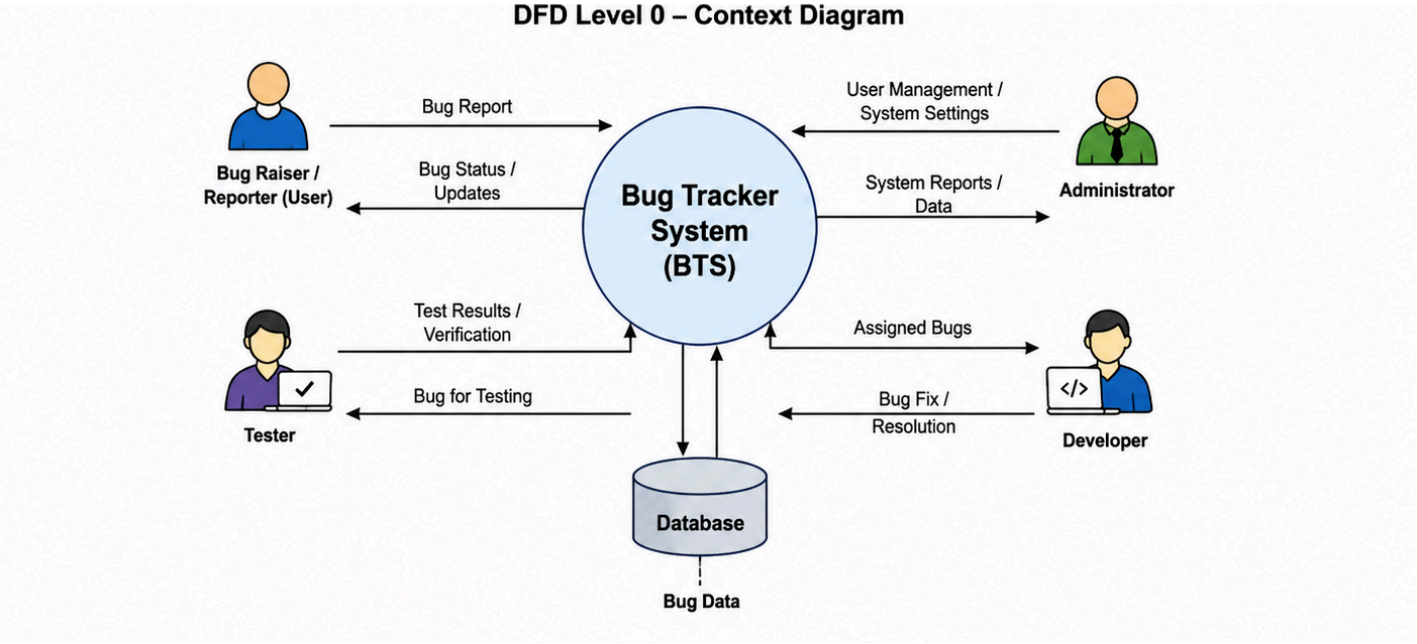


Figure 7.1.1 (DFD level 0)

1. Key Components

- Central Process: The large circle (Bug Tracker System) represents the entire system as a single process.
- External Entities: The people (Bug Raiser/Reporter, Tester, Developer, Administrator) and the Database that interact with the system.
- Data Flows: The arrows showing what information is being sent or received.

2. Interaction Breakdown

- Bug Raiser/Reporter (User): Submits bug reports and receives bug status updates.
- Administrator: Manages users, system settings and receives system reports and data.
- Developer: Receives assigned bugs and sends bug fix details/resolution.
- Tester: Receives bugs for testing and sends back the test results/verification.
- Database: Acts as the storage hub where the system stores bug data and retrieves it when needed.

7.1.2 Level 1 - Detailed Diagram

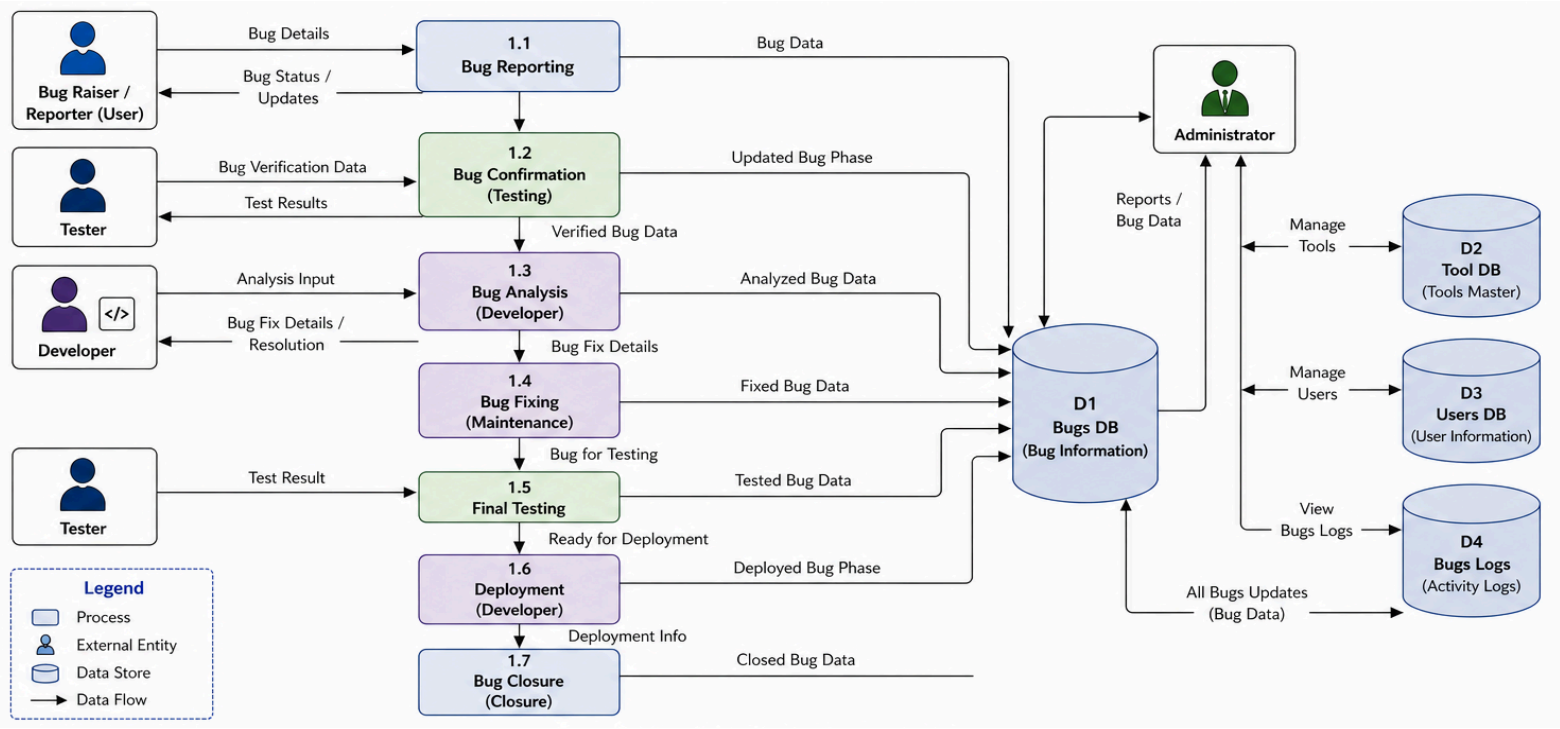


Figure 7.1.2 (DFD level 1)

1. Core Processes

- 1.1 Bug Reporting: Allows users to report bugs to the system.
- 1.2 Bug Confirmation (Testing): Testers verify and confirm reported bugs.
- 1.3 Bug Analysis (Developer): Developers analyze and understand the bugs.
- 1.4 Bug Fixing (Maintenance): Developers fix the reported bugs.
- 1.5 Final Testing: Testers test the fixed bugs and provide results.
- 1.6 Deployment (Developer): Fixed bugs are deployed to the system.
- 1.7 Bug Closure (Closure): Bugs are closed after successful deployment.

2. Data Stores (Databases)

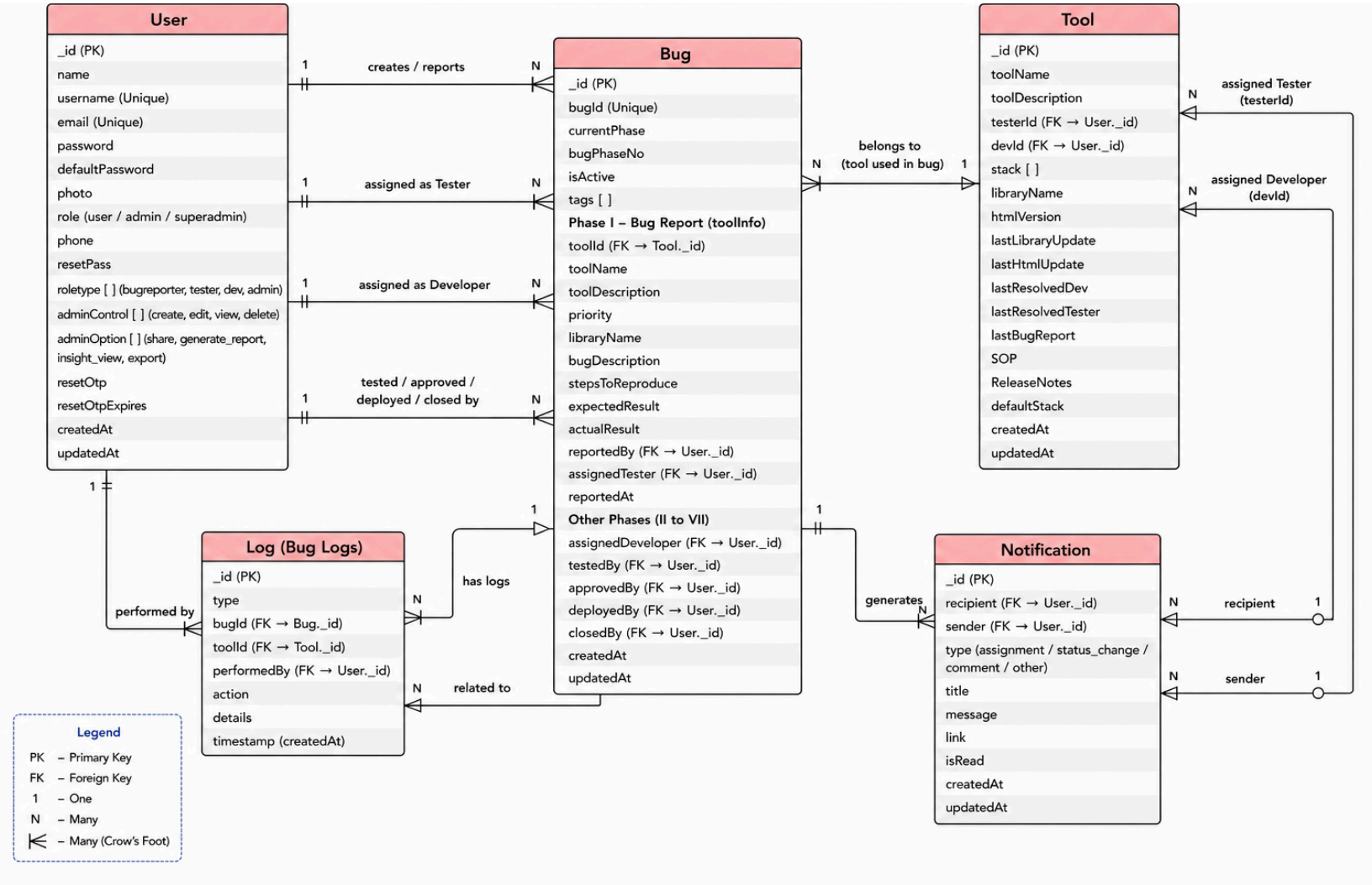
- D1 Bugs DB (Bug Information): Stores all bug-related information including reports, updates, fixes, testing, and closure.

- D2 Tool DB (Tools Master): Stores system tools and configurations.
- D3 Users DB (User Information): Stores user details and roles.
- D4 Bugs Logs: Stores all updates of bugs such as status changes, assignments, comments, and other activities.

3. External Entity

- Administrator: Manages tools, users, views bug reports/data, and monitors bugs logs.

7.2. E-R DIAGRAM



ER Diagram

The ER diagram represents the structure and relationships of the Bug Tracker System database. It consists of five main entities: User, Bug, Tool, Log (Bug Logs), and Notification.

- The User entity stores information about all users such as reporters, testers, developers, and administrators. A user can create bugs, work on them in different roles, and receive notifications.
- The Bug entity is the central component of the system. It contains all details related to a bug, including its phases (reporting, testing, analysis, fixing, deployment, and closure). Each bug is linked to a specific tool and multiple users involved in its lifecycle.
- The Tool entity represents the application or system where bugs are reported. A tool can have multiple bugs and is assigned to developers and testers.
- The Log (Bug Logs) entity records all activities performed on bugs, such as status changes, assignments, and updates. Each log is associated with a bug and the user who performed the action.
- The Notification entity manages communication between users. Notifications are generated for events like bug assignment, status changes, and comments, and are sent from one user to another.

Relationships:

- A User can create multiple Bugs (1:N).
- A Bug belongs to one Tool, while a tool can have many bugs (N:1).
- A Bug has multiple Logs (1:N).
- A User performs Logs and receives Notifications (1:N).
- A Bug generates Notifications for user interactions.

7.3. PROCESS FLOW DIAGRAM/FLOWCHART

7.3.1 Complete Bug Lifecycle Flowchart

The Flowchart represents the complete lifecycle of a bug in the Bug Tracker System, starting from user login to bug closure and reporting.

- The process begins with Log In, after which the user reports a bug.
- The bug is then tested by the tester.
- A decision is made:
    - If the bug test fails (reject), it is directly marked as Bug Closed.
    - If it passes, it moves to the analysis phase.
- During analysis, another decision is taken:
    - If the bug is rejected, it is again marked as Bug Closed.
    - If approved, it proceeds to maintenance (bug fixing).
- After fixing, the bug goes through final testing, followed by deployment.
- Once deployment is successful, the bug is marked as Bug Closed.
- After closure, the system performs log creation and generates reports.
- Finally, the user logs out, and the process ends.

Key Points

- Two rejection points: Testing and Analysis
- Both rejection paths directly lead to Bug Closed
- Ensures complete tracking from reporting to closure and reporting

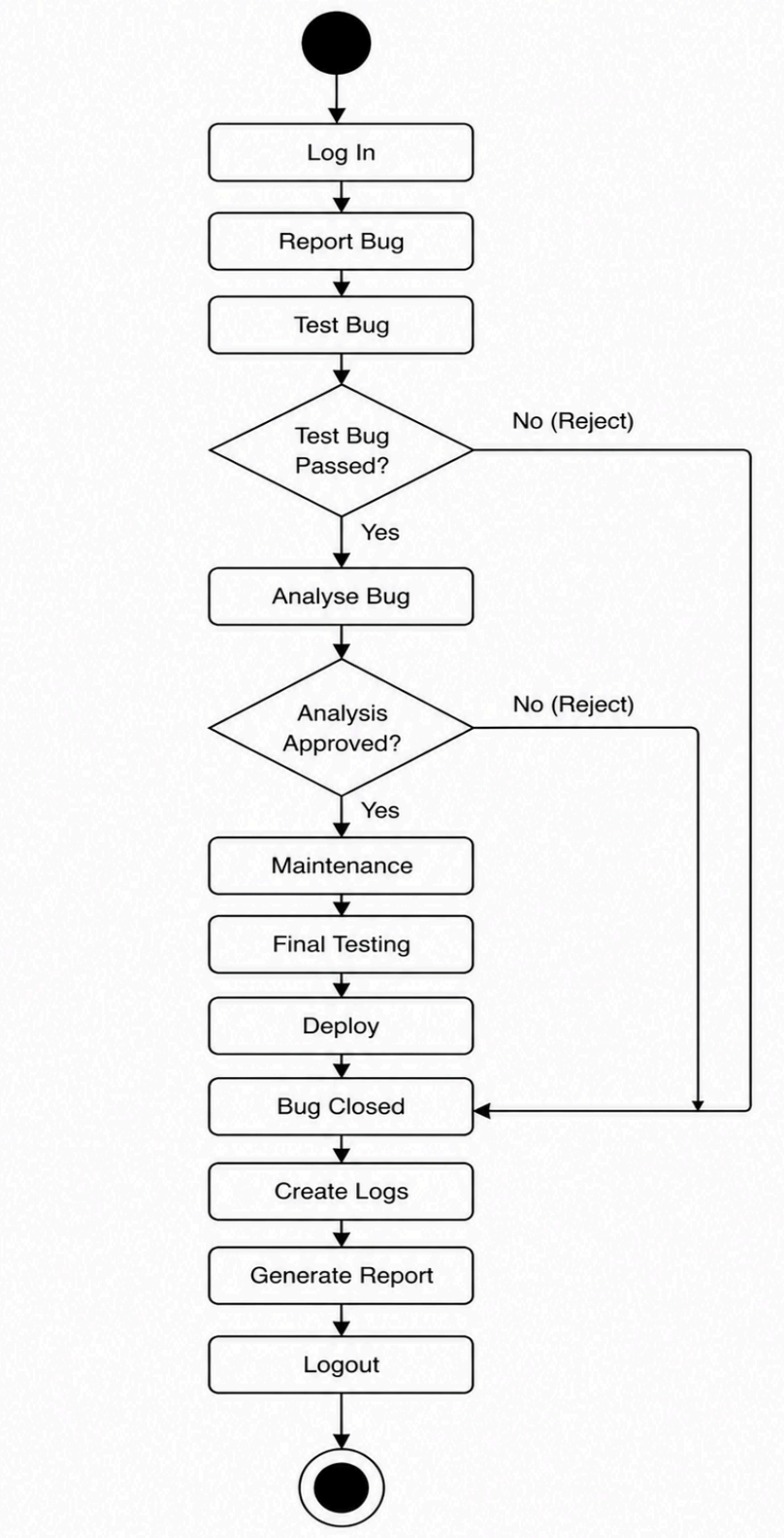


Figure 7.3. (Flow Diagram)

7.4. TABLE ARCHITECTURE

7.4.1 Bugs Collection Schema

Field Name	Data Type	Constraints	Description
_id	ObjectId	PRIMARY KEY	Unique document identifier
bugId	String	UNIQUE, REQUIRED	Bug identifier (e.g., BUG-2025-0001)
currentPhase	String	REQUIRED	Current phase name
bugPhaseNo	Number	REQUIRED	Current phase number (1–7)
phaseI_BugReport	Object	REQUIRED	Phase I data with tool info
phaseII_BugConfirmation	Object	CONDITIONAL	Phase II testing data
phaseIII_BugAnalysis	Object	CONDITIONAL	Phase III analysis data
phaseIV_Maintenance	Object	CONDITIONAL	Phase IV fix data
phaseV_FinalTesting	Object	CONDITIONAL	Phase V validation data
phaseVI_Deployment	Object	CONDITIONAL	Phase VI deployment data
phaseVII_Closure	Object	CONDITIONAL	Phase VII closure data
isActive	Boolean	REQUIRED	Bug active status
tags	Array	OPTIONAL	Bug classification tags
createdAt	Date	AUTO	Creation timestamp
updatedAt	Date	AUTO	Last update timestamp

Figure 7.4.1. (Bug Collection schema ~ MongoDB)

Bug Collection (Schema Description)

The **Bug Collection** is the core component of the Bug Tracker System. It stores all information related to bugs, including their lifecycle phases from reporting to closure.

1. Overview

Each bug is uniquely identified using a **bugId** and progresses through multiple phases such as reporting, testing, analysis, fixing, deployment, and closure. The schema is designed to support a **phase-based workflow**, ensuring structured tracking and updates.

2. Field Descriptions

- **_id (ObjectId, Primary Key):** Unique identifier automatically generated for each bug document.
- **bugId (String, Unique, Required):** A human-readable unique bug identifier (e.g., BUG-0001).

- **currentPhase** (**String, Required**): Represents the current stage of the bug in its lifecycle.
- **bugPhaseNo** (**Number, Required**): Indicates the numeric phase of the bug (1–7).

Phase-wise Data (**Embedded Objects**)

- **phaseI_BugReport** (**Object, Required**): Contains initial bug reporting details such as tool info, bug description, expected vs actual result, and reporter details.
- **phaseII_BugConfirmation** (**Object, Conditional**): Stores testing data including confirmation status, reproducibility, and tester remarks.
- **phaseIII_BugAnalysis** (**Object, Conditional**): Contains root cause analysis, estimated effort, affected modules, and developer remarks.
- **phaseIV_Maintenance** (**Object, Conditional**): Includes bug fix details, code changes, and maintenance-related updates.
- **phaseV_FinalTesting** (**Object, Conditional**): Stores validation results after fixing, including regression testing and approval status.
- **phaseVI_Deployment** (**Object, Conditional**): Contains deployment details such as environment, deployment type, and remarks.
- **phaseVII_Closure** (**Object, Conditional**): Final closure details including resolution summary and lessons learned.

Other Fields

- **isActive** (**Boolean, Required**): Indicates whether the bug is active or closed.
- **tags** (**Array, Optional**): Used for categorizing bugs (e.g., UI, Backend, Critical).
- **createdAt** (**Date, Auto**): Timestamp when the bug was created.
- **updatedAt** (**Date, Auto**): Timestamp of the last update made to the bug.

3. Key Features of Design

- **Phase-Based Structure:** Each bug follows a structured lifecycle divided into multiple phases.
- **Embedded Documents** (**MongoDB**): Phase data is embedded for better performance and atomic updates.
- **Scalability:** Optional fields allow flexibility for different bug scenarios.
- **Traceability:** Each phase stores detailed information for tracking bug progress.

7.4.2 Users Collection Schema

Field Name	Data Type	Constraints	Description
_id	ObjectId	PRIMARY KEY	Unique user identifier
username	String	UNIQUE, REQUIRED	Unique username
email	String	UNIQUE, REQUIRED	User email address
passwordHash	String	REQUIRED	Hashed password (bcrypt)
name	String	REQUIRED	Full name
mobile	String	REQUIRED	Mobile number
department	String	OPTIONAL	Department / team
role	String	REQUIRED	User role (bugraiser / tester / developer / admin)
isActive	Boolean	REQUIRED	Account status
createdAt	Date	AUTO	Registration timestamp
updatedAt	Date	AUTO	Last update timestamp

Figure 7.4.2. (Users Collection schema ~ MongoDB)

User Collection (Schema Description)

The **User Collection** stores all information related to system users, including authentication details, roles, and account status. It supports multiple user types such as bug raisers, testers, developers, and administrators.

1. Overview

Each user is uniquely identified and can perform different roles within the Bug Tracker System. The schema ensures secure authentication, role-based access, and proper user management.

2. Field Descriptions

- **_id (ObjectId, Primary Key):** Unique identifier for each user.
- **username (String, Unique, Required):** Unique username used for login.
- **email (String, Unique, Required):** User's email address for communication and authentication.
- **passwordHash (String, Required):** Securely stored hashed password using encryption (e.g., bcrypt).
- **name (String, Required):** Full name of the user.
- **mobile (String, Required):** User's contact number.
- **department (String, Optional):** Specifies the department or team of the user.
- **role (String, Required):** Defines the role of the user in the system:
  Bug Raiser / Tester / Developer / Admin
- **isActive (Boolean, Required):** Indicates whether the user account is active or deactivated.

- **createdAt (Date, Auto):** Timestamp when the user account was created.
- **updatedAt (Date, Auto):** Timestamp of the last update to user details.

3. Key Features of Design

- **Role-Based Access Control:** Users are assigned roles to control permissions and responsibilities.
- **Secure Authentication:** Passwords are stored in hashed format to enhance security.
- **Unique Identification:** Username and email are unique to prevent duplication.
- **Account Management:** The `isActive` field allows enabling/disabling user access.

7.4.3 Tools Collection Schema

Field Name	Data Type	Constraints	Description
_id	ObjectId	PRIMARY KEY	Unique tool identifier
toolName	String	REQUIRED	Tool / application name
toolDescription	String	REQUIRED	Detailed description
testerId	ObjectId	REQUIRED, FK	Default tester reference
devId	ObjectId	REQUIRED, FK	Default developer reference
stack	Array	REQUIRED	Supported technology stacks
libraryName	String	OPTIONAL	Primary library name
htmlVersion	String	OPTIONAL	HTML version
lastLibraryUpdate	Date	OPTIONAL	Last library update date
lastHtmlUpdate	Date	OPTIONAL	Last HTML update date
SOP	String	OPTIONAL	Standard Operating Procedure
releaseNotes	String	OPTIONAL	Release notes
createdAt	Date	AUTO	Creation timestamp
updatedAt	Date	AUTO	Last update timestamp

Figure 7.4.3. (Tool Collection schema ~ MongoDB)

Tool Collection (Schema Description)

The **Tool Collection** stores information about the applications or systems where bugs are reported. It maintains details about technology stacks, assigned users, and system configurations.

1. Overview

Each tool represents a specific software system or module. Bugs are associated with tools, and each tool is assigned a default tester and developer for handling issues efficiently.

2. Field Descriptions

- **_id (ObjectId, Primary Key):** Unique identifier for each tool.
- **toolName (String, Required):** Name of the tool or application.
- **toolDescription (String, Required):** Detailed description of the tool.
- **testerId (ObjectId, Required, FK):** Reference to the default tester assigned to the tool.
- **devId (ObjectId, Required, FK):** Reference to the default developer assigned to the tool.
- **stack (Array, Required):** List of technologies used in the tool (e.g., React, Node.js).
- **libraryName (String, Optional):** Name of the primary library used.
- **htmlVersion (String, Optional):** Specifies the HTML version used in the tool.
- **lastLibraryUpdate (Date, Optional):** Date when the library was last updated.
- **lastHtmlUpdate (Date, Optional):** Date when the HTML version was last updated.
- **SOP (String, Optional):** Standard Operating Procedure related to the tool.
- **releaseNotes (String, Optional):** Notes describing recent updates or releases.
- **createdAt (Date, Auto):** Timestamp when the tool was created.
- **updatedAt (Date, Auto):** Timestamp of the last update.

3. Key Features of Design

- **Tool-Based Bug Management:** Each bug is linked to a specific tool for better organization.
- **Default Role Assignment:** Assigning a default tester and developer ensures faster bug handling.
- **Technology Tracking:** The stack and library fields help track the technologies used.
- **Maintenance Tracking:** Update fields (library/HTML) help monitor system changes.

7.4.4 Configuration Collection Schema

Field Name	Data Type	Constraints	Description
_id	ObjectId	PRIMARY KEY	Configuration ID
stackList	Array	REQUIRED	Technology stacks list
stages	Object	REQUIRED	Phase number → phase name mapping
status	Array	REQUIRED	Available status types
priority	Array	REQUIRED	Priority levels
createdAt	Date	AUTO	Creation timestamp
updatedAt	Date	AUTO	Last update timestamp

Figure 7.4.4. (Configuration Collection schema ~ MongoDB)

Configuration Collection (Schema Description)

The **Configuration Collection** stores system-wide master data used in the Bug Tracker System. It defines standard values for bug stages, statuses, priorities, and supported platforms.

1. Overview

This collection acts as a **central configuration hub**, ensuring consistency across the system by maintaining predefined values used during bug tracking and management.

2. Field Descriptions

- **_id** (**ObjectId, Primary Key**)**:** Unique identifier for the configuration document.
- **stages** (**Object, Required**)**:** Defines the bug lifecycle stages mapped by numbers:
  1 → New Bug
  2 → Initial Testing
  3 → Bug Analysis
  4 → Under Maintenance
  5 → Final Testing
  6 → Deployment
  7 → Closed
- **status** (**Array, Required**)**:** List of possible bug statuses such as Pending, In Progress, Completed, Blocked, and Rejected.
- **priority** (**Array, Required**)**:** Defines severity levels of bugs such as Critical, High, Medium, and Low.
- **stacksList** (**Array, Optional**)**:** Contains supported platforms or system types like Web App, Android, iOS, etc.
- **createdAt** (**Date, Auto**)**:** Timestamp when the configuration was created.
- **updatedAt** (**Date, Auto**)**:** Timestamp of the last update.

3. Key Features of Design

- **Centralized Configuration:** All system constants are stored in one place.
- **Consistency:** Ensures uniform usage of stages, status, and priority across the system.
- **Flexibility:** Values can be updated dynamically without changing application logic.
- **Reusability:** Used across modules like Bug, Tool, and Reports.

7.4.5 Tools Collection Schema

Field Name	Data Type	Constraints	Description
_id	ObjectId	PRIMARY KEY	Log entry identifier
type	String	REQUIRED	Log type / category
bugId	ObjectId	OPTIONAL, FK	Associated bug reference
toolId	ObjectId	OPTIONAL, FK	Associated tool reference
performedBy	ObjectId	REQUIRED, FK	User who performed the action
action	String	OPTIONAL	Action performed
details	String	OPTIONAL	Detailed description
timestamp	Date	AUTO	Action timestamp

Figure 7.4.5. (Tool Collection schema ~ MongoDB)

Logs Collection (Schema Description)

The **Logs Collection** stores all activity records related to bugs, tools, and user actions within the Bug Tracker System. It helps in tracking system operations and maintaining a history of all changes.

1. Overview

Each log entry represents an action performed in the system, such as bug status updates, assignments, or modifications. Logs are essential for **audit tracking, debugging, and monitoring system activity**.

2. Field Descriptions

- **_id** (**ObjectId, Primary Key**): Unique identifier for each log entry.
- **type** (**String, Required**): Specifies the category of the log (e.g., bug activity, tool update).
- **bugId** (**ObjectId, FK, Optional**): References the bug associated with the log.
- **toolId** (**ObjectId, FK, Optional**): References the tool associated with the log.
- **performedBy** (**ObjectId, FK, Optional**): Identifies the user who performed the action.
- **action** (**String, Optional**): Describes the action performed (e.g., "status changed", "assigned to developer").
- **details** (**String, Optional**): Additional information about the action.
- **timestamp** (**Date, Auto**): Automatically records when the log entry was created.

3. Key Features of Design

- **Activity Tracking:** Records all system actions for better traceability.
- **Flexible References:** Logs can be linked to bugs, tools, or users.
- **Audit Support:** Helps in monitoring system changes and debugging issues.

Automatic Timestamping: Ensures accurate tracking of events.

7.5 OBJECT CLASS DIAGRAM

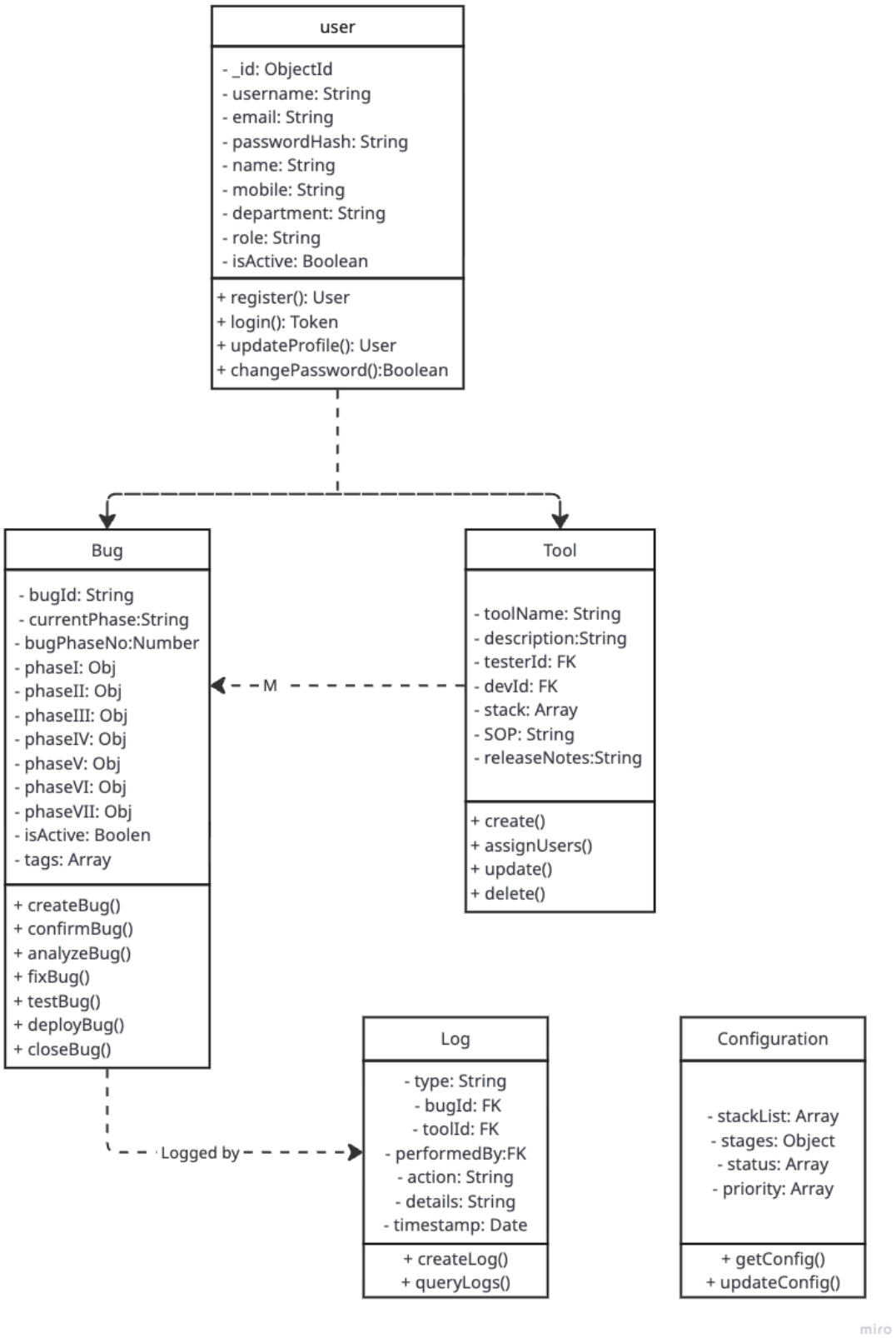


Figure 7.5. (OBJECT CLASS DIAGRAM)

Class Relationships:

1. User ↔ Bug: One-to-Many (User reports many Bugs); Many-to-One (Bug assigned to User as Tester/Developer)
2. User ↔ Tool: One-to-Many (User assigned to many Tools); Many-to-One (Tool has assigned Tester/Developer)
3. Bug ↔ Tool: Many-to-One (Many Bugs belong to one Tool)
4. User ↔ Log: One-to-Many (User performs many actions logged)
5. Bug ↔ Log: One-to-Many (Bug has many log entries)
6. Tool ↔ Log: One-to-Many (Tool has many log entries)

8. SOFTWARE MODULES AND DESCRIPTION (Summary)

Module 1: Authentication
User registration, login, JWT authentication, password hashing, and session handling.

Module 2: User Management
User profiles, role management, activation/deactivation, and user listing with search.

Module 3: Tool Management
Tool/application management, tester–developer assignment, tech stack, SOP, and release notes.

Module 4: Bug Reporting – Phase I
Bug submission, tool selection, file uploads, auto assignment, bug ID generation, and listing.

Module 5: Bug Confirmation – Phase II
Tester verification, reproduction workflow, confirmation status, developer assignment, notifications.

Module 6: Bug Analysis – Phase III
Root cause analysis, effort estimation, affected modules, and resolution planning.

Module 7: Bug Maintenance – Phase IV
Fix implementation, code change documentation, resolution notes, and attachments.

Module 8: Final Testing – Phase V
Validation testing, regression testing, approval or revision workflow.

Module 9: Deployment – Phase VI
Deployment tracking, environment details, deployment type, and SOP documentation.

Module 10: Closure – Phase VII
Bug closure, resolution summary, lessons learned, and knowledge documentation.

Module 11: Configuration Management
System configuration for stages, status, priorities, and technology stacks.

Module 12: Activity Logging
System-wide action logging, audit trail, and activity tracking.

Module 13: Notification System
Email notifications, templates, triggers, and notification preferences.

Module 14: Search & Filtering
Advanced search, multi-criteria filtering, sorting, and pagination.

Module 15: Dashboard & Analytics
Role-based dashboards, statistics, charts, and activity indicators.

Module 16: Admin Panel
Administrative controls, system monitoring, configuration UI, and logs.

Module 17: Testing & QA
Unit, integration, system testing, bug fixes, and performance optimization.

9. GANTT CHART & PERT CHART

9.1 Gantt Chart - Project Timeline

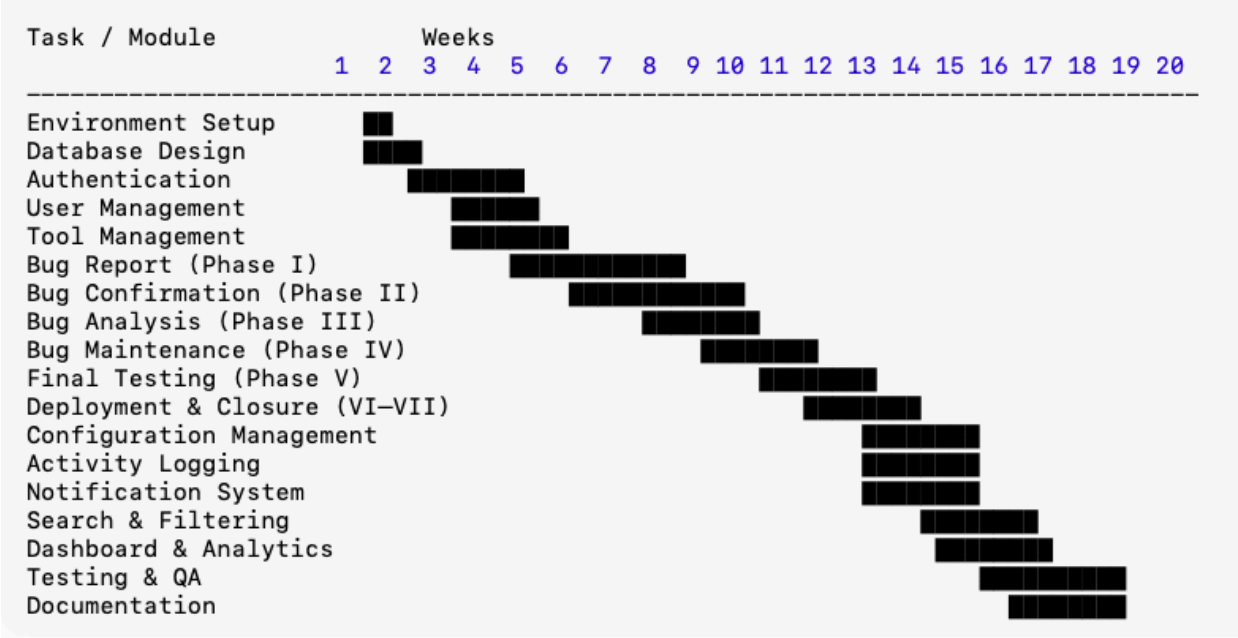


Figure 9.1 (Gantt Chart)

Figure 9.1 (Gantt Chart)

Milestones:

- ▼ Week 3: Authentication & User Management Complete
- ▼ Week 9: Core Bug Reporting & Confirmation Working
- ▼ Week 14: Complete Seven-Phase Workflow Implemented
- ▼ Week 17: Admin Features & Analytics Complete
- ▼ Week 20: Production Deployment & Project Completion

9.2 PERT Chart - Critical Path Analysis

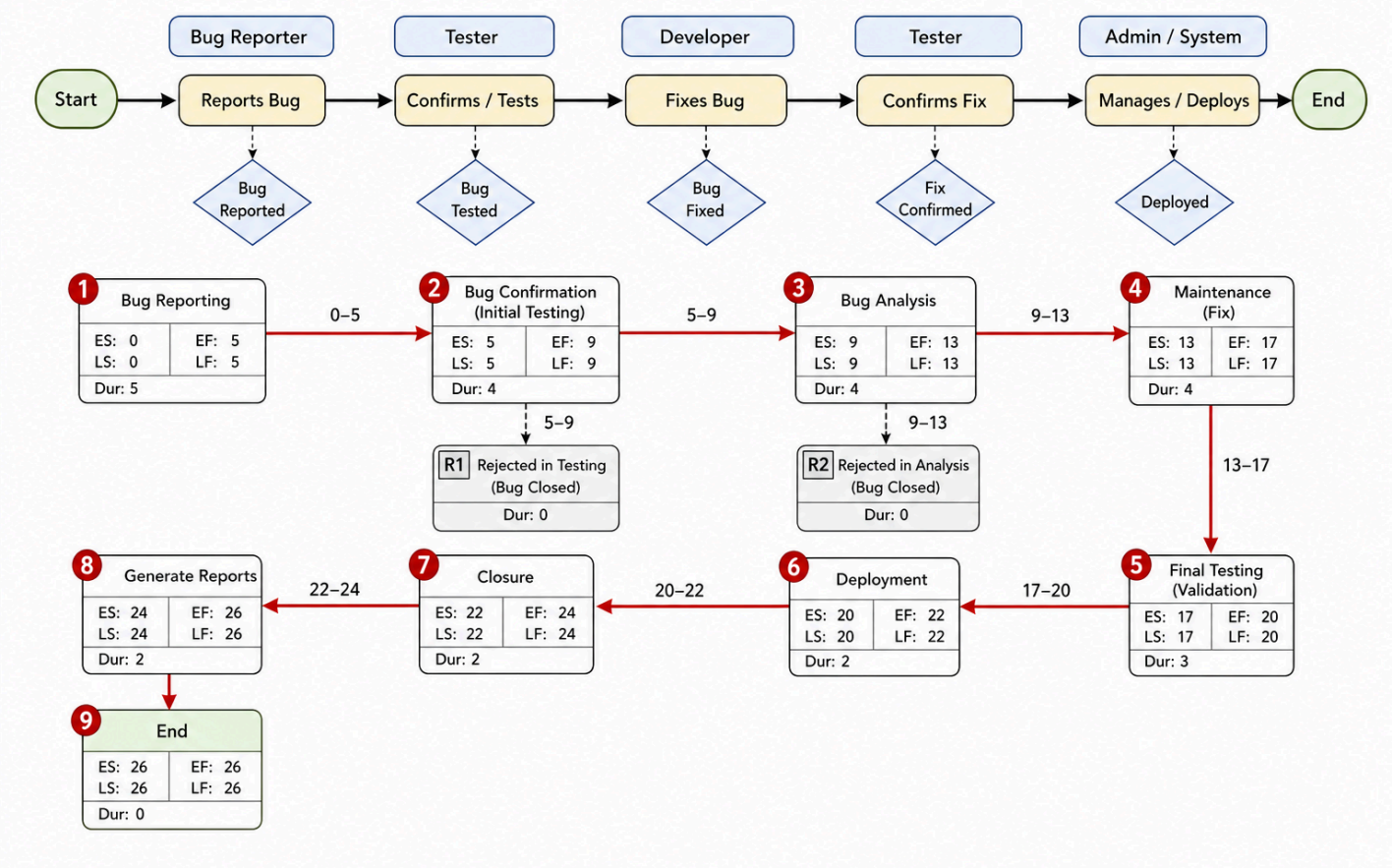


Figure 9.2（PERT Chart）

Critical Path

The **critical path** is the longest sequence of activities that determines the total project duration

: 1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9

Total Duration: 20 weeks

10. SOFTWARE TESTING TYPES & STRATEGIES

10.1 Unit Testing

Objective: Test individual components and functions in isolation.

Scope: Backend API functions, React components, utility functions, database operations

Tools: Jest, React Testing Library

Coverage Target: 80% code coverage

10.2 Integration Testing

Objective: Validate interaction between modules and APIs.

Scope: API routes, database integration, module interactions.

Tools: Jest.

10.3 System Testing

Objective: Ensure complete system functionality.

Scope: End-to-end workflows, all bug lifecycle phases, cross-module behavior.

Tools: Manual Testing.

10.4 Security Testing

Objective: Identify security vulnerabilities.

Scope: Authentication, authorization, input validation, injection prevention.

Tools: Manual Testing.

10.5 Performance Testing

Objective: Evaluate system performance under load.

Scope: Response time, concurrent users, database performance.

Tools: Manual ping check

Acceptance Criteria:

- Page load < 3 seconds
- API response < 2 seconds
- Supports 100 concurrent users

10.6 Usability Testing

Objective: Assess user experience and interface usability.

Scope: UI clarity, navigation, form usability, mobile responsiveness.

Method: User feedback sessions.

10.7 Compatibility Testing

Objective: Ensure cross-browser and device compatibility.

Scope: Major browsers and devices.

Tools: BrowserStack, Manual Testing.

10.8 Regression Testing

Objective: Ensure new changes do not affect existing functionality.

Strategy: Automated test execution, manual verification, smoke testing before releases.

10.9 Testing Strategy Summary

Testing was carried out in the following phases:

-Development Testing: Unit and integration testing during development

-System Testing: End-to-end, security, and performance testing

-User Acceptance Testing: Feedback-based validation

-Pre-Deployment Testing: Final regression and compatibility testing

11. SECURITY MECHANISM

11.1 Authentication Security

Password Security:

- Bcrypt hashing algorithm with 10 salt rounds
- Minimum password requirements (8 characters, mixed case, numbers)
- No plain text password storage
- Secure password reset mechanism with time-limited tokens

JWT Token Management:

- Secure token generation with secret key
- Token expiration (48 hours for access tokens)
- Token refresh mechanism for extended sessions
- HTTP-only cookies for token storage
- Token validation on every protected route request

11.2 Authorization Security

Role-Based Access Control (RBAC):

- Middleware verification for every protected endpoint
- Four distinct roles: Bug Raiser, Tester, Developer, Administrator
- Role checking before allowing specific actions
- Resource-level permissions
- Phase-specific permissions (only assigned tester can confirm bugs)

16.3 Data Security

Input Validation:

- Server-side validation for all user inputs
- express-validator for comprehensive validation rules
- Type checking and format validation
- Length restrictions on all fields
- Regex patterns for specific formats (email, mobile)

Injection Attack Prevention:

- NoSQL injection prevention through Mongoose schema validation
- Input sanitization for all user-provided data
- Parameterized queries using Mongoose ORM
- HTML entity encoding for displayed user content

XSS Prevention:
- Input sanitization removing script tags
- Output encoding before rendering user content
- Content Security Policy (CSP) headers
- React's built-in XSS protection

11.4 File Upload Security

File Type Validation:
- Whitelist of allowed file types (images, documents)
- MIME type verification
- File extension validation

File Size Restrictions:
- Maximum file size limit (10MB per file)
- Total upload size limits per request
- Prevention of disk space exhaustion attacks

Secure File Storage:
- Unique file naming to prevent overwrites
- Files stored outside web root directory
- Path traversal attack prevention
- Access control on uploaded files

11.5 Communication Security

HTTPS/TLS:
- SSL/TLS certificates for production environment
- Secure cookie flags (httpOnly, secure, sameSite)
- HSTS header forcing HTTPS

API Security Headers:
- Content Security Policy (CSP)
- X-Frame-Options preventing clickjacking

11.6 Database Security

MongoDB Security:
- MongoDB authentication enabled

- Database user with limited permissions
- Connection string stored in environment variables
- Network access restrictions (firewall rules)
- Regular automated backups

11.7 Application Security

CORS Configuration:
- Restricted origin list for API access
- Only allowed domains can make requests

Rate Limiting:
- Request rate limits to prevent brute force attacks
- Login attempt limiting (5 attempts per 15 minutes)
- API rate limiting per IP address

Error Handling:
- Generic error messages to users (no sensitive details exposed)
- Detailed error logging for debugging (server-side only)
- Stack traces never sent to clients
- Custom error pages

11.8 Logging and Monitoring

Security Event Logging:
- Failed login attempts logged with IP address
- All administrative actions logged

Audit Trail:
- User action tracking with timestamps
- Changes to critical data logged

12. FUTURE SCOPE OF THE PROJECT

12.1 Phase 1 Enhancements (6-12 months)

1. Advanced Analytics and Reporting:
- Interactive dashboards with real-time charts
- Trend analysis for bug patterns
- Predictive analytics for resolution time estimation
- Team performance metrics and KPIs
- Custom report generation with export functionality (PDF, Excel)

2. Enhanced Notification System:
- In-app notifications with badge counters
- SMS notifications for critical bugs
- Push notifications for mobile browsers
- Configurable notification rules per user
- Escalation mechanisms for overdue bugs
- Digest emails (daily/weekly summaries)

3. Improved Search and Filtering:
- Advanced search with complex boolean queries
- Saved search queries for frequently used filters
- Full-text search across all bug fields
- Search suggestions and autocomplete

12.2 Phase 2 Enhancements (12-24 months)

4. Integration Capabilities:
- Version control system integration (Git, GitHub, GitLab)
- CI/CD pipeline integration (Jenkins, GitLab CI, GitHub Actions)
- Project management tool integration (Jira, Asana, Trello)
- Communication platform integration (Slack, Microsoft Teams)
- RESTful API for third-party integrations
- Webhook support for real-time event notifications

5. Mobile Applications:
- Native iOS and Android applications
- Offline capability for bug reporting
- Push notifications for mobile devices
- Voice-to-text for bug descriptions

6. Artificial Intelligence Features:
- Automatic bug classification using NLP
- Duplicate bug detection using machine learning
- Intelligent assignment recommendations based on expertise
- Predicted resolution time estimation
- Smart notification filtering
- Auto-tagging of bugs based on content analysis
- Sentiment analysis of bug descriptions

12.3 Phase 3 Enhancements (24+ months)

7. Enterprise Features:
- Multi-tenant architecture for multiple organizations
- Single sign-on (SSO) integration (OAuth, SAML)
- Custom workflow designer with drag-and-drop interface
- White-label capabilities for branding
- Advanced compliance reporting (SOC 2, ISO 27001)
- Role hierarchy with delegation capabilities
- SLA (Service Level Agreement) tracking

8. Collaboration Enhancements:
- Real-time chat for bugs with typing indicators
- Video call integration for issue discussion
- Screen sharing for bug demonstration
- Collaborative document editing for analysis
- Knowledge base integration with articles
- Community forums for best practices

9. Advanced Reporting:
- Custom report builder with drag-and-drop
- Scheduled reports via email
- Real-time dashboards with auto-refresh
- Comparison reports (month-over-month, year-over-year)
- Heatmaps for bug distribution

10. Automation Features:
- Automated bug assignment based on keywords
- Automated status updates based on time
- Bulk operations on bugs

13. LIMITATIONS OF THE PROJECT

13.1 Technical Limitations

1. No Native Mobile Applications: Current version is web-based only, requires mobile browser for access, no offline capabilities

2. Limited Integration Capabilities: No direct integration with version control systems, CI/CD pipelines, or third-party project management tools

3. Single Language Support: Interface available in English only, no internationalization (i18n) support

4. File Storage Limitations: Maximum file size limited to 10MB per attachment, limited file type support, no video file attachments, local/server storage only

13.2 Functional Limitations

5. No Advanced Analytics: Basic reporting and statistics only, no predictive analytics or machine learning, limited data visualization options

6. No Real-Time Collaboration: No live chat functionality, no real-time co-editing, email-only notifications

7. Limited Workflow Customization: Fixed seven-phase workflow, cannot create custom phases without code changes, no visual workflow designer

8. No API for Third-Party Access: No public API for external integrations, no webhook support, no plugin architecture

13.3 Scalability Limitations

9. Single-Tenant Architecture: Designed for single organization use, no multi-tenancy support

10. Performance Constraints: Optimized for up to 1,000 concurrent users, may experience slowdown with 100,000+ bug records, limited horizontal scaling capabilities

13.4 Security Limitations

11. Basic Authentication Only: No single sign-on (SSO) integration, no multi-factor authentication (MFA), no biometric authentication support

12. Limited Audit Capabilities: Basic activity logging only, no advanced forensic analysis tools, no compliance reporting

13.5 User Experience Limitations

13. No Offline Support: Requires constant internet connection, no offline bug submission capability

14. Limited Accessibility Features: Basic accessibility compliance only, no screen reader optimization, limited keyboard navigation support

13.6 Deployment Limitations

15. Manual Deployment Process: No automated deployment pipeline, manual configuration required, no containerization (Docker) support

16. Infrastructure Dependencies: Requires separate MongoDB installation, needs Node.js runtime environment, email server dependency

13.7 Data Management Limitations

17. Limited Data Export Options: No bulk data export functionality, no scheduled backup automation, limited data migration tools

18. No Version Control for Bugs: No historical version tracking of bug changes, cannot rollback to previous bug state

13.8 Support and Documentation Limitations

19. Limited Documentation: Basic user manual only, no video tutorials, limited troubleshooting guides

20. No Advanced User Training: No built-in onboarding wizard, no interactive tutorials, limited help resources within application

14. BIBLIOGRAPHY & REFERENCES

Books

1. Node.js Design Patterns (3rd Ed.), Mario Casciaro & Luciano Mammino, Packt Publishing, 2020.
2. Learning React (2nd Ed.), Alex Banks & Eve Porcello, O'Reilly Media, 2020.
3. MongoDB: The Definitive Guide (3rd Ed.), Shannon Bradshaw et al., O'Reilly Media, 2019.
4. UML Distilled (3rd Ed.), Martin Fowler, Addison-Wesley, 2003.

Official Documentation

1. Node.js Official Documentation, https://nodejs.org/en/docs/, Accessed: October 2024.
2. React.js Official Documentation, https://react.dev/, Accessed: October 2024.
3. MongoDB & Mongoose Documentation, https://docs.mongodb.com/, https://mongoosejs.com/, Accessed: October 2024.
4. JWT (JSON Web Tokens) Documentation, https://jwt.io/, Accessed: October 2024.

Research Papers / Journals

1. Bug Tracking Systems: A Systematic Review, Journal of Software Engineering Research and Development, 2019.

Technical Articles / Guidelines

1. RESTful API Design Best Practices, Microsoft Azure Architecture Center, 2021.

PROJECT REPORT

(BUG TRACKER SYSTEM)

Table of content

1. TITLE OF THE PROJECT

This project titled **BUG TRACKER SYSTEM** is a web-based application designed to automate and streamline the complete lifecycle of software bug management. The system efficiently manages bugs through seven distinct phases, namely Bug Reporting, Bug Confirmation, Bug Analysis, Maintenance, Final Testing, Deployment, and Closure.

It helps development and testing teams to track, monitor, and resolve bugs systematically, thereby improving software quality, reducing development time, and ensuring better coordination among team members.

2. INTRODUCTION OF THE PROJECT

The Bug Tracker System is a comprehensive web-based application designed to revolutionize bug management processes in software development environments. This system provides a centralized, automated platform for managing the complete bug lifecycle through seven distinct phases ensuring systematic quality assurance at each stage of bug resolution.

2.1 Background

In contemporary software development environments, efficient bug tracking and management have become critical factors in ensuring software quality, timely project delivery, and effective team collaboration. Traditional bug management approaches involving manual processes, email communications, and spreadsheet-based tracking systems suffer from significant limitations including poor coordination, lack of accountability, inadequate workflow management, and insufficient audit trails.

2.2 Problem Statement

Current bug management practices in many software development organizations rely heavily on manual processes, fragmented tools, and inconsistent methodologies. These approaches create significant challenges:

1. Lack of Structured Workflow Management: Bugs may skip essential quality assurance steps without enforced workflows

2. Poor Coordination and Communication: Information silos where team members work with incomplete or outdated information

3. Inadequate Quality Assurance: Without mandatory testing phases, ensuring proper bug resolution becomes challenging

4. Insufficient Accountability: Lack of comprehensive audit trails makes it difficult to track actions and measure performance

5. Limited Scalability: Manual processes don't scale well with increasing bug volumes and team sizes

2.3 Proposed Solution

The Bug Tracker System addresses these challenges through a comprehensive, automated solution that implements systematic workflow management, enforces quality standards, and provides complete visibility into the bug resolution process.

Key Features:

1. Seven-Phase Workflow Architecture:

- Phase I: Bug Report - Initial submission with comprehensive details
- Phase II: Bug Confirmation - Tester verification and validation
- Phase III: Bug Analysis - Root cause identification by developer (phase III, IV merge)
- Phase IV: Maintenance - Fix implementation and documentation
- Phase V: Final Testing - Resolution validation and approval
- Phase VI: Deployment - Production deployment documentation
- Phase VII: Closure - Final resolution summary and lessons learned

2. Role-Based Access Control: Four distinct user roles (Bug Raiser, Tester, Developer, Administrator) with appropriate permissions

3. Tool-Based Organization: Bugs organized by tools/applications with pre-assigned testers and developers for automatic routing

4. Comprehensive Logging: Complete audit trail of all actions with timestamps and user information

5. Configurable System Settings: Dynamic configuration of stages, status types, priorities, and technology stacks

2.4 Scope

The system addresses critical challenges in software development by providing transparency, accountability, and efficiency in bug management processes while reducing resolution time through automated workflows and systematic quality assurance procedures.

3. OBJECTIVE OF THE PROJECT

3.1 Primary Objectives

1. Automate Bug Lifecycle Management
    - Implement a seven-phase automated workflow
    - Ensure systematic progression of bugs from reporting to closure
    - Built-in quality gates at each phase

2. Implement Role-Based Access Control- Design comprehensive user management system
    - Role-based permissions (Bug Raiser, Tester, Developer, Administrator)
    - Users can only perform actions appropriate to their roles

3. Ensure Quality Assurance
    - Establish mandatory testing and validation phases
    - Prevent bugs from being marked as resolved without proper verification
    - Include initial testing after confirmation and final testing before deployment

4. Provide Tool-Based Organization
    - Implement tool management system
    - Categorize bugs by application/module
    - Pre-assigned testers and developers for automatic routing

5. Maintain Comprehensive Audit Trails
    - Log all bug-related activities
    - Track creation, status changes, phase progressions, and user actions
    - Complete timestamp and user information for accountability

3.2 Secondary Objectives

1. Enhance Team Collaboration: Facilitate better communication between stakeholders through centralized information sharing and automated notifications

2. Reduce Resolution Time: Minimize bug resolution time through automated routing and clear responsibility assignment

3. Support Multiple Technology Stacks: Manage bugs across diverse platforms (Scripts, Web Apps, Android, iOS)

4. Enable Data-Driven Decision Making: Comprehensive reporting capabilities to identify bottlenecks and measure team performance

5. Ensure System Configurability: Allow administrators to customize system settings without code Modifications

6. Implement Secure Data Management: Ensure data integrity and security through proper authentication and authorization.

4. PROJECT CATEGORY

The Bug Tracker System falls under the Web Application Development category with strong elements of **Database Management Systems (DBMS) and Software Engineering.**

4.1 Primary Category

Web Application Development:

- Full-stack web application using MERN technology stack
- React.js for frontend development with single-page application (SPA) architecture
- Node.js with Express.js for backend services
- MongoDB for data persistence
- RESTful API architecture

4.2 Secondary Categories

Database Management:

- Document-oriented database (MongoDB)
- Complex schemas with embedded documents and references between collections
- Indexing strategies for performance optimization
- Data validation and referential integrity

Software Engineering Principles:

- Modular architecture design and separation of concerns
- Comprehensive requirements analysis and systematic design using UML diagrams
- Structured implementation methodology and rigorous testing strategies

Workflow Management:

- Sophisticated workflow automation with seven distinct phases
- Automated routing based on user roles and validation checkpoints at each phase transition

4.3 Project Complexity

This multi-faceted project requires integration of diverse technical skills including frontend development (React.js), backend programming (Node.js, Express.js), database design (MongoDB), API development (RESTful services), security implementation (JWT, bcrypt), and user experience design (UI/UX). The project demonstrates comprehensive understanding of full-stack development, making it an ideal project for MCA degree requirements.

5. DRAWBACK OF THE EXISTING SYSTEM

5.1 Traditional Bug Tracking Systems

1. Jira Software:

- Complex interface with steep learning curve
- Expensive licensing for growing teams
- Over-engineered for small to medium projects
- Lacks structured quality assurance workflow with no enforced testing phases3.

GitHub Issues:

- Basic bug tracking capabilities only
- Cannot enforce structured workflows
- Minimal reporting and analytics capabilities
- Tightly coupled with GitHub ecosystem

5.2 Common Drawbacks of Manual/Spreadsheet-Based Systems

1. Communication Gaps: Manual tracking creates information silos with email chains difficult to track and important details getting lost

2. Lack of Structured Workflows: No enforced quality assurance steps, bugs skip essential validation, inconsistent handling procedures

3. Inadequate Documentation: Missing knowledge management with no standardized documentation format, resolution steps not captured

4. Poor Accountability: No comprehensive audit trails, difficult to identify bottlenecks and measure team performance

5. Scalability Limitations: Cannot handle growing bug volumes, manual coordination becomes overwhelming, performance degrades with scale

6. Quality Assurance Challenges: No mandatory testing checkpoints, bugs marked resolved without proper testing, leading to production issues

5.3 Key Missing Features in Existing Systems

1. Enforced seven-phase workflow with mandatory multi-phase quality assurance
2. Tool-based organization with automatic routing based on tool/application assignment
3. Phase-specific documentation requirements at each phase
4. Comprehensive audit trail with system-wide action logging across all entities
5. Dynamic configuration without requiring code changes for workflow modifications
6. Integrated testing phases including confirmation testing, final testing, and deployment validation.

6. System Analysis

6.1 Software Requirement Analysis (SRA)

The project proposes a comprehensive, web-based Bug Tracker System designed to automate and streamline the complete lifecycle of software bug management.[1]

- **System Goal:** To provide transparency, accountability, and efficiency in bug management processes while reducing resolution time through automated workflows and systematic quality assurance procedures.[1]
- **Architecture:** The system uses a three-tier client-server architecture consisting of a Presentation Layer (client-side), an Application Layer (server-side), and a Data Layer (MongoDB).[1]
- **User Base:** The system is built to serve four distinct user roles, with permissions governing their access and actions: Bug Raiser, Tester, Developer, and Administrator.[1]
- **Technology Stack (MERN):**
    - **Frontend:** React.js v19.x (UI Framework).[1]
    - **Backend:** Node.js v20+ LTS with Express.js v4.x (Framework).[1]
    - **Database:** MongoDB v5.0+ (Document-oriented database) using Mongoose v6.x (ODM).[1]
- **Hardware & Software:**
    - **Development OS:** Windows 10/11, macOS 11+, or Ubuntu 20.04+.[1]
    - **Production Server OS:** Ubuntu Server 20.04 LTS or higher.[1]
    - **Minimum Development RAM:** 8GB.[1]
    - **Minimum Production Server RAM:** 16GB.[1]

6.2 Functional Requirements

Functional requirements are defined by the core objectives and module descriptions of the system:

- **Bug Lifecycle Management (Seven-Phase Workflow):** The system must automate the progression of bugs through seven mandatory phases:[1]
    - **Phase I (Bug Report):** Initial submission with comprehensive details, automatic ID generation, and tool selection.[1]
    - **Phase II (Confirmation):** Tester verification, validation, and confirmation of reproducibility.[1]
    - **Phase III (Analysis):** Root cause identification by the developer, effort estimation, and resolution planning.[1]
    - **Phase IV (Maintenance):** Fix implementation and documentation of code changes.[1]
    - **Phase V (Final Testing):** Validation and regression testing of the fix by the tester.[1]
    - **Phase VI (Deployment):** Documentation and tracking of deployment to the production environment.[1]
    - **Phase VII (Closure):** Formal closure with resolution summary and documentation of lessons learned.[1]

- **Role-Based Access Control (RBAC):** Users can only perform actions appropriate to their defined roles (Bug Raiser, Tester, Developer, Administrator).1
- **Tool Management:** Provide a system to categorize bugs by application/module and automatically route bugs to pre-assigned testers and developers.1
- **User Management:** Functionality for user registration, login, role assignment, and account activation/deactivation.1
- **Configuration Management:** Allow administrators to dynamically configure stages, status types (Pending, In Progress, Rejected), priorities (Critical, High, Medium, Low), and technology stacks without requiring code modifications.1
- **Search and Filtering:** Implement advanced search capabilities, multi-criteria filtering, sorting, and pagination across all bugs.1

6.3 Non-functional Requirements

Non-functional requirements cover security, performance, and other quality attributes:1. Security (Section 11)

- **Authentication:** Must implement JWT-based stateless authentication and use Bcrypt hashing (10 salt rounds) for password security.1
- **Authorization:** Must enforce Role-Based Access Control (RBAC) using middleware verification on protected routes, including phase-specific permissions (e.g., only assigned tester can confirm bugs).1
- **Data Security:** Server-side input validation/sanitization (using express-validator), NoSQL injection prevention via Mongoose schema validation, and XSS prevention through input sanitization and output encoding.1
- **Communication:** Requires HTTPS/TLS for the production environment and use of secure cookie flags.1

2. Performance and Scalability (Sections 10.5, 13.3)

- **Performance Acceptance Criteria:**
  - Page load time: Less than 3 seconds.1
  - API response time: Less than 2 seconds.1
  - Must support 100 concurrent users.1
- **Limitation (Scalability):** The current architecture is optimized for up to 1,000 concurrent users and may experience slowdown with over 100,000 bug records.1

3. Usability and Compatibility (Sections 6.2, 10.6)

- **Usability:** The system must be tested to ensure UI clarity, form usability, and mobile responsiveness.1
- **Browser Support:** Must support Google Chrome 90+, Mozilla Firefox 88+, Safari 14+, and Microsoft Edge 90+.1

4. Audit and Logging

- **Traceability:** Must maintain a comprehensive audit trail of all actions with user information and timestamps (Logs Collection).1

5. Identified Limitations (Gaps in NFRs)
The project identifies the following areas not implemented in the current version:1

- No Single Sign-On (SSO) or Multi-Factor Authentication (MFA).1
- No native mobile applications (web-based only, no offline capabilities).1
- Limited integration capabilities with version control (Git) or third-party project management tools (Jira, Trello).1
- Single language support (English only).1

6.4 Feasibility Study (Synthesized Analysis)

6.4.1 Technical Feasibility

FEASIBLE. The system relies on the well-defined and modern MERN technology stack (MongoDB, Express.js, React.js, Node.js). The architecture is a standard three-tier client-server model. Specified hardware and software requirements (e.g., 8GB min RAM for development, Ubuntu OS) are standard and available.1

6.4.2 Economic Feasibility

NOT DETERMINED. The document does not provide a Cost Estimation, financial metrics, cost-benefit analysis, or budget details. However, the use of open-source technologies (MERN stack) suggests a potentially cost-effective implementation.1

6.4.3 Operational Feasibility

HIGHLY FEASIBLE. The entire project is explicitly designed to address severe operational flaws in existing systems, such as a lack of structured workflow, poor coordination, and insufficient accountability. The proposed system—with its seven-phase workflow, Role-Based Access Control (RBAC), and comprehensive audit trails—directly solves the stated business problems.

6.4.4 Time Feasibility

PLANNED. The project has a clearly defined schedule. The Gantt Chart section outlines a **Critical Path** with a **Total Duration** of **20 weeks** from start to production deployment, indicating that the timeline has been planned and is manageable.

6.4.5 Legal Feasibility

PARTIALLY FEASIBLE. The system incorporates robust security mechanisms like JWT-based authentication, Bcrypt hashing, and server-side validation to protect data. However, it lacks advanced enterprise features like Single Sign-On (SSO) or Multi-Factor Authentication (MFA), and advanced compliance reporting (e.g., SOC 2) is listed only in the future scope.1

7. SYSTEM DESIGN

* **System Architecture**

The Bug Tracker System follows a three-tier client-server architecture:

Presentation Layer (Client-Side):

- Built with React.js providing component-based UI
- Responsive design for multi-device support
- Handles user interactions and client-side validation
- Communicates with backend via REST API

Application Layer (Server-Side):

- Express.js framework handling HTTP requests
- Implements business logic for bug lifecycle
- Enforces security through authentication/authorization
- Validates data integrity and business rules
- Provides RESTful API endpoints
- Manages notifications and activity logging

Data Layer:

- MongoDB document database for flexible storage
- Mongoose ODM for schema definition and validation
- Supports complex nested documents
- Implements indexes for performance optimization
- Handles file storage for attachments

* **Database Design**

Core Collections:

1. Bugs Collection: Stores complete bug lifecycle data with seven embedded phase objects, user references for assignments, and audit trail information

2. Users Collection: User authentication credentials, role-based permissions, contact information, and activity tracking

3. Tools Collection: Tool/application configuration, tester and developer assignments, technology stack information, SOP and release notes

4. Configuration Collection: System-wide settings including stages, status types, priorities, and technology stack list

5. Logs Collection: Comprehensive action logging, user activity tracking, and bug/tool history

* **Security Design**

Authentication:
- JWT-based stateless authentication
- Bcrypt password hashing (10 salt rounds)
- Secure token generation and validation

Authorization:
- Role-based access control (RBAC)
- Middleware verification on protected routes
- Resource-level permissions

Data Protection:
- Input validation and sanitization
- CORS configuration and rate limiting

7.1. DFD (DATA FLOW DIAGRAM)

7.1.1 Level 0 - Context Diagram

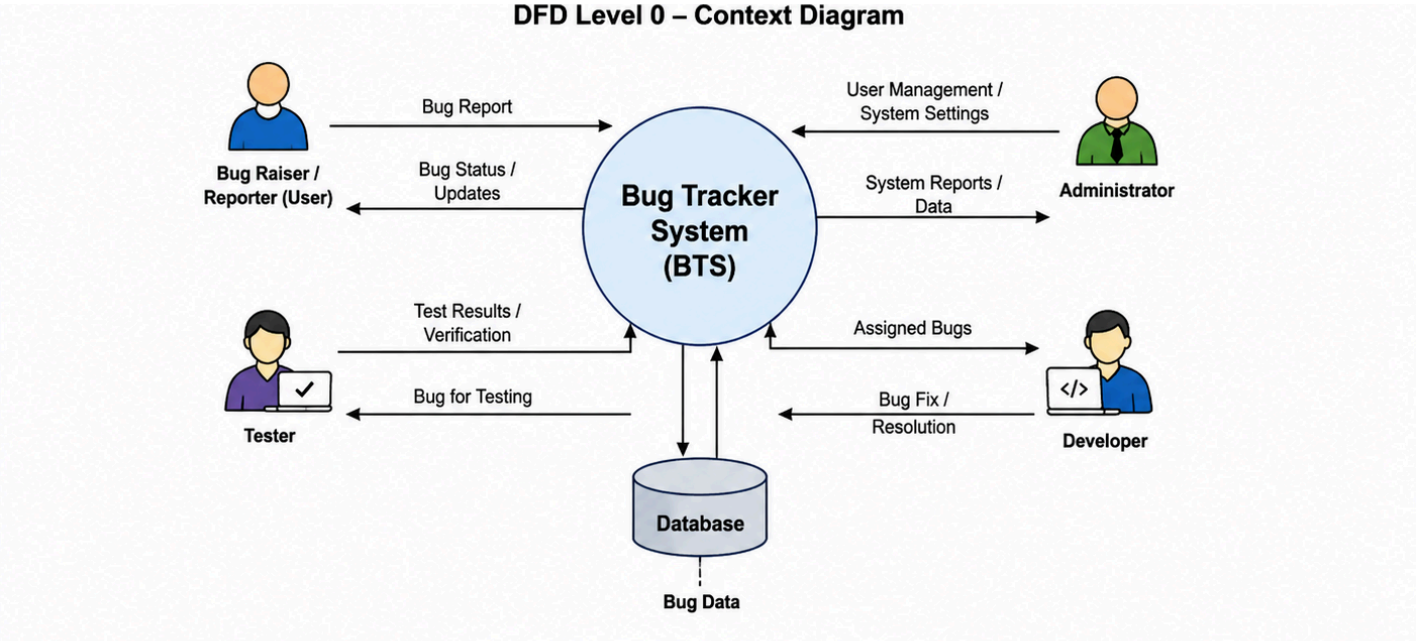


Figure 7.1.1 (DFD level 0)

1. Key Components

- Central Process: The large circle (Bug Tracker System) represents the entire system as a single process.
- External Entities: The people (Bug Raiser/Reporter, Tester, Developer, Administrator) and the Database that interact with the system.
- Data Flows: The arrows showing what information is being sent or received.

2. Interaction Breakdown

- Bug Raiser/Reporter (User): Submits bug reports and receives bug status updates.
- Administrator: Manages users, system settings and receives system reports and data.
- Developer: Receives assigned bugs and sends bug fix details/resolution.
- Tester: Receives bugs for testing and sends back the test results/verification.
- Database: Acts as the storage hub where the system stores bug data and retrieves it when needed.

7.1.2 Level 1 - Detailed Diagram

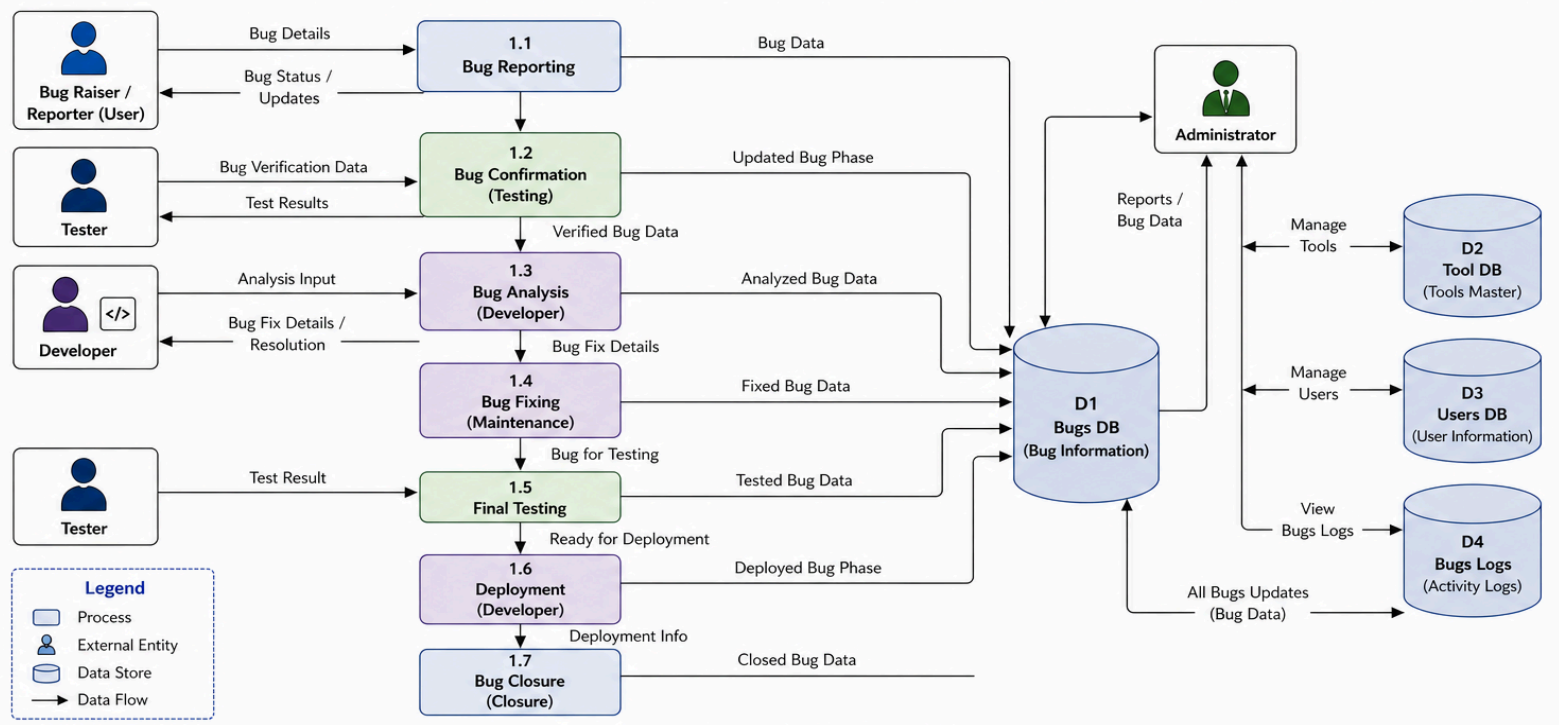


Figure 7.1.2 (DFD level 1)

1. Core Processes

- 1.1 Bug Reporting: Allows users to report bugs to the system.
- 1.2 Bug Confirmation (Testing): Testers verify and confirm reported bugs.
- 1.3 Bug Analysis (Developer): Developers analyze and understand the bugs.
- 1.4 Bug Fixing (Maintenance): Developers fix the reported bugs.
- 1.5 Final Testing: Testers test the fixed bugs and provide results.
- 1.6 Deployment (Developer): Fixed bugs are deployed to the system.
- 1.7 Bug Closure (Closure): Bugs are closed after successful deployment.

2. Data Stores (Databases)

- D1 Bugs DB (Bug Information): Stores all bug-related information including reports, updates, fixes, testing, and closure.
- D2 Tool DB (Tools Master): Stores system tools and configurations.
- D3 Users DB (User Information): Stores user details and roles.
- D4 Bugs Logs: Stores all updates of bugs such as status changes, assignments, comments, and other activities.

3. External Entity

- Administrator: Manages tools, users, views bug reports/data, and monitors bugs logs.

7.2. E-R DIAGRAM

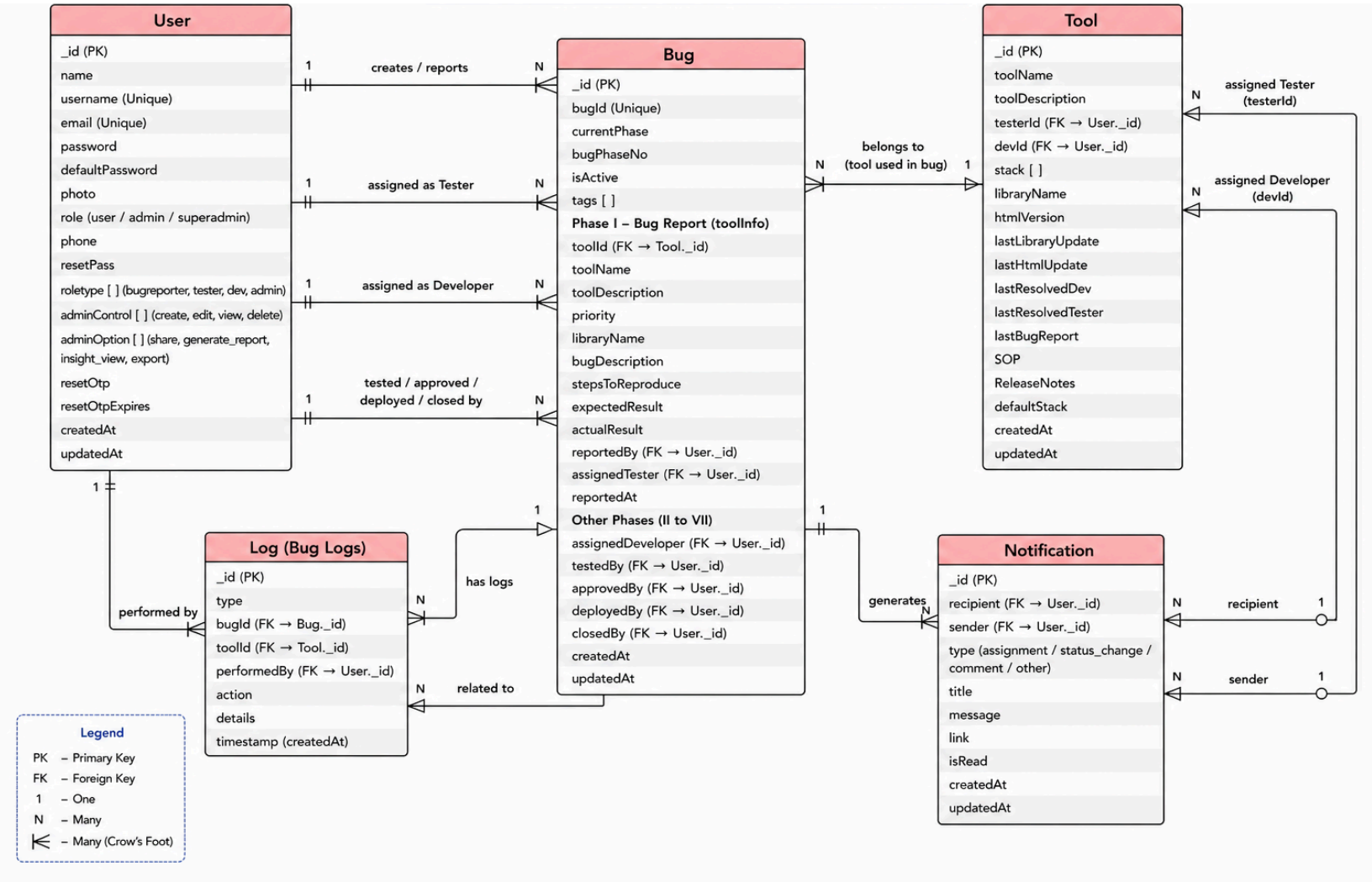


Figure 7.2. (E-R diagram)

ER Diagram

The ER diagram represents the structure and relationships of the Bug Tracker System database. It consists of five main entities: User, Bug, Tool, Log (Bug Logs), and Notification.

- The User entity stores information about all users such as reporters, testers, developers, and administrators. A user can create bugs, work on them in different roles, and receive notifications.
- The Bug entity is the central component of the system. It contains all details related to a bug, including its phases (reporting, testing, analysis, fixing, deployment, and closure). Each bug is linked to a specific tool and multiple users involved in its lifecycle.
- The Tool entity represents the application or system where bugs are reported. A tool can have multiple bugs and is assigned to developers and testers.
- The Log (Bug Logs) entity records all activities performed on bugs, such as status changes, assignments, and updates. Each log is associated with a bug and the user who performed the action.
- The Notification entity manages communication between users. Notifications are generated for events like bug assignment, status changes, and comments, and are sent from one user to another.

Relationships:

- A User can create multiple Bugs (1:N).
- A Bug belongs to one Tool, while a tool can have many bugs (N:1).
- A Bug has multiple Logs (1:N).
- A User performs Logs and receives Notifications (1:N).
- A Bug generates Notifications for user interactions.

7.3. PROCESS FLOW DIAGRAM/FLOWCHART

7.3.1 Complete Bug Lifecycle Flowchart

The Flowchart represents the complete lifecycle of a bug in the Bug Tracker System, starting from user login to bug closure and reporting.

- The process begins with Log In, after which the user reports a bug.
- The bug is then tested by the tester.
- A decision is made:
    - If the bug test fails (reject), it is directly marked as Bug Closed.
    - If it passes, it moves to the analysis phase.
- During analysis, another decision is taken:
    - If the bug is rejected, it is again marked as Bug Closed.
    - If approved, it proceeds to maintenance (bug fixing).
- After fixing, the bug goes through final testing, followed by deployment.
- Once deployment is successful, the bug is marked as Bug Closed.
- After closure, the system performs log creation and generates reports.
- Finally, the user logs out, and the process ends.

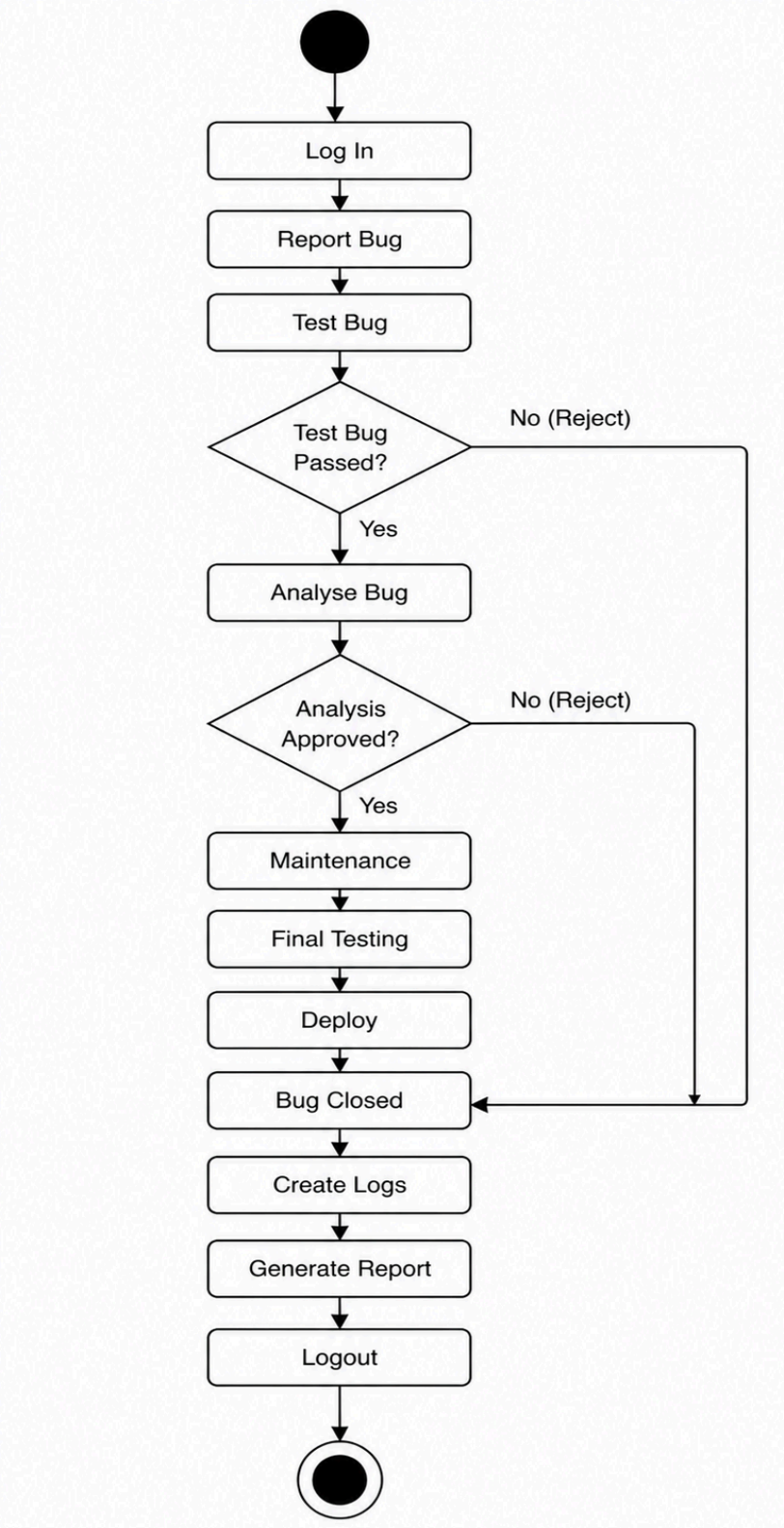


Figure 7.3. (Flow Diagram)

Key Points

- Two rejection points: Testing and Analysis
- Both rejection paths directly lead to Bug Closed
- Ensures complete tracking from reporting to closure and reporting

7.4. TABLE ARCHITECTURE

7.4.1 Bugs Collection Schema

Field Name	Data Type	Constraints	Description
_id	ObjectId	PRIMARY KEY	Unique document identifier
bugId	String	UNIQUE, REQUIRED	Bug identifier (e.g., BUG-0001)
currentPhase	String	REQUIRED	Current phase name
bugPhaseNo	Number	REQUIRED	Current phase number (1–7)
phaseI_BugReport	Object	REQUIRED	Phase I data with tool info
phaseII_BugConfirmation	Object	CONDITIONAL	Phase II testing data
phaseIII_BugAnalysis	Object	CONDITIONAL	Phase III analysis data
phaseIV_Maintenance	Object	CONDITIONAL	Phase IV fix data
phaseV_FinalTesting	Object	CONDITIONAL	Phase V validation data
phaseVI_Deployment	Object	CONDITIONAL	Phase VI deployment data
phaseVII_Closure	Object	CONDITIONAL	Phase VII closure data
isActive	Boolean	REQUIRED	Bug active status
tags	Array	OPTIONAL	Bug classification tags
createdAt	Date	AUTO	Creation timestamp
updatedAt	Date	AUTO	Last update timestamp

Figure 7.4.1. (Bug Collection schema ~ MongoDB)

Bug Collection (Schema Description)

The **Bug Collection** is the core component of the Bug Tracker System. It stores all information related to bugs, including their lifecycle phases from reporting to closure.

1. Overview

Each bug is uniquely identified using a **bugId** and progresses through multiple phases such as reporting, testing, analysis, fixing, deployment, and closure. The schema is designed to support a **phase-based workflow**, ensuring structured tracking and updates.

2. Field Descriptions

- **_id** (**ObjectId, Primary Key**): Unique identifier automatically generated for each bug document.
- **bugId** (**String, Unique, Required**): A human-readable unique bug identifier (e.g., BUG-0001).
- **currentPhase** (**String, Required**): Represents the current stage of the bug in its lifecycle.
- **bugPhaseNo** (**Number, Required**): Indicates the numeric phase of the bug (1–7).

Phase-wise Data (**Embedded Objects**)

- **phaseI_BugReport** (**Object, Required**): Contains initial bug reporting details such as tool info, bug description, expected vs actual result, and reporter details.
- **phaseII_BugConfirmation** (**Object, Conditional**): Stores testing data including confirmation status, reproducibility, and tester remarks.
- **phaseIII_BugAnalysis** (**Object, Conditional**): Contains root cause analysis, estimated effort, affected modules, and developer remarks.
- **phaseIV_Maintenance** (**Object, Conditional**): Includes bug fix details, code changes, and maintenance-related updates.
- **phaseV_FinalTesting** (**Object, Conditional**): Stores validation results after fixing, including regression testing and approval status.
- **phaseVI_Deployment** (**Object, Conditional**): Contains deployment details such as environment, deployment type, and remarks.
- **phaseVII_Closure** (**Object, Conditional**): Final closure details including resolution summary and lessons learned.

Other Fields

- **isActive** (**Boolean, Required**): Indicates whether the bug is active or closed.
- **tags** (**Array, Optional**): Used for categorizing bugs (e.g., UI, Backend, Critical).
- **createdAt** (**Date, Auto**): Timestamp when the bug was created.
- **updatedAt** (**Date, Auto**): Timestamp of the last update made to the bug.

3. Key Features of Design

- **Phase-Based Structure:** Each bug follows a structured lifecycle divided into multiple phases.
- **Embedded Documents** (**MongoDB**): Phase data is embedded for better performance and atomic updates.
- **Scalability:** Optional fields allow flexibility for different bug scenarios.
- **Traceability:** Each phase stores detailed information for tracking bug progress.

7.4.2 Users Collection Schema

Field Name	Data Type	Constraints	Description
_id	ObjectId	PRIMARY KEY	Unique user identifier
username	String	UNIQUE, REQUIRED	Unique username
email	String	UNIQUE, REQUIRED	User email address
passwordHash	String	REQUIRED	Hashed password (bcrypt)
name	String	REQUIRED	Full name
mobile	String	REQUIRED	Mobile number
department	String	OPTIONAL	Department / team
role	String	REQUIRED	User role (bugraiser / tester / developer / admin)
isActive	Boolean	REQUIRED	Account status
createdAt	Date	AUTO	Registration timestamp
updatedAt	Date	AUTO	Last update timestamp

Figure 7.4.2. (Users Collection schema ~ MongoDB)

User Collection (Schema Description)

The **User Collection** stores all information related to system users, including authentication details, roles, and account status. It supports multiple user types such as bug raisers, testers, developers, and administrators.

1. Overview

Each user is uniquely identified and can perform different roles within the Bug Tracker System. The schema ensures secure authentication, role-based access, and proper user management.

2. Field Descriptions

- **_id (ObjectId, Primary Key):** Unique identifier for each user.
- **username (String, Unique, Required):** Unique username used for login.
- **email (String, Unique, Required):** User's email address for communication and authentication.
- **passwordHash (String, Required):** Securely stored hashed password using encryption (e.g., bcrypt).
- **name (String, Required):** Full name of the user.
- **mobile (String, Required):** User's contact number.
- **department (String, Optional):** Specifies the department or team of the user.
- **role (String, Required):** Defines the role of the user in the system:
  Bug Raiser / Tester / Developer / Admin
- **isActive (Boolean, Required):** Indicates whether the user account is active or deactivated.

- **createdAt (Date, Auto):** Timestamp when the user account was created.
- **updatedAt (Date, Auto):** Timestamp of the last update to user details.

3. Key Features of Design

- **Role-Based Access Control:** Users are assigned roles to control permissions and responsibilities.
- **Secure Authentication:** Passwords are stored in hashed format to enhance security.
- **Unique Identification:** Username and email are unique to prevent duplication.
- **Account Management:** The `isActive` field allows enabling/disabling user access.

7.4.3 Tools Collection Schema

Field Name	Data Type	Constraints	Description
_id	ObjectId	PRIMARY KEY	Unique tool identifier
toolName	String	REQUIRED	Tool / application name
toolDescription	String	REQUIRED	Detailed description
testerId	ObjectId	REQUIRED, FK	Default tester reference
devId	ObjectId	REQUIRED, FK	Default developer reference
stack	Array	REQUIRED	Supported technology stacks
libraryName	String	OPTIONAL	Primary library name
htmlVersion	String	OPTIONAL	HTML version
lastLibraryUpdate	Date	OPTIONAL	Last library update date
lastHtmlUpdate	Date	OPTIONAL	Last HTML update date
SOP	String	OPTIONAL	Standard Operating Procedure
releaseNotes	String	OPTIONAL	Release notes
createdAt	Date	AUTO	Creation timestamp
updatedAt	Date	AUTO	Last update timestamp

Figure 7.4.3. (Tool Collection schema ~ MongoDB)

Tool Collection (Schema Description)

The **Tool Collection** stores information about the applications or systems where bugs are reported. It maintains details about technology stacks, assigned users, and system configurations.

1. Overview

Each tool represents a specific software system or module. Bugs are associated with tools, and each tool is assigned a default tester and developer for handling issues efficiently.

2. Field Descriptions

- **_id (ObjectId, Primary Key):** Unique identifier for each tool.
- **toolName (String, Required):** Name of the tool or application.
- **toolDescription (String, Required):** Detailed description of the tool.
- **testerId (ObjectId, Required, FK):** Reference to the default tester assigned to the tool.
- **devId (ObjectId, Required, FK):** Reference to the default developer assigned to the tool.
- **stack (Array, Required):** List of technologies used in the tool (e.g., React, Node.js).
- **libraryName (String, Optional):** Name of the primary library used.
- **htmlVersion (String, Optional):** Specifies the HTML version used in the tool.
- **lastLibraryUpdate (Date, Optional):** Date when the library was last updated.
- **lastHtmlUpdate (Date, Optional):** Date when the HTML version was last updated.
- **SOP (String, Optional):** Standard Operating Procedure related to the tool.
- **releaseNotes (String, Optional):** Notes describing recent updates or releases.
- **createdAt (Date, Auto):** Timestamp when the tool was created.
- **updatedAt (Date, Auto):** Timestamp of the last update.

3. Key Features of Design

- **Tool-Based Bug Management:** Each bug is linked to a specific tool for better organization.
- **Default Role Assignment:** Assigning a default tester and developer ensures faster bug handling.
- **Technology Tracking:** The stack and library fields help track the technologies used.
- **Maintenance Tracking:** Update fields (library/HTML) help monitor system changes.

7.4.4 Configuration Collection Schema

Field Name	Data Type	Constraints	Description
_id	ObjectId	PRIMARY KEY	Configuration ID
stackList	Array	REQUIRED	Technology stacks list
stages	Object	REQUIRED	Phase number → phase name mapping
status	Array	REQUIRED	Available status types
priority	Array	REQUIRED	Priority levels
createdAt	Date	AUTO	Creation timestamp
updatedAt	Date	AUTO	Last update timestamp

Figure 7.4.4. (Configuration Collection schema ~ MongoDB)

Configuration Collection (Schema Description)

The **Configuration Collection** stores system-wide master data used in the Bug Tracker System. It defines standard values for bug stages, statuses, priorities, and supported platforms.

1. Overview

This collection acts as a **central configuration hub**, ensuring consistency across the system by maintaining predefined values used during bug tracking and management.

2. Field Descriptions

- **_id** (**ObjectId, Primary Key**)**:** Unique identifier for the configuration document.
- **stages** (**Object, Required**)**:** Defines the bug lifecycle stages mapped by numbers:
  1 → New Bug
  2 → Initial Testing
  3 → Bug Analysis
  4 → Under Maintenance
  5 → Final Testing
  6 → Deployment
  7 → Closed
- **status** (**Array, Required**)**:** List of possible bug statuses such as Pending, In Progress, Completed, Blocked, and Rejected.
- **priority** (**Array, Required**)**:** Defines severity levels of bugs such as Critical, High, Medium, and Low.
- **stacksList** (**Array, Optional**)**:** Contains supported platforms or system types like Web App, Android, iOS, etc.
- **createdAt** (**Date, Auto**)**:** Timestamp when the configuration was created.
- **updatedAt** (**Date, Auto**)**:** Timestamp of the last update.

3. Key Features of Design

- **Centralized Configuration:** All system constants are stored in one place.
- **Consistency:** Ensures uniform usage of stages, status, and priority across the system.
- **Flexibility:** Values can be updated dynamically without changing application logic.
- **Reusability:** Used across modules like Bug, Tool, and Reports.

7.4.5 Tools Collection Schema

Field Name	Data Type	Constraints	Description
_id	ObjectId	PRIMARY KEY	Log entry identifier
type	String	REQUIRED	Log type / category
bugId	ObjectId	OPTIONAL, FK	Associated bug reference
toolId	ObjectId	OPTIONAL, FK	Associated tool reference
performedBy	ObjectId	REQUIRED, FK	User who performed the action
action	String	OPTIONAL	Action performed
details	String	OPTIONAL	Detailed description
timestamp	Date	AUTO	Action timestamp

Figure 7.4.5. (Tool Collection schema ~ MongoDB)

Logs Collection (Schema Description)

The **Logs Collection** stores all activity records related to bugs, tools, and user actions within the Bug Tracker System. It helps in tracking system operations and maintaining a history of all changes.

1. Overview

Each log entry represents an action performed in the system, such as bug status updates, assignments, or modifications. Logs are essential for **audit tracking, debugging, and monitoring system activity**.

2. Field Descriptions

- **_id (ObjectId, Primary Key):** Unique identifier for each log entry.
- **type (String, Required):** Specifies the category of the log (e.g., bug activity, tool update).
- **bugId (ObjectId, FK, Optional):** References the bug associated with the log.
- **toolId (ObjectId, FK, Optional):** References the tool associated with the log.
- **performedBy (ObjectId, FK, Optional):** Identifies the user who performed the action.
- **action (String, Optional):** Describes the action performed (e.g., "status changed", "assigned to developer").
- **details (String, Optional):** Additional information about the action.
- **timestamp (Date, Auto):** Automatically records when the log entry was created.

3. Key Features of Design

- **Activity Tracking:** Records all system actions for better traceability.
- **Flexible References:** Logs can be linked to bugs, tools, or users.
- **Audit Support:** Helps in monitoring system changes and debugging issues.

- **Automatic Timestamping:** Ensures accurate tracking of events.

7.5 OBJECT CLASS DIAGRAM

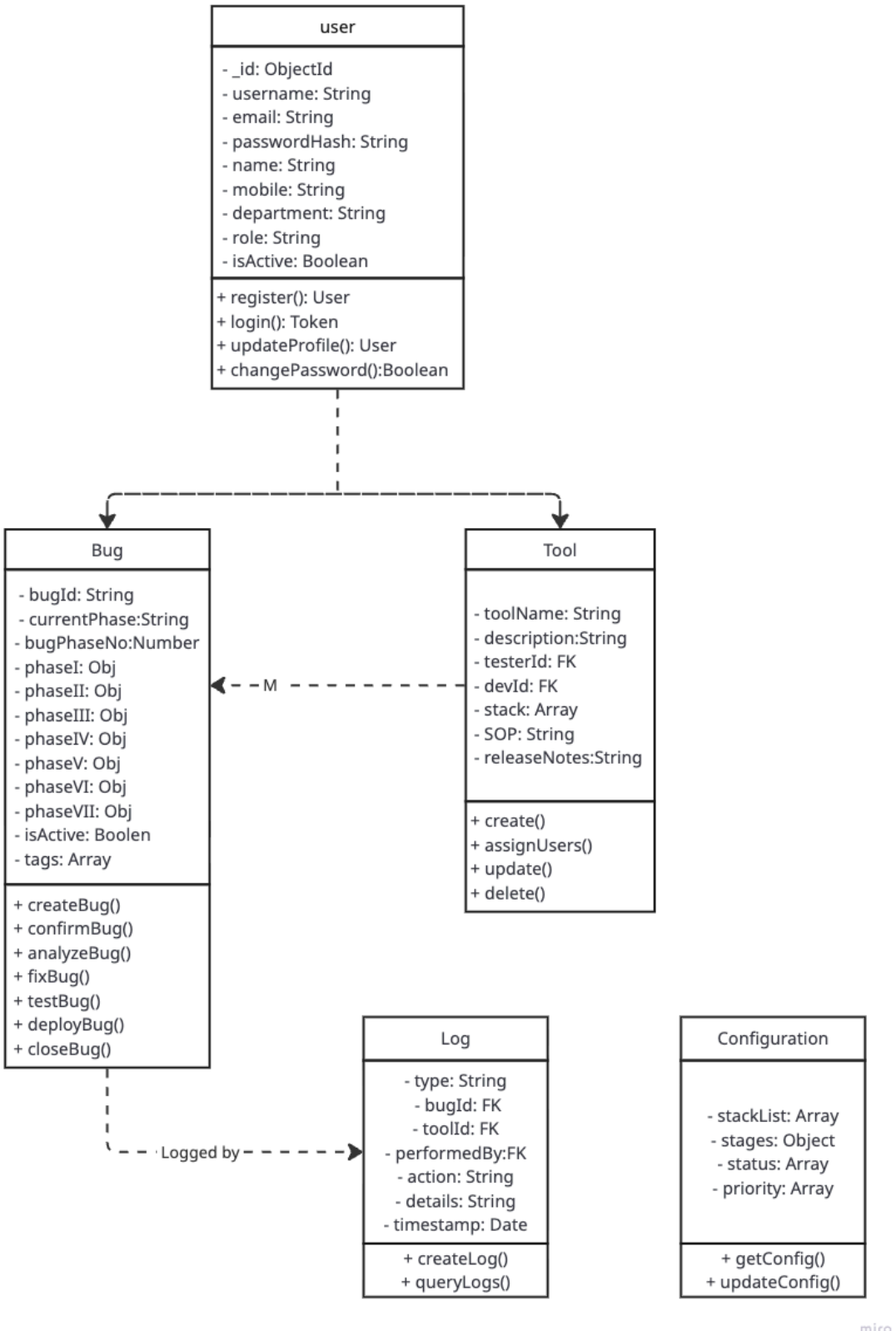


Figure 7.5. (OBJECT CLASS DIAGRAM)

Class Relationships:

1. User ↔ Bug: One-to-Many (User reports many Bugs); Many-to-One (Bug assigned to User as Tester/Developer)
2. User ↔ Tool: One-to-Many (User assigned to many Tools); Many-to-One (Tool has assigned Tester/Developer)
3. Bug ↔ Tool: Many-to-One (Many Bugs belong to one Tool)
4. User ↔ Log: One-to-Many (User performs many actions logged)
5. Bug ↔ Log: One-to-Many (Bug has many log entries)
6. Tool ↔ Log: One-to-Many (Tool has many log entries)

7.6. GANTT CHART & PERT CHART

7.6.1 Gantt Chart - Project Timeline

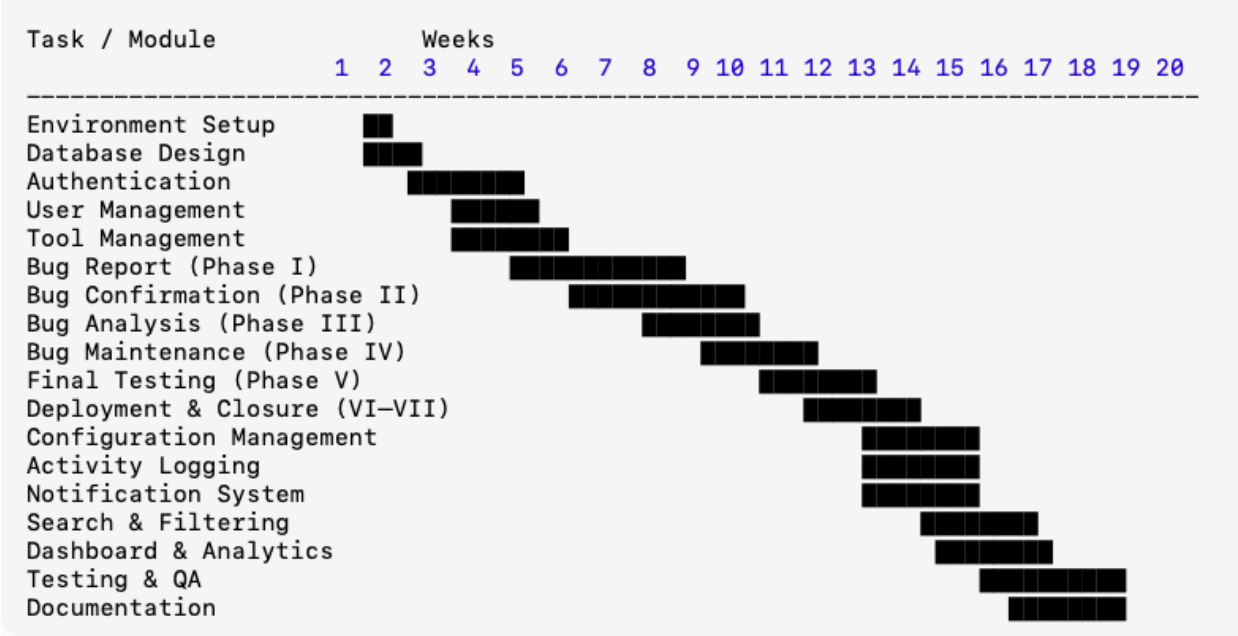


Figure 9.1 (Gantt Chart)

Milestones:

- ▼ Week 3: Authentication & User Management Complete
- ▼ Week 9: Core Bug Reporting & Confirmation Working
- ▼ Week 14: Complete Seven-Phase Workflow Implemented
- ▼ Week 17: Admin Features & Analytics Complete
- ▼ Week 20: Production Deployment & Project Completion

7.6.2 PERT Chart - Critical Path Analysis

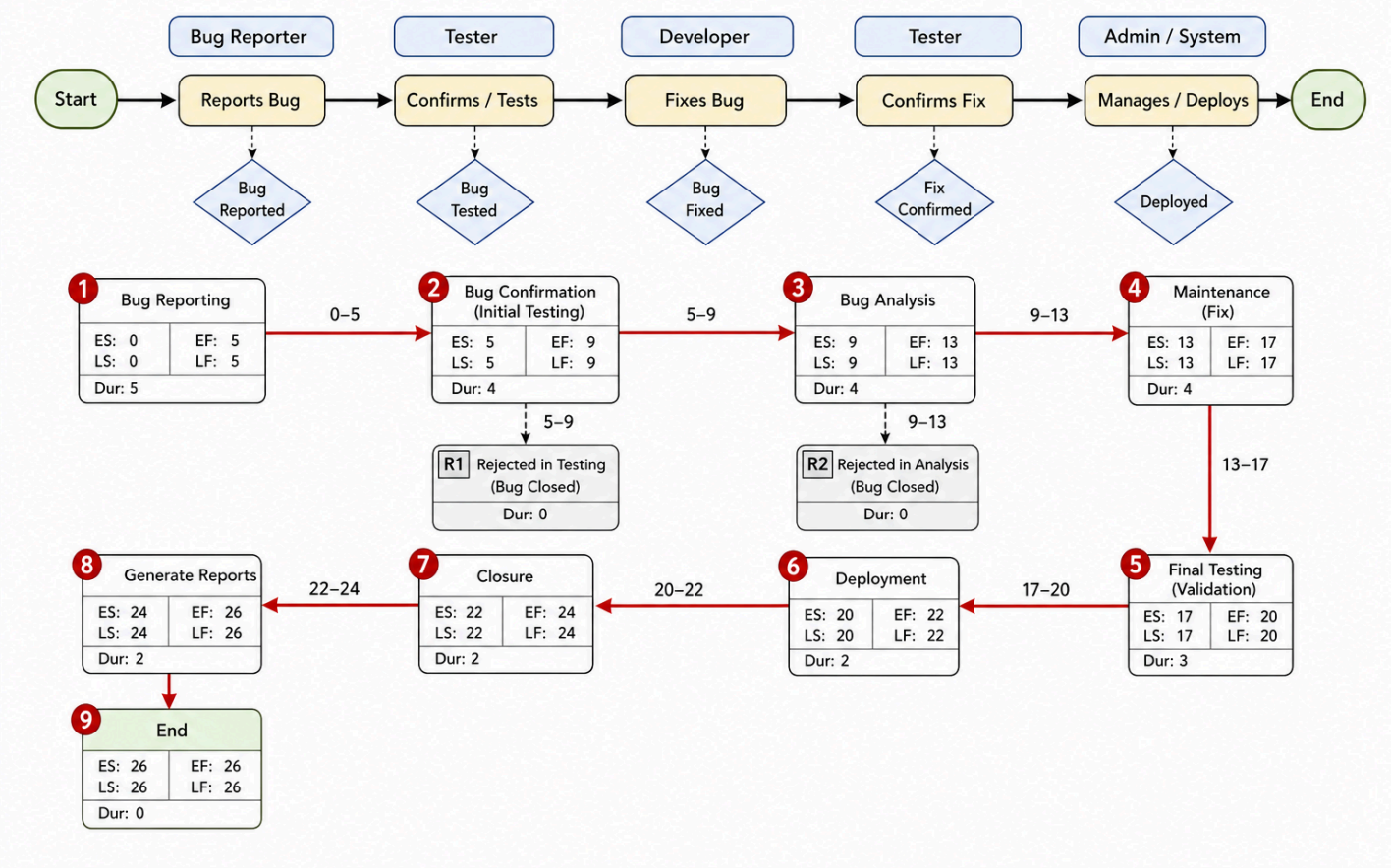


Figure 9.1（PERT Chart）

Critical Path

The **critical path** is the longest sequence of activities that determines the total project duration

: **1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9**

- Total Duration: 20 weeks

8. MODULE DESCRIPTIONS

1. Authentication & Security Module

This module ensures secure access to the system and manages user identities and permissions.

Key Features:
User Registration & Login: Secure signup and sign-in using hashed passwords (bcrypt).
JWT-Based Authentication: Stateless session management using JSON Web Tokens.
Role-Based Access Control (RBAC): Restricting access to specific features based on user roles (bugreporter, tester, dev, admin).
Password Reset Flow: OTP-based password recovery and secure reset mechanisms.

2. User Management

User profiles, role management, activation/deactivation, and user listing with search.

3. Tool Management

Tool/application management, tester–developer assignment, tech stack, SOP, and release notes

3. Bug Lifecycle Management Module

The core engine of the system, managing the transition of bugs through seven distinct phases.

Phases Covered:
Phase I (Reporting): Capturing technical context, priority, and reproduction steps.
Phase II (Confirmation): Validation by testers to confirm the issue exists.
Phase III (Analysis): Identifying root causes and estimating fix effort.
Phase IV (Maintenance): Implementing the fix and documenting code changes.
Phase V (Testing): Final validation and regression testing of the fix.
Phase VI (Deployment): Managing the release to staging or production environments.
Phase VII (Closure): Formal closure with lessons learned and resolution summaries.

4. Tool & Tech Stack Configuration Module

Manages the inventory of software tools and technologies being tracked.

Key Features:
Tool Inventory: Creating and managing records for different software products.
Stack Management: Defining technology stacks (e.g., Node.js, React, Python) associated with tools.
Resource Mapping: Assigning default developers and testers to specific tools for automated routing.
Standard Operating Procedures (SOPs): Linking technical documentation to specific tools.

5. Activity Logging & Audit Trail Module

Provides a forensic record of all actions performed within the system.

Key Features:
Audit Logs: Automatically capturing who made what change and when (e.g., status changes, comment additions).
Change Tracking: Maintaining a history of bug updates for accountability.
Log Retrieval: Admin interface to view and filter system logs for troubleshooting.

6. Notification & Alert Module
Keeps stakeholders informed about critical updates and status changes.

Key Features:
In-App Notifications: Real-time alerts for bug assignments or phase updates.
Role-Specific Alerts: Ensuring developers are notified of new fixes and testers of new confirmations.
Read/Unread Tracking: Allowing users to manage their notification queue.

7. Analytical Dashboard & Visualization Module
Transforms raw data into actionable insights through visual representations.

Key Features:
Project Health Metrics: High-level view of active vs. closed bugs.
Priority Distribution: Visualizing bugs by severity (Critical, High, Medium, Low).
Phase Progress: Tracking the distribution of bugs across the 7-phase lifecycle.
Interactive Charts: Using Recharts to provide dynamic data visualization.

8. Search & Filtering
Advanced search, multi-criteria filtering, sorting, and pagination.

9. Dashboard & Analytics
Role-based dashboards, statistics, charts, and activity indicators.

10. Admin Panel
Administrative controls, system monitoring, configuration UI, and logs.

11. Testing & QA
Unit, integration, system testing, bug fixes, and performance optimization.

9. Folder Structure & CODING

Folder Structure :

```
Bug_Bunters/
├── backend/
│   ├── src/
│   │   ├── controllers/          # Controller layer for handling API logic
│   │   │   ├── auth.controller.ts
│   │   │   ├── authExtend.controller.ts
│   │   │   ├── bug.controller.ts
│   │   │   ├── tool.controller.ts
│   │   │   ├── credential.controller.ts
│   │   │   ├── logs.controller.ts
│   │   │   ├── resetpass.controller.ts
│   │   │   ├── stack.controller.ts
│   │   │   ├── user.controller.ts
│   │   │   └── notification.controller.ts
│   │   ├── models/               # Mongoose schemas for MongoDB
│   │   │   ├── bug.models.ts
│   │   │   ├── user.model.ts
│   │   │   ├── tool.models.ts
│   │   │   ├── logs.models.ts
│   │   │   └── notification.models.ts
│   │   ├── routes/               # Express route definitions
│   │   │   ├── authExtend.routes.ts
│   │   │   ├── bug.routes.ts
│   │   │   ├── credential.routes.ts
│   │   │   ├── auth.routes.ts
│   │   │   ├── tool.routes.ts
│   │   │   └── index.ts
│   │   │   └── notification.routes.ts
│   │   │   └── user.routes.ts
│   │   │   └── resetpass.routes.ts
│   │   ├── middlewares/          # Security and processing middlewares
│   │   │   ├── authMiddleware.ts
│   │   │   ├── rateLimiter.ts
│   │   │   └── multer.ts
│   │   ├── utils/                # Shared utility functions
│   │   │   ├── mail.ts
│   │   │   ├── cloudinary.ts
│   │   │   └── asyncHandler.ts
│   │   │   └── bugActivity.ts
│   │   │   └── notification.ts
│   │   │   └── cron.ts
│   │   │   └── errorcodes.ts
│   │   │   └── emailTemplates.ts
│   │   │   └── errorHandler.ts
│   │   │   └── logger.ts
```

```
│   │   ├── tests/              # Automated test suites (Jest)
│   │   │   ├── unit/
│   │   │   └── integration/
│   │   ├── app.ts              # App configuration
│   │   └── server.ts           # Server entry point
│   ├── logs/                   # Error and access logs
│   ├── package.json            # Project dependencies
│   └── tsconfig.json           # TypeScript config
│
│
├── frontend/
│   ├── src/
│   │   ├── API_Call/           # Service layer for API interactions
│   │   │   ├── Auth.js
│   │   │   ├── Bug.js
│   │   │   ├── Tool.js
│   │   │   ├── Config.js
│   │   │   ├── Notification.js
│   │   │   └── User.js
│   │   ├── components/         # Reusable React components
│   │   │   ├── Dashboard/      # Dashboard-specific widgets
│   │   │   ├── Authorization/  # Login and Auth forms
│   │   │   ├── Phases/         # Bug phase components
│   │   │   ├── My_Account/
│   │   │   ├── Settings/
│   │   │   ├── Dropdowns/
│   │   │   ├── Collapsible/
│   │   │   └── ui/             # Shadcn-like base UI components
│   │   ├── context/            # Global state (Context API)
│   │   │   ├── AuthContext.jsx
│   │   │   └── NotificationContext.jsx
│   │   ├── hooks/              # Custom React hooks
│   │   ├── pages/              # Main view components
│   │   │   ├── Dashboard.jsx
│   │   │   ├── BugsPage.jsx
│   │   │   ├── Account.jsx
│   │   │   ├── Settings.jsx
│   │   │   └── BugActivityLog.jsx
│   │   ├── assets/             # Static media and icons
│   │   ├── lib/                # Library configurations
│   │   ├── App.jsx             # Routing and main layout
│   │   └── main.jsx            # DOM entry point
│   ├── public/                 # Static public assets
│   ├── vite.config.js          # Vite build tool config
│   ├── tailwind.config.js      # Tailwind CSS config
│   └── package.json            # Frontend dependencies
└── README.md                   # Project documentation
```

Frontend (Folder)

Index.html

```
<!doctype html>
<html lang="en">
<head>
 <meta charset="UTF-8" />
 <link rel="icon" type="image/svg+xml" href="data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg'
viewBox='0 0 24 24' fill='%230ea5e9'%3E%3Cpath d='M19 8h-1.81a5.985 5.985 0 0 0-1.82-1.96L17 4.41 15.59
3l-2.17 2.17a6.002 6.002 0 0 0-2.83 0L8.41 3 7 4.41l1.62 1.63C7.88 6.55 7.26 7.22 6.81
8H5v2h1.09c-.05.33-.09.66-.09 1v1H5v2h1v1c0 .34.04.67.09 1H5v2h1.81c1.04 1.79 2.97 3 5.19 3s4.15-1.21
5.19-3H19v-2h-1.09c.05-.33.09-.66.09-1v-1h1v-2h-1v-1c0-.34-.04-.67-.09-1H19V8zM13
16h-2v-2h2v2zm0-4h-2v-2h2v2z'/%3E%3C/svg%3E" />
 <meta name="viewport" content="width=device-width, initial-scale=1.0" />
 <title>Bug Tracker</title>
 <link rel="preconnect" href="https://fonts.googleapis.com" />
 <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />
<link href="https://fonts.googleapis.com/css2?family=Plus+Jakarta+Sans:ital,
       wght@0,300;0,400;0,500;0,600;0,700;0,800;1,400&display=swap" rel="stylesheet" />
</head>
<body>
 <div id="root"></div>
 <script type="module" src="/src/main.jsx"></script>
</body>
</html>
```

eslint.config.js

```
import js from '@eslint/js'
import globals from 'globals'
import reactHooks from 'eslint-plugin-react-hooks'
import reactRefresh from 'eslint-plugin-react-refresh'
import { defineConfig, globalIgnores } from 'eslint/config'
export default defineConfig([
 globalIgnores(['dist']),
 {
   files: ['**/*.{js,jsx}'],
   extends: [
     js.configs.recommended, reactHooks.configs['recommended-latest'], reactRefresh.configs.vite,
   ],
   languageOptions: {
     ecmaVersion: 2020, globals: globals.browser,
     parserOptions: { ecmaVersion: 'latest', ecmaFeatures: { jsx: true }, sourceType: 'module',},
   },
   rules: { 'no-unused-vars': ['error', { varsIgnorePattern: '^[A-Z_]' }],},
 },
])
```

components.json

```
{
 "$schema": "https://ui.shadcn.com/schema.json",
 "style": "new-york",
 "rsc": false,
 "tsx": false,
```

```
 "tailwind": { "config": "tailwind.config.js", "css": "src/index.css", "baseColor": "zinc",
   "cssVariables": true, "prefix": ""
 },
 "iconLibrary": "lucide",
 "aliases": { "components": "@/components", "utils": "@/lib/utils", "ui": "@/components/ui",
   "lib": "@/lib", "hooks": "@/hooks"
 },
 "registries": {}
}
```

package.json

```
{
 "name": "frontend",
 "private": true,
 "version": "0.0.0",
 "type": "module",
 "scripts": { "dev": "vite", "build": "vite build",  "lint": "eslint .", "preview": "vite preview" },
 "dependencies": {
   "@radix-ui/react-accordion": "^1.2.12",  "@radix-ui/react-avatar": "^1.1.11",
   "@radix-ui/react-checkbox": "^1.3.3", "@radix-ui/react-collapsible": "^1.1.12",
   "@radix-ui/react-context-menu": "^2.2.16",  "@radix-ui/react-dialog": "^1.1.15",
   "@radix-ui/react-dropdown-menu": "^2.1.16", "@radix-ui/react-label": "^2.1.8",
   "@radix-ui/react-popover": "^1.1.15", "@radix-ui/react-radio-group": "^1.3.8",
   "@radix-ui/react-select": "^2.2.6", "@radix-ui/react-separator": "^1.1.8",
   "@radix-ui/react-slot": "^1.2.4", "@radix-ui/react-tooltip": "^1.2.8",
   "axios": "^1.13.2", "class-variance-authority": "^0.7.1",
   "clsx": "^2.1.1", "cmdk": "^1.1.1",
   "cookie-parser": "^1.4.7",  "date-fns": "^4.1.0",
   "lucide-react": "^0.553.0", "motion": "^12.23.24",
   "next": "16.0.2", "react": "^19.2.0",
   "react-dom": "^19.2.0",  "react-hot-toast": "^2.4.1",
   "react-icons": "^4.12.0", "react-router-dom": "^7.9.6",
   "recharts": "^3.8.1", "shadcn": "3.5.0",
   "tailwind-merge": "^3.4.0", "tailwindcss-animate": "^1.0.7"
 },
 "devDependencies": {
   "@eslint/js": "^9.39.1", "@types/node": "24.10.1",
   "@types/react": "^19.2.2", "@types/react-dom": "^19.2.2",
   "@vitejs/plugin-react": "^5.1.0",  "autoprefixer": "^10.4.16",
   "eslint": "^9.39.1",  "eslint-plugin-react-hooks": "^5.2.0",
   "eslint-plugin-react-refresh": "^0.4.24", "globals": "^16.5.0",
   "postcss": "^8.4.32", "tailwindcss": "^3.4.17",
   "typescript": "5.9.3", "vite": "^7.2.2"
 }
}
```

postcss.config.js

```
export default { plugins: {tailwindcss: {}, autoprefixer: {},},}
```

tailwind.config.js

```
/** @type {import('tailwindcss').Config} */
export default {
```

```
darkMode: ["class"],
content: ["./index.html", "./src/**/*.{js,ts,jsx,tsx}", "*.{js,ts,jsx,tsx,mdx}"],
theme: {
  extend: {
    colors: {
      border: 'hsl(var(--border))',input: 'hsl(var(--input))',  ring: 'hsl(var(--ring))',
      background: 'hsl(var(--background))', foreground: 'hsl(var(--foreground))',
      primary: { DEFAULT: 'hsl(var(--primary))', foreground: 'hsl(var(--primary-foreground))' },
      secondary: { DEFAULT: 'hsl(var(--secondary))',  foreground: 'hsl(var(--secondary-foreground))'  },
      destructive: {DEFAULT: 'hsl(var(--destructive))',foreground: 'hsl(var(--destructive-foreground))'},
      muted: { DEFAULT: 'hsl(var(--muted))', foreground: 'hsl(var(--muted-foreground))'  },
      accent: { DEFAULT: 'hsl(var(--accent))', foreground: 'hsl(var(--accent-foreground))'  },
      popover: {  DEFAULT: 'hsl(var(--popover))',foreground: 'hsl(var(--popover-foreground))'},
      card: { DEFAULT: 'hsl(var(--card))',foreground: 'hsl(var(--card-foreground))' },
      chart: {'1': 'hsl(var(--chart-1))', '2': 'hsl(var(--chart-2))', '3': 'hsl(var(--chart-3))',
        '4': 'hsl(var(--chart-4))', '5': 'hsl(var(--chart-5))'
      },
      sidebar: { DEFAULT: 'hsl(var(--sidebar-background))', foreground: 'hsl(var(--sidebar-foreground))',
        primary: 'hsl(var(--sidebar-primary))',
        'primary-foreground': 'hsl(var(--sidebar-primary-foreground))',accent: 'hsl(var(--sidebar-accent))',
        'accent-foreground': 'hsl(var(--sidebar-accent-foreground))',
        border: 'hsl(var(--sidebar-border))', ring: 'hsl(var(--sidebar-ring))'
      }
    },
    borderRadius: { lg: 'var(--radius)', md: 'calc(var(--radius) - 2px)',sm: 'calc(var(--radius) - 4px)' },
    keyframes: { slideInRight: {'0%': {  transform: 'translateX(100%)',  opacity: '0'},
        '100%': {  transform: 'translateX(0)',  opacity: '1'}
      },
      slideInLeft: {'0%': {  transform: 'translateX(-100%)',  opacity: '0'},
        '100%': {   transform: 'translateX(0)',   opacity: '1' }
      },
      slideOutLeft: {'0%': {  transform: 'translateX(0)',    opacity: '1'},
        '100%': { transform: 'translateX(-100%)', opacity: '0' }
      },
      slideOutRight: { '0%': {  transform: 'translateX(0)',   opacity: '1' },
        '100%': { transform: 'translateX(100%)', opacity: '0' }},
      fadeIn: { '0%': {opacity: '0'},'100%': {  opacity: '1'} },
      'accordion-down': { from: {   height: '0' }, to: {  height: 'var(--radix-accordion-content-height)' }
      },
      'accordion-up': {  from: {  height: 'var(--radix-accordion-content-height)'},to: {  height: '0'}}
    },
    animation: {
      'slide-in-right': 'slideInRight 0.5s ease-out', 'slide-in-left': 'slideInLeft 0.5s ease-out',
      'slide-out-left': 'slideOutLeft 0.5s ease-out', 'slide-out-right': 'slideOutRight 0.5s ease-out',
      'fade-in': 'fadeIn 0.3s ease-out', 'accordion-down': 'accordion-down 0.2s ease-out',
      'accordion-up': 'accordion-up 0.2s ease-out'
    }
  }
},
plugins: [require("tailwindcss-animate")],
}
```

vite.config.js

```
import { defineConfig } from 'vite'
```

```
import react from '@vitejs/plugin-react'
import path from 'path'
// https://vite.dev/config/
export default defineConfig({
 plugins: [react()], resolve: {alias: {"@": path.resolve(__dirname, "src"),},},
})
```

/src (folder)

main.jsx

```
import { createRoot } from "react-dom/client";
import { Toaster } from "react-hot-toast";
import "./index.css";
import App from "./App.jsx";
createRoot(document.getElementById("root")).render(
 <><Toaster position="top-right" /><App /></>
);
```

Index.css

```
/* Added CSS variables for shadcn/ui theming and custom animations */
@tailwind base;
@tailwind components;
@tailwind utilities;
@layer base {
 :root {
   --background: 0 0% 100%;--foreground: 240 10% 3.9%;  --card: 0 0% 100%; --card-foreground: 240 10% 3.9%;
   --popover: 0 0% 100%; --popover-foreground: 240 10% 3.9%; --muted: 240 4.8% 95.9%;
   --muted-foreground: 240 3.8% 46.1%; --accent: 240 4.8% 95.9%; --accent-foreground: 240 5.9% 10%;
   --destructive: 0 84.2% 60.2%; --destructive-foreground: 0 0% 98%; --border: 240 5.9% 90%;
   --input: 240 5.9% 90%; --primary: 240 5.9% 10%;  --primary-foreground: 0 0% 98%;
   --ring: 240 10% 3.9%;--radius: 0.5rem;--secondary: 240 4.8% 95.9%; --secondary-foreground: 240 5.9% 10%;
   --chart-1: 12 76% 61%; --chart-2: 173 58% 39%;  --chart-3: 197 37% 24%;  --chart-4: 43 74% 66%;
   --chart-5: 27 87% 67%; /* ── Sidebar: dark teal/navy theme ── */  --sidebar-background: 200 60% 10%;
   /* deep navy-teal  #0a2233 */ --sidebar-foreground: 195 30% 90%;
   /* soft white-teal         */ --sidebar-primary: 187 80% 50%;
   /* bright cyan             */ --sidebar-primary-foreground: 0 0% 100%; --sidebar-accent: 195 40% 18%;
   /* hover / active item     */  --sidebar-accent-foreground: 187 80% 70%;
   /* cyan text on hover       */--sidebar-border: 200 30% 18%;
   /* subtle border           */--sidebar-ring: 187 80% 50%;
 }

 .dark {
   --background: 240 10% 3.9%; --foreground: 0 0% 98%; --card: 240 10% 3.9%; --card-foreground: 0 0% 98%;
   --popover: 240 10% 3.9%; --popover-foreground: 0 0% 98%; --muted: 240 3.7% 15.9%;
   --muted-foreground: 240 5% 64.9%;  --accent: 240 3.7% 15.9%; --accent-foreground: 0 0% 98%;
   --destructive: 0 62.8% 30.6%; --destructive-foreground: 0 0% 98%;  --border: 240 3.7% 15.9%;
   --input: 240 3.7% 15.9%;  --primary: 0 0% 98%; --primary-foreground: 240 5.9% 10%; --ring: 240 4.9% 83.9%;
   --secondary: 240 3.7% 15.9%; --secondary-foreground: 0 0% 98%;--chart-1: 220 70% 50%; --chart-2: 160 60%
45%;
   --chart-3: 30 80% 55%; --chart-4: 280 65% 60%; --chart-5: 340 75% 55%; --sidebar-background: 240 5.9% 10%;
   --sidebar-foreground: 240 4.8% 95.9%; --sidebar-primary: 224.3 76.3% 48%;
   --sidebar-primary-foreground: 0 0% 100%; --sidebar-accent: 240 3.7% 15.9%;
   --sidebar-accent-foreground: 240 4.8% 95.9%; --sidebar-border: 240 3.7% 15.9%;
   --sidebar-ring: 217.2 91.2% 59.8%;
```

```
 }
}

@layer base {
 * {  @apply border-border;}
 body {   @apply bg-background text-foreground; }
}
* { margin: 0; padding: 0; box-sizing: border-box;}
html,body {height: 100%; width: 100%;}
#root {height: 100vh;width: 100%;}
body {
 font-family: 'Plus Jakarta Sans', system-ui, sans-serif; -webkit-font-smoothing: antialiased;
 -moz-osx-font-smoothing: grayscale;
}
h1,h2,h3,h4,h5,h6 { font-family: 'Plus Jakarta Sans', system-ui, sans-serif;letter-spacing: -0.02em;}
/* Custom scrollbar */
::-webkit-scrollbar { width: 4px; height: 8px;}
::-webkit-scrollbar-track { background: #f1f1f1; height: 99%; margin: 8px; border-radius: 22px;}
::-webkit-scrollbar-thumb { background: #0891b2; border-radius: 4px;}
::-webkit-scrollbar-thumb:hover { background: #0e7490;}
/* ═══════════════════════════════════════════
  Sidebar dark teal override
  ═══════════════════════════════════════════ */
/* Main sidebar panel */
[data-sidebar="sidebar"] {
 background: linear-gradient(180deg, #0a2233 0%, #0c2d42 60%, #0a2233 100%) !important;
 border-right: 1px solid rgba(6, 182, 212, 0.12) !important;
}
/* Sidebar header (TeamSwitcher area) */
[data-sidebar="sidebar"] [data-sidebar="header"] {
 border-bottom: 1px solid rgba(6, 182, 212, 0.1);padding-bottom: 0.5rem;
}
/* Sidebar footer (NavUser area) */
[data-sidebar="sidebar"] [data-sidebar="footer"] { border-top: 1px solid rgba(6, 182, 212, 0.1);}
/* All sidebar text */
[data-sidebar="sidebar"],[data-sidebar="sidebar"] span,[data-sidebar="sidebar"] a {
 color: rgba(200, 235, 245, 0.85);
}
[data-sidebar="sidebar"] [data-sidebar="menu-button"]:hover {
 background: rgba(6, 182, 212, 0.1) !important; color: #67e8f9 !important;
}
/* Active / selected state */
[data-sidebar="sidebar"] [data-sidebar="menu-button"][data-active="true"] {
 background: rgba(6, 182, 212, 0.18) !important;color: #67e8f9 !important;
}
/* Section group labels */
[data-sidebar="sidebar"] [data-sidebar="group-label"] {
 color: rgba(103, 232, 249, 0.45) !important;font-size: 10px; letter-spacing: 0.1em; font-weight: 700;
}
/* Subtle separators */
[data-sidebar="sidebar"] [data-sidebar="separator"] { background: rgba(6, 182, 212, 0.12) !important;}
/* TeamSwitcher button */
[data-sidebar="sidebar"] [data-sidebar="menu-button"][size="lg"] {
```

```
 background: rgba(6, 182, 212, 0.08) !important; border-radius: 12px;
}
/* User footer button */
[data-sidebar="sidebar"] [data-sidebar="footer"] [data-sidebar="menu-button"] {
 background: rgba(6, 182, 212, 0.06) !important; border-radius: 10px;
}
```

App.jsx

```
import { BrowserRouter, Routes, Route, Navigate } from "react-router-dom";
import { AuthProvider } from "@/context/AuthContext";
import AuthContainer from "./components/Authorization/AuthContainer";
import ProtectedRoute from "./components/ProtectedRoute";
import DashboardLayout from "./components/Dashboard/DashboardLayout";
import Dashboard from "./pages/Dashboard";
import BugsPage from "./pages/BugsPage";
import Settings from "./pages/Settings";
import "./App.css";
import Account from "./pages/Account";
import BugActivityLog from "./pages/BugActivityLog";
import { TooltipProvider } from "@/components/ui/tooltip";
function App() {
 return (
   <AuthProvider>
     <TooltipProvider>
       <BrowserRouter>
       <Routes>
         <Route path="/login" element={<AuthContainer />} />
         <Routepath="/" element={ <ProtectedRoute> <DashboardLayout /></ProtectedRoute>}  >
           <Route index element={<Dashboard />} />
           <Route path="bugs" element={<BugsPage />} />
           <Route path="bugs/:id/activity" element={<BugActivityLog />} />
           <Route path="account" element={<Account />} />
           <Route path="settings/:tab?" element={<Settings />} />
         </Route>
         <Route path="*" element={<Navigate to="/login" replace />} />
       </Routes>
     </BrowserRouter>
     </TooltipProvider>
   </AuthProvider>
 );
}
export default App;
```

App.css

```
/* Updated with new authentication UI styles */
#root {
 height: 100vh;
 width: 100%;
}
/* Decorative shapes */
.shape-top-left {
       position: fixed; top: -50px; left: -50px; width: 200px; height: 200px; opacity: 0.1;z-index: 0;
}
```

```
.shape-bottom-right {
 position: fixed; bottom: -50px; right: -50px;width: 200px; height: 200px; opacity: 0.1; z-index: 0;
}
/* Input focus states */
input:focus { outline: none;}
/* Smooth transitions for forms */
.form-transition { transition: all 0.5s cubic-bezier(0.4, 0, 0.2, 1);}
.fade-in {
 opacity: 1; animation-name: fadeInOpacity; animation-iteration-count: 1;
 animation-timing-function: ease-in; animation-duration: 1.5s;
  transition: all 0.5s cubic-bezier(0.4, 0, 0.2, 1);
}
@keyframes fadeInOpacity { 0% {   opacity: 0; } 100% {   opacity: 1; } }
@keyframes slide-in {
 from { transform: translateX(-100%); } to { transform: translateX(0); }
}
.animate-slide-in { animation: slide-in 0.25s ease-out; }
```

API_Call (folder)

API.js

```
import axios from "axios";
// ("process.env.REACT_APP_API_BASE_URL");
const API_BASE_URL = "http://localhost:8000/api";
const API_TIMEOUT = 10000; // 10 seconds
const API_HEADERS = { "Content-Type": "application/json", };
// Create an Axios instance with default configurations
const api = axios.create({
 baseURL: API_BASE_URL,timeout: API_TIMEOUT,headers: API_HEADERS,
 withCredentials: true, // Add this line to send credentials with every request
});
export default api;
```

Auth.js

```
import api from "./API";
// REGISTER — user creation
// REGISTER — public user self-creation

export const register = async (userData) => {
 try {
   const res = await api.post("/auth/register", userData);
   return { success: true, message: res.data.message, user: res.data.user, };
 } catch (error) {
   return { success: false,
     message: error.response?.data?.message || "Something went wrong during registration",
   };
 }
};

// REGISTER BY ADMIN — protected admin-side user creation
```

```
export const registerByAdmin = async (userData) => {
 try {
   const res = await api.post("/authextend/register", userData);
   return { success: true, message: res.data.message, user: res.data.user, };
 } catch (error) {
   return {
     success: false, message: error.response?.data?.message || "Something went wrong during user creation",
   };
 }
};

// CHECK IF USER LOGGED IN
export const checkAuth = async () => {
 try {
   const res = await api.get("/authextend/me", {withCredentials: true, });
   console.log("Auth check response:", res);
   return res.data.user; // authenticated user
 } catch (error) {
   return null; // not authenticated
 }
};
// LOGIN — cookie automatically stored
export const login = async (credentials) => {
 try {const res = await api.post("/auth/login", credentials, { withCredentials: true,});
   return { success: true, message: res.data.message, user: res.data.user,};
 } catch (error) {
   return {success: false, message: error.response?.data?.message || "Something went wrong",};
 }
};

// PASSWORD RESET (OTP)
export const requestOtpReset = async (username_email) => {
 try {
   const res = await api.post("/reset/request", { username_email });
   return { success: true, message: res.data.message };
 } catch (error) {
   return { success: false,  message: error.response?.data?.message || "Failed to request OTP",
   };
 }
};
export const verifyOtpReset = async (data) => {
 try {
   const res = await api.post("/reset/verify", data);
   return { success: true, message: res.data.message, resetToken: res.data.resetToken };
 } catch (error) {
   return {success: false,message: error.response?.data?.message || "Invalid or expired OTP",};
 }
};
export const resetPassword = async (data) => {
 try {
   const res = await api.post("/reset/password", data);
   return { success: true, message: res.data.message };
 } catch (error) {
```

```
    return {success: false,message: error.response?.data?.message || "Failed to reset password",};
  }
};
// LOGOUT
export const logout = async () => {
  try {
    const res = await api.post(`/auth/logout`, {credentials: true,});
    return { success: true,message: res.data.message,};
  } catch (error) {
    return {success: false,message: error.response?.data?.message || "Something went wrong",};
  }
};
```

Bug.js

```
import api from "./API";
// BUG CRUD OPERATIONS
export const createBug = async (bugData) => {
  try {
    const res = await api.post("/bug/bug_create", bugData);
    return { success: true, data: res.data };
  } catch (error) {
    return { success: false, message: error.response?.data?.message || "Failed to create bug" };
  }
};
export const updateBug = async (bugData) => {
  try {
    const res = await api.post("/bug/update_Bug", bugData);
    return { success: true, data: res.data };
  } catch (error) {
    return { success: false, message: error.response?.data?.message || "Failed to update bug" };
  }
};
export const deleteBug = async (bugData) => {
  try {
    const res = await api.post("/bug/delete_Bug", bugData);
    return { success: true, data: res.data };
  } catch (error) {
    return { success: false, message: error.response?.data?.message || "Failed to delete bug" };
  }
};

export const getBugs = async (filters = {}) => {
  try {
    const res = await api.post("/bug/get_bugs", filters);
    return { success: true, data: res.data };
  } catch (error) {
    return { success: false, message: error.response?.data?.message || "Failed to get bugs" };
  }
};

export const getBugById = async (bugData) => {
```

```
 try {
   const res = await api.post("/bug/get_BugById", bugData);
   return { success: true, data: res.data };
 } catch (error) {
   return { success: false, message: error.response?.data?.message || "Failed to get bug by ID" };
 }
};
export const getBugStatistics = async (reqData = {}) => {
 try {
   const res = await api.post("/bug/get_BugStatistics", reqData);
   return { success: true, data: res.data };
 } catch (error) {
   return { success: false, message: error.response?.data?.message || "Failed to get bug statistics" };
 }
};
export const getBugLogs = async (bugData) => {
 try {
   const res = await api.post("/bug/get_BugLogs", bugData);
   return { success: true, data: res.data };
 } catch (error) {
   return { success: false, message: error.response?.data?.message || "Failed to get activity logs" };
 }
};
// BUG LIFECYCLE OPERATIONS
export const confirmBug = async (bugData) => {
 try {
   const res = await api.post("/bug/bug_confirm", bugData);
   return { success: true, data: res.data };
 } catch (error) {
   return { success: false, message: error.response?.data?.message || "Failed to confirm bug" };
 }
};

export const analyzeBug = async (bugData) => {
 try {
   const res = await api.post("/bug/bug_Analyze", bugData);
   return { success: true, data: res.data };
 } catch (error) {
   return { success: false, message: error.response?.data?.message || "Failed to analyze bug" };
 }
};
export const fixBug = async (bugData) => {
 try {
   const res = await api.post("/bug/bug_Fix", bugData);
   return { success: true, data: res.data };
 } catch (error) {
   return { success: false, message: error.response?.data?.message || "Failed to fix bug" };
 }
};
export const finalTestBug = async (bugData) => {
 try {
   const res = await api.post("/bug/bug_FinalTest", bugData);
   return { success: true, data: res.data };
```

```
 } catch (error) {
   return { success: false, message: error.response?.data?.message || "Failed to final test bug" };
 }
};
export const deployBug = async (bugData) => {
 try {
   const res = await api.post("/bug/bug_Deploy", bugData);
   return { success: true, data: res.data };
 } catch (error) {
   return { success: false, message: error.response?.data?.message || "Failed to deploy bug" };
 }
};
export const closeBug = async (bugData) => {
 try {
   const res = await api.post("/bug/bug_Close", bugData);
   return { success: true, data: res.data };
 } catch (error) {
   return { success: false, message: error.response?.data?.message || "Failed to close bug" };
 }
};

export const uploadFiles = async (files) => {
 try {
   const formData = new FormData();
   files.forEach((file) => { formData.append("files", file);});
   const res = await api.post("/bug/upload", formData, {
     headers: { "Content-Type": "multipart/form-data",},
   });
   return { success: true, data: res.data };
 } catch (error) {
   return { success: false, message: error.response?.data?.message || "Upload failed" };
 }
};
```

Config.js

```
import api from "./API";
/**
* Fetch the global system configuration, primarily containing the stack lists.
* @returns {Promise<{success: boolean, data: any} | {success: boolean, message: string}>}
*/
export const systemConfig = async () => {
 try {
   const res = await api.get("/stack/system_config");
   return { success: true, data: res.data,};
 } catch (error) {
   console.error("Fetch System Config Error:", error);
   return { success: false, message: error.response?.data?.message || "Failed to fetch system
configuration.",};
 }
};
```

Notification.js

```
import api from "./API";
```

```
export const getNotifications = async () => {
 try {
   const res = await api.get("/notifications", { withCredentials: true });
   return res.data;
 } catch (error) {
   console.error("Error fetching notifications:", error);
   return { notifications: [], unreadCount: 0 };
 }
};
export const markAsRead = async (id) => {
 try {
   const res = await api.patch(`/notifications/${id}/read`, {}, { withCredentials: true });return res.data;
 } catch (error) {
   console.error("Error marking notification as read:", error); return null;
 }
};
export const markAllAsRead = async () => {
 try {
   const res = await api.patch("/notifications/read-all", {}, { withCredentials: true }); return res.data;
 } catch (error) {
   console.error("Error marking all notifications as read:", error); return null;
 }
};
export const deleteNotification = async (id) => {
 try {
   const res = await api.delete(`/notifications/${id}`, { withCredentials: true }); return res.data;
 } catch (error) {
   console.error("Error deleting notification:", error);return null;
 }
};
```

Tool.js

```
import api from "./API";
/*** Fetch all tools natively from the backend.
* @returns {Promise<{success: boolean, data: any} | {success: boolean, message: string}>}
*/
export const getTools = async () => {
 try { const res = await api.get("/tools/getTools"); return { success: true, data: res.data };}
 catch (error) {
   console.error("Error fetching tools:", error);
   return { success: false, message: error.response?.data?.message || "Failed to fetch tools",};
 }
};
/*** Creates or Updates a tool payload leveraging the native batch operation processor logic.
* @param {Object} toolData
* @returns {Promise<{success: boolean, data: any} | {success: boolean, message: string}>}
*/
export const updateTool = async (toolData) => {
 try {
   const res = await api.post("/tools/update_tool", { tools: [toolData] });
   return { success: true, data: res.data };
 } catch (error) {
   return { success: false,  message: error.response?.data?.message || "Failed to update tool",};
 }
```

```
};
```

User.js

```
import api from "./API";
export const getAllUsers = async () => {
 try {
   const res = await api.get("/user/get_all_users"); return { success: true, data: res.data.data };
 } catch (error) {
   console.error("Error fetching users:", error);
   return {  success: false, message: error.response?.data?.message || "Failed to fetch users",};
 }
};
// authextend
export const updateUser = async (id, userData) => {
 try {
   const res = await api.put(`/user/update_user/${id}`, userData);
   return { success: true, data: res.data.data, message: res.data.message };
 } catch (error) {
   return { success: false,  message: error.response?.data?.message || "Failed to update user",
   };
 }
};
export const deleteUser = async (id) => {
 try {
   const res = await api.delete(`/user/delete_user/${id}`); return { success: true, message: res.data.message
};
 } catch (error) {
   console.error("Error deleting user:", error);
   return {  success: false, message: error.response?.data?.message || "Failed to delete user",};
 }
};

export const changePassword = async (id, { oldPassword, newPassword }) => {
 try {
   const res = await api.post(`/user/change_password/${id}`, { oldPassword, newPassword });
   return { success: true, message: res.data.message };
 } catch (error) {
   console.error("Error changing password:", error);
   return { success: false, message: error.response?.data?.message || "Failed to change password",};
 }
};
```

components (folder)

AuthContainer.css

```
/* Floating animated background blobs */
@keyframes blob-float {
 0%, 100% { transform: translate(0, 0) scale(1);}
 33% {   transform: translate(20px, -20px) scale(1.05); }
 66% {   transform: translate(-15px, 15px) scale(0.97); }
}
@keyframes fade-in-up {
 from {   opacity: 0;   transform: translateY(18px); }to {   opacity: 1;   transform: translateY(0); }
}
@keyframes slide-left-in {
```

```
 from {opacity: 0;transform: translateX(40px); } to {opacity: 1;transform: translateX(0); }
}
@keyframes shimmer-move { 0% {   background-position: -200% 0; } 100% {   background-position: 200% 0; }}
@keyframes pulse-ring {
 0% {   box-shadow: 0 0 0 0 rgba(6, 182, 212, 0.4); }
 70% {   box-shadow: 0 0 0 10px rgba(6, 182, 212, 0); }
 100% {   box-shadow: 0 0 0 0 rgba(6, 182, 212, 0); }
}
@keyframes spin-slow { from { transform: rotate(0deg);  to {   transform: rotate(360deg); } }
/* Fade-in animation for form content */
.fade-in { animation: fade-in-up 0.45s cubic-bezier(0.22, 1, 0.36, 1) both;}
.slide-left-in { animation: slide-left-in 0.4s cubic-bezier(0.22, 1, 0.36, 1) both;}
/* Background blob decorators */
.blob { position: absolute; border-radius: 50%; filter: blur(60px);
 animation: blob-float 8s ease-in-out infinite;
}
.blob-1 { width: 350px; height: 350px; background: rgba(6, 182, 212, 0.25); top: -80px;
 left: -80px; animation-delay: 0s;
}
.blob-2 { width: 280px; height: 280px; background: rgba(99, 102, 241, 0.2); bottom: -60px;
 right: -60px; animation-delay: -3s;
}
.blob-3 { width: 200px; height: 200px; background: rgba(6, 182, 212, 0.15); bottom: 100px;
 left: 60px; animation-delay: -5s;
}
/* Form field focus glow */
.input-group:focus-within { box-shadow: 0 0 0 3px rgba(6, 182, 212, 0.2); border-color: #06b6d4 !important;}
/* Smooth panel transitions */
.form-panel { will-change: transform, opacity; backface-visibility: hidden; transform: translateZ(0);
 transition: all 0.5s cubic-bezier(0.25, 0.46, 0.45, 0.94);
}
/* Primary action button */
.btn-primary-shine {
 background: linear-gradient(135deg, #164e63 0%, #0891b2 50%, #06b6d4 100%); background-size: 200% 200%;
 background-position: left center; letter-spacing: 0.02em;
 transition: background-position 0.5s ease, transform 0.15s ease, box-shadow 0.25s ease;
}
.btn-primary-shine:hover:not(:disabled) { background-position: right center; transform: translateY(-2px);
 box-shadow: 0 10px 30px rgba(8, 145, 178, 0.4), 0 2px 8px rgba(8, 145, 178, 0.2);
}
.btn-primary-shine:active:not(:disabled) {
 transform: translateY(0px);
 box-shadow: 0 4px 12px rgba(8, 145, 178, 0.3);
}
/* OTP digit boxes */
.otp-box { width: 48px; height: 56px; border: 2px solid #e5e7eb; border-radius: 12px; text-align: center;
 font-size: 1.25rem; font-weight: 700; transition: border-color 0.2s ease, box-shadow 0.2s ease;
 outline: none; background: #f9fafb; color: #111827;
}
.otp-box:focus { border-color: #06b6d4; box-shadow: 0 0 0 3px rgba(6, 182, 212, 0.2); background: #fff;}
.otp-box.filled { border-color: #06b6d4; background: #ecfeff;}
/* Social button hover lift */
.social-btn { transition: transform 0.2s ease, box-shadow 0.2s ease, background 0.2s ease;}
```

```
.social-btn:hover { transform: translateY(-3px); box-shadow: 0 8px 20px rgba(0, 0, 0, 0.12);}
/* Mode toggle tabs */
.mode-tab { position: relative; transition: color 0.2s ease;}
.mode-tab.active::after { content: ''; position: absolute; bottom: -2px; left: 0; right: 0;
 height: 2px; background: #06b6d4; border-radius: 2px; animation: fade-in-up 0.2s ease both;
}
/* Left panel decorative grid */
.auth-grid-bg {
 background-image: linear-gradient(rgba(255, 255, 255, 0.05) 1px, transparent 1px),
   linear-gradient(90deg, rgba(255, 255, 255, 0.05) 1px, transparent 1px);
 background-size: 30px 30px;
}
/* Feature badges on left panel */
.feature-badge { animation: fade-in-up 0.5s ease both;}
.feature-badge:nth-child(2) { animation-delay: 0.1s;}
.feature-badge:nth-child(3) { animation-delay: 0.2s;}
.feature-badge:nth-child(4) { animation-delay: 0.3s;}
/* Spinning decorative ring */
.spin-ring { animation: spin-slow 12s linear infinite;}
/* Pulse for security indicator */
.pulse-security { animation: pulse-ring 2s ease-in-out infinite;}
```

AuthContainer.jsx

```
"use client";
import { useState } from "react";
import { Bug, Rocket } from "lucide-react";
import Login from "./Login";
import Register from "./Register";
import PasswordReset from "./PasswordReset";
import "./AuthContainer.css";
function AuthContainer() {
 const [authMode, setAuthMode] = useState("login");
 const isRegister = authMode === "register";
 return (
   <div className="relative w-full h-screen overflow-hidden flex items-center justify-center p-4"
     style={{ background: "linear-gradient(135deg, #f0f9ff 0%, #e0f2fe 50%, #f8fafc 100%)" }}>
     {/* ── Subtle ambient blobs on light bg ── */}
     <div className="absolute top-[-80px] left-[-80px] w-72 h-72 rounded-full blur-3xl opacity-40"
       style={{ background: "radial-gradient(circle, #06b6d4 0%, transparent 70%)" }}/>
     <div className="absolute bottom-[-60px] right-[-60px] w-60 h-60 rounded-full blur-3xl opacity-30"
       style={{ background: "radial-gradient(circle, #818cf8 0%, transparent 70%)" }} />
     {/* ── Subtle grid overlay ── */}
     <div className="absolute inset-0 opacity-[0.03]"
       style={{ backgroundImage: "linear-gradient(#0ea5e9 1px, transparent 1px),
       linear-gradient(90deg, #0ea5e9 1px, transparent 1px)",backgroundSize: "40px 40px",
       }} />
     {/* ═══════ Main Card ═══════ */}
     <div className="relative w-full max-w-5xl rounded-3xl shadow-2xl overflow-hidden bg-white flex
       min-h-[620px] z-10"
       style={{ boxShadow: "0 25px 60px rgba(8, 145, 178, 0.12), 0 8px 20px rgba(0,0,0,0.06)" }}>
       {/* ═══ LEFT PANEL ═══ */}
       <div className="relative hidden lg:flex flex-col justify-between p-10 w-[46%] flex-shrink-0
                 overflow-hidden text-white"
         style={{ background: "linear-gradient(145deg, #0891b2 0%, #0e7490 45%, #164e63 100%)", }} >
```

```
        {/* Decorative rings */}
        <div className="absolute top-10 right-8 w-32 h-32 border-4 border-white/10 rounded-full spin-ring"
/>
        <div className="absolute top-16 right-14 w-20 h-20 border-2 border-white/5 rounded-full" />
        {/* Bottom glow */}
        <div className="absolute bottom-20 left-4 w-44 h-44 rounded-full blur-3xl"
          style={{ background: "rgba(6,182,212,0.2)" }} />
        {/* ── Brand ── */}
        <div className="relative z-10 fade-in">
          <div className="flex items-center gap-3 mb-6">
            <div className="w-10 h-10 bg-white/20 rounded-xl flex items-center justify-center">
              <Bug className="w-6 h-6 text-white" />
            </div>
            <div>
              <h2 className="text-lg font-bold tracking-tight leading-none"
                  style={{ fontFamily: "'Poppins', sans-serif" }}> Bug Hunter
              </h2>
              <p className="text-cyan-200 text-xs font-medium">Internal Platform</p>
            </div>
          </div>
          <h3 className="text-[1.6rem] font-bold leading-snug text-white mb-1"
            style={{ fontFamily: "'Poppins', sans-serif" }}>
            {isRegister ? "Join the community" : "Squash bugs faster."}
          </h3>
          <div className="flex items-center gap-2">
            <h3 className="text-[1.6rem] font-bold leading-snug text-cyan-200"
              style={{ fontFamily: "'Poppins', sans-serif" }}>
              {isRegister ? "of bug hunters" : "Ship better software."}
            </h3>
            {isRegister && <Rocket className="w-6 h-6 text-cyan-200" />}
          </div>
        </div>
        {/* ── Illustration image ── */}
        <div className="relative z-10 flex justify-center items-center flex-1">
          <img src="/bug-hunter-illustration.png" alt="Bug Hunter at Work"
            className="w-full max-w-[280px] drop-shadow-xl"
            style={{ filter: "drop-shadow(0 8px 24px rgba(0,0,0,0.2))" }}
          />
        </div>
      </div>
      {/* ═══════ RIGHT PANEL ═══════ */}
      <div className="flex-1 flex flex-col justify-center items-center px-8 sm:px-12 py-10 bg-white">
        <div className="w-full max-w-md">
          {authMode === "login" && (
            <Login onSwitchToRegister={() => setAuthMode("register")}
              onSwitchToReset={() => setAuthMode("reset")} />
          )}
          {authMode === "register" && ( <Register onSwitchToLogin={() => setAuthMode("login")} /> )}
          {authMode === "reset" && ( <PasswordReset onSwitchToLogin={() => setAuthMode("login")} />)}
        </div>
      </div>
    </div>
```

```
      {/* ── Footer ── */}
      <p className="absolute bottom-3 text-center text-xs z-10"
        style={{ color: "#94a3b8", fontFamily: "'Inter', sans-serif" }}>
        © {new Date().getFullYear()} Bug Hunter · Software Quality Assurance
      </p>
    </div>
  );
}
export default AuthContainer;
```

Login.jsx

```
import { useEffect, useState } from "react";
import { useAuth } from "@/context/AuthContext";
import { useNavigate } from "react-router-dom";
import { Mail, Lock, Eye, EyeOff, Hand, PartyPopper} from "lucide-react";
import toast from "react-hot-toast";
import { login } from "@/API_Call/Auth";
/* ─────────────── InputField ─────────────── */
function InputField({ icon: Icon, label, children, extra }) {
  return (
    <div className="space-y-1.5">
      <div className="flex items-center justify-between">
        <label className="text-sm font-semibold text-gray-600 tracking-wide"> {label} </label>  {extra}
      </div>
      <div className="input-group flex items-center gap-3 bg-gray-50 border-2 border-gray-200 rounded-2xl
          px-4 py-3.5 transition-all duration-200">
        <Icon className="text-cyan-500 text-xl flex-shrink-0" />
        {children}
      </div>
    </div>
  );
}

/* ─────────────── Main Login Component ─────────────── */
function Login({ onSwitchToRegister, onSwitchToReset }) {
  const navigate = useNavigate();
  const [username_email, setUsernameEmail] = useState("");
  const [password, setPassword] = useState("");
  const [showPassword, setShowPassword] = useState(false);
  const [rememberMe, setRememberMe] = useState(false);
  const { user, setUser, loading, setLoading } = useAuth();
  /* ── redirect if already logged in ── */
  useEffect(() => {
    if (user && !username_email) { toast.success("Already logged in!");  navigate("/");}
  }, [user]);
  /* ── handlers ── */
  const handlePasswordLogin = async (e) => {
    e.preventDefault();
    if (!username_email || !password)  return toast.error("Please fill in all fields");
    setLoading(true);
    const userData = { username_email, password, keepMeSignedIn: rememberMe };
    const login_check = await login(userData);
    if (login_check.success) {
```

```
    setUser(login_check.user);
    toast.success(
      <div className="flex items-center gap-2">
        Welcome back! <PartyPopper className="w-5 h-5 text-yellow-500" />
      </div>
    );
    navigate("/");
  } else { toast.error(login_check.message); }
  setLoading(false);
};

return (
  <div className="w-full fade-in">
    {/* ── Header ── */}
    <div className="mb-8">
      <h1 className="text-3xl font-extrabold text-gray-900 leading-tight flex items-center gap-3"
        style={{ fontFamily: "'Poppins', sans-serif" }}>
        Welcome back <Hand className="text-yellow-400 w-8 h-8" />
      </h1>
      <p className="text-gray-500 mt-1.5 text-sm" style={{ fontFamily: "'Inter', sans-serif" }}>
        Sign in to continue tracking &amp;amp; reporting bugs
      </p>
    </div>
    {/* ═══════ LOGIN FORM ═══════ */}
    <form onSubmit={handlePasswordLogin} className="space-y-5">
      {/* Email / Username */}
      <InputField icon={Mail} label="Email or Username">
        <input type="text" placeholder="you@company.com"
          value={username_email} onChange={(e) => setUsernameEmail(e.target.value)}
          className="w-full bg-transparent outline-none text-gray-800 text-sm placeholder:text-gray-400"
          autoComplete="username" />
      </InputField>
      {/* Password */}
      <InputField icon={Lock} label="Password"
        extra={
          <button type="button" onClick={onSwitchToReset}
            className="text-xs text-cyan-500 hover:text-cyan-700 font-semibold transition">
            Forgot password?
          </button>
        }>
        <input type={showPassword ? "text" : "password"} placeholder="••••••••••••" value={password}
          onChange={(e) => setPassword(e.target.value)} autoComplete="current-password"
          className="w-full bg-transparent outline-none text-gray-800 text-sm placeholder:text-gray-400" />
        <button type="button"  onClick={() => setShowPassword(!showPassword)}
          className="text-gray-400 hover:text-cyan-500 transition flex-shrink-0">
          {showPassword ? ( <EyeOff size={18} />) : (<Eye size={18} />)}
        </button>
      </InputField>
      {/* Remember me */}
      <label className="flex items-center gap-2.5 cursor-pointer select-none group">
        <div onClick={() => setRememberMe(!rememberMe)}
          className={`w-5 h-5 rounded-md border-2 flex items-center justify-center transition-all
            duration-200 ${
```

```
            rememberMe ? "bg-cyan-500 border-cyan-500": "border-gray-300 group-hover:border-cyan-400" }`}>
          {rememberMe && (
            <svg className="w-3 h-3 text-white" fill="none" viewBox="0 0 24 24" stroke="currentColor"
                strokeWidth={3}>
              <path strokeLinecap="round" strokeLinejoin="round" d="M5 1314 4L19 7" />
            </svg>
          )}
        </div>
        <span className="text-sm text-gray-600">Keep me signed in</span>
      </label>
      {/* Submit */}
      <button  type="submit" disabled={loading}
        className="btn-primary-shine w-full text-white font-semibold py-3.5 rounded-2xl text-sm
tracking-wide
                   disabled:opacity-60 disabled:cursor-not-allowed flex items-center justify-center gap-2
mt-2">
        {loading ? (
          <>
            <svg className="animate-spin w-4 h-4" viewBox="0 0 24 24" fill="none">
              <circle className="opacity-25" cx="12" cy="12" r="10" stroke="currentColor" strokeWidth="4" />
              <path className="opacity-75" fill="currentColor" d="M4 12a8 8 0 018-8v8H4z" />
            </svg>
            Signing in…
          </>
        ) : ( "Sign In" )}
      </button>
    </form>
    {/* ── Register Link ── */}
    <p className="text-center mt-7 text-sm text-gray-500">
      New to Bug Tracler?
      <button onClick={onSwitchToRegister}
        className="text-cyan-600 font-bold hover:text-cyan-800 transition underline underline-offset-2">
        Create an account </button>
    </p>
  </div>
 );
}
export default Login;
```

PasswordReset.jsx

```
"use client";
import React, { useState, useRef, useEffect } from "react";
import { Mail, Lock, Eye, EyeOff, ChevronLeft, MailQuestion, PartyPopper } from "lucide-react";
import toast from "react-hot-toast";
import { requestOtpReset, verifyOtpReset, resetPassword } from "@/API_Call/Auth";
/* ─────────────── Shared OTP Input ─────────────── */
function OtpInput({ value, onChange }) {
 const digits = 6;
 const inputRefs = useRef([]);
 useEffect(() => { inputRefs.current = inputRefs.current.slice(0, digits); }, []);
 const arr = value.padEnd(digits, "").split("").slice(0, digits);
 const handleKey = (i, e) => {
   if (e.key === "Backspace") { if (!arr[i] && i > 0) { inputRefs.current[i - 1]?.focus(); }}
 };
 const handleChange = (i, e) => { const val = e.target.value; const char = val.slice(-1).replace(/\D/g, "");
```

```
  const nextArr = [...arr]; nextArr[i] = char; const nextStr = nextArr.join("").slice(0, digits);
  onChange(nextStr); if (char && i < digits - 1) { inputRefs.current[i + 1]?.focus();}
};
const handlePaste = (e) => {
  e.preventDefault(); onChange(pasted);
  const pasted = e.clipboardData.getData("text").replace(/\D/g, "").slice(0, digits);
};
return (
  <div className="flex gap-2.5 justify-center py-2" onPaste={handlePaste}>
    {Array.from({ length: digits }).map((_, i) => (
      <input  key={i} ref={(el) => (inputRefs.current[i] = el)} type="text" inputMode="numeric"
        maxLength={1} value={arr[i] || ""}  onChange={(e) => handleChange(i, e)}
        onKeyDown={(e) => handleKey(i, e)}
        className={`w-11 h-14 text-center text-xl font-bold border-2 rounded-2xl transition-all
                  duration-200 outline-none
      ${ arr[i] ? "border-cyan-500 bg-cyan-50 text-cyan-700 shadow-[0_0_15px_-5px_rgba(6,182,212,0.3)]"
            : "border-gray-200 bg-gray-50 text-gray-800"
        }`}
      />
    ))}
  </div>
);
}
/* ———————— Shared Input Field ———————— */
function InputField({ icon: Icon, label, children, extra }) {
return (
  <div className="space-y-1.5 w-full">
    <div className="flex items-center justify-between">
      <label className="text-sm font-semibold text-gray-600 tracking-wide">{label}</label>{extra}
    </div>
    <div className="input-group flex items-center gap-3 bg-gray-50 border-2 border-gray-200 rounded-2xl
       px-4 py-3.5 transition-all duration-200 focus-within:border-cyan-500 focus-within:bg-white
shadow-sm">
      <Icon className="text-cyan-500 text-xl flex-shrink-0" />
      {children}
    </div>
  </div>
);
}
function PasswordReset({ onSwitchToLogin }) {
const [step, setStep] = useState(1); // 1: email, 2: otp, 3: new password
const [email, setEmail] = useState("");
const [otp, setOtp] = useState("");
const [newPassword, setNewPassword] = useState("");
const [confirmPassword, setConfirmPassword] = useState("");
const [showPassword, setShowPassword] = useState(false);
const [showConfirmPassword, setShowConfirmPassword] = useState(false);
const [loading, setLoading] = useState(false);
const [resetToken, setResetToken] = useState("");
const handleSendOtp = async (e) => {
  e.preventDefault();
  if (!email) return toast.error("Please enter your email");
  setLoading(true);
```

```
  const result = await requestOtpReset(email);
  if (result.success) {
    toast.success(
      (t) => (
        <div className="flex items-center gap-3">
          <div className="w-10 h-10 bg-cyan-100 rounded-full flex items-center justify-center">
            <MailQuestion className="text-cyan-600 w-6 h-6" />
          </div>
          <div className="flex flex-col text-left">
            <span className="font-bold text-gray-900">OTP Sent!</span>
            <span className="text-xs text-gray-500">Check your inbox at
             <span className="font-semibold text-cyan-600">{email}</span></span>
          </div>
        </div>
      ),
      { duration: 5000 }
    );
    setStep(2);
  } else { toast.error(result.message);}
  setLoading(false);
};
const handleVerifyOtp = async (e) => {
  e.preventDefault();
  if (otp.length < 6) return toast.error("Please enter 6-digit OTP");
  setLoading(true);
  const result = await verifyOtpReset({ username_email: email, otp });
  if (result.success) {
    toast.success("Identity verified! Set your new password.");
    setResetToken(result.resetToken); setStep(3);
  } else {  toast.error(result.message); }
  setLoading(false);
};
const handleUpdatePassword = async (e) => {
  e.preventDefault();
  if (!newPassword || !confirmPassword) return toast.error("Please fill in all fields");
  if (newPassword !== confirmPassword) return toast.error("Passwords do not match");
  if (newPassword.length < 6) return toast.error("Password must be at least 6 characters");
  setLoading(true);
  const result = await resetPassword({ resetToken, newPassword });
  if (result.success) {
    toast.success(
      <div className="flex items-center gap-2">
        Password updated successfully! <PartyPopper className="w-5 h-5 text-yellow-500" />
      </div>
    );
    onSwitchToLogin();
  } else { toast.error(result.message); }
  setLoading(false);
};
return (
  <div className="w-full fade-in">
    {/* ── Header ── */}
    <div className="mb-8">
```

```
        <h1 className="text-3xl font-extrabold text-gray-900 leading-tight"> Reset Password  </h1>
        <div className="flex items-center gap-2 mt-2">
          <div className="flex gap-1">
            {[1, 2, 3].map((s) => (
              <div key={s} className={`h-1.5 rounded-full transition-all duration-300 ${
                  s === step ? "w-6 bg-cyan-500" : s < step ? "w-2 bg-cyan-200" : "w-2 bg-gray-200"
                }`}
              />
            ))}
          </div>
          <span className="text-xs text-gray-400 font-medium ml-1">Step {step} of 3</span>
        </div>
      </div>
      {/* ── Step 1: Email ── */}
      {step === 1 && (
        <form onSubmit={handleSendOtp} className="space-y-6">
          <InputField icon={Mail} label="Email Address">
            <input type="email" placeholder="thisuser@mail.com" value={email}
              onChange={(e) => setEmail(e.target.value)}
              className="w-full bg-transparent outline-none text-gray-800 text-sm placeholder:text-gray-400"/>
          </InputField>
          <button type="submit" disabled={loading}
            className="btn-primary-shine w-full text-white font-semibold py-3.5 rounded-2xl text-sm
tracking-wide
                    disabled:opacity-60 disabled:cursor-not-allowed flex items-center justify-center gap-2"
>
            {loading ? "Sending OTP..." : "Send Reset Code"}
          </button>
        </form>
      )}
      {/* ── Step 2: OTP Verification ── */}
      {step === 2 && (
        <form onSubmit={handleVerifyOtp} className="space-y-6 text-center">
          <div className="space-y-3 text-left">
            <label className="text-sm font-semibold text-gray-600 tracking-wide block">
              Enter Verification Code
            </label>
            <OtpInput value={otp} onChange={setOtp} />
          </div>
          <button type="submit" disabled={loading || otp.length < 6}
            className="btn-primary-shine w-full text-white font-semibold py-3.5 rounded-2xl text-sm
tracking-wide
                  disabled:opacity-60 disabled:cursor-not-allowed flex items-center justify-center gap-2
mt-4" >
            {loading ? "Verifying..." : "Verify OTP"}
          </button>
          <button type="button" onClick={() => setStep(1)} className="text-xs text-cyan-500 hover:underline">
            Use a different email?
          </button>
        </form>
      )}
      {/* ── Step 3: New Password ── */}
      {step === 3 && (
```

```
      <form onSubmit={handleUpdatePassword} className="space-y-5">
        <InputField icon={Lock} label="New Password">
          <input type={showPassword ? "text" : "password"} placeholder="••••••••••••" value={newPassword}
            onChange={(e) => setNewPassword(e.target.value)}
            className="w-full bg-transparent outline-none text-gray-800 text-sm placeholder:text-gray-400"
/>
          <button  type="button" onClick={() => setShowPassword(!showPassword)}
            className="text-gray-400 hover:text-cyan-500 transition flex-shrink-0">
            {showPassword ? <EyeOff size={20} /> : <Eye size={20} />}
          </button>
        </InputField>
        <InputField icon={Lock} label="Confirm Password">
          <input type={showConfirmPassword ? "text" : "password"}  placeholder="••••••••••••"
            value={confirmPassword}onChange={(e) => setConfirmPassword(e.target.value)}
            className="w-full bg-transparent outline-none text-gray-800 text-sm placeholder:text-gray-400"/>
          <button type="button" onClick={() => setShowConfirmPassword(!showConfirmPassword)}
            className="text-gray-400 hover:text-cyan-500 transition flex-shrink-0" >
            {showConfirmPassword ? <EyeOff size={20} /> : <Eye size={20} />}
          </button>
        </InputField>
        <button type="submit" disabled={loading}
          className="btn-primary-shine w-full text-white font-semibold py-3.5 rounded-2xl text-sm
tracking-wide
              disabled:opacity-60 disabled:cursor-not-allowed flex items-center justify-center gap-2 mt-2"
>
          {loading ? "Updating..." : "Reset Password"}
        </button>
      </form>
    )}
    {/* ── Footer Link ── */}
    <div className="text-center mt-10">
      <button onClick={onSwitchToLogin}
        className="group inline-flex items-center gap-1.5 text-sm font-semibold text-gray-400
                      hover:text-cyan-600 transition-all duration-200" >
        <ChevronLeft className="text-xl group-hover:-translate-x-1 transition-transform" />
        Back to Login
      </button>
    </div>
  </div>
 );
}
export default PasswordReset;
```

Register.jsx

```
"use client";
import { useState } from "react";
import { Mail, Lock, Eye, EyeOff, User, CheckCircle2, Sparkles, PartyPopper} from "lucide-react";
import toast from "react-hot-toast";
import { register } from "@/API_Call/Auth";
/* ── Reusable input field wrapper ── */
function InputField({ icon: Icon, label, children, hint }) {
 return (
   <div className="space-y-1.5">
     <label className="text-sm font-semibold text-gray-600 tracking-wide block"> {label}</label>
```

```
      <div className="input-group flex items-center gap-3 bg-gray-50 border-2 border-gray-200 rounded-2xl px-4
           py-3.5 transition-all duration-200">
        <Icon className="text-cyan-500 text-xl flex-shrink-0" />{children}
      </div> {hint && <p className="text-xs text-gray-400 pl-1">{hint}</p>}
    </div>
  );
}
/* ── Password strength ── */
function PasswordStrength({ password }) {
  if (!password) return null;
  const strength = (() => {
    if (password.length < 6) return { label: "Weak", color: "bg-red-400", w: "w-1/4", text: "text-red-500" };
    if (password.length < 10) return { label: "Fair", color: "bg-yellow-400", w: "w-2/4", text:
"text-yellow-500" };
    if (!/[A-Z]/.test(password) || !/[0-9]/.test(password))
      return { label: "Good", color: "bg-blue-400", w: "w-3/4", text: "text-blue-500" };
    return { label: "Strong", color: "bg-green-400", w: "w-full", text: "text-green-500" };
  })();
  return (
    <div className="space-y-1 px-1">
      <div className="h-1.5 bg-gray-200 rounded-full overflow-hidden">
        <div className={`h-full rounded-full transition-all duration-500 ${strength.color} ${strength.w}`} />
      </div>
      <p className="text-xs text-gray-400"> Strength:
        <span className={`font-semibold ${strength.text}`}>{strength.label}</span>
      </p>
    </div>
  );
}
/* ── Password match indicator ── */
function MatchIndicator({ password, confirm }) {
  if (!confirm) return null;
  const match = password === confirm;
  return (
    <div className={`flex items-center gap-1.5 text-xs px-1 ${match ? "text-green-500" : "text-red-400"}`}>
      <CheckCircle2 size={14} className={`${match ? "opacity-100" : "opacity-40"}`} />
      {match ? "Passwords match" : "Passwords do not match"}
    </div>
  );
}
/* ── Main Register Component ── */
function Register({ onSwitchToLogin }) {
  const [username, setUsername] = useState("");
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [confirmPassword, setConfirmPassword] = useState("");
  const [showPassword, setShowPassword] = useState(false);
  const [showConfirmPassword, setShowConfirmPassword] = useState(false);
  const [agreed, setAgreed] = useState(false);
  const [loading, setLoading] = useState(false);
  const handleRegister = async (e) => {
    e.preventDefault();
```

```
   if (!username||!email || !password || !confirmPassword){ toast.error("Please fill in all fields");
return;}
   if (password !== confirmPassword) { toast.error("Passwords do not match"); return;}
   if (password.length < 6) { toast.error("Password must be at least 6 characters"); return;}
   if (!agreed) { toast.error("Please accept the terms to continue"); return;}
   setLoading(true);
   const res = await register({ username, email, password });
   if (res.success) {
     toast.success(
       <div className="flex items-center gap-2">
         Account created! Welcome aboard <PartyPopper className="w-5 h-5 text-yellow-500" />
       </div>
     );
     setUsername(""); setEmail(""); setPassword(""); setConfirmPassword(""); setAgreed(false);
     // Switch to login after successful registration
     setTimeout(() => onSwitchToLogin(), 1500);
   } else {     toast.error(res.message);   }
   setLoading(false);
 };
 return (
   <div className="w-full fade-in">
     {/* ── Header ── */}
     <div className="mb-7">
       <h1 className="text-3xl font-extrabold text-gray-900 leading-tight flex items-center gap-3">
         Create account <Sparkles className="text-yellow-400 w-8 h-8" />
       </h1>
       <p className="text-gray-500 mt-1.5 text-sm"> Join Bug Hunter and start squashing bugs today </p>
     </div>
     {/* ── Form ── */}
     <form onSubmit={handleRegister} className="space-y-4">
       {/* Username */}
       <InputField icon={User} label="Username">
         <input type="text" placeholder="johndoe" value={username} onChange={(e) =>
setUsername(e.target.value)}
           className="w-full bg-transparent outline-none text-gray-800 text-sm placeholder:text-gray-400"
           autoComplete="username"/>
       </InputField>
       {/* Email */}
       <InputField icon={Mail} label="Email Address">
         <input type="email" placeholder="you@company.com" value={email}
           onChange={(e) => setEmail(e.target.value)}  autoComplete="email"
           className="w-full bg-transparent outline-none text-gray-800 text-sm placeholder:text-gray-400"/>
       </InputField>
       {/* Password */}
       <div className="space-y-2">
         <InputField icon={Lock} label="Password" hint="Min. 8 chars, one uppercase & number recommended">
           <input type={showPassword ? "text" : "password"} placeholder="••••••••••••"
             value={password} onChange={(e) => setPassword(e.target.value)} autoComplete="new-password"
             className="w-full bg-transparent outline-none text-gray-800 text-sm placeholder:text-gray-400"
/>
           <button type="button"  onClick={() => setShowPassword(!showPassword)}
             className="text-gray-400 hover:text-cyan-500 transition flex-shrink-0">
             {showPassword ? <EyeOff size={18} /> : <Eye size={18} />}
```

```
          </button>
        </InputField>
        <PasswordStrength password={password} />
      </div>
      {/* Confirm Password */}
      <div className="space-y-2">
        <InputField icon={Lock} label="Confirm Password">
          <input type={showConfirmPassword ? "text" : "password"} placeholder="••••••••••••"
            value={confirmPassword} onChange={(e) => setConfirmPassword(e.target.value)}
            className="w-full bg-transparent outline-none text-gray-800 text-sm placeholder:text-gray-400"
            autoComplete="new-password" />
          <button type="button" onClick={() => setShowConfirmPassword(!showConfirmPassword)}
            className="text-gray-400 hover:text-cyan-500 transition flex-shrink-0">
            {showConfirmPassword ? <EyeOff size={18} /> : <Eye size={18} />}
          </button>
        </InputField>
        <MatchIndicator password={password} confirm={confirmPassword} />
      </div>
      {/* Terms & Conditions */}
      <label className="flex items-start gap-2.5 cursor-pointer select-none group pt-1">
        <div onClick={() => setAgreed(!agreed)}
          className={`w-5 h-5 rounded-md border-2 flex items-center justify-center mt-0.5
                    flex-shrink-0 transition-all duration-200
      ${ agreed ? "bg-cyan-500 border-cyan-500" : "border-gray-300 group-hover:border-cyan-400"}`}>
          {agreed && (
            <svg className="w-3 h-3 text-white" fill="none" viewBox="0 0 24 24"
                 stroke="currentColor" strokeWidth={3}>
              <path strokeLinecap="round" strokeLinejoin="round" d="M5 13l4 4L19 7" />
            </svg>
          )}
        </div>
        <span className="text-sm text-gray-600 leading-relaxed"> I agree to the
          <button type="button" className="text-cyan-600 font-semibold hover:underline"> Terms of Service
          </button> and
          <button type="button" className="text-cyan-600 font-semibold hover:underline"> Privacy Policy
          </button>
        </span>
      </label>
      {/* Submit */}
      <button type="submit" disabled={loading}
        className="btn-primary-shine w-full text-white font-bold py-3.5 rounded-2xl text-sm tracking-wide
          disabled:opacity-60 disabled:cursor-not-allowed flex items-center justify-center gap-2 mt-2" >
        {loading ? (
          <>
            <svg className="animate-spin w-4 h-4" viewBox="0 0 24 24" fill="none">
              <circle className="opacity-25" cx="12" cy="12" r="10" stroke="currentColor" strokeWidth="4" />
              <path className="opacity-75" fill="currentColor" d="M4 12a8 8 0 018-8v8H4z" />
            </svg>
            Creating account…
          </>
        ) : ("Create Account →" )}
      </button>
    </form>
```

```
    {/* ── Login Link ── */}
    <p className="text-center mt-6 text-sm text-gray-500">
      Already have an account?{" "}
      <button onClick={onSwitchToLogin}
        className="text-cyan-600 font-bold hover:text-cyan-800 transition underline underline-offset-2" >
        Sign in
      </button>
    </p>
  </div>
 );
}
export default Register;
```

Collapsible (folder)

Collapsible.jsx

```
import { Accordion, AccordionContent, AccordionItem, AccordionTrigger,} from "@/components/ui/accordion";
export function Collapsible() {
 return (
   <Accordion type="single" collapsible className="w-full">
     <AccordionItem value="item-1">
       <AccordionTrigger>Is it accessible?</AccordionTrigger>
       <AccordionContent> Yes. It adheres to the WAI-ARIA design pattern.</AccordionContent>
     </AccordionItem>
   </Accordion>
 );
}
export default Collapsible;
```

Dashboard (folder)
DashboardLayout.jsx

```
import { Outlet } from "react-router-dom";
import { useAuth } from "@/context/AuthContext";
import { Bell, PanelLeft } from "lucide-react";
import { useNavigate } from "react-router-dom";
import { useState } from "react";
import Sidebar from "./Sidebar";
function DashboardLayout() {
 const { user } = useAuth();
 const navigate = useNavigate();
 const [collapsed, setCollapsed] = useState(false);
 return (
   <div className="flex h-screen overflow-hidden">
     {/* ── Custom Sidebar — receives collapse state from here ── */}
     <Sidebar collapsed={collapsed} setCollapsed={setCollapsed} />
     {/* ── Main Content Area ── */}
     <div className="flex flex-col flex-1 overflow-hidden">
       {/* ── Top Header Bar ── */}
       <header className="flex h-14 shrink-0 items-center justify-between gap-3 bg-white border-b
        border-gray-100 px-4">
         {/* Left: toggle + greeting */}
         <div className="flex items-center gap-3">
           {/* Sidebar toggle in header */}
           <button onClick={() => setCollapsed(!collapsed)} className="w-8 h-8 rounded-lg flex items-center
```

```
              justify-center transition-all duration-200 hover:bg-gray-100 text-gray-500
hover:text-cyan-600"
            title={collapsed ? "Expand sidebar" : "Collapse sidebar"}>
            <PanelLeft size={17} />
          </button>
          <div className="h-5 w-px bg-gray-200" />
          <div>
            <p className="text-sm font-semibold text-gray-800 leading-none">
              Hi, {user?.name?.split(" ")[0] ?? "there"} 👋
            </p>
            <p className="text-xs text-gray-400 mt-0.5">Bug Tracker Platform</p>
          </div>
        </div>
        {/* Right: avatar */}
        <div className="flex items-center gap-2">
          <button onClick={() => navigate("/account")}
            className="w-7 h-7 rounded-full flex items-center justify-center text-white text-xs font-bold
                       transition-transform hover:scale-105"
            style={{ background: "linear-gradient(135deg, #0891b2, #0e7490)" }}>      {user?.name?.[0] ??
"U"}
          </button>
        </div>
      </header>
      {/* ── Page Content ── */}
      <main className="flex-1 overflow-auto p-5"  style={{ background: "#f8fafc" }} >
        <Outlet />
      </main>
    </div>
  </div>
 );
}
export default DashboardLayout;
```

NotificationPopover.jsx

```
import { useState, useEffect } from "react";
import { Bell, Check, Trash2, X } from "lucide-react";
import { Popover, PopoverContent, PopoverTrigger,} from "@/components/ui/popover";
import { getNotifications, markAsRead, markAllAsRead, deleteNotification,} from "@/API_Call/Notification";
import { formatDistanceToNow } from "date-fns";
import { useNavigate } from "react-router-dom";
export function NotificationPopover() {
 const [notifications, setNotifications] = useState([]);
 const [unreadCount, setUnreadCount] = useState(0);
 const [isOpen, setIsOpen] = useState(false);
 const navigate = useNavigate();
 const fetchNotifications = async () => {
   const data = await getNotifications();
   setNotifications(data.notifications || []);
   setUnreadCount(data.unreadCount || 0);
 };
 useEffect(() => {
   fetchNotifications();
   // Poll for notifications every 30 seconds
   const interval = setInterval(fetchNotifications, 30000);
```

```
    return () => clearInterval(interval);
  }, []);

  const handleMarkRead = async (id) => { await markAsRead(id); fetchNotifications(); };
  const handleMarkAllRead = async () => {  await markAllAsRead(); fetchNotifications(); };
  const handleDelete = async (id, e) => {
    e.stopPropagation(); await deleteNotification(id);  fetchNotifications();
  };
  const handleNotificationClick = async (notification) => {
    if (!notification.isRead) { await handleMarkRead(notification._id);   }
    if (notification.link) {     navigate(notification.link);   }
    setIsOpen(false);
  };
  return (
    <Popover open={isOpen} onOpenChange={setIsOpen}>
      <PopoverTrigger asChild>
        <button className="relative w-8 h-8 rounded-lg bg-gray-50 hover:bg-gray-100 flex items-center
                justify-center transition-colors">
          <Bell size={15} className="text-gray-500" />
          {unreadCount > 0 && (
            <span className="absolute top-1 right-1 flex h-4 w-4 items-center justify-center rounded-full
                bg-cyan-500 text-[10px] font-bold text-white border-2 border-white">
              {unreadCount > 9 ? "9+" : unreadCount}
            </span>
          )}
        </button>
      </PopoverTrigger>
      <PopoverContent className="w-80 p-0" align="end">
        <div className="flex items-center justify-between px-4 py-2 border-b">
          <h3 className="font-semibold text-sm">Notifications</h3>
          {unreadCount > 0 && (
          <button onClick={handleMarkAllRead} className="text-xs text-cyan-600 hover:text-cyan-700
font-medium">
            Mark all as read
          </button>
          )}
        </div>
        <div className="max-h-[400px] overflow-auto">
          {notifications.length === 0 ? (
            <div className="p-8 text-center text-gray-400 text-sm">No notifications yet</div>
          ) : (
            notifications.map((n) => (
              <div key={n._id} onClick={() => handleNotificationClick(n)}
                 className={`relative group px-4 py-3 border-b hover:bg-gray-50 transition-colors
cursor-pointer
                ${!n.isRead ? "bg-cyan-50/50" : ""}`}>
                 <div className="flex justify-between items-start gap-2">
                   <div className="flex-1">
                     <p className={`text-sm ${!n.isRead ? "font-semibold" : "text-gray-700"}`}> {n.title}</p>
                     <p className="text-xs text-gray-500 mt-1 line-clamp-2">{n.message}</p>
                     <p className="text-[10px] text-gray-400 mt-1">
                       {formatDistanceToNow(new Date(n.createdAt), { addSuffix: true })}
                     </p>
```

```
                </div>
                <div className="flex flex-col gap-1 opacity-0 group-hover:opacity-100 transition-opacity">
                  {!n.isRead && (
                    <button onClick={(e) => {  e.stopPropagation(); handleMarkRead(n._id);}}
                      className="p-1 hover:bg-gray-200 rounded text-gray-400 hover:text-cyan-600"
                      title="Mark as read"> <Check size={12} />
                    </button>
                  )}
                  <button onClick={(e) => handleDelete(n._id, e)}
                    className="p-1 hover:bg-gray-200 rounded text-gray-400
hover:text-red-600"title="Delete">
                    <Trash2 size={12} />
                  </button>
                </div>
              </div>
              {!n.isRead && (
                <div className="absolute left-1 top-1/2 -translate-y-1/2 w-1 h-8 bg-cyan-500 rounded-full"
/>
              )}
            </div>
          ))
        )}
      </div>
    </PopoverContent>
  </Popover>
 );
}
```

Sidebar.jsx

```
import { useState } from "react";
import { useLocation, useNavigate } from "react-router-dom";
import { useAuth } from "@/context/AuthContext";
import { logout } from "@/API_Call/Auth";
import toast from "react-hot-toast";
import { LayoutDashboard, Bug, Settings, LogOut, BugIcon, ChevronDown, ChevronRight, Users, Wrench,
 FileBarChart,} from "lucide-react";
const NAV = [
 { label: "Overview",   items: [ { name: "Dashboard", href: "/", icon: LayoutDashboard },],},
 {   label: "Work",   items: [{ name: "Bugs", href: "/bugs", icon: Bug }], },
 {   label: "Admin",
   items: [
     {
       name: "Settings", href: "/settings", icon: Settings,
       subItems: [
         { name: "Users", href: "/settings/users", icon: Users },
         { name: "Tools", href: "/settings/tools", icon: Wrench },
         // { name: "Reports", href: "/settings/reports", icon: FileBarChart },
       ],
     },
   ],
 },
];
export default function Sidebar({ collapsed, setCollapsed }) {
 const { pathname } = useLocation();
```

```
const navigate = useNavigate();
const { user, setUser, setLoading } = useAuth();
const [expanded, setExpanded] = useState({ Settings: true });
const toggleExpand = (name) => { setExpanded(prev => ({ ...prev, [name]: !prev[name] })); };
const hasSettingsAccess = user?.role === "superadmin" || (Array.isArray(user?.adminControl) &&
       user.adminControl.some(r => ["create", "edit", "view", "delete"].includes(r?.toLowerCase())));
const handleLogout = async () => {
  setLoading(true);
  const res = await logout();
  if (res.success) { setUser(""); toast.success("Logged out"); navigate("/login");}
  else { toast.error(res.message); }
  setLoading(false);
};
const initial = user?.name?.charAt(0)?.toUpperCase() || user?.username?.charAt(0)?.toUpperCase() || "U";
const isActive = (href) => {
  if (href === "/settings") return pathname.startsWith("/settings");
  if (href === "/bugs") return pathname.startsWith("/bugs");
  return pathname === href;
};
return (
  <aside className="relative flex flex-col h-screen flex-shrink-0 transition-all duration-300
       ease-in-out overflow-hidden"
    style={{width: collapsed ? "64px" : "220px", borderRight: "1px solid rgba(6,182,212,0.12)",
      background: "linear-gradient(165deg, #071e2e 0%, #0a2d44 50%, #071e2e 100%)",}} >
    {/* Ambient glow blobs */}
    <div className="absolute top-0 left-0 w-48 h-48 rounded-full pointer-events-none"
      style={{ background: "radial-gradient(circle, rgba(6,182,212,0.1) 0%, transparent 70%)",
       transform: "translate(-40%,-40%)" }} />
    <div className="absolute bottom-0 right-0 w-40 h-40 rounded-full pointer-events-none"
      style={{ background: "radial-gradient(circle, rgba(99,102,241,0.08) 0%, transparent 70%)",
       transform: "translate(40%,40%)" }} />

    {/* ── Brand header ── */}
    <div className={`flex items-center h-14 flex-shrink-0 px-3 ${collapsed ? "justify-center"
             : "gap-3 px-4"}`}>
      <div className="w-8 h-8 rounded-xl flex items-center justify-center flex-shrink-0"
        style={{ background: "linear-gradient(135deg, #06b6d4, #0891b2)" }}>
        <BugIcon size={16} className="text-white" />
      </div>
      {!collapsed && (
        <div className="overflow-hidden">
          <p className="text-white font-bold text-sm leading-none whitespace-nowrap">Bug Tracker</p>
          <p className="text-cyan-400/60 text-[10px] mt-0.5 whitespace-nowrap">by CEOITBOX</p>
        </div>
      )}
    </div>
    {/* Divider */}
    <div className="mx-3 h-px flex-shrink-0" style={{ background: "rgba(6,182,212,0.12)" }} />
    {/* ── Nav ── */}
    <nav className="flex-1 overflow-y-auto overflow-x-hidden py-3 px-2" style={{ scrollbarWidth: "none" }}>
      {NAV.map((group) => {
        if (group.label === "Admin" && !hasSettingsAccess) return null;
        return (
```

```
        <div key={group.label} className="mb-1">
          {/* Section label — only when expanded */}
          {!collapsed && (
            <p className="px-3 pt-3 pb-1.5 text-[10px] font-bold tracking-widest uppercase select-none"
              style={{ color: "rgba(103,232,249,0.35)" }}> {group.label}
            </p>
          )}
          {collapsed && <div className="h-3" />}
          {group.items.map((item) => {
            const active = isActive(item.href);
            const isExpanded = expanded[item.name];
            const hasSubItems = item.subItems && item.subItems.length > 0;
            return (
              <div key={item.name}>
                <div onClick={hasSubItems ? () => toggleExpand(item.name) : () => navigate(item.href)}
                  className="group relative flex items-center rounded-xl mb-0.5 transition-all duration-200
                             overflow-hidden cursor-pointer"
                  style={{
                    justifyContent: collapsed ? "center" : "flex-start",  gap: collapsed ? 0 : "0.75rem",
                    padding: collapsed ? "0.625rem" : "0.625rem 0.75rem",
                    ...(active && !hasSubItems ? {
                      background: "linear-gradient(135deg, rgba(6,182,212,0.2) 0%, rgba(6,182,212,0.08)
100%)",
                      borderLeft: collapsed ? "none" : "3px solid #22d3ee",
                      borderRadius: collapsed ? "12px" : undefined,
                      boxShadow: collapsed ? "0 0 0 1.5px rgba(34,211,238,0.5)" : "none",
                    } : { borderLeft: collapsed ? "none" : "3px solid transparent",}),
                  }}
                >
                  {active && !hasSubItems && (
                    <div className="absolute inset-0 pointer-events-none"
                      style={{ background: "radial-gradient(ellipse at left center, rgba(6,182,212,0.15) 0%,
                      transparent 60%)" }} />
                  )}

                  <item.icon size={18}
                    className="flex-shrink-0 relative z-10 transition-colors duration-200"
                    style={{ color: active ? "#22d3ee" : "rgba(186,230,253,0.55)" }}
                  />

                  {!collapsed && (
                    <span className="text-sm font-medium truncate relative z-10"
                      style={{ color: active ? "#e0f2fe" : "rgba(186,230,253,0.7)" }}> {item.name}
                    </span>
                  )}
                  {hasSubItems && !collapsed && (
                    <span className="ml-auto relative z-10">
                      {isExpanded ? <ChevronDown size={14} className="text-cyan-400/50" />
                                  : <ChevronRight size={14} className="text-cyan-400/50" />}
                    </span>
                  )}
                  {active && !hasSubItems && !collapsed && (
                    <span className="ml-auto w-1.5 h-1.5 rounded-full flex-shrink-0 relative z-10"
```

```
                        style={{ background: "#22d3ee", boxShadow: "0 0 6px #22d3ee" }} />
                    )}
                  </div>

                  {/* Sub items */}
                  {hasSubItems && isExpanded && !collapsed && (
                    <div className="ml-4 pl-4 border-l border-cyan-900/50 space-y-0.5 mt-1 mb-2">
                      {item.subItems.map((sub) => {
                        const subActive = pathname === sub.href;
                        return (
                          <a key={sub.name} href={sub.href}
                           className="flex items-center gap-2.5 px-3 py-2 rounded-lg transition-all
duration-200"
                           style={{ background: subActive ? "rgba(6,182,212,0.1)" : "transparent",}}>
                           <sub.icon size={14} style={{ color: subActive ? "#22d3ee" :
"rgba(186,230,253,0.4)"
                           }} />
                           <span className="text-xs font-medium" style={{ color: subActive ? "#e0f2fe"
                           : "rgba(186,230,253,0.6)" }}>{sub.name}
                           </span>
                          </a>
                        );
                      })}
                    </div>
                  )}
                </div>
              );
            })}
          </div>
          );
        })}
      </nav>
      {/* Divider */}
      <div className="mx-3 h-px flex-shrink-0" style={{ background: "rgba(6,182,212,0.12)" }} />

      {/* ── User footer ── */}
     <div className={`flex items-center py-4 flex-shrink-0 ${collapsed ? "justify-center px-3" : "gap-3
px-3"}`}>
        <buttononClick={() => navigate("/account")} title={user?.name}
          className="w-8 h-8 rounded-xl flex items-center justify-center text-white text-xs font-bold
                flex-shrink-0 hover:scale-105 transition-transform"
          style={{ background: "linear-gradient(135deg, #0891b2, #0e7490)" }}> {initial}
        </button>
        {!collapsed && (
          <>
            <div className="flex-1 overflow-hidden">
              <p className="text-sm font-semibold text-white/90 truncate leading-none">{user?.name}</p>
              <p className="text-[10px] mt-0.5 truncate" style={{ color: "rgba(103,232,249,0.5)" }}>
                {user?.email || user?.username}
              </p>
            </div>
            <button onClick={handleLogout}  title="Log out" className="flex-shrink-0 w-7 h-7 rounded-lg flex
               items-center justify-center hover:bg-red-500/20 transition-all">
```

```
            <LogOut size={14} style={{ color: "rgba(248,113,113,0.7)" }} />
          </button>
        </>
      )}
    </div>
  </aside>
 );
}
```

Dropdowns (folder)

Dropdown.jsx

```
import * as React from "react";
import { Select, SelectContent, SelectGroup, SelectItem, SelectLabel, SelectTrigger, SelectValue,
} from "@/components/ui/select";
function DynamicSelect({
 value, onChange, placeholder = "Select an option", items = [], groupLabel = null, className = "w-full",
 disabled = false,
}) {
 return (
   <Select value={value} onValueChange={onChange} disabled={disabled}>
     <SelectTrigger className={className}> <SelectValue placeholder={placeholder} /> </SelectTrigger>
     <SelectContent>
       <SelectGroup>
         {groupLabel && <SelectLabel>{groupLabel}</SelectLabel>}
         {items.map((item) => ( <SelectItem key={item.value} value={item.value}> {item.label}
</SelectItem>))}
       </SelectGroup>
     </SelectContent>
   </Select>
 );
}
export default DynamicSelect;
```

SearchableSelect.jsx

```
// SearchableToolDropdown.jsx
import React, { useState, useEffect } from "react";
import { Check, ChevronsUpDown } from "lucide-react";
import { Button } from "@/components/ui/button";
import { Command, CommandEmpty, CommandInput, CommandItem,} from "@/components/ui/command";
import { Popover, PopoverContent, PopoverTrigger,} from "@/components/ui/popover";
import { cn } from "@/lib/utils";
function SearchableToolDropdown({ value, onChange, placeholder = "Select Tool",
 items = [], isLoading = false, classAdd,  // Accept items as prop  // Accept loading state as prop
}) {
 const [open, setOpen] = useState(false);
 const [filteredTools, setFilteredTools] = useState([]);
 const [searchQuery, setSearchQuery] = useState("");
 // Update filtered tools when items change or popover opens
 useEffect(() => {
   if (open) {setSearchQuery(""); setFilteredTools(items);
   } else if (!searchQuery) {setFilteredTools(items);}
```

```
 }, [items, open]);
 const handleSearch = (search) => {
   setSearchQuery(search);
   if (!search) { setFilteredTools(items);
   } else {
     const lower = search.toLowerCase();
     setFilteredTools(
       items.filter(
         (t) =>t.label.toLowerCase().includes(lower) ||
           (t.sublabel && t.sublabel.toLowerCase().includes(lower))
         )
     );
   }
 };
 const selectedLabel = () => {
   const found = items.find((i) => i.value === value);
   return found ? `${found.label}${found.sublabel ? ` (${found.sublabel})` : ""}` : placeholder;
 };
 return (
   <Popover open={open} onOpenChange={setOpen}>
     <PopoverTrigger asChild className={classAdd}>
       <Button  variant="outline" role="combobox" aria-expanded={open}
         className="w-full justify-between border-2 h-10" > {selectedLabel()}
         <ChevronsUpDown className="opacity-50 h-4 w-4" />
       </Button>
     </PopoverTrigger>
     <PopoverContent align="start" sideOffset={5}  className="p-0 w-[var(--radix-popover-trigger-width)]">
       <Command shouldFilter={false}>
         <CommandInput placeholder="Search..." className="h-9" onValueChange={handleSearch}/>
         <div className="max-h-[300px] overflow-y-auto"
           onWheel={(e) => { e.currentTarget.scrollTop += e.deltaY;}}>
           <CommandEmpty>No item found.</CommandEmpty>
           {filteredTools.map((item) => (
             <CommandItem key={item.value} value={item.label}
               onSelect={() => { onChange(item.value);  setOpen(false);}} >
               <Check className={cn( "mr-2 h-4 w-4", value === item.value ? "opacity-100" : "opacity-0")} />
               <div className="flex flex-col">
                <span className="text-sm font-medium">{item.label}</span>
                {item.sublabel && ( <span className="text-xs text-muted-foreground"> {item.sublabel} </span>
)}
               </div>
             </CommandItem>
           ))}
           {isLoading && ( <div className="py-3 text-center text-sm text-muted-foreground"> Loading... </div>
)}
         </div>
       </Command>
     </PopoverContent>
   </Popover>
 );
}
export default SearchableToolDropdown;
```

My_Account (folder)
Admin_Control.jsx

```
import React from "react";
import { useAuth } from "@/context/AuthContext";
import { Users, ShieldCheck, Wrench, Key, AlertTriangle, BadgeCheck,} from "lucide-react";
function roleColor(role) {
 const map = {
   superadmin: "bg-purple-100 text-purple-700 border-purple-200",
   admin:      "bg-cyan-100 text-cyan-700 border-cyan-200",
   bugreporter:"bg-blue-100 text-blue-700 border-blue-200",
   tester:     "bg-indigo-100 text-indigo-700 border-indigo-200",
   dev:        "bg-emerald-100 text-emerald-700 border-emerald-200",
 };
 return map[role] || "bg-gray-100 text-gray-600 border-gray-200";
}
function roleLabel(r) {
 const map = { bugreporter: "Bug Reporter", tester:"Tester", dev: "Developer",admin: "Admin",
       superadmin:  "Super Admin", };
 return map[r] || r;
}
function InfoRow({ label, value, icon: Icon, mono }) {
 return (
   <div className="flex flex-col sm:flex-row sm:items-start gap-1 sm:gap-4 py-3 border-b border-gray-100
       last:border-0">
     <div className="flex items-center gap-1.5 sm:w-44 flex-shrink-0">
       {Icon && <Icon className="w-3.5 h-3.5 text-gray-400 flex-shrink-0" />}
       <span className="text-xs text-gray-400 uppercase font-semibold tracking-wide"> {label}  </span>
     </div>
     <div className={`text-sm font-medium text-gray-800 ${mono ? "font-mono text-gray-500 text-xs" : ""}`}>
       {value || "—"}
     </div>
   </div>
 );
}
function AdminControl() {
 const { user, allUsers, toolList } = useAuth();
 const isAdmin = user?.role === "admin" || user?.role === "superadmin";
 const allRoles = [
   ...(user?.role ? [user.role] : []),
   ...(Array.isArray(user?.roletype) ? user.roletype : user?.roletype  ? [user.roletype] : []),
 ];
 const uniqueRoles = [...new Set(allRoles)];
 return (
   <div className="space-y-5">
     {/* ── Account Authority Card ── */}
     <div className="bg-white rounded-2xl border border-gray-100 shadow-sm p-6">
       <div className="flex items-center gap-2 mb-5">
         <div className="w-8 h-8 bg-cyan-50 rounded-lg flex items-center justify-center">
           <ShieldCheck className="w-4 h-4 text-cyan-600" />
         </div>
         <h3 className="text-sm font-bold text-gray-700">Account Authority</h3>
       </div>
       <InfoRow label="Account ID"  value={user?._id || user?.id} icon={Key} mono />
       <InfoRow label="Name"        value={user?.name} icon={BadgeCheck} />
```

```
      <InfoRow label="Username"    value={user?.username} icon={BadgeCheck} />
      <InfoRow label="Email"       value={user?.email} icon={BadgeCheck} />
      {/* Roles badges */}
      <div className="flex flex-col sm:flex-row sm:items-start gap-1 sm:gap-4 py-3 border-b
border-gray-100">
        <div className="flex items-center gap-1.5 sm:w-44 flex-shrink-0">
         <ShieldCheck className="w-3.5 h-3.5 text-gray-400 flex-shrink-0" />
         <span className="text-xs text-gray-400 uppercase font-semibold tracking-wide"> Assigned Roles
</span>
        </div>
        <div className="flex flex-wrap gap-2">
          {uniqueRoles.length > 0 ? (
            uniqueRoles.map((r) => (
              <span  key={r}  className={`inline-flex items-center gap-1 text-xs font-semibold px-2.5
                     py-0.5 rounded-full border ${roleColor(r)}`} >{roleLabel(r)} </span>
            ))
          ) : (  <span className="text-sm text-gray-400">No roles assigned</span> )}
        </div>
      </div>
      <div className="flex flex-col sm:flex-row sm:items-start gap-1 sm:gap-4 py-3">
        <div className="flex items-center gap-1.5 sm:w-44 flex-shrink-0">
          <ShieldCheck className="w-3.5 h-3.5 text-gray-400 flex-shrink-0" />
          <span className="text-xs text-gray-400 uppercase font-semibold tracking-wide"> Admin Access
</span>
        </div>
        <span className={`inline-flex items-center gap-1.5 text-xs font-bold px-3 py-1 rounded-full ${
            isAdmin ? "bg-emerald-50 text-emerald-700 border border-emerald-200"
                    : "bg-gray-100 text-gray-500 border border-gray-200" }`}>
          {isAdmin ? "✓ Granted" : "✗ Not an Admin"}
        </span>
      </div>
    </div>
      {isAdmin && (
        <div className="flex flex-col sm:flex-row sm:items-start gap-1 sm:gap-4 py-3 border-t
border-gray-100">
          <div className="flex items-center gap-1.5 sm:w-44 flex-shrink-0">
            <Clock className="w-3.5 h-3.5 text-gray-400 flex-shrink-0" />
            <span className="text-xs text-gray-400 uppercase font-semibold tracking-wide">
              Daily Report Time
            </span>
          </div>
          <div className="text-sm font-medium text-gray-800"> 8:00 PM</div>
        </div>
      )}
    </div>
    {/* ── System Stats (admin-only) ── */}
    {isAdmin && (
      <div className="bg-white rounded-2xl border border-gray-100 shadow-sm p-6">
        <div className="flex items-center gap-2 mb-5">
          <div className="w-8 h-8 bg-violet-50 rounded-lg flex items-center justify-center">
            <Wrench className="w-4 h-4 text-violet-600" />
          </div>
          <h3 className="text-sm font-bold text-gray-700">System Overview</h3>
```

```
            <span className="ml-auto text-xs text-gray-400">Admin only</span>
          </div>
          <div className="grid grid-cols-2 sm:grid-cols-3 gap-4">
            {/* Total Users */}
            <div className="bg-gradient-to-br from-cyan-50 to-blue-50 border border-cyan-100 rounded-xl p-4">
              <div className="w-8 h-8 bg-cyan-500 rounded-lg flex items-center justify-center mb-3">
                <Users className="w-4 h-4 text-white" />
              </div>
              <p className="text-2xl font-extrabold text-cyan-700">{allUsers?.length ?? "—"}</p>
              <p className="text-xs font-semibold text-cyan-600 mt-0.5">Total Users</p>
            </div>
            {/* Total Tools */}
            <div className="bg-gradient-to-br from-violet-50 to-purple-50 border border-violet-100
                 rounded-xl p-4">
              <div className="w-8 h-8 bg-violet-500 rounded-lg flex items-center justify-center mb-3">
                <Wrench className="w-4 h-4 text-white" />
              </div>
              <p className="text-2xl font-extrabold text-violet-700"> {toolList?.length ?? "—"}</p>
              <p className="text-xs font-semibold text-violet-600 mt-0.5">Registered Tools</p>
            </div>
            {/* Admins */}
            <div className="bg-gradient-to-br from-emerald-50 to-teal-50 border border-emerald-100 rounded-xl
                 p-4">
              <div className="w-8 h-8 bg-emerald-500 rounded-lg flex items-center justify-center mb-3">
                <ShieldCheck className="w-4 h-4 text-white" />
              </div>
              <p className="text-2xl font-extrabold text-emerald-700">
                {allUsers?.filter( (u) => u.role === "admin" || u.role === "superadmin" ).length ?? "—"}
              </p>
              <p className="text-xs font-semibold text-emerald-600 mt-0.5">Admins</p>
            </div>
          </div>
          {/* User role breakdown */}
          <div className="mt-5">
            <p className="text-xs text-gray-400 uppercase font-semibold tracking-wide mb-3"> Role
Breakdown</p>
            <div className="space-y-2">
              {["bugreporter", "tester", "dev"].map((r) => {
                const count =allUsers?.filter((u) =>Array.isArray(u.roletype)? u.roletype.includes(r)
                      : u.roletype === r).length ?? 0;
                const pct = allUsers?.length? Math.round((count / allUsers.length) * 100): 0;
                const colors = {
                  bugreporter: { bar: "bg-blue-500",    text: "text-blue-700",    bg: "bg-blue-50" },
                  tester:      { bar: "bg-indigo-500",  text: "text-indigo-700",  bg: "bg-indigo-50" },
                  dev:         { bar: "bg-emerald-500", text: "text-emerald-700", bg: "bg-emerald-50" },
                };
                const c = colors[r];
                return (
                  <div key={r} className="flex items-center gap-3">
                    <span className={`text-xs font-semibold w-28 ${c.text}`}>{roleLabel(r)} </span>
                    <div className="flex-1 h-2 bg-gray-100 rounded-full overflow-hidden">
                      <div  className={`h-full rounded-full ${c.bar} transition-all duration-700`}
                        style={{ width: `${pct}%` }}/>
```

```
                  </div>
                  <span className="text-xs font-bold text-gray-500 w-8 text-right">
                    {count}
                  </span>
                </div>
              );
            })}
          </div>
        </div>
      </div>
    )}
    {/* Non-admin notice */}
    {!isAdmin && (
      <div className="bg-amber-50 border border-amber-200 rounded-2xl p-5 flex items-start gap-3">
        <AlertTriangle className="w-5 h-5 text-amber-500 flex-shrink-0 mt-0.5" />
        <div>
          <p className="text-sm font-semibold text-amber-800"> Admin access required</p>
          <p className="text-xs text-amber-600 mt-0.5">
            System overview and admin controls are only visible to Admin and
            Super Admin accounts.
          </p>
        </div>
      </div>
    )}
  </div>
 );
}
export default AdminControl;
```

General.jsx

```
import React, { useState, useEffect } from "react";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Button } from "@/components/ui/button";
import { useAuth } from "@/context/AuthContext";
import { updateUser } from "@/API_Call/User";
import toast from "react-hot-toast";
import { Pencil, Check, Loader2,} from "lucide-react";
function getRoleLabel(user) {
 if (!user) return "—";
 const roleArr = Array.isArray(user.roletype) ? user.roletype : [user.roletype || "bugreporter"];
 const labelMap = {   bugreporter: "Bug Reporter",   tester: "Tester",   dev: "Developer",
   admin: "Admin",   superadmin: "Super Admin", };  return roleArr.map((r) => labelMap[r] || r).join(", ");
}
function General() {
 const { user, setUser } = useAuth();
 const [isEditing, setIsEditing] = useState(false);
 const [saving, setSaving] = useState(false);
 const [form, setForm] = useState({ name: "", username: "", phone: "" });
 useEffect(() => {
   if (user) {
     setForm({ name: user.name || "", username: user.username || "",  phone: user.phone || user.mobile ||
"",});
```

```
  }
}, [user]);
const handleSave = async () => {
  setSaving(true);
  try {
    const res = await updateUser(user.id || user._id, form);
    if (res.success) {
      setUser({ ...user, ...form }); toast.success("Profile updated");  setIsEditing(false);
    } else toast.error(res.message);
  } catch (err) { toast.error("Error saving profile");
  } finally {setSaving(false);  }
};
return (
  <div className="max-w-2xl mx-auto py-4">
    <div className="flex items-center justify-between mb-8">
      <div>
        <h2 className="text-2xl font-bold text-gray-900">Profile Information</h2>
        <p className="text-sm text-gray-500 mt-1">Update your account details and how others see you.</p>
      </div>
      {!isEditing ? (
        <Button variant="outline" onClick={() => setIsEditing(true)} className="gap-2">
          <Pencil size={16} /> Edit
        </Button>
      ) : (
        <div className="flex gap-2">
          <Button variant="ghost" onClick={() => setIsEditing(false)} disabled={saving}>Cancel</Button>
          <Button onClick={handleSave} disabled={saving} className="bg-cyan-600 hover:bg-cyan-700">
            {saving ? <Loader2 size={16} className="animate-spin" /> : <Check size={16} />} Save
          </Button>
        </div>
      )}
    </div>
    <div className="space-y-6">
      <div className="grid grid-cols-1 gap-6 p-6 border border-gray-100 rounded-2xl bg-gray-50/30">
        <div className="space-y-2">
          <Label className="text-gray-500">Full Name</Label>
          {isEditing ? (
            <Input value={form.name} onChange={e => setForm({...form, name: e.target.value})}
              className="bg-white" />
          ) : (
            <p className="text-base font-medium text-gray-900 px-3 py-2 bg-white border border-gray-100
              rounded-lg">{user?.name || "—"}</p>
          )}
        </div>
        <div className="grid grid-cols-1 md:grid-cols-2 gap-6">
          <div className="space-y-2">
            <Label className="text-gray-500">Username</Label>
            {isEditing ? (
              <Input value={form.username} onChange={e => setForm({...form, username: e.target.value})}
               className="bg-white" />
            ) : (
              <p className="text-base font-medium text-gray-900 px-3 py-2 bg-white border border-gray-100
               rounded-lg">@{user?.username}</p>
```

```
            )}
          </div>
          <div className="space-y-2">
            <Label className="text-gray-500">Phone</Label>
            {isEditing ? (
              <Input value={form.phone} onChange={e => setForm({...form, phone: e.target.value})}
               className="bg-white" />
            ) : (
              <p className="text-base font-medium text-gray-900 px-3 py-2 bg-white border border-gray-100
              rounded-lg">{user?.phone || user?.mobile || "—"}</p>
            )}
          </div>
        </div>
        <div className="space-y-2">
          <Label className="text-gray-500">Email Address</Label>
          <p className="text-base font-medium text-gray-400 px-3 py-2 bg-gray-100/50 border border-dashed
               border-gray-200 rounded-lg cursor-not-allowed">{user?.email}
          </p>
        </div>
        <div className="pt-4 border-t border-gray-100">
          <div className="flex items-center gap-3">
            <div className="p-2 bg-cyan-50 text-cyan-600 rounded-lg"> <Shield size={20} /></div>
            <div>
              <p className="text-xs font-bold text-gray-400 uppercase tracking-wider">Account Role</p>
              <p className="text-sm font-semibold text-gray-700">{getRoleLabel(user)}</p>
            </div>
          </div>
        </div>
      </div>
    </div>
  </div>
 );
}

export default General;
```

Password.jsx

```
import React, { useState } from "react";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Button } from "@/components/ui/button";
import { useAuth } from "@/context/AuthContext";
import { changePassword } from "@/API_Call/User";
import toast from "react-hot-toast";
import { KeyRound, Eye, EyeOff, Lock, ShieldCheck, Loader2, CheckCircle2, XCircle,} from "lucide-react";
function StrengthBar({ password }) {
 const checks = [
   { label: "At least 6 characters",    pass: password.length >= 6 },
   { label: "Contains uppercase letter", pass: /[A-Z]/.test(password) },
   { label: "Contains number",           pass: /[0-9]/.test(password) },
   { label: "Contains special character",pass: /[^A-Za-z0-9]/.test(password) },
 ];
 const score = checks.filter((c) => c.pass).length;
```

```
 const colors = ["bg-red-400", "bg-orange-400", "bg-yellow-400", "bg-emerald-400"];
 const labels = ["Weak", "Fair", "Good", "Strong"];
 if (!password) return null;
 return (
   <div className="mt-3 space-y-2">
     {/* Bar */}
     <div className="flex gap-1">
       {[0, 1, 2, 3].map((i) => (
         <div key={i} className={`h-1.5 flex-1 rounded-full transition-all duration-300 ${
             i < score ? colors[score - 1] : "bg-gray-200"}`}/>
       ))}
     </div>
     <p className={`text-xs font-semibold ${score < 2 ? "text-red-500" : score < 4 ? "text-yellow-600"
        : "text-emerald-600"}`}>{labels[score - 1] || "Too weak"}
     </p>
     {/* Checklist */}
     <ul className="space-y-1 pt-1">
       {checks.map((c) => (
         <li key={c.label} className="flex items-center gap-2">
           {c.pass ? <CheckCircle2 className="w-3.5 h-3.5 text-emerald-500 flex-shrink-0" />
             : <XCircle className="w-3.5 h-3.5 text-gray-300 flex-shrink-0" />}
           <span className={`text-xs ${c.pass ? "text-emerald-600" : "text-gray-400"}`}>{c.label}</span>
         </li>
       ))}
     </ul>
   </div>
 );
}
function PasswordInput({ id, label, value, onChange, placeholder, icon: Icon }) {
 const [show, setShow] = useState(false);
 return (
   <div className="space-y-1.5">
     <Label htmlFor={id} className="text-xs text-gray-400 uppercase font-semibold tracking-wide flex
        items-center gap-1.5"> {Icon && <Icon className="w-3.5 h-3.5" />}{label}
     </Label>
     <div className="relative">
       <Input id={id} type={show ? "text" : "password"} value={value} onChange={onChange}
        placeholder={placeholder} autoComplete="off"
        className="bg-gray-50 border-gray-200 pr-10 focus:border-cyan-400 focus:ring-cyan-100 transition-all"
/>
       <button type="button" onClick={() => setShow((s) => !s)}
         className="absolute right-3 top-1/2 -translate-y-1/2 text-gray-400 hover:text-gray-600
          transition-colors" tabIndex={-1} >
         {show ? <EyeOff className="w-4 h-4" /> : <Eye className="w-4 h-4" />}
       </button>
     </div>
   </div>
 );
}
function PasswordControl() {
 const { user } = useAuth();
 const [form, setForm] = useState({ old: "", new: "", confirm: "" });
 const [saving, setSaving] = useState(false);
```

```
const set = (key) => (e) => setForm((p) => ({ ...p, [key]: e.target.value }));
const newPasswordStrong = form.new.length >= 6;
const passwordsMatch   = form.new === form.confirm;
const canSubmit        = form.old && form.new && form.confirm && newPasswordStrong && passwordsMatch;
const handleSubmit = async (e) => {
  e.preventDefault();
  if (!passwordsMatch) {     toast.error("New passwords do not match");     return;}
  if (!newPasswordStrong) {  toast.error("New password must be at least 6 characters"); return;}
  const userId = user?._id || user?.id;
  if (!userId) { toast.error("User session not found"); return; }
  setSaving(true);
  try {
    const res = await changePassword(userId, { oldPassword: form.old, newPassword: form.new, });
    if (res.success) {
      toast.success("Password changed successfully");
      setForm({ old: "", new: "", confirm: "" });
    } else { toast.error(res.message || "Failed to change password"); }
  } catch { toast.error("Something went wrong");
  } finally {setSaving(false);}
};

return (
  <div className="space-y-5">
    {/* ── Header card ── */}
    <div className="bg-white rounded-2xl border border-gray-100 shadow-sm p-6">
      <div className="flex items-center gap-3 mb-1">
        <div className="w-10 h-10 rounded-xl bg-gradient-to-br from-cyan-500 to-indigo-600 flex items-center
justify-center shadow-md">
          <KeyRound className="w-5 h-5 text-white" />
        </div>
        <div>
          <h3 className="text-base font-bold text-gray-900">Change Password</h3>
          <p className="text-xs text-gray-400 mt-0.5">
            You must enter your current password before setting a new one.
          </p>
        </div>
      </div>
    </div>
    {/* ── Form card ── */}
    <form onSubmit={handleSubmit}>
      <div className="bg-white rounded-2xl border border-gray-100 shadow-sm p-6 space-y-5">
        {/* Current (old) password */}
        <div className="pb-5 border-b border-gray-100">
          <PasswordInput id="old-password" label="Current Password" value={form.old}
            onChange={set("old")} placeholder="Enter your current password" icon={Lock} />
        </div>
        {/* New password */}
        <PasswordInput id="new-password" label="New Password"  value={form.new} onChange={set("new")}
          placeholder="Enter a new password" icon={ShieldCheck} />
        <StrengthBar password={form.new} />
        {/* Confirm new password */}
        <PasswordInput  id="confirm-password" label="Confirm New Password" value={form.confirm}
          onChange={set("confirm")} placeholder="Re-enter your new password" icon={ShieldCheck}/>
```

```
        {/* Mismatch warning */}
        {form.confirm && !passwordsMatch && (
          <p className="flex items-center gap-1.5 text-xs text-red-500 font-medium">
            <XCircle className="w-3.5 h-3.5 flex-shrink-0" /> Passwords do not match
          </p>
        )}
        {form.confirm && passwordsMatch && form.confirm.length > 0 && (
          <p className="flex items-center gap-1.5 text-xs text-emerald-500 font-medium">
            <CheckCircle2 className="w-3.5 h-3.5 flex-shrink-0" /> Passwords match
          </p>
        )}
        {/* Submit */}
        <div className="pt-2 flex justify-end">
          <Button type="submit" disabled={!canSubmit || saving}
            className="h-10 px-6 gap-2 bg-cyan-600 hover:bg-cyan-700 text-white transition-all
             disabled:opacity-50">
            {saving ? ( <Loader2 className="w-4 h-4 animate-spin" />) : ( <KeyRound className="w-4 h-4" />)}
            {saving ? "Updating…" : "Update Password"}
          </Button>
        </div>
      </div>
    </form>
  </div>
 );
}
export default PasswordControl;
```

Phases (folder)

Phasel (folder)
Attachment.jsx

```
import { Accordion, AccordionContent, AccordionItem, AccordionTrigger,} from "@/components/ui/accordion";
import { Label } from "@radix-ui/react-dropdown-menu";
import { Input } from "@/components/ui/input";
import { Upload, X } from "lucide-react";
function Attachment({ selectedImage, setSelectedImage, selectedVideo, setSelectedVideo,}) {
 const handleImageUpload = (e) => { const file = e.target.files[0]; if (file) { setSelectedImage(file); } };
 const handleVideoUpload = (e) => { const file = e.target.files[0]; if (file) {setSelectedVideo(file);}};
 const removeImage = () => {   setSelectedImage(null); };
 const removeVideo = () => {   setSelectedVideo(null); };
 return (
   <Accordion type="single" collapsible>
     <AccordionItem value="item-1">
       <AccordionTrigger>Attachments/Url ?</AccordionTrigger>
       <AccordionContent className="space-y-4">
         <div className="grid grid-cols-2 gap-4">
           {/* Upload Image */}
           <div className="space-y-2">
             <Label className="text-sm font-medium">Upload Image</Label>
             <div className="border-2 border-dashed rounded-lg p-4 text-center hover:border-primary
                 transition-colors cursor-pointer">
               <input type="file" accept="image/*" onChange={handleImageUpload} className="hidden"
                 id="image-upload"/>
               <label htmlFor="image-upload" className="cursor-pointer">
```

```
                <Upload className="w-6 h-6 mx-auto mb-2 text-gray-400" />
                <p className="text-sm text-gray-600">Image</p>
              </label>
            </div>
            {selectedImage && (
              <div className="flex items-center justify-between p-2 bg-gray-50 rounded border">
                <span className="text-sm truncate">{selectedImage.name}</span>
                <button  onClick={removeImage}  className="text-red-500 hover:text-red-700" >
                  <X className="w-4 h-4" />
                </button>
              </div>
            )}
          </div>
          {/* Upload Video */}
          <div className="space-y-2">
            <Label className="text-sm font-medium">Upload Video</Label>
            <div className="border-2 border-dashed rounded-lg p-4 text-center hover:border-primary
                transition-colors cursor-pointer">
              <input type="file"  accept="video/*"  onChange={handleVideoUpload} className="hidden"
                id="video-upload" />
              <label htmlFor="video-upload" className="cursor-pointer">
                <Upload className="w-6 h-6 mx-auto mb-2 text-gray-400" />
                <p className="text-sm text-gray-600">Video</p>
              </label>
            </div>
            {selectedVideo && (
              <div className="flex items-center justify-between p-2 bg-gray-50 rounded border">
                <span className="text-sm truncate">{selectedVideo.name}</span>
                <button onClick={removeVideo} className="text-red-500 hover:text-red-700">
                  <X className="w-4 h-4" />
                </button>
              </div>
            )}
          </div>
        </div>
        <div className="space-y-2">
          <Label htmlFor="pageUrl" className="text-sm font-medium"> Page URL or Path</Label>
          <Input id="pageUrl" placeholder="/dashboard/profile"
            className="border-2 focus:border-primary rounded-lg"/>
        </div>
      </AccordionContent>
    </AccordionItem>
  </Accordion>
);
}
export default Attachment;
```

BugForm.jsx

```
// BugForm.jsx
import React, { useEffect, useState } from "react";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Textarea } from "@/components/ui/textarea";
import { Asterisk, Upload, Bug, Menu, Loader2 } from "lucide-react";
```

```
import SearchableToolDropdown from "@/components/Dropdowns/SearchableSelect";
import DynamicSelect from "@/components/Dropdowns/Dropdown";
import AnimatedCheckbox from "./Checkbox";
import SOPChecklist from "./SopChecks";
import FileUploader from "../../Shared/FileUploader";
import { SheetContent, SheetDescription, SheetHeader, SheetTitle, SheetFooter,} from "@/components/ui/sheet";
import { Button } from "@/components/ui/button";
import toast from "react-hot-toast";
import { createBug } from "@/API_Call/Bug";
import { useAuth } from "@/context/AuthContext";
function BugForm({ setIsOpen }) {
 const { toolList, allUsers, bugsList, setBugsList } = useAuth();
 const [errors, setErrors] = useState({});
 const [isSubmitting, setIsSubmitting] = useState(false);
 const [selectedTool, setSelectedTool] = useState("");
 const [selectedTester, setSelectedTester] = useState("");
 const [selectedPriority, setSelectedPriority] = useState("");
 const [tools, setTools] = useState([]);
 const [testers, setTesters] = useState([]);
 const [isLoadingTools, setIsLoadingTools] = useState(false);
 const [isLoadingTesters, setIsLoadingTesters] = useState(false);
 const [descriptionType, setDescriptionType] = useState(true);
 const [facedByMe, setFacedByMe] = useState(false);
 const [facedByClient, setFacedByClient] = useState(false);
 const [parentChecked, setParentChecked] = useState(false);
 const [sopChecks, setSopChecks] = useState([false, false, false, false]);
 const [attachments, setAttachments] = useState([]);
 const priorityOptions = [
   { value: "low", label: "Low" }, { value: "medium", label: "Medium" },
   { value: "high", label: "High" }, { value: "critical", label: "Critical" },
 ];
 const selectedToolData = tools.find(t => t.value === selectedTool);
 const sopList = selectedToolData?.sop ? selectedToolData.sop.split('\n').filter(s => s.trim() !== '') : [];
 useEffect(() => {
   setSopChecks(new Array(sopList.length || 0).fill(false)); setParentChecked(false);
 }, [selectedTool, tools]);
 // Clear facedBy error when either checkbox is checked
 useEffect(() => {
   if (facedByMe || facedByClient) {
     setErrors((prev) => {  const newErrors = { ...prev }; delete newErrors.facedBy; return newErrors; });
   }
 }, [facedByMe, facedByClient]);
 // Clear SOP error when any checkbox is checked
 useEffect(() => {
   if (sopChecks.some(Boolean)) {
     setErrors((prev) => { const newErrors = { ...prev };  delete newErrors.sop; return newErrors; });
   }
 }, [sopChecks]);
 useEffect(() => { loadTesters(); }, [allUsers, selectedTool, toolList]);
 // Sync Context tools to Dropdown format implicitly whenever context updates
 useEffect(() => {
   if (toolList && toolList.length > 0) {
     const formattedTools = toolList.map((tool) => ({
```

```
      value: tool.id || tool._id, label: tool.toolName,
      sublabel: tool.stack[0] || "No category", sop: tool.SOP || ""
    }));
    setTools(formattedTools);
  } else { setTools([]); }
}, [toolList]);
  const loadTesters = async () => {
  setIsLoadingTesters(true);
  try {
    if (allUsers && allUsers.length > 0) {
      let defaultTesterId = null;
      if (selectedTool && toolList) {
        const matchedTool = toolList.find(t => (t.id || t._id) === selectedTool);
        if (matchedTool && matchedTool.testerId) {
           defaultTesterId = typeof matchedTool.testerId === "object" ? (matchedTool.testerId._id ||
           matchedTool.testerId.id) : matchedTool.testerId;
        }
      }
      const actualTesters = allUsers
        .filter((u) => Array.isArray(u.roletype) ? u.roletype.includes("tester") : u.roletype === "tester")
        .map((tester) => ({ value: tester.id,  label: tester.id === defaultTesterId ?
              `${tester.name || tester.username || tester.email}
              (default)` : (tester.name || tester.username || tester.email),
        }));
      setTesters([ ...actualTesters ]);
    } else { setTesters([]);}
  } catch (error) { setTesters([]);
  } finally { setIsLoadingTesters(false);}
};
const checkValidation = () => {
  let newErrors = {}; let description = ""; let expectedResult = "";  let actualResult = "";
  let clientName = ""; let companyName = "";
  if (!selectedTool) newErrors.tool = "Tool Name is required.";
  if (!selectedTester) newErrors.tester = "Tester is required.";
  if (!selectedPriority) newErrors.priority = "Priority is required.";
  if (descriptionType) {
    const desc = document.getElementById("description")?.value;
    if (!desc?.trim()) { newErrors.description = "Description is required."; }
    else {  description = desc.trim();}
  } else {
    const expected = document.getElementById("expected_Result")?.value;
    const actual = document.getElementById("actual_Result")?.value;
    if (!expected?.trim()) { newErrors.expected = "Expected Result is required."; }
    else { expectedResult = expected.trim();}
    if (!actual?.trim()) { newErrors.actual = "Actual Result is required."; }
    else { actualResult = actual.trim();}
  }
  // Faced by validation
  if (!facedByMe && !facedByClient) { newErrors.facedBy = "Select who faced the issue.";}
  if (facedByClient) {
    const cname = document.getElementById("client_name")?.value;
    const comp = document.getElementById("company_name")?.value;
    if (!cname?.trim()) { newErrors.client_name = "Client Name is required.";}
```

```
      else { clientName = cname.trim();}
    if (!comp?.trim()) { newErrors.company_name = "Company Name is required.";}
      else {companyName = comp.trim(); }
  }
  // SOP Check validation
  if (sopList.length > 0 && !sopChecks.some(Boolean)){newErrors.sop = "SOP checklist items must be
checked.";}
  setErrors(newErrors);
  return { newErrors,        description, expectedResult, actualResult,clientName, companyName,};
};
const handleSubmit = async () => {
  const { newErrors, description, expectedResult, actualResult, clientName, companyName,} =
checkValidation();
// stop submit
  if (Object.keys(newErrors).length > 0) {  console.log("❌ Validation Failed:", newErrors);  return; }
  // START LOADING
  setIsSubmitting(true);
  // Prepare SOP data with questions
  const sopData = sopList.reduce((acc, question, index) => {
    acc[question] = sopChecks[index];
    return acc;
  }, {});
  const selectedItem = tools.find((t) => t.value === selectedTool);
  // Prepare complete data object for API formatted to match backend expectations
  const bugReportData = {
    toolInfo: {
      toolId: selectedTool,
      toolName: selectedItem?.label || "Unknown Tool",
      toolDescription: selectedItem?.sublabel || "Unknown Category",
      stack: selectedItem?.sublabel || "Unknown",
      priority: selectedPriority,
      // Map Description vs Expected/Actual cleanly for the backend
      bugDescription: descriptionType ? description : "Detailed Bug Report",
      expectedResult: descriptionType ? "Not Applicable (Simple Description)" : expectedResult,
      actualResult: descriptionType ? "Not Applicable (Simple Description)" : actualResult,
      attachments: attachments.map(f => ({  url: f.url, fileName: f.name,
        size: f.size,  uploadedAt: new Date()
      })),
    },
    tags: [],
    // Extra frontend context, ignored by the current backend schema but safe to pass
    clientContext: { facedByMe,facedByClient, clientName,  companyName,  sopChecklist: sopData,}
  };
 if (selectedTester && selectedTester !== "none"){ bugReportData.assignedTester = { userId: selectedTester
};}
  // Log to console
  console.log(" Bug Report Data Ready for API:");
  console.log(bugReportData);
  try {
    // API call execution
    const apiResponse = await createBug(bugReportData);
    // Real-time synchronization without refetch
    if (apiResponse.success && apiResponse.data && apiResponse.data.results) {
```

```
        const newBugs = apiResponse.data.results.filter(r => r.bug).map(r => r.bug);
        if (newBugs.length > 0) { setBugsList((prev) => [...prev, ...newBugs]);}
      }
      await new Promise((r) => setTimeout(r, 1500));
      // setIsOpen(false); // close sheet
    } catch (err) {
      console.error("API Submit Error:", err);
      toast.error(err.message || "Bug Report failed", { position: "top-center",});
    } finally {
      // STOP LOADING
      setIsSubmitting(false); toast.success("Bug Report successfully", { position: "top-center",});
      resetForm();
    }
  };
  const resetForm = () => { setErrors({}); setSelectedTool(""); setSelectedTester("");
setSelectedPriority("");
    // setDescriptionType(true);
    setFacedByMe(false); setFacedByClient(false);  setParentChecked(false);  setAttachments([]);
    setSopChecks(new Array(sopList.length || 0).fill(false));
      // also clear input boxes manually
    const simpleDesc = document.getElementById("description");
    if (simpleDesc) simpleDesc.value = "";
    const expected = document.getElementById("expected_Result");
    if (expected) expected.value = "";
    const actual = document.getElementById("actual_Result");
    if (actual) actual.value = "";
    const cname = document.getElementById("client_name");
    if (cname) cname.value = ""
    const comp = document.getElementById("company_name");
    if (comp) comp.value = "";
  };

  const handleCancel = () => {setIsOpen(false);};
  return (
    <SheetContent className="justify-between w-[96%] sm:!max-w-none sm:w-[550px] lg:w-[550px] overflow-y-auto
        rounded-xl sm:rounded-2xl border border-gray-300 ring-2 ring-gray-200 ring-offset-2 my-[3.2%]
        mx-[1.8%] sm:m-[1.2%] h-[97%]">
      <SheetHeader>
        <div className="flex items-center gap-3 pb-4 border-b">
          <div className="w-12 h-12 bg-red-500 rounded-lg flex items-center justify-center">
            <Bug className="w-6 h-6 text-white" />
          </div>
          <div>
            <h3 className="font-semibold text-lg">Bug Report</h3>
            <p className="text-[12px] sm:text-md text-gray-500">
              Found a problem? Let us know so we can fix it.
            </p>
          </div>
        </div>
        <div className="hidden">
          <SheetTitle>  <Bug className="w-6 h-6 text-white" /> Create New Bug </SheetTitle>
          <SheetDescription> Fill in the details below to report a new bug.   </SheetDescription>
        </div>
```

```
    </SheetHeader>
    {/* Bug Form Component */}
    <div className="flex flex-col justify-between h-[92%]">
      {/* bug form start here */}
      <div className="space-y-6 py-4">
        {/* Tool Name */}
        <div className="space-y-2">
          <Label htmlFor="tool" className="text-sm font-medium flex items-center">
            <Asterisk size={12} />Tool Name
            <span className="block text-gray-500 text-xs">(Category)</span>
          </Label>
          <SearchableToolDropdown value={selectedTool}
            onChange={(val) => { setSelectedTool(val);
              if (errors.tool) setErrors((prev) => ({ ...prev, tool: "" }));
              // Auto-assign tester if configured on the tool
              const matchedTool = toolList?.find(t => (t.id || t._id) === val);
              if (matchedTool && matchedTool.testerId) {
                const tId = typeof matchedTool.testerId === "object"
                   ? (matchedTool.testerId._id || matchedTool.testerId.id) : matchedTool.testerId;
                setSelectedTester(tId);
                if (errors.tester) setErrors((prev) => ({ ...prev, tester: "" }));
              } else {
                setSelectedTester("none");
                if (errors.tester) setErrors((prev) => ({ ...prev, tester: "" }));
              }
            }}
            placeholder="Select a tool" items={tools}  isLoading={isLoadingTools}
            classAdd={errors.tool ? "border-red-300" : ""}
          />
          {errors.tool && ( <p className="text-red-400 text-xs">{errors.tool}</p> )}
        </div>
        {/* Assign Tester */}
        <div className="grid grid-cols-2 gap-4">
          <div className="space-y-2">
            <Label htmlFor="tester" className="text-sm font-medium flex gap-0 items-center" >
              <Asterisk size={12} /> Assign Tester
            </Label>
            <SearchableToolDropdown value={selectedTester}
              onChange={(val) => {
                setSelectedTester(val);
                if (errors.tester) setErrors((prev) => ({ ...prev, tester: "" })); }}
              placeholder="Select tester" items={testers}
              isLoading={isLoadingTesters} classAdd={errors.tester ? "border-red-300" : ""}/>
            {errors.tester && ( <p className="text-red-400 text-xs">{errors.tester}</p>)}
          </div>
          <div className="space-y-2">
            <Label htmlFor="priority"className="flex text-sm font-medium items-center">
              <Asterisk size={12} /> Priority
            </Label>
            <DynamicSelect value={selectedPriority}  placeholder="Select priority"  items={priorityOptions}
              onChange={(val) => {
                setSelectedPriority(val);
                if (errors.priority)  setErrors((prev) => ({ ...prev, priority: "" }));}}
```

```
              className={`w-full border-2 !h-[40px] ${ errors.priority ? "border-red-300" : ""}`} />
            {errors.priority && (<p className="text-red-400 text-xs">{errors.priority}</p>)}
          </div>
        </div>

        {/* Bug Description */}
        <div className="space-y-2">
          <LabelhtmlFor="description" onClick={() => setDescriptionType(!descriptionType)}
            className="flex text-sm font-medium underline underline-offset-4 cursor-pointer items-center">
            <Asterisk size={12} />{descriptionType ? "Description" : "Expected Result"}
          </Label>
          {descriptionType ? (
            <>
              <Textarea id="description"  rows={3}
                placeholder="Describe the issue you encountered. Include steps to reproduce,
                     screen affected, and any error messages."
                className={`text-sm border-2 rounded-lg ${errors.description ? "border-red-300" : "" }`}
                onInput={() => {
                  if (errors.description) setErrors((prev) => ({ ...prev, description: "" }));}}  />
              {errors.description && (<p className="text-red-400 text-xs">{errors.description}</p>)}
            </>
          ) : (
            <div className="flex flex-col gap-2">
              <Textarea id="expected_Result"
                placeholder="Explain what should have happened. Example: The form should submit
                 successfully and display a confirmation message."
                rows={3} className={`text-sm border-2 rounded-lg ${errors.expected ? "border-red-300" : ""
}`}
                onInput={() => setErrors((p) => ({ ...p, expected: "", actual: "" })) }/>
              {errors.expected && ( <p className="text-red-400 text-xs">{errors.expected}</p>)}

              <Label htmlFor="actual_Result" className="flex text-sm font-medium items-center">
                <Asterisk size={12} />  Actual Result
              </Label>
              <Textarea id="actual_Result"
                placeholder="Explain what actually happened. Example: Clicking submit reloads the page
                   without saving data or showing any confirmation."  rows={3}
                className={`text-sm border-2 rounded-lg ${ errors.actual ? "border-red-300" : "" }`}
                onInput={() => setErrors((p) => ({ ...p, expected: "", actual: "" }))}/>
              {errors.actual && ( <p className="text-red-500 text-xs">{errors.actual}</p>)}
            </div>
          )}
        </div>
        {/* Attachments Section */}
        <div className="pt-2 pb-4 border-t border-gray-50 mt-4">
          <FileUploader files={attachments} onChange={setAttachments}  label="Bug Attachments" />
        </div>
        {/* Faced By Section */}
        <div className="space-y-2">
          <div className="flex justify-between mr-[2rem]">
            <Label htmlFor="facedBy"  className="flex text-sm font-medium items-center" >
              <Asterisk size={12} /> Issue Faced By?
            </Label>
```

```
            <div className="flex gap-4">
              <AnimatedCheckbox textSize="" checkedVal={facedByMe} setCheckedVal={setFacedByMe} title="Me"/>
              <AnimatedCheckbox textSize="" checkedVal={facedByClient}
                setCheckedVal={setFacedByClient}title="Client" />
            </div>
          </div>
          {errors.facedBy && ( <p className="text-red-400 text-xs">{errors.facedBy}</p>)}
        </div>
        {facedByClient && (
          <div className="grid grid-cols-2 gap-4">
            <div className="space-y-2">
              <Label htmlFor="client_name" className="text-sm font-medium"> Client Name </Label>
              <Input  id="client_name"  type="text"  placeholder="Client Name"
                className={`border-2 rounded-lg ${   errors.client_name ? "border-red-300" : ""  }`}
                onInput={() => {
                  if (errors.client_name)  setErrors((prev) => ({ ...prev, client_name: "" })); }} />
              {errors.client_name && ( <p className="text-red-400 text-xs">{errors.client_name}</p>  )}
            </div>
            <div className="space-y-2">
              <Label htmlFor="company_name" className="text-sm font-medium">Company Name</Label>
              <Input  id="company_name" type="text" placeholder="Company Name"
                className={`border-2 rounded-lg ${  errors.company_name ? "border-red-300" : "" }`}
                onInput={() => {
                  if (errors.company_name)  setErrors((prev) => ({ ...prev, company_name: "" })); }}/>
              {errors.company_name && ( <p className="text-red-400 text-xs">{errors.company_name}</p> )}
            </div>
          </div>
        )}
        <SOPChecklist checks={sopChecks}  setChecks={setSopChecks} sopError={errors.sop} sopList={sopList}
          parentChecked={parentChecked} setParentChecked={setParentChecked} />
      </div>
      {/* bug form end here */}
      <SheetFooter className="gap-3 py-2">
        <Button variant="outline" onClick={handleCancel}> Cancel</Button>
        <Button disabled={isSubmitting} onClick={handleSubmit}  // className="w-full">
          {isSubmitting ? (
            <div className="flex items-center gap-2">
              <Loader2 className="animate-spin w-4 h-4" />  Submitting...
            </div>
          ) : ("Submit Bug")}
        </Button>
      </SheetFooter>
    </div>
  </SheetContent>
);
}
export default BugForm;
```

Checkbox.jsx

```
"use client";
import { useId, useEffect, useRef } from "react";
import { motion, AnimatePresence, easeOut } from "motion/react";
import { Checkbox } from "@/components/ui/checkbox";
```

```
import { Label } from "@/components/ui/label";
const particleAnimation = (index) => {
 const angle = Math.random() * Math.PI * 2;
 const distance = 30 + Math.random() * 20;
 return {
   initial: { x: "50%", y: "50%", scale: 0, opacity: 0 },
   animate: {
     x: `calc(50% + ${Math.cos(angle) * distance}px)`,
     y: `calc(50% + ${Math.sin(angle) * distance}px)`,
     scale: [0, 1, 0], opacity: [0, 1, 0],
   },
   transition: { duration: 0.4, delay: index * 0.05, ease: easeOut },
 };
};
const ConfettiPiece = ({ index }) => {
 const colors = [   "#FF0000",   "#00FF00",   "#0000FF",   "#FFFF00",   "#FF00FF",   "#00FFFF", ];
 const color = colors[index % colors.length];
 return (
   <motion.div className="absolute size-1 rounded-full"  style={{ backgroundColor: color }}
     {...particleAnimation(index)} />
 );
};
const AnimatedCheckbox = ({checkedVal, setCheckedVal, indeterminate = false, title, textSize, errors,}) => {
 const id = useId();
 const checkboxRef = useRef(null);
 // Apply indeterminate state to checkbox DOM
 useEffect(() => {
   if (checkboxRef.current) {  checkboxRef.current.indeterminate = indeterminate;}
 }, [indeterminate]);
 return (
   <div className="relative flex items-center gap-2">
     <Checkbox  ref={checkboxRef} id={id}checked={checkedVal} onCheckedChange={(v) => setCheckedVal(!!v)}/>
     <Label htmlFor={id} className={textSize}>{title}</Label>
     <AnimatePresence>
       {checkedVal && (
         <div className="pointer-events-none absolute inset-0">
           {[...Array(12)].map((_, i) => (  <ConfettiPiece key={i} index={i} /> ))}
         </div>
       )}
     </AnimatePresence>
   </div>
 );
};
export default AnimatedCheckbox;
```

CreateBug.jsx

```
import React, { useState } from "react";
import { BadgePlus } from "lucide-react";
import MenuOption from "./MenuOption";
import BugForm from "./BugForm";
import BugViewToggle from "../Shared/BugViewToggle";
import { useAuth } from "@/context/AuthContext";
import { filterBugsForRaiser, getUserRoleFlags } from "@/utils/bugRoleFilter";
```

```
import { Sheet } from "@/components/ui/sheet";
function CreateBug() {
 const { bugsList, user } = useAuth();
 const [isOpen, setIsOpen] = useState(false);
 const { isAdmin, isBugRaiser } = getUserRoleFlags(user);
 const canCreate = isAdmin || isBugRaiser;
 const editableColumnKeys = (isAdmin || isBugRaiser)
   ? ["toolName", "priority", "stack", "description", "assignedTester", "attachments"] : ["currentPhase"];
 const rawPhaseBugs = bugsList   ? bugsList.filter((bug) => bug.currentPhase === "Bug Reported")    : [];
 // Show only bugs this user raised (admins see all)
 const phaseBugs = filterBugsForRaiser(rawPhaseBugs, user);
 return (
   <div>
     <BugViewToggle title="Create Bug" phaseBugs={phaseBugs} phaseId="create"
       editableColumnKeys={editableColumnKeys}
       actionButton={
         canCreate ? (
           <MenuOption onClick={() => setIsOpen(true)} title=" New Bug"
             icon={<BadgePlus className="h-4 w-4" />}/>
         ) : null
       }
     />
     <Sheet open={isOpen} onOpenChange={setIsOpen}> <BugForm setIsOpen={setIsOpen} /> </Sheet>
   </div>
 );
}
export default CreateBug;
```

MenuOption.jsx

```
"use client";
import * as React from "react";
import { Button } from "@/components/ui/button";
export default function ButtonGroupDemo({ onClick, title, icon }) {
 return ( <Button variant="outline" onClick={onClick} className="gap-2">{icon}{title}</Button>);
}
```

SopChecks.jsx

```
"use client";
import { useState } from "react";
import AnimatedCheckbox from "./Checkbox";
import { Accordion, AccordionContent, AccordionItem, AccordionTrigger,} from "@/components/ui/accordion";
import { Asterisk } from "lucide-react";
export default function SOPChecklist({ checks, setChecks, sopError, sopList,parentChecked,
setParentChecked,})
{
 const [parentIndeterminate, setParentIndeterminate] = useState(false);
 const [isOpen, setIsOpen] = useState(false); // ← Track accordion open/close
 const handleParentChange = (val) => {
   setParentChecked(val); setParentIndeterminate(false); setChecks(checks.map(() => val));
 };
 const handleChildChange = (index, val) => {
   const updated = [...checks]; updated[index] = val;
```

```
  setChecks(updated); const allChecked = updated.every(Boolean);
  const noneChecked = updated.every((v) => !v);
  if (allChecked) { setParentChecked(true); setParentIndeterminate(false);}
  else if (noneChecked) { setParentChecked(false);  setParentIndeterminate(false);}
  else {  setParentChecked(false);  setParentIndeterminate(true);}
};
return (
  <Accordion type="single" collapsible
    onValueChange={(val) => setIsOpen(val === "item-1")} // Track open/close>
    <AccordionItem value="item-1">
      <div className="flex justify-between items-center py-2">
        <label className="flex text-sm font-medium gap-3 items-center">
          <div className="flex items-center"> <Asterisk size={12} /> SOP Followed checkpoints </div>
          <AnimatedCheckbox textSize="text-xs" checkedVal={parentChecked}
            indeterminate={parentIndeterminate} setCheckedVal={handleParentChange} />
        </label>
        <AccordionTrigger className="flex gap-2"> {isOpen ? "Collapse" : "Expand"} </AccordionTrigger>
      </div>
      {sopError && ( <p className="text-red-400 text-xs !mt-[-10px] pb-4">{sopError}</p>)}
      <AccordionContent>
        <div className="grid grid-cols gap-4">
          <div className="grid grid-cols gap-[11px]">
            {sopList.map((text, index) => (
              <AnimatedCheckbox  key={index} title={text} textSize="text-xs"  checkedVal={checks[index]}
                setCheckedVal={(val) => handleChildChange(index, val)}  />
            ))}
          </div>
        </div>
      </AccordionContent>
    </AccordionItem>
  </Accordion>
);
}
```

PhaseII (folder)

BugTesting.jsx

```
import React from "react";
import BugViewToggle from "../Shared/BugViewToggle";
import { useAuth } from "@/context/AuthContext";
import { filterBugsForTester, getUserRoleFlags } from "@/utils/bugRoleFilter";
function BugTesting() {
 const { bugsList, user } = useAuth();
 const rawPhaseBugs = bugsList ? bugsList
       .filter( (bug) =>bug.currentPhase === "Bug Testing" || bug.currentPhase === "Bug Confirmation"): [];
 // Show only bugs assigned to this tester (admins see all)
 const phaseBugs = filterBugsForTester(rawPhaseBugs, user);
 const { isAdmin, isTester } = getUserRoleFlags(user);
 const editableColumnKeys = (isAdmin || isTester)
   ? ["bugStatus", "sopFollowed", "remarks", "testingAttachment", "assignedDev", "currentPhase",
"testingFlag", "delayedReason"] : ["currentPhase"];
 return (
   <div>
     <BugViewToggle title="Bug Testing"  phaseBugs={phaseBugs} phaseId="testing"
```

```
        editableColumnKeys={editableColumnKeys} />
    </div>
  );
}
export default BugTesting;
```

PhaseIII (folder)
BugAnalyze.jsx

```
import React from "react";
import BugViewToggle from "../Shared/BugViewToggle";
import { useAuth } from "@/context/AuthContext";
import { filterBugsForDev, getUserRoleFlags } from "@/utils/bugRoleFilter";
function AnalyzeBug() {
 const { bugsList, user } = useAuth();
 const rawPhaseBugs = bugsList   ? bugsList.filter((bug) => bug.currentPhase === "Bug Analysis")    : [];
 // Show only bugs assigned to this developer (admins see all)
 const phaseBugs = filterBugsForDev(rawPhaseBugs, user);
 const { isAdmin, isDev } = getUserRoleFlags(user);
 const editableColumnKeys = (isAdmin || isDev)
   ? ["analyzeRemark", "sopForSolution", "delayedReason", "analyzeAttachment", "rootCause", "currentPhase"]
   : ["currentPhase"];
 return (
   <div className="flex flex-col gap-16 pb-10">
     <div>
       <BugViewToggle title="Analyze Bug" phaseBugs={phaseBugs} phaseId="analyze"
         editableColumnKeys={editableColumnKeys}/>
     </div>
   </div>
 );
}
export default AnalyzeBug;
```

PhaseIV (folder)

Maintenance.jsx

```
import React from "react";
import BugViewToggle from "../Shared/BugViewToggle";
import { useAuth } from "@/context/AuthContext";
import { filterBugsForDev, getUserRoleFlags } from "@/utils/bugRoleFilter";
function Maintenance() {
 const { bugsList, user } = useAuth();
 const rawPhaseBugs = bugsList   ? bugsList.filter((bug) => bug.currentPhase === "Maintenance")    : [];
 // Maintenance is typically handled by Developers
 const phaseBugs = filterBugsForDev(rawPhaseBugs, user);
 const { isAdmin, isDev } = getUserRoleFlags(user);
 const editableColumnKeys = (isAdmin || isDev)
   ? ["finalTestingRemark", "testingFlag", "currentPhase"]    : ["currentPhase"];
 return (
   <div className="flex flex-col gap-16 pb-10">
     <div>
       <BugViewToggle title="Maintenance"  phaseBugs={phaseBugs} phaseId="maintenance"
         editableColumnKeys={editableColumnKeys} />
     </div>
```

```
    </div>
 );
}
export default Maintenance;
```

Phase V (folder)
ReadyToTesting.jsx

```
import React from "react";
import BugViewToggle from "../Shared/BugViewToggle";
import { useAuth } from "@/context/AuthContext";
import { filterBugsForTester } from "@/utils/bugRoleFilter";
function ReadyToTesting() {
 const { bugsList, user } = useAuth();
 const rawPhaseBugs = bugsList ? bugsList.filter( (bug) => bug.currentPhase === "Ready to Test" ): [];
 // Show only bugs assigned to this tester (admins see all)
 const phaseBugs = filterBugsForTester(rawPhaseBugs, user);
 return (
   <div> <BugViewToggle title="Ready for Testing" phaseBugs={phaseBugs} phaseId="rtt" />  </div>
 );
}
export default ReadyToTesting;
```

PhaseVI (folder)

ReadyToDeploy.jsx

```
import React from "react";
import BugViewToggle from "../Shared/BugViewToggle";
import { useAuth } from "@/context/AuthContext";
import { filterBugsForDev } from "@/utils/bugRoleFilter";
function ReadyToDeploy() {
 const { bugsList, user } = useAuth();
 const rawPhaseBugs = bugsList
   ? bugsList.filter(
       (bug) => bug.currentPhase === "Ready to Deploy" ||bug.currentPhase === "Final Testing" ) : [];
 // Show only bugs assigned to this developer (admins see all)
 const phaseBugs = filterBugsForDev(rawPhaseBugs, user);
 return ( <div> <BugViewToggle title="Ready for Deployment" phaseBugs={phaseBugs} phaseId="rtd" /></div> );
}
export default ReadyToDeploy;
```

PhaseVII (folder)

Deployed.jsx

```
import React from "react";
import BugViewToggle from "../Shared/BugViewToggle";
import { useAuth } from "@/context/AuthContext";
function Deployed() {
 const { bugsList } = useAuth();
 const phaseBugs = bugsList ? bugsList.filter(bug => bug.currentPhase === "Closure") : [];
 return (
   <div> <BugViewToggle title="Deployed Bugs" phaseBugs={phaseBugs} phaseId="deployed" /></div>
 );
}
export default Deployed;
```

Shared (folder)

AllBugsStage.jsx

```
import React, { useState } from "react";
import BugViewToggle from "./BugViewToggle";
import { useAuth } from "@/context/AuthContext";
import { DropdownMenu, DropdownMenuContent, DropdownMenuItem, DropdownMenuTrigger,
} from "@/components/ui/dropdown-menu";
import { Button } from "@/components/ui/button";
import { ChevronDown, Filter, BadgePlus } from "lucide-react";
import { isUserInvolvedInBug, getUserRoleFlags } from "@/utils/bugRoleFilter";
import BugForm from "../PhaseI/BugForm";
import { Sheet } from "@/components/ui/sheet";
function getEditableColumnKeys(user) {
 if (!user) return [];
 if (user.role === "admin" || user.role === "superadmin") return null; // null = all editable
 const roleArr = Array.isArray(user.roletype) ? user.roletype : (user.roletype ? [user.roletype] : []);
 const isBugRaiser = roleArr.includes("bugreporter");
 const isTester    = roleArr.includes("tester");
 const isDev       = roleArr.includes("dev");
 const editable = new Set();
 if (isBugRaiser) {
   ["toolName", "priority", "stack", "issueFacedBy", "description", "sopFollowedRaiser", "assignedTester",
"attachments", "currentPhase"].forEach(k => editable.add(k));
 }
 if (isTester) {
   ["bugStatus", "sopFollowed", "remarks", "testingAttachment", "assignedDev", "delayedReason",
"currentPhase", "testingFlag"].forEach(k => editable.add(k));
 }
 if (isDev) {
   ["analyzeRemark", "sopForSolution", "delayedReason", "analyzeAttachment", "testingFlag",
"finalTestingRemark", "deploymentStatus", "deployRemark", "finalSop", "bugStatus", "currentPhase"]
     .forEach(k => editable.add(k));
 }
 // bugId, bugRaiser, reportedDate, rootCause are always read-only
 return [...editable];
}
export default function AllBugsStage() {
 const { bugsList, user } = useAuth();
 const [selectedPhase, setSelectedPhase] = useState("All Phases");
 const [showFullDetails, setShowFullDetails] = useState(false);
 const [isCreateOpen, setIsCreateOpen] = useState(false);
 const { isAdmin, isBugRaiser } = getUserRoleFlags(user);
 const canCreate = isAdmin || isBugRaiser;
 const editableColumnKeys = getEditableColumnKeys(user);
 // Row-level edit gate: user can only edit rows they are involved in.
 // Admins (editableColumnKeys === null) can edit any row.// null signals BugTable: all rows editable
 const canEditRow = editableColumnKeys === null ? null : (bug) => isUserInvolvedInBug(bug, user);
 const filterOptions = [   "All Phases",   "Bug Reported",   "Bug Testing",   "Bug Analysis",  "Maintenance"
   "Ready to Test",   "Ready to Deploy",   "Closure" ];
 let filteredBugs = bugsList || [];
 if (selectedPhase !== "All Phases") {
   filteredBugs = filteredBugs.filter(bug => {
     if (selectedPhase === "Ready to Test")   return bug.currentPhase === "Ready to Test"   ||
```

```
        bug.currentPhase === "Maintenance";
    if (selectedPhase === "Ready to Deploy") return bug.currentPhase === "Ready to Deploy" ||
        bug.currentPhase === "Final Testing";
    if (selectedPhase === "Bug Testing")     return bug.currentPhase === "Bug Testing"     ||
        bug.currentPhase === "Bug Confirmation";
    return bug.currentPhase === selectedPhase;
  });
}
return (
  <div className="w-full">
    <BugViewToggle title="All Bugs" phaseBugs={filteredBugs} phaseId={showFullDetails ? "all_full" : "all"}
      editableColumnKeys={editableColumnKeys} canEditRow={canEditRow}
      actionButton={
        <div className="flex items-center gap-3">
          <Button variant={showFullDetails ? "default" : "outline"}
             className="h-9 px-3 text-sm shadow-sm transition-all focus:ring-0"
             onClick={() => setShowFullDetails(!showFullDetails)} >
             {showFullDetails ? "Compact View" : "Full Details"}
          </Button>
          <DropdownMenu>
          <DropdownMenuTrigger asChild>
            <Button variant="outline" className="flex items-center gap-2 h-9 px-3 border-gray-200
                 shadow-sm transition-all hover:bg-gray-50 focus:ring-0">
              <Filter className="w-4 h-4 text-gray-500" />
              <span className="text-sm font-medium text-gray-700">{selectedPhase}</span>
              <ChevronDown className="w-4 h-4 text-gray-400" />
            </Button>
          </DropdownMenuTrigger>
          <DropdownMenuContent align="end" className="w-48 z-[100] shadow-lg border-gray-100 rounded-xl">
            {filterOptions.map(opt => (
              <DropdownMenuItem key={opt} onClick={() => setSelectedPhase(opt)}
                 className={`cursor-pointer rounded-lg mx-1 my-0.5 px-3 py-2 text-sm
                 ${selectedPhase === opt ? "bg-blue-50 font-semibold text-blue-700"
                 : "text-gray-700 hover:bg-gray-50 hover:text-gray-900"}`} >
                 {opt}
              </DropdownMenuItem>
            ))}
          </DropdownMenuContent>
        </DropdownMenu>
        {canCreate && (
           <Button onClick={() => setIsCreateOpen(true)}
             className="h-9 px-4 bg-cyan-600 hover:bg-cyan-700 text-white font-semibold rounded-lg
                  shadow-md transition-all flex items-center gap-2 group">
             <BadgePlus className="w-4 h-4 transition-transform group-hover:scale-110" />
             <span className="hidden sm:inline">New Bug</span>
           </Button>
        )}
        </div>
      }
    />
    <Sheet open={isCreateOpen} onOpenChange={setIsCreateOpen}>
      <BugForm setIsOpen={setIsCreateOpen} />
    </Sheet>
```

```
    </div>
  );
}
```

BugModal.jsx

```
import React, { useState, useEffect } from "react";
import { Dialog, DialogContent, DialogHeader, DialogTitle } from "@/components/ui/dialog";
import { Button } from "@/components/ui/button";
import { ChevronLeft, ChevronRight, X } from "lucide-react";
export default function BugModal({ isOpen, onClose, bugData, phaseBugs, onNavigate}) {
 const [currentIndex, setCurrentIndex] = useState(-1);
 useEffect(() => {
   if (bugData && phaseBugs) {
     const idx = phaseBugs.findIndex(b => b._id === bugData._id || b.bugId === bugData.bugId);
     setCurrentIndex(idx);
   }
 }, [bugData, phaseBugs]);
 const handlePrev = () => { if (currentIndex > 0) { onNavigate(phaseBugs[currentIndex - 1]);} };
 const handleNext = () => {
   if (currentIndex < phaseBugs.length - 1) { onNavigate(phaseBugs[currentIndex + 1]); }
 };
 if (!bugData) return null;
 const toolName = bugData.phaseI_BugReport?.toolInfo?.toolName || "-";
 const priority = bugData.phaseI_BugReport?.toolInfo?.priority || "low";
 const desc = bugData.phaseI_BugReport?.toolInfo?.bugDescription || "No description provided";
 const bugId = bugData.bugId || "N/A";
  const raiser = bugData.phaseI_BugReport?.reportedBy?.name || "Unknown";
 const tester = bugData.phaseI_BugReport?.assignedTester?.name || "Unassigned";
 const developer = bugData.phaseII_BugConfirmation?.assignedDeveloper?.name || "Unassigned";
 return (
   <Dialog open={isOpen} onOpenChange={(open) => !open && onClose()}>
     <DialogContent className="max-w-3xl p-0 overflow-hidden bg-slate-50 flex flex-col h-[85vh]">
       {/* Header Ribbon */}
       <div className="flex items-center justify-between px-6 py-4 bg-white border-b sticky top-0 z-10">
         <div className="flex items-center gap-4">
           <div className={`px-3 py-1.5 rounded-full text-[11px] font-bold tracking-wider uppercase
             ${priority === 'critical' ? 'bg-red-100 text-red-700' :
               priority === 'high' ? 'bg-orange-100 text-orange-700' :
               priority === 'medium' ? 'bg-yellow-100 text-yellow-700' : 'bg-green-100 text-green-700' }`}>
               {priority}
           </div>
           <span className="text-sm font-bold text-gray-400">{bugId}</span>
         </div>
         <div className="flex items-center gap-1.5">
            <Button variant="outline"size="sm" onClick={handlePrev} disabled={currentIndex <= 0}
              className="h-8 px-2.5 text-gray-500 hover:text-gray-900" >
              <ChevronLeft className="w-4 h-4 mr-1" />
              Prev
            </Button>
            <span className="text-xs font-semibold text-gray-400 px-2">
              {currentIndex + 1} of {phaseBugs.length}
            </span>
            <Button variant="outline" size="sm" disabled={currentIndex >= phaseBugs.length - 1}
              className="h-8 px-2.5 text-gray-500 hover:text-gray-900"  onClick={handleNext} >
```

```
              Next<ChevronRight className="w-4 h-4 ml-1" />
            </Button>
          </div>
        </div>
        {/* Body Content */}
        <div className="p-6 overflow-y-auto flex-1">
          <h2 className="text-2xl font-black text-gray-900 mb-6">{toolName}</h2>
          <div className="bg-white rounded-2xl p-5 border shadow-sm mb-6">
            <h3 className="text-xs font-bold text-gray-400 uppercase tracking-widest mb-3">Description</h3>
            <p className="text-sm text-gray-700 leading-relaxed whitespace-pre-wrap">{desc}</p>
          </div>
          <div className="grid grid-cols-1 md:grid-cols-3 gap-4 mb-6">
            <div className="bg-white rounded-xl p-4 border shadow-sm flex flex-col items-start">
              <span className="text-[10px] font-bold text-gray-400 uppercase tracking-widest mb-1">
               Bug Raiser</span>
              <span className="text-sm font-semibold text-gray-800">{raiser}</span>
            </div>
            <div className="bg-white rounded-xl p-4 border shadow-sm flex flex-col items-start">
              <span className="text-[10px] font-bold text-gray-400 uppercase tracking-widest mb-1">
                Assigned Tester</span>
              <span className="text-sm font-semibold text-gray-800">{tester}</span>
            </div>
            <div className="bg-white rounded-xl p-4 border shadow-sm flex flex-col items-start">
              <span className="text-[10px] font-bold text-gray-400 uppercase tracking-widest mb-1">
                Assigned Dev</span>
              <span className="text-sm font-semibold text-gray-800">{developer}</span>
            </div>
          </div>
          {/* Results Section - Only show if not N/A */}
          {(bugData.phaseI_BugReport?.toolInfo?.expectedResult !== "Not Applicable (Simple Description)" ||
            bugData.phaseI_BugReport?.toolInfo?.actualResult !== "Not Applicable (Simple Description)") && (
            <div className={`grid gap-4 ${
              (bugData.phaseI_BugReport?.toolInfo?.expectedResult !== "Not Applicable (Simple Description)" &&
               bugData.phaseI_BugReport?.toolInfo?.actualResult !== "Not Applicable (Simple Description)")
               ? "grid-cols-1 md:grid-cols-2" : "grid-cols-1"}`}>
               {bugData.phaseI_BugReport?.toolInfo?.expectedResult !== "Not Applicable (Simple Description)"
&&
                (
                 <div className="bg-white rounded-2xl p-5 border shadow-sm">
                   <h3 className="text-xs font-bold text-gray-400 uppercase tracking-widest mb-3">
                   Expected Result</h3>
                   <p className="text-sm text-gray-700">{bugData.phaseI_BugReport?.toolInfo?.expectedResult ||
                    "-"}</p>
                 </div>
               )}
               {bugData.phaseI_BugReport?.toolInfo?.actualResult !== "Not Applicable (Simple Description)"
             && (
                 <div className="bg-white rounded-2xl p-5 border shadow-sm">
                   <h3 className="text-xs font-bold text-gray-400 uppercase tracking-widest mb-3">
                   Actual Result</h3>
                   <p className="text-sm text-gray-700">{bugData.phaseI_BugReport?.toolInfo?.actualResult ||
                     "-"}</p>
                 </div>
```

```
              )}
            </div>
          )}
          {/* Add Forms here to actually update the bug states going forward! */}
        </div>
        {/* Footer */}
        <div className="p-4 bg-white border-t flex justify-end gap-3">
          <Button variant="outline" onClick={onClose}>Close</Button>
          <Button>Save Changes</Button>
        </div>
      </DialogContent>
    </Dialog>
  );
}
```

BugRightClickMenu.jsx

```
import { ContextMenu, ContextMenuContent, ContextMenuItem, ContextMenuSeparator, ContextMenuTrigger,
} from "@/components/ui/context-menu";
import { Edit2, Trash2, Archive, CheckSquare, Settings } from "lucide-react";
import { useAuth } from "@/context/AuthContext";
export default function BugRightClickMenu({ children, item, onEdit, onDelete, onSaveAll, onEditSelected,
 onDeleteSelected, onCancelAll,}) {
 const { user } = useAuth();
 const isAdmin = user?.role === "admin" || user?.role === "superadmin"
 return (
   <ContextMenu>
     <ContextMenuTrigger asChild>{children}</ContextMenuTrigger>
     <ContextMenuContent className="w-56">
       <ContextMenuItem onClick={() => navigator.clipboard.writeText(item.bugId)}>
         <CheckSquare className="w-4 h-4 mr-2 text-slate-400" /> Copy Bug ID
       </ContextMenuItem>
       <ContextMenuItem onClick={onEdit}> <Edit2 className="w-4 h-4 mr-2" />Edit Bug</ContextMenuItem>
       {isAdmin && (
         <>
           <ContextMenuItem onClick={() => onArchive?.(item)}><Archive className="w-4 h-4 mr-2" />Archive Bug
           </ContextMenuItem>
           <ContextMenuItem className="text-red-600" onClick={onDelete}>
             <Trash2 className="w-4 h-4 mr-2" />Delete Bug
           </ContextMenuItem>
         </>
       )}
       <ContextMenuSeparator />
       <div className="px-2 py-1.5 text-xs font-semibold text-gray-500">Bulk Actions</div>
       <ContextMenuItem onClick={onEditSelected}> <CheckSquare className="w-4 h-4 mr-2" /> Edit Selected
       </ContextMenuItem>
       {isAdmin && (
         <ContextMenuItem className="text-red-600" onClick={onDeleteSelected}>
           <Trash2 className="w-4 h-4 mr-2" /> Delete Selected
         </ContextMenuItem>
       )}
       <ContextMenuSeparator />
       <ContextMenuItem onClick={onSaveAll} className="text-green-600">
         <Settings className="w-4 h-4 mr-2" /> Save All Changes
```

```
      </ContextMenuItem>
      <ContextMenuItem onClick={onCancelAll}><Archive className="w-4 h-4 mr-2" /> Discard Chages
      </ContextMenuItem>
    </ContextMenuContent>
  </ContextMenu>
);
}
```

BugTable.jsx

```
import React, { useState, useRef, useEffect } from "react";
import { ArrowUp, ArrowDown, Pin, ChevronLeft, ChevronRight, ChevronDown, Flag, Check, X, LibraryBig } from
"lucide-react";
import { useNavigate } from "react-router-dom";
import FileUploader from "../../Shared/FileUploader";
import AttachmentGallery from "../../Shared/AttachmentGallery";
import { Button } from "@/components/ui/button";
import { Checkbox } from "@/components/ui/checkbox";
import { Input } from "@/components/ui/input";
import { DropdownMenu, DropdownMenuContent, DropdownMenuItem, DropdownMenuTrigger, DropdownMenuCheckboxItem,
} from "@/components/ui/dropdown-menu";
import { Table, TableBody, TableCell, TableHead, TableHeader, TableRow,} from "@/components/ui/table";
import BugRightClickMenu from "./BugRightClickMenu";
import BugTableRowMenu from "./BugTableRowMenu";
import { updateBug, deleteBug } from "@/API_Call/Bug";
import { useAuth } from "@/context/AuthContext";
import toast from "react-hot-toast";
import { Tooltip, TooltipContent, TooltipTrigger } from "@/components/ui/tooltip";
/* ———————————— Tooltip Helper ———————————— */
const CellTooltip = ({ children, content }) => {
 if (!content) return children;
 return (
   <Tooltip delayDuration={300}>
     <TooltipTrigger asChild>{children}</TooltipTrigger>
     <TooltipContent className="max-w-[300px] break-words">{content} </TooltipContent>
   </Tooltip>
 );
};
// Returns column definitions based on the current phase
const getTableColumns = (phaseId) => {
 const commonEnd = [
   { key: "reportedDate", label: "Reported", type: "date", width: 160 },
   { key: "currentPhase", label: "Change Phase", type: "phaseSelect", width: 150 },
 ];
  if (phaseId === "create") {
   return [
     { key: "bugId", label: "Bug ID", type: "text", width: 140 },
     { key: "toolName", label: "Tool Name", type: "text", width: 180 },
     { key: "priority", label: "Priority", type: "prioritySelect", width: 120 },
     { key: "stack", label: "Stack", type: "stackSelect", width: 120 },
     { key: "bugRaiser", label: "Bug Raiser", type: "text", width: 150 },
     { key: "issueFacedBy", label: "Issue Faced By", type: "facedByCard", width: 160 },
     { key: "assignedTester", label: "Assigned Tester", type: "text", width: 150 },
     { key: "description", label: "Bug Description", type: "description", width: 250 },
```

```
    { key: "sopFollowedRaiser", label: "SOP Followed by Raiser", type: "sopFollowedRaiserCard", width: 220
},
    { key: "attachments", label: "Bug Attachments", type: "attachment", width: 150 },
    ...commonEnd
  ];
}
 if (phaseId === "testing") {
  return [
    { key: "bugId", label: "Bug ID", type: "text", width: 140 },
    { key: "toolName", label: "Tool Name", type: "text", width: 180 },
    { key: "priority", label: "Priority", type: "prioritySelect", width: 120 },
    { key: "bugRaiser", label: "Bug Raiser", type: "text", width: 150 },
    { key: "issueFacedBy", label: "Issue Faced By", type: "facedByCard", width: 160 },
    { key: "description", label: "Bug Description", type: "description", width: 250 },
    { key: "sopFollowedRaiser", label: "SOP Followed by Raiser", type: "sopFollowedRaiserCard", width: 220
},
    { key: "attachments", label: "Bug Attachments", type: "attachment", width: 150 },
    { key: "bugStatus", label: "Bug Status", type: "statusSelect", width: 150 },
    { key: "sopFollowed", label: "SOP Followed", type: "sopText", width: 160 },
    { key: "remarks", label: "Testing Remark", type: "remarksText", width: 200 },
    { key: "testingAttachment", label: "Testing Attachment", type: "testingAttachment", width: 140 },
    { key: "assignedDev", label: "Forward to Developer", type: "devSelect", width: 180 },
    ...commonEnd
  ];
}
if (phaseId === "analyze") {
  return [
    { key: "bugId", label: "Bug ID", type: "text", width: 140 },
    { key: "toolName", label: "Tool Name", type: "text", width: 180 },
    { key: "priority", label: "Priority", type: "prioritySelect", width: 120 },
    { key: "bugRaiser", label: "Bug Raiser", type: "text", width: 150 },
    { key: "issueFacedBy", label: "Issue Faced By", type: "facedByCard", width: 160 },
    { key: "assignedTester", label: "Bug Tester", type: "text", width: 150 },
    { key: "description", label: "Bug Description", type: "description", width: 250 },
    { key: "sopFollowedRaiser", label: "SOP Followed by Raiser", type: "sopFollowedRaiserCard", width: 220
},
    { key: "attachments", label: "Bug Attachments", type: "attachment", width: 150 },
    { key: "sopFollowed", label: "SOP Followed", type: "sopText", width: 160 },
    { key: "remarks", label: "Testing Remark", type: "remarksText", width: 200 },
    { key: "testingAttachment", label: "Testing Attachment", type: "testingAttachment", width: 140 },
    { key: "assignedDev", label: "Forward to Developer", type: "devSelect", width: 180 },
    { key: "bugStatus", label: "Bug Status", type: "statusSelect", width: 150 },
    { key: "analyzeRemark", label: "Analyse Remark", type: "analyzeRemarkText", width: 200 },
    { key: "sopForSolution", label: "SOP for Solution", type: "sopSolutionText", width: 200 },
    { key: "delayedReason", label: "Delayed Reason", type: "delayedReasonText", width: 200 },
    { key: "analyzeAttachment", label: "Analyse Attachment", type: "analyzeAttachment", width: 140 },
    { key: "reportedDate", label: "Reported", type: "date", width: 180 },
    { key: "bugTestedDate", label: "Bug Tested", type: "date", width: 180 },
    { key: "currentPhase", label: "Change Phase", type: "phaseSelect", width: 150 }
  ];
}
if (phaseId === "reanalyze") {
  return [
```

```
      { key: "bugId", label: "Bug ID", type: "text", width: 140 },
      { key: "toolName", label: "Tool Name", type: "text", width: 180 },
      { key: "priority", label: "Priority", type: "prioritySelect", width: 120 },
      { key: "bugRaiser", label: "Bug Raiser", type: "text", width: 150 },
      { key: "issueFacedBy", label: "Issue Faced By", type: "facedByCard", width: 160 },
      { key: "assignedTester", label: "Bug Tester", type: "text", width: 150 },
      { key: "description", label: "Bug Description", type: "description", width: 250 },
      { key: "sopFollowedRaiser", label: "SOP Followed by Raiser", type: "sopFollowedRaiserCard", width: 220
},
      { key: "attachments", label: "Bug Attachments", type: "attachment", width: 150 },
      { key: "sopFollowed", label: "SOP Followed", type: "sopText", width: 160 },
      { key: "remarks", label: "Testing Remark", type: "remarksText", width: 200 },
      { key: "testingAttachment", label: "Testing Attachment", type: "testingAttachment", width: 140 },
      { key: "assignedDev", label: "Forward to Developer", type: "devSelect", width: 180 },
      { key: "bugStatus", label: "Bug Status", type: "statusSelect", width: 150 },
      { key: "analyzeRemark", label: "Analyse Remark", type: "analyzeRemarkText", width: 200 },
      { key: "sopForSolution", label: "SOP for Solution", type: "sopSolutionText", width: 200 },
      { key: "delayedReason", label: "Delayed Reason", type: "delayedReasonText", width: 200 },
      { key: "analyzeAttachment", label: "Analyse Attachment", type: "analyzeAttachment", width: 140 },
      { key: "finalTestingRemark", label: "Final Testing Remark", type: "finalTestingRemarkText", width: 200
},
      { key: "testingFlag", label: "Testing Flag", type: "testingFlagSelect", width: 160 },
      { key: "reportedDate", label: "Reported", type: "date", width: 180 },
      { key: "bugTestedDate", label: "Bug Tested", type: "date", width: 180 },
      { key: "bugAnalysedDate", label: "Bug Analysed At", type: "date", width: 180 },
      { key: "currentPhase", label: "Change Phase", type: "phaseSelect", width: 150 }
    ];
  }
  if (phaseId === "rtt") {
    return [
      { key: "bugId", label: "Bug ID", type: "text", width: 140 },
      { key: "toolName", label: "Tool Name", type: "text", width: 180 },
      { key: "priority", label: "Priority", type: "prioritySelect", width: 120 },
      { key: "bugRaiser", label: "Bug Raiser", type: "text", width: 150 },
      { key: "issueFacedBy", label: "Issue Faced By", type: "facedByCard", width: 160 },
      { key: "assignedTester", label: "Bug Tester", type: "text", width: 150 },
      { key: "description", label: "Bug Description", type: "description", width: 250 },
      { key: "sopFollowedRaiser", label: "SOP Followed by Raiser", type: "sopFollowedRaiserCard", width: 220
},
      { key: "attachments", label: "Bug Attachments", type: "attachment", width: 150 },
      { key: "sopFollowed", label: "SOP Followed", type: "sopText", width: 160 },
      { key: "remarks", label: "Testing Remark", type: "remarksText", width: 200 },
      { key: "testingAttachment", label: "Testing Attachment", type: "testingAttachment", width: 140 },
      { key: "assignedDev", label: "Forward to Developer", type: "devSelect", width: 180 },
      { key: "bugStatus", label: "Bug Status", type: "statusSelect", width: 150 },
      { key: "analyzeRemark", label: "Analyse Remark", type: "analyzeRemarkText", width: 200 },
      { key: "sopForSolution", label: "SOP for Solution", type: "sopSolutionText", width: 200 },
      { key: "delayedReason", label: "Delayed Reason", type: "delayedReasonText", width: 200 },
      { key: "analyzeAttachment", label: "Analyse Attachment", type: "analyzeAttachment", width: 140 },
      { key: "finalTestingRemark", label: "Final Testing Remark", type: "finalTestingRemarkText", width: 200
},
      { key: "testingFlag", label: "Testing Flag", type: "testingFlagSelect", width: 160 },
      { key: "reportedDate", label: "Reported", type: "date", width: 180 },
```

```
    { key: "bugTestedDate", label: "Bug Tested", type: "date", width: 180 },
    { key: "bugAnalysedDate", label: "Bug Analysed At", type: "date", width: 180 },
    { key: "currentPhase", label: "Change Phase", type: "phaseSelect", width: 150 }
  ];
}
if (phaseId === "rtd") {
  return [
    { key: "bugId", label: "Bug ID", type: "text", width: 140 },
    { key: "toolName", label: "Tool Name", type: "text", width: 180 },
    { key: "priority", label: "Priority", type: "prioritySelect", width: 120 },
    { key: "bugRaiser", label: "Bug Raiser", type: "text", width: 150 },
    { key: "issueFacedBy", label: "Issue Faced By", type: "facedByCard", width: 160 },
    { key: "assignedTester", label: "Bug Tester", type: "text", width: 150 },
    { key: "description", label: "Bug Description", type: "description", width: 250 },
    { key: "sopFollowedRaiser", label: "SOP Followed by Raiser", type: "sopFollowedRaiserCard", width: 220
},
    { key: "attachments", label: "Bug Attachments", type: "attachment", width: 150 },
    { key: "sopFollowed", label: "SOP Followed", type: "sopText", width: 160 },
    { key: "remarks", label: "Testing Remark", type: "remarksText", width: 200 },
    { key: "testingAttachment", label: "Testing Attachment", type: "testingAttachment", width: 140 },
    { key: "assignedDev", label: "Forward to Developer", type: "devSelect", width: 180 },
    { key: "bugStatus", label: "Bug Status", type: "statusSelect", width: 150 },
    { key: "analyzeRemark", label: "Analyse Remark", type: "analyzeRemarkText", width: 200 },
    { key: "sopForSolution", label: "SOP for Solution", type: "sopSolutionText", width: 200 },
    { key: "delayedReason", label: "Delayed Reason", type: "delayedReasonText", width: 200 },
    { key: "analyzeAttachment", label: "Analyse Attachment", type: "analyzeAttachment", width: 140 },
    { key: "finalTestingRemark", label: "Final Testing Remark", type: "finalTestingRemarkText", width: 200
},
    { key: "testingFlag", label: "Testing Flag", type: "testingFlagSelect", width: 160 },
    { key: "deploymentStatus", label: "Deployment Status", type: "deploymentStatusSelect", width: 170 },
    { key: "deployRemark", label: "Deploy Remark", type: "deployRemarkText", width: 200 },
    { key: "finalSop", label: "Final SOP", type: "finalSopText", width: 200 },
    { key: "reportedDate", label: "Reported", type: "date", width: 180 },
    { key: "bugTestedDate", label: "Bug Tested", type: "date", width: 180 },
    { key: "bugAnalysedDate", label: "Bug Analysed At", type: "date", width: 180 },
    { key: "currentPhase", label: "Change Phase", type: "phaseSelect", width: 150 }
  ];
}
if (phaseId === "all_full" || phaseId === "deployed") {
  return [
    { key: "bugId", label: "Bug ID", type: "text", width: 140 },
    { key: "toolName", label: "Tool Name", type: "text", width: 180 },
    { key: "priority", label: "Priority", type: "prioritySelect", width: 120 },
    { key: "stack", label: "Stack", type: "stackSelect", width: 120 },
    { key: "bugRaiser", label: "Bug Raiser", type: "text", width: 150 },
    { key: "issueFacedBy", label: "Issue Faced By", type: "facedByCard", width: 160 },
    { key: "assignedTester", label: "Assigned Tester", type: "text", width: 150 },
    { key: "assignedDev", label: "Assigned Dev", type: "text", width: 150 },
    { key: "description", label: "Bug Description", type: "description", width: 250 },
    { key: "sopFollowedRaiser", label: "SOP by Raiser", type: "sopFollowedRaiserCard", width: 220 },
    { key: "attachments", label: "Bug Attachments", type: "attachment", width: 150 },
    { key: "bugStatus", label: "Bug Status", type: "statusSelect", width: 150 },
    { key: "sopFollowed", label: "Tester SOP", type: "sopText", width: 160 },
```

```
    { key: "remarks", label: "Testing Remark", type: "remarksText", width: 200 },
    { key: "testingAttachment", label: "Testing Attachment", type: "testingAttachment", width: 140 },
    { key: "rootCause", label: "Root Cause", type: "text", width: 160 },
    { key: "analyzeRemark", label: "Analyse Remark", type: "analyzeRemarkText", width: 200 },
    { key: "sopForSolution", label: "SOP for Solution", type: "sopSolutionText", width: 200 },
    { key: "delayedReason", label: "Delayed Reason", type: "delayedReasonText", width: 200 },
    { key: "analyzeAttachment", label: "Analyse Attachment", type: "analyzeAttachment", width: 140 },
    { key: "finalTestingRemark", label: "Final Testing Remark", type: "finalTestingRemarkText", width: 200
},
    { key: "testingFlag", label: "Testing Flag", type: "testingFlagSelect", width: 160 },
    { key: "deploymentStatus", label: "Deployment Status", type: "deploymentStatusSelect", width: 170 },
    { key: "deployRemark", label: "Deploy Remark", type: "deployRemarkText", width: 200 },
    { key: "finalSop", label: "Final SOP", type: "finalSopText", width: 200 },
    { key: "reportedDate", label: "Reported", type: "date", width: 180 },
    { key: "bugTestedDate", label: "Bug Tested", type: "date", width: 180 },
    { key: "bugAnalysedDate", label: "Bug Analysed At", type: "date", width: 180 },
    { key: "currentPhase", label: "Change Phase", type: "phaseSelect", width: 150 }
  ];
 }
 // Default fallback
 return [
   { key: "bugId", label: "Bug ID", type: "text", width: 140 },
   { key: "toolName", label: "Tool Name", type: "text", width: 180 },
   { key: "priority", label: "Priority", type: "prioritySelect", width: 120 },
   { key: "stack", label: "Stack", type: "stackSelect", width: 120 },
   { key: "bugRaiser", label: "Bug Raiser", type: "text", width: 150 },
   { key: "assignedTester", label: "Assigned Tester", type: "text", width: 150 },
   { key: "assignedDev", label: "Assigned Dev", type: "text", width: 150 },
   { key: "description", label: "Bug Description", type: "description", width: 250 },
   { key: "attachments", label: "Bug Attachments", type: "attachment", width: 150 },
   ...commonEnd
 ];
};
const priorityOptions = ["critical", "high", "medium", "low"];
const stackOptions = ["Web", "App", "Any"];
const ResizableHeader = ({ children, columnKey, colWidths, setColWidths }) => {
 const startX = useRef(0);
 const startWidth = useRef(0);
 const handleMouseDown = (e) => {
   e.preventDefault();
   startX.current = e.clientX; startWidth.current = colWidths[columnKey];
   document.addEventListener("mousemove", handleMouseMove);
   document.addEventListener("mouseup", handleMouseUp);
 };
 const handleMouseMove = (e) => {
   const diff = e.clientX - startX.current;
   setColWidths((prev) => ({ ...prev,[columnKey]: Math.max(60, startWidth.current + diff),}));
 };
 const handleMouseUp = () => {
   document.removeEventListener("mousemove", handleMouseMove);
   document.removeEventListener("mouseup", handleMouseUp);
 };
 return (
```

```
    <div className="flex items-center group" style={{ width: colWidths[columnKey], minWidth: 48 }}>
      <div className="flex-1">{children}</div>
      <div onMouseDown={handleMouseDown}
        className="w-4 h-6 cursor-col-resize ml-1 flex items-center justify-center"
        title="Resize"  style={{ fontSize: "18px", color: "#cbc8c8" }}> |
      </div>
    </div>
  );
};
const ColumnHeader = ({ children, onPin, isPinned, columnKey }) => {
  const [sortOrder, setSortOrder] = useState(null);
  return (
    <DropdownMenu>
      <DropdownMenuTrigger asChild>
        <div className="flex items-center gap-2 cursor-pointer group select-none">
          <span>{children}</span>
          {isPinned && <Pin className="w-4 h-4 text-cyan-500" fill="currentColor" />}
          <div className="opacity-0 group-hover:opacity-100 transition-opacity">
            <svg className="w-4 h-4" fill="none" stroke="currentColor" viewBox="0 0 24 24">
              <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2}
                d="M19 13l-7 7-7-7m0-6l7-7 7 7" />
            </svg>
          </div>
        </div>
      </DropdownMenuTrigger>
      <DropdownMenuContent align="start">
        <DropdownMenuItem onClick={() => setSortOrder(sortOrder === "asc" ? null : "asc")}>
          <ArrowUp className="w-4 h-4 mr-2" /> Sort Ascending
        </DropdownMenuItem>
        <DropdownMenuItem onClick={() => setSortOrder(sortOrder === "desc" ? null : "desc")}>
          <ArrowDown className="w-4 h-4 mr-2" /> Sort Descending
        </DropdownMenuItem>
        {columnKey !== "bugId" && (
          <DropdownMenuItem onClick={() => onPin(columnKey)}>
            <Pin className="w-4 h-4 mr-2" /> {isPinned ? "Unpin" : "Pin"}
          </DropdownMenuItem>
        )}
      </DropdownMenuContent>
    </DropdownMenu>
  );
};
const PHASE_EDITABLE_MAP = {
  "Bug Reported": ["toolName", "priority", "stack", "description", "assignedTester", "currentPhase",
"attachments"],
  "Bug Testing": ["bugStatus", "sopFollowed", "remarks", "testingAttachment", "assignedDev", "currentPhase"],
  "Bug Confirmation": ["bugStatus", "sopFollowed", "remarks", "testingAttachment", "assignedDev",
"currentPhase"],
  "Bug Analysis": ["analyzeRemark", "sopForSolution", "delayedReason", "analyzeAttachment", "currentPhase",
"rootCause"],
  "Ready to Test": ["finalTestingRemark", "testingFlag", "currentPhase"],
  "Maintenance": ["finalTestingRemark", "testingFlag", "currentPhase"],
  "Ready to Deploy": ["deploymentStatus", "deployRemark", "finalSop", "currentPhase"],
  "Final Testing": ["deploymentStatus", "deployRemark", "finalSop", "currentPhase"],
```

```
 "Deployed": ["currentPhase"],
 "Closure": ["currentPhase"],
};
/**
* Returns true if this column is editable given the restriction list.
* @param {string} colKey        - column key from tableDetails
* @param {object} item          - the bug object
* @param {string[] | null | undefined} editableColumnKeys
*/
function isColEditable(colKey, item, editableColumnKeys) {
 if (colKey === "bugId") return false;
 // 1. Determine columns allowed for the bug's current phase
 const currentPhase = item?.currentPhase;
 const phaseAllowed = PHASE_EDITABLE_MAP[currentPhase] || [];
  if (!phaseAllowed.includes(colKey)) return false;
 // 2. Check role-based restrictions
 if (editableColumnKeys === null || editableColumnKeys === undefined) return true;
 return editableColumnKeys.includes(colKey);
}
export default function BugTable({ phaseBugs, onClickCardToModal, phaseId, editableColumnKeys, canEditRow })
{
 const navigate = useNavigate();
 const { user } = useAuth();
 const tableDetails = getTableColumns(phaseId);
 const { bugsList, setBugsList, allUsers, toolList } = useAuth();
 const [editingIds, setEditingIds] = useState([]);
 const [editData, setEditData] = useState({});
 const [selectedRows, setSelectedRows] = useState([]);
 const [pinnedColumns, setPinnedColumns] = useState(["bugId"]);
 const [colWidths, setColWidths] = useState(
   Object.fromEntries(tableDetails.map((col) => [col.key, col.width || 160]))
 );
 // Synchronize colWidths whenever tableDetails changes (switching phases)
 // This ensures new columns have an initial width so resizing doesn't fail with NaN
 useEffect(() => {
   setColWidths((prev) => {
     const updated = { ...prev }; let changed = false;
     tableDetails.forEach((col) => {
       if (!(col.key in updated)) {  updated[col.key] = col.width || 160; changed = true;  }
     });
     return changed ? updated : prev;
   });
 }, [tableDetails]);
 const [currentPage, setCurrentPage] = useState(1);
 const [rowsPerPage, setRowsPerPage] = useState(10);
  const totalPages = Math.max(1, Math.ceil((phaseBugs?.length || 0) / rowsPerPage));
 const paginatedData = phaseBugs ? phaseBugs.slice((currentPage - 1) * rowsPerPage, currentPage *
rowsPerPage) : [];
 const handlePinColumn = (columnKey) => {
   // Bug ID is always pinned
   if (columnKey === "bugId") return;
   setPinnedColumns((prev) =>
     prev.includes(columnKey) ? prev.filter((col) => col !== columnKey) : [...prev, columnKey]
```

```
  );
};
const getPinnedStyle = (columnKey) => {
  if (!pinnedColumns.includes(columnKey)) return {};
  let left = colWidths.checkbox || 48;
  for (const col of tableDetails) {
    if (col.key === columnKey) break;
    if (pinnedColumns.includes(col.key)) left += colWidths[col.key] || 160;
  }
  if (columnKey === "checkbox") left = 0;
  return { position: "sticky", left: `${left}px`, zIndex: 10, boxShadow: "2px 0 2px -2px #eee",   };
};
// Returns true when the current user is allowed to edit this specific row.
const isRowEditable = (item) => {
  if (canEditRow === null || canEditRow === undefined) return true; // admin or phase-scoped (all editable)
  if (typeof canEditRow === "function") return canEditRow(item);
  return true;
};
const handleEdit = (item) => {setEditingIds([item._id]);setEditData({ [item._id]: { ...item } }); };
const handleSave = async (id) => {
  const dataToSave = editData[id];
  // We update the deeply nested values dynamically if they changed
  // In our backend schema for Priority/Stack, they live in phaseI_BugReport.toolInfo
  let fallbackDevId = dataToSave.phaseII_BugConfirmation?.assignedDeveloper?._id ||
dataToSave.phaseII_BugConfirmation?.assignedDeveloper?.id;
  if (!fallbackDevId && toolList && dataToSave.phaseI_BugReport?.toolInfo?.toolId) {
      const mappedTool = toolList.find(t => (t.id || t._id) ===
dataToSave.phaseI_BugReport.toolInfo.toolId);
      if (mappedTool && mappedTool.devId) {
          fallbackDevId = typeof mappedTool.devId === "object" ? (mappedTool.devId._id ||
mappedTool.devId.id) : mappedTool.devId;
      }
  }
  const updatePayload = {
    "phaseI_BugReport.toolInfo.toolName": dataToSave.toolNameExtracted ||
       dataToSave.phaseI_BugReport?.toolInfo?.toolName,
    "phaseI_BugReport.toolInfo.toolId": dataToSave.toolIdExtracted ||
      dataToSave.phaseI_BugReport?.toolInfo?.toolId,
    "phaseI_BugReport.toolInfo.priority": dataToSave.priorityExtracted ||
      dataToSave.phaseI_BugReport?.toolInfo?.priority,
    "phaseI_BugReport.toolInfo.stack": dataToSave.stackExtracted ||
      dataToSave.phaseI_BugReport?.toolInfo?.stack,
    "phaseI_BugReport.toolInfo.bugDescription": dataToSave.bugDescriptionExtracted !== undefined ?
      dataToSave.bugDescriptionExtracted : dataToSave.phaseI_BugReport?.toolInfo?.bugDescription,
    "phaseI_BugReport.toolInfo.expectedResult": dataToSave.expectedResultExtracted !== undefined ?
      dataToSave.expectedResultExtracted : dataToSave.phaseI_BugReport?.toolInfo?.expectedResult,
    "phaseI_BugReport.toolInfo.actualResult": dataToSave.actualResultExtracted !== undefined ?
      dataToSave.actualResultExtracted : dataToSave.phaseI_BugReport?.toolInfo?.actualResult,
    "phaseI_BugReport.assignedTester": dataToSave.testerExtracted !== undefined ? dataToSave.testerExtracted
      : (dataToSave.phaseI_BugReport?.assignedTester?._id ||
dataToSave.phaseI_BugReport?.assignedTester?.id
      || dataToSave.phaseI_BugReport?.assignedTester),
    "phaseII_BugConfirmation.testingInfo.status": dataToSave.statusExtracted !== undefined ?
```

```
        dataToSave.statusExtracted : dataToSave.phaseII_BugConfirmation?.testingInfo?.status,
      "phaseII_BugConfirmation.testingInfo.sopFollowed": dataToSave.sopFollowedExtracted !== undefined ?
        dataToSave.sopFollowedExtracted : dataToSave.phaseII_BugConfirmation?.testingInfo?.sopFollowed,
      "phaseII_BugConfirmation.testingInfo.remarks": dataToSave.remarksExtracted !== undefined ?
        dataToSave.remarksExtracted : dataToSave.phaseII_BugConfirmation?.testingInfo?.remarks,
      "phaseII_BugConfirmation.assignedDeveloper": dataToSave.devExtracted !== undefined ?
        dataToSave.devExtracted : fallbackDevId,
      "phaseIII_BugAnalysis.analysisInfo.remarks": dataToSave.analyzeRemarkExtracted !== undefined ?
        dataToSave.analyzeRemarkExtracted : dataToSave.phaseIII_BugAnalysis?.analysisInfo?.remarks,
      "phaseIII_BugAnalysis.analysisInfo.sopProvided": dataToSave.sopSolutionExtracted !== undefined ?
        dataToSave.sopSolutionExtracted : dataToSave.phaseIII_BugAnalysis?.analysisInfo?.sopProvided,
      "phaseIII_BugAnalysis.analysisInfo.delayedReason": dataToSave.delayedReasonExtracted !== undefined ?
        dataToSave.delayedReasonExtracted : dataToSave.phaseIII_BugAnalysis?.analysisInfo?.delayedReason,
      "phaseIV_Maintenance.maintenanceInfo.testingFlag": dataToSave.testingFlagExtracted !== undefined ?
        dataToSave.testingFlagExtracted : dataToSave.phaseIV_Maintenance?.maintenanceInfo?.testingFlag,
      "phaseIV_Maintenance.maintenanceInfo.remarks": dataToSave.finalTestingRemarkExtracted !== undefined ?
        dataToSave.finalTestingRemarkExtracted : dataToSave.phaseIV_Maintenance?.maintenanceInfo?.remarks,
      "phaseV_FinalTesting.testingInfo.deploymentStatus": dataToSave.deploymentStatusExtracted !== undefined ?
        dataToSave.deploymentStatusExtracted : dataToSave.phaseV_FinalTesting?.testingInfo?.deploymentStatus,
      "phaseV_FinalTesting.testingInfo.deployRemark": dataToSave.deployRemarkExtracted !== undefined ?
        dataToSave.deployRemarkExtracted : dataToSave.phaseV_FinalTesting?.testingInfo?.deployRemark,
      "phaseV_FinalTesting.testingInfo.finalSop": dataToSave.finalSopExtracted !== undefined ?
        dataToSave.finalSopExtracted : dataToSave.phaseV_FinalTesting?.testingInfo?.finalSop,
      // Attachments for all phases
      "phaseI_BugReport.toolInfo.attachments": dataToSave.attachmentsExtracted !== undefined ?
        dataToSave.attachmentsExtracted.map(f => ({ url: f.url, fileName: f.name || f.fileName, uploadedAt:
         f.uploadedAt || new Date() })) : dataToSave.phaseI_BugReport?.toolInfo?.attachments,
      "phaseII_BugConfirmation.testingInfo.attachments": dataToSave.testingAttachmentsExtracted !== undefined
?
         dataToSave.testingAttachmentsExtracted.map(f => ({ url: f.url, fileName: f.name || f.fileName,
        uploadedAt: f.uploadedAt || new Date() }))
:dataToSave.phaseII_BugConfirmation?.testingInfo?.attachments,
      "phaseIII_BugAnalysis.analysisInfo.attachments": dataToSave.analyzeAttachmentsExtracted !== undefined ?
        dataToSave.analyzeAttachmentsExtracted.map(f => ({ url: f.url, fileName: f.name || f.fileName,
        uploadedAt: f.uploadedAt || new Date() })) :
dataToSave.phaseIII_BugAnalysis?.analysisInfo?.attachments,
    };
    if (dataToSave.currentPhaseExtracted) {
      updatePayload.currentPhase = dataToSave.currentPhaseExtracted;
      updatePayload.bugPhaseNo = dataToSave.bugPhaseNoExtracted;
      const bugToUpdate = phaseBugs.find(b => (b.id || b._id) === id);
      if (bugToUpdate?.currentPhase === "Bug Testing" && dataToSave.currentPhaseExtracted === "Bug Analysis")
{
         updatePayload["phaseII_BugConfirmation.testedAt"] = new Date().toISOString();
      }
      // Automatically set "Go back to dev" flag if moving from RTT/Maintenance to Analysis
      if ((bugToUpdate?.currentPhase === "Ready to Test" || bugToUpdate?.currentPhase === "Maintenance") &&
          dataToSave.currentPhaseExtracted === "Bug Analysis") {
         updatePayload["phaseIV_Maintenance.maintenanceInfo.testingFlag"] = "Go back to dev";
      }
      // Automatically set "Done" deployment status if moving to Closure
      if (dataToSave.currentPhaseExtracted === "Closure") {
         updatePayload["phaseV_FinalTesting.testingInfo.deploymentStatus"] = "Done";
```

```
    }
    // Automatically capture "Bug Analysed At" if moving out of Analysis
    if (bugToUpdate?.currentPhase === "Bug Analysis" &&
        dataToSave.currentPhaseExtracted &&
        dataToSave.currentPhaseExtracted !== "Bug Analysis") {
       if (!bugToUpdate.phaseIII_BugAnalysis?.analyzedAt) {
          updatePayload["phaseIII_BugAnalysis.analyzedAt"] = new Date().toISOString();
       }
    }
  }
  const finalTestingFlag = updatePayload["phaseIV_Maintenance.maintenanceInfo.testingFlag"] !== undefined
              ? updatePayload["phaseIV_Maintenance.maintenanceInfo.testingFlag"]
              : dataToSave.phaseIV_Maintenance?.maintenanceInfo?.testingFlag;
  try {
    const bugToUpdate = phaseBugs.find(b => (b.id || b._id) === id);
    const isTransitioningToTesting = bugToUpdate?.currentPhase === "Bug Reported"
              && updatePayload.currentPhase === "Bug Testing";
    const isTransitioningToAnalysis = bugToUpdate?.currentPhase === "Bug Testing"
               && updatePayload.currentPhase === "Bug Analysis";
    const res = await updateBug({ id, ...updatePayload });
    if (isTransitioningToTesting || isTransitioningToAnalysis) {
      const tName = updatePayload["phaseI_BugReport.toolInfo.toolName"] ||
              bugToUpdate.phaseI_BugReport?.toolInfo?.toolName || "-";
      const priority = updatePayload["phaseI_BugReport.toolInfo.priority"] ||
              bugToUpdate.phaseI_BugReport?.toolInfo?.priority || "low";
      const rName = bugToUpdate.phaseI_BugReport?.reportedBy?.name || "Unknown";
      const bId = bugToUpdate.bugId || "N/A";
      // Determine assigned person
      let assignedName = "Unknown";
      if (isTransitioningToTesting) {
        const testerId = updatePayload["phaseI_BugReport.assignedTester"];
        assignedName = allUsers?.find(u => (u.id || u._id) === testerId)?.name || "Unassigned";
      } else {
        const devId = updatePayload["phaseII_BugConfirmation.assignedDeveloper"];
        assignedName = allUsers?.find(u => (u.id || u._id) === devId)?.name || "Unassigned";
      }
      toast.custom((t) => (
        <div className={`${t.visible ? 'animate-in fade-in slide-in-from-top-5' : 'animate-out fade-out'}
              max-w-sm w-full bg-white shadow-2xl rounded-2xl pointer-events-auto flex flex-col border
              border-gray-100 overflow-hidden z-[9999]`}>
          <div className="p-4 bg-slate-50 border-b border-gray-100 flex items-center justify-between">
             <div className="flex items-center gap-2">
               <div className="w-2 h-2 rounded-full bg-cyan-500 animate-pulse" />
               <span className="text-[10px] font-black uppercase tracking-widest text-gray-500 italic">
                    Assignment Alert</span>
             </div>
             <span className="text-xs font-bold text-gray-400">{bId}</span>
          </div>
          <div className="p-4 flex flex-col">
             <h4 className="text-sm font-black text-gray-900 leading-tight">
               Bug {bId} Assigned to <span className="text-cyan-600">{assignedName}</span>
             </h4>
             <div className="flex items-center gap-2 mt-2">
```

```
              <span className="text-xs font-bold text-gray-700 truncate max-w-[120px]">{tName}</span>
              <span className="w-1 h-1 rounded-full bg-gray-300" />
              <span className={`text-[9px] px-2 py-0.5 rounded-md font-black uppercase ${
                 priority === 'critical' ? 'bg-red-50 text-red-600' :
                 priority === 'high' ? 'bg-orange-50 text-orange-600' :
                 'bg-emerald-50 text-emerald-600' }`}>
                {priority}
              </span>
            </div>
            <div className="grid grid-cols-2 gap-2 mt-3 pt-3 border-t border-gray-50">
              <div className="flex flex-col">
                <span className="text-[9px] text-gray-400 font-bold uppercase tracking-tighter">
                   Reported By</span>
                <span className="text-[11px] text-gray-600 font-black truncate">{rName}</span>
              </div>
              <div className="flex flex-col">
                <span className="text-[9px] text-gray-400 font-bold uppercase tracking-tighter">
                   Assigned To</span>
                <span className="text-[11px] text-cyan-600 font-black truncate">{assignedName}</span>
              </div>
            </div>
          </div>
        </div>
      ), { duration: 6000 });
    } else { toast.success("Bug updated successfully!");}
    setBugsList((prev) =>
      prev.map((bug) => {
        if (bug._id === id) {
          const newBug = {...bug};
          newBug.phaseI_BugReport.toolInfo.toolName = updatePayload["phaseI_BugReport.toolInfo.toolName"];
          if (updatePayload["phaseI_BugReport.toolInfo.toolId"]) {
            newBug.phaseI_BugReport.toolInfo.toolId = updatePayload["phaseI_BugReport.toolInfo.toolId"];
          }
          newBug.phaseI_BugReport.toolInfo.priority = updatePayload["phaseI_BugReport.toolInfo.priority"];
          newBug.phaseI_BugReport.toolInfo.stack = updatePayload["phaseI_BugReport.toolInfo.stack"];
          newBug.phaseI_BugReport.toolInfo.bugDescription =
             updatePayload["phaseI_BugReport.toolInfo.bugDescription"];
          newBug.phaseI_BugReport.toolInfo.expectedResult =
             updatePayload["phaseI_BugReport.toolInfo.expectedResult"];
          newBug.phaseI_BugReport.toolInfo.actualResult =
             updatePayload["phaseI_BugReport.toolInfo.actualResult"];
          newBug.phaseI_BugReport.toolInfo.attachments =
             updatePayload["phaseI_BugReport.toolInfo.attachments"];
          // Sync assignedTester
          if (updatePayload["phaseI_BugReport.assignedTester"] !== undefined) {
            const testerId = updatePayload["phaseI_BugReport.assignedTester"];
            const matchedTester = allUsers?.find(u => u.id === testerId || u._id === testerId);
            newBug.phaseI_BugReport.assignedTester = matchedTester
              ? { _id: matchedTester._id || matchedTester.id, name: matchedTester.name, email:
               matchedTester.email }: testerId;
          }
          if (!newBug.phaseII_BugConfirmation) newBug.phaseII_BugConfirmation = { testingInfo: {} };
          if (!newBug.phaseII_BugConfirmation.testingInfo) newBug.phaseII_BugConfirmation.testingInfo = {};
```

```
        if (updatePayload["phaseII_BugConfirmation.testingInfo.status"] !== undefined) {
            newBug.phaseII_BugConfirmation.testingInfo.status =
            updatePayload["phaseII_BugConfirmation.testingInfo.status"];
        }
        if (updatePayload["phaseII_BugConfirmation.testingInfo.sopFollowed"] !== undefined) {
            newBug.phaseII_BugConfirmation.testingInfo.sopFollowed =
            updatePayload["phaseII_BugConfirmation.testingInfo.sopFollowed"];
        }
        if (updatePayload["phaseII_BugConfirmation.testingInfo.remarks"] !== undefined) {
            newBug.phaseII_BugConfirmation.testingInfo.remarks =
            updatePayload["phaseII_BugConfirmation.testingInfo.remarks"];
        }
        if (updatePayload["phaseII_BugConfirmation.testingInfo.attachments"] !== undefined) {
            newBug.phaseII_BugConfirmation.testingInfo.attachments =
            updatePayload["phaseII_BugConfirmation.testingInfo.attachments"];
        }
        if (updatePayload["phaseII_BugConfirmation.assignedDeveloper"] !== undefined) {
            const devId = updatePayload["phaseII_BugConfirmation.assignedDeveloper"];
            const matchedUser = allUsers?.find(u => u.id === devId || u._id === devId);
            if (matchedUser) { newBug.phaseII_BugConfirmation.assignedDeveloper = { _id: matchedUser._id
                  || matchedUser.id, name: matchedUser.name, email: matchedUser.email };
            } else {
                newBug.phaseII_BugConfirmation.assignedDeveloper = devId; // fallback
            }
        }
        if (updatePayload["phaseII_BugConfirmation.testedAt"] !== undefined) {
            newBug.phaseII_BugConfirmation.testedAt = updatePayload["phaseII_BugConfirmation.testedAt"];
        }
        if (!newBug.phaseIII_BugAnalysis) newBug.phaseIII_BugAnalysis = { analysisInfo: {} };
        if (!newBug.phaseIII_BugAnalysis.analysisInfo) newBug.phaseIII_BugAnalysis.analysisInfo = {};
        if (updatePayload["phaseIII_BugAnalysis.analysisInfo.remarks"] !== undefined) {
            newBug.phaseIII_BugAnalysis.analysisInfo.remarks =
            updatePayload["phaseIII_BugAnalysis.analysisInfo.remarks"];
        }
        if (updatePayload["phaseIII_BugAnalysis.analysisInfo.attachments"] !== undefined) {
            newBug.phaseIII_BugAnalysis.analysisInfo.attachments =
            updatePayload["phaseIII_BugAnalysis.analysisInfo.attachments"];
        }
        if (updatePayload["phaseIII_BugAnalysis.analysisInfo.sopProvided"] !== undefined) {
            newBug.phaseIII_BugAnalysis.analysisInfo.sopProvided =
            updatePayload["phaseIII_BugAnalysis.analysisInfo.sopProvided"];
        }
        if (updatePayload["phaseIII_BugAnalysis.analysisInfo.delayedReason"] !== undefined) {
            newBug.phaseIII_BugAnalysis.analysisInfo.delayedReason =
            updatePayload["phaseIII_BugAnalysis.analysisInfo.delayedReason"];
        }
        if (!newBug.phaseIV_Maintenance) newBug.phaseIV_Maintenance = { maintenanceInfo: {} };
        if (!newBug.phaseIV_Maintenance.maintenanceInfo) newBug.phaseIV_Maintenance.maintenanceInfo = {};

        if (updatePayload["phaseIV_Maintenance.maintenanceInfo.testingFlag"] !== undefined) {
            newBug.phaseIV_Maintenance.maintenanceInfo.testingFlag =
            updatePayload["phaseIV_Maintenance.maintenanceInfo.testingFlag"];
        }
```

```
            if (updatePayload["phaseIV_Maintenance.maintenanceInfo.remarks"] !== undefined) {
               newBug.phaseIV_Maintenance.maintenanceInfo.remarks =
              updatePayload["phaseIV_Maintenance.maintenanceInfo.remarks"];
            }
            if (!newBug.phaseV_FinalTesting) newBug.phaseV_FinalTesting = { testingInfo: {} };
            if (!newBug.phaseV_FinalTesting.testingInfo) newBug.phaseV_FinalTesting.testingInfo = {};
            if (updatePayload["phaseV_FinalTesting.testingInfo.deploymentStatus"] !== undefined) {
               newBug.phaseV_FinalTesting.testingInfo.deploymentStatus =
              updatePayload["phaseV_FinalTesting.testingInfo.deploymentStatus"];
            }
            if (updatePayload["phaseV_FinalTesting.testingInfo.deployRemark"] !== undefined) {
               newBug.phaseV_FinalTesting.testingInfo.deployRemark =
              updatePayload["phaseV_FinalTesting.testingInfo.deployRemark"];
            }
            if (updatePayload["phaseV_FinalTesting.testingInfo.finalSop"] !== undefined) {
               newBug.phaseV_FinalTesting.testingInfo.finalSop =
              updatePayload["phaseV_FinalTesting.testingInfo.finalSop"];
            }
            if (dataToSave.currentPhaseExtracted) {
              newBug.currentPhase = dataToSave.currentPhaseExtracted;
              newBug.bugPhaseNo = dataToSave.bugPhaseNoExtracted;
           }
           return newBug;
        }
        return bug;
      })
    );
  } catch (error) { toast.error("Failed to update bug");}
  setEditingIds((prev) => prev.filter((eid) => eid !== id));
  setEditData((prev) => { const newData = { ...prev }; delete newData[id];  return newData;});
};
const handleCancel = (id) => {
  setEditingIds((prev) => prev.filter((eid) => eid !== id));
  setEditData((prev) => { const newData = { ...prev }; delete newData[id]; return newData; });
};
const handleRowSelect = (id) => {
  setSelectedRows((prev) =>prev.includes(id) ? prev.filter((rowId) => rowId !== id) : [...prev, id]);
};
const handleEditSelected = () => {
  if (selectedRows.length > 0) {
    setEditingIds(selectedRows);
    const newEditData = {};
    selectedRows.forEach((id) => {
      const item = bugsList.find((i) => i._id === id);
      if (item) newEditData[id] = { ...item };
    });
    setEditData(newEditData);
  }
};
const handleDeleteSelected = async () => {
  for (const id of selectedRows) { // await deleteBug({ id }); // Backend soft delete   }
  toast.success("Feature Mock (Bugs Deleted)");
  setSelectedRows([]);
```

```
};
const handleDeleteSingle = async (id) => { toast.success("Feature Mock (Bug Archive)"); };
const handleSaveAll = async () => {
  for (const id of editingIds) { if (editData[id]) await handleSave(id);}
  setEditingIds([]);  setEditData({}); setSelectedRows([]);
};
const handleCancelAll = () => {  setEditingIds([]);  setEditData({}); setSelectedRows([]); };
const renderCell = (col, rawItem, isEditing, displayData, setRowEditData) => {
  // Because data is nested deeply, we extract them virtually for editing
  const toolName = displayData.toolNameExtracted || rawItem.phaseI_BugReport?.toolInfo?.toolName || "-";
  const priority = displayData.priorityExtracted || rawItem.phaseI_BugReport?.toolInfo?.priority || "low";
  let stack = displayData.stackExtracted || rawItem.phaseI_BugReport?.toolInfo?.stack;
  if (!stack || stack === "-" || stack === "") {
      const foundTool = toolList?.find(t => (t.id || t._id) === rawItem.phaseI_BugReport?.toolInfo?.toolId);
      if (foundTool && Array.isArray(foundTool.stack) && foundTool.stack.length > 0) {
          stack = foundTool.stack[0];
      } else { stack = "-"; }
  }
  // Description extractions
  const bugDesc = displayData.bugDescriptionExtracted !== undefined
    ? displayData.bugDescriptionExtracted
    : rawItem.phaseI_BugReport?.toolInfo?.bugDescription || "";
  const expRes = displayData.expectedResultExtracted !== undefined
    ? displayData.expectedResultExtracted
    : rawItem.phaseI_BugReport?.toolInfo?.expectedResult || "";
  const actRes = displayData.actualResultExtracted !== undefined
    ? displayData.actualResultExtracted
    : rawItem.phaseI_BugReport?.toolInfo?.actualResult || "";
  const isExpectedMode = expRes && expRes !== "Not Applicable (Simple Description)";
  // User mappings - Add email handling
  let raiserObj = rawItem.phaseI_BugReport?.reportedBy;
  if (raiserObj && (typeof raiserObj !== "object" || !raiserObj.name)) {
      const idToCheck = typeof raiserObj === "object" ? (raiserObj._id || raiserObj.id) : raiserObj;
      const found = allUsers?.find(u => (u.id || u._id) === idToCheck);
      if (found) raiserObj = found;
  }
  const raiser = raiserObj?.name || "Unknown";
  const raiserEmail = raiserObj?.email || "No email provided";
  // Bug Tester uses assignedTester in Phase I, or testedBy in Phase II, default to assignedTester
  let testerObj = rawItem.phaseI_BugReport?.assignedTester || rawItem.phaseII_BugConfirmation?.testedBy;
  if (testerObj && (typeof testerObj !== "object" || !testerObj.name)) {
      const idToCheck = typeof testerObj === "object" ? (testerObj._id || testerObj.id) : testerObj;
      const found = allUsers?.find(u => (u.id || u._id) === idToCheck);
      if (found) testerObj = found;
  }
  const tester = testerObj?.name || "";
  const testerEmail = testerObj?.email || "No email provided";
  // Developer extraction uses edit data dynamically, or default from Phase 2
  let developerObj = rawItem.phaseII_BugConfirmation?.assignedDeveloper;
  if (developerObj && (typeof developerObj !== "object" || !developerObj.name)) {
      const idToCheck = typeof developerObj === "object" ? (developerObj._id || developerObj.id)
       : developerObj;
      const found = allUsers?.find(u => (u.id || u._id) === idToCheck);
```

```
        if (found) developerObj = found;
    }
    if (isEditing && displayData.devExtracted) {
        developerObj = allUsers?.find(u => (u.id || u._id) === displayData.devExtracted);
    }
    const developer = developerObj?.name || (typeof developerObj === "string" ? developerObj : "");
    const developerEmail = developerObj?.email || "No email provided";
    // New testing fields
    const bugStatusOptions = ["No Bug", "Minor Bugs", "Error Unlocatted", "Actual Issue", "Resolved"];
    const bugStatus = displayData.statusExtracted ?? rawItem.phaseII_BugConfirmation?.testingInfo?.status ??
"-";
    const sopFollowedArr = displayData.sopFollowedExtracted !== undefined ? displayData.sopFollowedExtracted
        : rawItem.phaseII_BugConfirmation?.testingInfo?.sopFollowed;
    const sopFollowedStr = displayData.sopFollowedRaw !== undefined
      ? displayData.sopFollowedRaw: (sopFollowedArr && Array.isArray(sopFollowedArr)
         && sopFollowedArr.length > 0 ? sopFollowedArr.join(', ') : '');
    const remarks = displayData.remarksExtracted !== undefined ? displayData.remarksExtracted
        : rawItem.phaseII_BugConfirmation?.testingInfo?.remarks || "";
    // New analysis fields
    const analyzeRemark = displayData.analyzeRemarkExtracted !== undefined
        ? displayData.analyzeRemarkExtracted : rawItem.phaseIII_BugAnalysis?.analysisInfo?.remarks || "";
    const sopSolutionArr = displayData.sopSolutionExtracted !== undefined
        ? displayData.sopSolutionExtracted : rawItem.phaseIII_BugAnalysis?.analysisInfo?.sopProvided;
    const sopSolutionStr = displayData.sopSolutionRaw !== undefined
        ? displayData.sopSolutionRaw : (sopSolutionArr && Array.isArray(sopSolutionArr)
        && sopSolutionArr.length > 0 ? sopSolutionArr.join(', ') : '');
    const delayedReason = displayData.delayedReasonExtracted !== undefined
        ? displayData.delayedReasonExtracted : rawItem.phaseIII_BugAnalysis?.analysisInfo?.delayedReason ||
"";
    if (col.key === "bugId") {
       return (
         <CellTooltip content={`Click to view details for ${rawItem.bugId}`}>
           <span className="font-medium text-cyan-600 cursor-pointer"
                onClick={() => onClickCardToModal(rawItem)}> {rawItem.bugId || "N/A"}
           </span>
         </CellTooltip>
       );
    }
    if (col.key === "reportedDate") {
        return <span className="text-gray-500 whitespace-nowrap">
          {rawItem.phaseI_BugReport?.reportedAt ? new Date(rawItem.phaseI_BugReport.reportedAt)
                .toLocaleString('en-GB', { day: '2-digit', month: 'short', year: 'numeric',
                hour: '2-digit', minute: '2-digit', hour12: true }) : "-"}
        </span>;
    }
    if (col.key === "bugTestedDate") {
        return <span className="text-gray-500 whitespace-nowrap">
          {rawItem.phaseII_BugConfirmation?.testedAt ? new Date(rawItem.phaseII_BugConfirmation.testedAt)
                .toLocaleString('en-GB', { day: '2-digit', month: 'short', year: 'numeric',
                hour: '2-digit', minute: '2-digit', hour12: true }) : "-"}
        </span>;
    }
    if (col.key === "bugAnalysedDate") {
```

```
      return <span className="text-gray-500 whitespace-nowrap">
        {rawItem.phaseIII_BugAnalysis?.analyzedAt ? new Date(rawItem.phaseIII_BugAnalysis.analyzedAt)
              .toLocaleString('en-GB', { day: '2-digit', month: 'short', year: 'numeric',
               hour: '2-digit', minute: '2-digit', hour12: true }) : "-"}
      </span>;
  }
  if (col.key === "toolName") {
   if (isEditing) {
      // Build dropdown from the global tool list
    const toolsForDropdown = toolList || [];
    const selectedToolId  = displayData.toolIdExtracted ?? rawItem.phaseI_BugReport?.toolInfo?.toolId;
    const selectedToolLabel = toolsForDropdown.find(t => (t.id || t._id) === selectedToolId)?.name ||
toolName;
      return (
        <DropdownMenu>
          <DropdownMenuTrigger asChild>
            <Button variant="outline" className="h-8 w-full justify-start text-left shrink-0 text-xs px-2"
                    onClick={(e) => e.stopPropagation()}>
              <span className="truncate">{selectedToolLabel}</span>
              <ChevronDown className="ml-1 h-3 w-3 opacity-50 flex-shrink-0" />
            </Button>
          </DropdownMenuTrigger>
          <DropdownMenuContent className="w-52 max-h-52 overflow-auto z-50">
            {toolsForDropdown.map((tool) => (
              <DropdownMenuItem key={tool.id || tool._id} className="text-xs py-1.5 cursor-pointer"
                onClick={(e) => {
                  e.stopPropagation();
                  setRowEditData({toolNameExtracted: tool.name, toolIdExtracted: tool.id || tool._id,
                    // Clear stale stack so relative options are re-evaluated
                    stackExtracted: undefined,
                  });
                }}
              >
                {tool.name}
              </DropdownMenuItem>
            ))}
          </DropdownMenuContent>
        </DropdownMenu>
      );
    }
    return (
      <CellTooltip content={toolName}>
        <span className="font-medium text-gray-900 truncate block">{toolName}</span>
      </CellTooltip>
    );
  }
  if (col.key === "bugRaiser") {
    return (
      <CellTooltip content={`${raiser} (${raiserEmail})`}>
        <span className="text-gray-600 truncate block w-full">{raiser}</span>
      </CellTooltip>
    );
  }
```

```
  if (col.type === "facedByCard") {
    const clientCtx = rawItem.phaseI_BugReport?.clientContext;
    if (!clientCtx) return <span className="text-gray-400">-</span>;
    if (clientCtx.facedByMe) {
      return (
        <CellTooltip content="Issue faced by the raiser themselves">
          <span className="text-gray-700 font-medium">Me</span>
        </CellTooltip>
      );
    } else if (clientCtx.facedByClient) {
      return (
        <div className="flex flex-col text-xs text-gray-700 truncate min-w-[120px]">
          <span className="font-semibold truncate">{clientCtx.clientName || "Unknown Client"}</span>
          <span className="text-gray-500 truncate">{clientCtx.companyName || "Unknown Company"}</span>
        </div>
      );
    }
    return <span className="text-gray-400">-</span>;
  }
  if (col.type === "sopFollowedRaiserCard") {
    const sopChecklist = rawItem.phaseI_BugReport?.clientContext?.sopChecklist;
    if (!sopChecklist || Object.keys(sopChecklist).length === 0) {
      return <span className="text-gray-400 px-2 block w-full">-</span>;
    }
    const followedItems = Object.entries(sopChecklist)
      .filter(([_, isChecked]) => isChecked).map(([text, _]) => text);
    if (followedItems.length === 0) return <span className="text-gray-400 px-2 block w-full">None</span>;
    return (
      <CellTooltip content={followedItems.join('\n')}>
        <ul className="list-disc pl-5 text-xs text-gray-700 m-0 max-h-20 overflow-y-auto block w-full pr-1">
          {followedItems.map((item, idx) => ( <li key={idx} className="truncate">{item}</li> ))}
        </ul>
      </CellTooltip>
    );
  }
  if (col.key === "assignedTester") {
    if (isEditing) {
      const testers = allUsers?.filter(u =>
        Array.isArray(u.roletype) ? u.roletype.includes("tester") : u.roletype === "tester"
      ) || [];

      let selectedTesterId = displayData.testerExtracted ?? testerObj?._id ?? testerObj?.id;
      const selectedTesterLabel = testers.find(t => (t.id || t._id) === selectedTesterId)
             ?.name|| tester|| "Select Tester";

      return (
        <DropdownMenu>
          <DropdownMenuTrigger asChild>
            <Button variant="outline" className="h-8 w-full justify-start text-left shrink-0 text-xs px-2"
               onClick={(e) => e.stopPropagation()}>
              <span className="truncate">{selectedTesterLabel}</span>
              <ChevronDown className="ml-1 h-3 w-3 opacity-50 flex-shrink-0" />
            </Button>
```

```
          </DropdownMenuTrigger>
          <DropdownMenuContent className="w-48 max-h-48 overflow-auto z-50">
            {testers.map((t) => (
              <DropdownMenuItem key={t.id || t._id} className="text-xs py-1.5 cursor-pointer"
               onClick={(e) => { e.stopPropagation();  setRowEditData({ testerExtracted: t.id || t._id });
}}>
                {t.name || t.username || t.email}
              </DropdownMenuItem>
            ))}
          </DropdownMenuContent>
        </DropdownMenu>
      );
    }
    return (
      <CellTooltip content={`${tester} (${testerEmail})`}>
        <span className="text-gray-600 truncate block w-full">{tester}</span>
      </CellTooltip>
    );
  }

  if (col.key === "assignedDev") {
    if (isEditing && col.type === "devSelect") {
      const devs = allUsers?.filter(u => (Array.isArray(u.roletype) ? u.roletype.includes("dev")
      : u.roletype === "dev") || (u.roles && u.roles.includes("Developer"))) || [];
      let selectedDevId = displayData.devExtracted ?? developerObj?._id ?? developerObj?.id;
      // Auto-select tool dev if unset
      if (!selectedDevId && toolList && (rawItem.phaseI_BugReport?.toolInfo?.toolId)) {
        const mappedTool = toolList.find(t => (t.id || t._id) ===
rawItem.phaseI_BugReport.toolInfo.toolId);
        if (mappedTool && mappedTool.devId) {
          const dId = typeof mappedTool.devId === "object"
            ? (mappedTool.devId._id || mappedTool.devId.id) : mappedTool.devId;
            selectedDevId = dId;
        }
      }
      const selectedDevLabel = devs.find(d => (d.id || d._id) === selectedDevId)?.name || "Select
Developer";
      return (
        <DropdownMenu>
          <DropdownMenuTrigger asChild>
            <Button variant="outline" className="h-8 w-full justify-start text-left shrink-0 text-xs px-2"
              onClick={(e) => e.stopPropagation()}>
              <span className="truncate">{selectedDevLabel}</span>
              <ChevronDown className="ml-1 h-3 w-3 opacity-50 flex-shrink-0" />
            </Button>
          </DropdownMenuTrigger>
          <DropdownMenuContent className="w-48 max-h-48 overflow-auto z-50">
            {devs.map((dev) => (
              <DropdownMenuItem key={dev.id || dev._id} className="text-xs py-1.5 cursor-pointer"
                onClick={(e) => { e.stopPropagation(); setRowEditData({ devExtracted: dev.id || dev._id
});}}>
                {dev.name || dev.username || dev.email}
              </DropdownMenuItem>
```

```
            ))}
          </DropdownMenuContent>
        </DropdownMenu>
      );
    } else {
      return (
        <CellTooltip content={developerEmail}>
          <span className="text-gray-600 truncate block w-full">{developer || "-"}</span>
        </CellTooltip>
      );
    }
  }
 if (col.key === "attachments" || col.type === "attachment") {
 const attachments = displayData.attachmentsExtracted || rawItem.phaseI_BugReport?.toolInfo?.attachments ||
[];
    return (
      <AttachmentGallery files={attachments} isEditable={isEditing}  label="Bug Attachments"
        onUpdate={(newFiles) => setRowEditData({ attachmentsExtracted: newFiles })} />
    );
  }

  if (col.key === "testingAttachment" || col.type === "testingAttachment") {
    const attachments = displayData.testingAttachmentsExtracted ||
       rawItem.phaseII_BugConfirmation?.testingInfo?.attachments || [];
    return (
      <AttachmentGallery files={attachments} isEditable={isEditing}  label="Testing Attachments"
        onUpdate={(newFiles) => setRowEditData({ testingAttachmentsExtracted: newFiles })} />
    );
  }

  if (col.key === "analyzeAttachment" || col.type === "analyzeAttachment") {
    const attachments = displayData.analyzeAttachmentsExtracted ||
rawItem.phaseIII_BugAnalysis?.analysisInfo?.attachments || [];
    return (
      <AttachmentGallery files={attachments}  isEditable={isEditing}  label="Analysis Attachments"
        onUpdate={(newFiles) => setRowEditData({ analyzeAttachmentsExtracted: newFiles })}       />
    );
  }
  if (col.type === "analyzeRemarkText") {
     if (isEditing) {
        return (
          <textarea className="w-full text-xs border rounded p-1 resize-y" rows={2} placeholder="Add Analyse
              Remarks..." value={analyzeRemark} onChange={(e) =>
              setRowEditData({ analyzeRemarkExtracted: e.target.value })} />
        );
     }
     return (
       <CellTooltip content={analyzeRemark !== "-" ? analyzeRemark : ""}>
         <span className="text-gray-600 truncate block w-full">{analyzeRemark || "-"}</span>
       </CellTooltip>
     );
  }
  if (col.type === "delayedReasonText") {
```

```
      if (isEditing) {
         return (
            <Input className="h-8 w-full text-xs" placeholder="Delayed Reason..." value={delayedReason}
                onChange={(e) => setRowEditData({ delayedReasonExtracted: e.target.value })} />
         );
      }
      return (
        <CellTooltip content={delayedReason !== "-" ? delayedReason : ""}>
          <span className="text-gray-600 truncate block w-full">{delayedReason || "-"}</span>
        </CellTooltip>
      );
   }
   if (col.type === "sopSolutionText") {
     if (isEditing) {
       return (
         <textarea  className="w-full text-xs border rounded p-1 resize-y" rows={2}
           placeholder="SOP for solution (comma separated)"value={sopSolutionStr}
           onChange={(e) => { const raw = e.target.value;
             const arr = raw ? raw.split(',').map(s => s.trim()).filter(Boolean) : [];
             setRowEditData({sopSolutionExtracted: arr,sopSolutionRaw: raw});
           }}
         />
       );
     } else {
       return (
         <CellTooltip content={sopSolutionStr !== "-" ? sopSolutionStr : ""}>
           <span className="text-gray-600 text-xs truncate block w-full">{sopSolutionStr || "-"}</span>
         </CellTooltip>
       );
     }
   }
   if (col.key === "remarks" || col.type === "remarksText") {
      if (isEditing) {
         return (
          <textarea className="w-full text-xs border rounded p-1 resize-y" rows={2} value={remarks}
           placeholder="Add remarks..." onChange={(e) => setRowEditData({ remarksExtracted: e.target.value
}))}/>
         );
      }
      return (
        <CellTooltip content={remarks !== "-" ? remarks : ""}>
          <span className="text-gray-600 truncate block w-full">{remarks || "-"}</span>
        </CellTooltip>
      );
   }

   const finalTestingRemark = displayData.finalTestingRemarkExtracted !== undefined ?
displayData.finalTestingRemarkExtracted : rawItem.phaseIV_Maintenance?.maintenanceInfo?.remarks || "";

   if (col.type === "finalTestingRemarkText") {
      if (isEditing) {
         return (
```

```
          <textarea className="w-full text-xs border rounded p-1 resize-y" rows={2} placeholder="Add Final
Testing Remarks..." value={finalTestingRemark} onChange={(e) => setRowEditData({ finalTestingRemarkExtracted:
e.target.value })} />
        );
      }
      return (
        <CellTooltip content={finalTestingRemark !== "-" ? finalTestingRemark : ""}>
          <span className="text-gray-600 truncate block w-full">{finalTestingRemark || "-"}</span>
        </CellTooltip>
      );
    }

    const deployRemark = displayData.deployRemarkExtracted !== undefined ? displayData.deployRemarkExtracted :
rawItem.phaseV_FinalTesting?.testingInfo?.deployRemark || "";
    if (col.type === "deployRemarkText") {
      if (isEditing) {
        return (
          <textarea className="w-full text-xs border rounded p-1 resize-y bg-emerald-50/30" rows={2}
placeholder="Add Deploy Remarks..." value={deployRemark} onChange={(e) => setRowEditData({
deployRemarkExtracted: e.target.value })} />
        );
      }
      return (
        <CellTooltip content={deployRemark || ""}>
          <span className="text-gray-600 truncate block w-full">{deployRemark || "-"}</span>
        </CellTooltip>
      );
    }

    const finalSop = displayData.finalSopExtracted !== undefined ? displayData.finalSopExtracted :
rawItem.phaseV_FinalTesting?.testingInfo?.finalSop || "";
    if (col.type === "finalSopText") {
      if (isEditing) {
        return (
          <textarea className="w-full text-xs border rounded p-1 resize-y bg-emerald-50/30" rows={2}
placeholder="Add Final SOP..." value={finalSop} onChange={(e) => setRowEditData({ finalSopExtracted:
e.target.value })} />
        );
      }
      return (
        <CellTooltip content={finalSop || ""}>
          <span className="text-gray-600 truncate block w-full">{finalSop || "-"}</span>
        </CellTooltip>
      );
    }

    if (col.type === "statusSelect") {
      if (isEditing) {
        return (
          <DropdownMenu>
            <DropdownMenuTrigger asChild>
              <Button variant="outline" className="h-8 w-full justify-start text-left shrink-0 text-[11px]
px-2" onClick={(e) => e.stopPropagation()}>
```

```
              <span className="truncate">{bugStatus}</span>
              <ChevronDown className="ml-1 h-3 w-3 opacity-50 flex-shrink-0" />
            </Button>
          </DropdownMenuTrigger>
          <DropdownMenuContent className="w-36 z-50">
            {bugStatusOptions.map((opt) => (
              <DropdownMenuCheckboxItem
                key={opt}
                checked={bugStatus === opt}
                className="text-[11px]"
                onClick={(e) => {
                  e.stopPropagation();
                  setRowEditData({ statusExtracted: opt });
                }}
              >
                {opt}
              </DropdownMenuCheckboxItem>
            ))}
          </DropdownMenuContent>
        </DropdownMenu>
      );
    } else {
      return <span className="text-gray-700 text-xs truncate block w-full">{bugStatus}</span>;
    }
  }

  if (col.type === "sopText") {
    if (isEditing) {
      return (
        <textarea
          className="w-full text-xs border rounded p-1 resize-y"
          rows={2}
          placeholder="SOP Followed (comma separated)"
          value={sopFollowedStr}
          onChange={(e) => {
            const raw = e.target.value;
            const arr = raw ? raw.split(',').map(s => s.trim()).filter(Boolean) : [];
            setRowEditData({
              sopFollowedExtracted: arr,
              sopFollowedRaw: raw
            });
          }}
        />
      );
    } else {
      return (
        <CellTooltip content={sopFollowedStr !== "-" ? sopFollowedStr : ""}>
          <span className="text-gray-600 text-xs truncate block w-full">{sopFollowedStr || "-"}</span>
        </CellTooltip>
      );
    }
  }
  if (col.key === "rootCause") {
```

```
      const rootCause = rawItem.phaseIII_BugAnalysis?.analysisInfo?.rootCause || "-";
      return (
        <CellTooltip content={rootCause}>
          <span className="text-gray-600 truncate block w-full">{rootCause}</span>
        </CellTooltip>
      );
    }
    if (col.type === "description") {
      if (isEditing) {
        if (isExpectedMode) {
          return (
            <div className="flex flex-col gap-2 w-full p-1 max-w-full">
              <textarea className="w-[230px] text-[14px] border border-gray-300 rounded p-1.5 resize-y
bg-white
                focus:outline-none focus:ring-1 focus:ring-blue-300 transition-all placeholder:text-gray-400"
                rows={2} placeholder="Expected result..." value={expRes}
                onChange={(e) => setRowEditData({ expectedResultExtracted: e.target.value })}
              />
              <textarea className="w-[230px] text-[14px] border border-gray-300 rounded p-1.5 resize-y
bg-white
                focus:outline-none focus:ring-1 focus:ring-blue-300 transition-all placeholder:text-gray-400"
                rows={2}  placeholder="Actual result..." value={actRes}
                onChange={(e) => setRowEditData({ actualResultExtracted: e.target.value })}
              />
            </div>
          );
        } else {
          return (
            <textarea className="w-[230px] text-[14px] border border-gray-300 rounded p-1.5 resize-y
              min-h-[4.5rem] bg-white focus:outline-none focus:ring-1 focus:ring-blue-300 transition-all
              placeholder:text-gray-400" rows={3} placeholder="Bug description..." value={bugDesc}
               onChange={(e) => setRowEditData({ bugDescriptionExtracted: e.target.value })} />
          );
        }
      } else {
        if (isExpectedMode) {
          const maxLines = "line-clamp-4";
          return (
            <CellTooltip content={`Expected: ${expRes}\n\nActual: ${actRes}`}>
              <div className={`text-[14px] text-gray-700 whitespace-pre-wrap ${maxLines} overflow-hidden`}>
                <span className="font-semibold text-gray-900">Expected:</span> {expRes}  <br/><br/>
                <span className="font-semibold text-gray-900">Actual:</span> {actRes}
              </div>
            </CellTooltip>
          );
        } else {
          return (
            <CellTooltip content={bugDesc}>
              <div className="text-[14px] text-gray-700 whitespace-pre-wrap line-clamp-4 overflow-hidden">
                {bugDesc}
              </div>
            </CellTooltip>
          );
```

```
      }
    }
  }
  if (col.type === "prioritySelect") {
    return isEditing ? (
      <DropdownMenu>
        <DropdownMenuTrigger asChild>
          <Button variant="outline" className="h-8 w-full justify-start text-left shrink-0">
             {priority.toUpperCase()}
          </Button>
        </DropdownMenuTrigger>
        <DropdownMenuContent className="w-32">
          {priorityOptions.map((opt) => (
             <DropdownMenuCheckboxItem key={opt} checked={priority === opt}
               onClick={() => setRowEditData({ priorityExtracted: opt })} > {opt.toUpperCase()}
             </DropdownMenuCheckboxItem>
          ))}
        </DropdownMenuContent>
      </DropdownMenu>
    ) : (
      <span className={`px-2 py-1 rounded-full text-[10px] font-semibold tracking-wide uppercase
        ${priority === 'critical' ? 'bg-red-100 text-red-700' :
          priority === 'high' ? 'bg-orange-100 text-orange-700' :
           priority === 'medium' ? 'bg-yellow-100 text-yellow-700' : 'bg-green-100 text-green-700'}`}>
        {priority}
      </span>
    );
  }
  if (col.type === "stackSelect") {
    // Determine which tool is selected (from edit data or from saved bug)
    const activeToolId = displayData.toolIdExtracted ?? rawItem.phaseI_BugReport?.toolInfo?.toolId;
    const activeTool   = toolList?.find(t => (t.id || t._id) === activeToolId);
    const toolStacks   = activeTool && Array.isArray(activeTool.stack) && activeTool.stack.length > 0
      ? activeTool.stack : stackOptions; // fallback to global defaults
    return isEditing ? (
      <DropdownMenu>
        <DropdownMenuTrigger asChild>
          <Button variant="outline" className="h-8 w-full justify-start text-left">{stack} </Button>
        </DropdownMenuTrigger>
        <DropdownMenuContent className="w-32">
          {toolStacks.map((opt) => (
             <DropdownMenuCheckboxItem key={opt} checked={stack === opt}
               onClick={() => setRowEditData({ stackExtracted: opt })}> {opt}
             </DropdownMenuCheckboxItem>
          ))}
        </DropdownMenuContent>
      </DropdownMenu>
    ) : ( <span className="text-gray-600" title={stack}>{stack}</span> );
  }

  if (col.type === "phaseSelect") {
    const currentPhaseName = displayData.currentPhaseExtracted || rawItem.currentPhase || "-";
    const availablePhases = [
```

```
      { label: "Bug Testing", value: "testing", phaseNo: 2 },
      { label: "Bug Analysis", value: "analyze", phaseNo: 3 },
      { label: "Ready to Test", value: "rtt", phaseNo: 4 },
      { label: "Ready to Deploy", value: "rtd", phaseNo: 5 },
      { label: "Closure", value: "deployed", phaseNo: 6 }
    ];

    return isEditing ? (
      <DropdownMenu>
        <DropdownMenuTrigger asChild>
          <Button variant="outline" className="h-8 w-full justify-between text-left shrink-0 text-xs px-2"
             onClick={(e) => e.stopPropagation()}>
             <span className="truncate">{currentPhaseName}</span>
             <ChevronDown className="ml-1 h-3 w-3 opacity-50 flex-shrink-0" />
          </Button>
        </DropdownMenuTrigger>
        <DropdownMenuContent className="w-40 z-50">
          {availablePhases.map((opt) => (
             <DropdownMenuItem key={opt.value} className="text-xs py-1.5 cursor-pointer"
               onClick={(e) => {
                 e.stopPropagation();
                 setRowEditData({ currentPhaseExtracted: opt.label,  bugPhaseNoExtracted: opt.phaseNo });
               }} > {opt.label}
             </DropdownMenuItem>
          ))}
        </DropdownMenuContent>
      </DropdownMenu>
    ) : (  <span className="text-gray-600 truncate block w-full"
       title={`Phase: ${currentPhaseName}`}>{currentPhaseName}</span>
    );
  }
  if (col.type === "testingFlagSelect") {
    const tsFlagOptions = ["Go back to dev", "Test by bug raiser", "Ready for rollout", "-"];
    const flagVal = displayData.testingFlagExtracted ??
          rawItem.phaseIV_Maintenance?.maintenanceInfo?.testingFlag ?? "-";
    const getFlagColor = (val) => {
       if (val === 'Go back to dev' || val === 'Red flag') return 'text-red-500';
       if (val === 'Test by bug raiser' || val === 'Blue Flag') return 'text-blue-500';
       if (val === 'Ready for rollout' || val === 'Green Flag') return 'text-green-500';
       return 'text-gray-300';
    };
    if (isEditing) {
      return (
        <DropdownMenu>
          <DropdownMenuTrigger asChild>
            <Button variant="outline" className="w-full justify-between h-8 px-2 text-xs">
               {flagVal !== "-" ? (
                 <div className="flex items-center gap-1.5 truncate">
                    <Flag className={`w-3.5 h-3.5 shrink-0 ${getFlagColor(flagVal)}`} fill="currentColor" />
                    <span className="truncate text-[11px]">{flagVal}</span>
                 </div>
               ) : (<span className="text-gray-400">Select Flag...</span>)}
               <ChevronDown className="ml-1 w-3 shrink-0" />
```

```
            </Button>
          </DropdownMenuTrigger>
          <DropdownMenuContent className="z-50 min-w-44">
            {tsFlagOptions.map(opt => (
              <DropdownMenuItem key={opt} onClick={() => setRowEditData({ testingFlagExtracted: opt })}
                 className="cursor-pointer">
                 {opt !== "-" ? (
                    <div className="flex items-center text-xs">
                       <Flag className={`w-3.5 h-3.5 mr-2 ${getFlagColor(opt)}`} fill="currentColor" />
                       {opt}
                    </div>
                 ) : ( <span className="text-xs text-gray-400 pl-6">Clear Flag</span> )}
              </DropdownMenuItem>
            ))}
          </DropdownMenuContent>
        </DropdownMenu>
      );
    }
    if (flagVal === "-") return <span className="text-gray-400 w-full px-2 block">-</span>;
    let flagBgClasses = 'bg-gray-50 border-gray-100 text-gray-700';
    if (flagVal === 'Go back to dev' || flagVal === 'Red flag')
       flagBgClasses = 'bg-red-50 border-red-200 text-red-700';
    else if (flagVal === 'Test by bug raiser' || flagVal === 'Blue Flag')
      flagBgClasses = 'bg-blue-50 border-blue-200 text-blue-700';
    else if (flagVal === 'Ready for rollout' || flagVal === 'Green Flag')
      flagBgClasses = 'bg-green-50 border-green-200 text-green-700';
    return (
      <span className={`text-[11px] px-2.5 py-1 font-medium rounded-full border inline-flex items-center
          gap-1.5 whitespace-nowrap ${flagBgClasses}`}>
        <Flag className={`w-3 h-3 ${getFlagColor(flagVal)}`} fill="currentColor" /> {flagVal}
      </span>
    );
  }
  if (col.type === "deploymentStatusSelect") {
    const deployOptions = ["Done", "Hold", "For date", "Approval For deploy", "-"];
    const statusVal = displayData.deploymentStatusExtracted ??
          rawItem.phaseV_FinalTesting?.testingInfo?.deploymentStatus ?? "-";
    if (isEditing) {
      return (
        <DropdownMenu>
          <DropdownMenuTrigger asChild>
            <Button variant="outline" className="w-full justify-between h-8 px-2 text-xs">
              <span className="truncate">{statusVal !== "-" ? statusVal : "Select Status..."}</span>
              <ChevronDown className="ml-1 w-3 shrink-0" />
            </Button>
          </DropdownMenuTrigger>
          <DropdownMenuContent className="z-50 min-w-44">
            {deployOptions.map(opt => (
              <DropdownMenuItem key={opt} onClick={() => setRowEditData({ deploymentStatusExtracted: opt
})}
                 className="cursor-pointer">
                 {opt !== "-" ? opt : <span className="text-xs text-gray-400">Clear</span>}
              </DropdownMenuItem>
```

```
              ))}
            </DropdownMenuContent>
          </DropdownMenu>
        );
      }
      if (statusVal === "-") return <span className="text-gray-400 w-full px-2 block">-</span>;
      return (
        <span className="text-[11px] px-2.5 py-1 font-medium rounded-full border border-emerald-200
bg-emerald-50
          text-emerald-700 inline-flex items-center break-words"> {statusVal}
        </span>
      );
    }
    return null;
  };
  return (
    <div className="w-full">
      <div className="[&>div]:rounded-sm [&>div]:border overflow-x-auto overflow-y-auto
max-h-[calc(100vh-320px)]
          min-h-[160px] mt-4 shadow-sm bg-white relative">
        <Table className="min-w-full">
          <TableHeader>
            <TableRow className="hover:bg-transparent h-14">
              <TableHead className="p-0 sticky left-0 top-0 z-40 bg-white shadow-[2px_0_2px_-2px_#eee]"
                 style={{ width: colWidths.checkbox || 48, minWidth: 48, maxWidth: 120 }}>
                <div className="flex items-center justify-center h-14 w-full">
                  <Checkbox checked={paginatedData?.length > 0 && selectedRows.length ===
paginatedData.length}
                    onCheckedChange={(checked) => {
                      if (checked) {  setSelectedRows(paginatedData.map(item => item._id || item.id)); }
                      else {setSelectedRows([]); }
                    }}
                  />
                </div>
              </TableHead>
              {tableDetails.map((col) => {
                const isPinned = pinnedColumns.includes(col.key);
                const pinStyle = getPinnedStyle(col.key);
                const isPhase2Target = ['testing', 'analyze', 'rtt', 'reanalyze', 'rtd'].includes(phaseId)
                && ['bugStatus', 'sopFollowed', 'remarks', 'testingAttachment',
'assignedDev'].includes(col.key);
                const isPhase3Target = ['analyze', 'rtt', 'reanalyze', 'rtd'].includes(phaseId)
                && ['analyzeRemark', 'sopForSolution', 'delayedReason',
'analyzeAttachment'].includes(col.key);
                const isPhase4Target = ['analyze', 'rtt', 'reanalyze', 'rtd'].includes(phaseId)
                 && ['testingFlag', 'finalTestingRemark'].includes(col.key);
                const isPhase5Target = ['rtd'].includes(phaseId)
                 && ['deploymentStatus', 'deployRemark', 'finalSop'].includes(col.key);
                let textColor = "";
                let bgActiveColor = "white";
                if (isPhase5Target) { textColor = "text-[#059669]"; bgActiveColor = "#ecfdf5";} // emerald-50
                else if (isPhase4Target) { textColor = "text-[#ea580c]"; bgActiveColor = "#fffbeb";    }
                else if (isPhase3Target) {textColor = "text-[#7c3aed]"; bgActiveColor = "#f5f3ff";}
```

```
              else if (isPhase2Target) { textColor = "text-[#4f46e5]"; bgActiveColor = "#eef2ff"; }
              return (
              <TableHead key={col.key} className={`sticky top-0 ${textColor} border-b border-gray-100
                shadow-[2px_0_2px_-2px_#eee]`} style={{ ...pinStyle, width: colWidths[col.key], minWidth:
60,
              maxWidth: 600, zIndex: isPinned ? 50 : 30, backgroundColor: isPinned ? '#fff' : bgActiveColor
}}>
                <ResizableHeader columnKey={col.key} colWidths={colWidths} setColWidths={setColWidths}>
                  <ColumnHeader columnKey={col.key} onPin={handlePinColumn} isPinned={isPinned}> {col.label}
                  </ColumnHeader>
                </ResizableHeader>
              </TableHead>
              );
            })}
            <TableHead className="sticky right-0 top-0 bg-white shadow-[-2px_0_2px_-2px_#eee]"
              style={{ width: 120, minWidth: 60, maxWidth: 600, zIndex: 40 }}>Actions
            </TableHead>
          </TableRow>
        </TableHeader>
        <TableBody>
          {(!paginatedData || paginatedData.length === 0) ? (
            <TableRow>
              <TableCell colSpan={tableDetails.length + 2} className="text-center py-10 text-gray-500">
                No bugs in this phase yet.
              </TableCell>
            </TableRow>
          ) : (
            paginatedData.map((item) => {
              const isEditing = editingIds.includes(item._id);
              const displayData = isEditing && editData[item._id] ? editData[item._id] : item;
              return (
                <BugRightClickMenu key={item._id} item={item} onCancelAll={handleCancelAll}
                  onEdit={() => isRowEditable(item) ? handleEdit(item) : null}
                  onDelete={() => handleDeleteSingle(item._id)} onSaveAll={handleSaveAll}
                  onEditSelected={handleEditSelected} onDeleteSelected={handleDeleteSelected}>
                  <TableRow className="group cursor-pointer hover:bg-slate-50 transition-colors">
                    <TableCell className="p-0 sticky left-0 z-20 !bg-white group-hover:!bg-slate-50
                          transition-colors border-r border-gray-100"
                        style={{ width: colWidths.checkbox || 48, minWidth: 48, maxWidth: 120 }}>
                      <div className="flex items-center justify-center h-full min-h-[64px] w-full">
                        <Checkbox checked={selectedRows.includes(item._id)}
                                  onCheckedChange={() => handleRowSelect(item._id)} />
                      </div>
                    </TableCell>
                    {tableDetails.map((col) => {
                      const isPhase2Target = ['testing', 'analyze', 'rtt',
'reanalyze','rtd'].includes(phaseId)
                          && ['bugStatus', 'sopFollowed', 'remarks', 'testingAttachment', 'assignedDev']
                        .includes(col.key);
                      const isPhase3Target = ['analyze', 'rtt', 'reanalyze', 'rtd'].includes(phaseId)
                && ['analyzeRemark', 'sopForSolution', 'delayedReason',
'analyzeAttachment'].includes(col.key);
                      const isPhase4Target = ['analyze', 'rtt', 'reanalyze', 'rtd'].includes(phaseId)
```

```
                            && ['testingFlag', 'finalTestingRemark'].includes(col.key);
                      const isPhase5Target = ['rtd'].includes(phaseId)
                            && ['deploymentStatus', 'deployRemark', 'finalSop'].includes(col.key);

                      let cellBg = undefined;
                      if (isPhase5Target) {cellBg = "bg-emerald-50/30";}
                      else if (isPhase4Target) { cellBg = "#fffbeb";}
                      else if (isPhase3Target) { cellBg = "#f5f3ff";}
                      else if (isPhase2Target) { cellBg = "#eef2ff";}
                      // if column is pinned, override background to white so hover doesn't break, unless it's
a phase target but testing phase target won't be pinned usually.
                      // Respect editableColumnKeys restriction AND current phase restriction.
                      const colIsEditable = isColEditable(col.key, item, editableColumnKeys);
                      const effectiveIsEditing = isEditing && colIsEditable;
                      const isPinned = pinnedColumns.includes(col.key);
                      const pinnedClasses = isPinned ? "!bg-white group-hover:!bg-slate-50
transition-colors"
                            : "";
                      // Visual cue for locked columns when row is in edit mode
                      // We avoid 'opacity' on sticky columns to prevent transparency issues
                      const lockedStyle = isEditing && !colIsEditable
                        ? { backgroundColor: isPinned ? "#f1f5f9" : "#f8fafc", color: "#94a3b8" }: {};
                      return (
                      <TableCell key={col.key}
                        style={{ ...getPinnedStyle(col.key), width: colWidths[col.key], minWidth: 60,
                          maxWidth: 600,  backgroundColor: !isPinned ? cellBg : undefined,
                          zIndex: isPinned ? 20 : 1,  ...lockedStyle }}
                         className={`${pinnedClasses} ${isPinned ? 'border-r border-gray-100' : ''}`}>
                        {renderCell(col, item, effectiveIsEditing, displayData, (val) =>
                          setEditData((prev) => ({
                            ...prev, [item._id]: { ...(prev[item._id] || item), ...val },
                          }))
                        )}
                      </TableCell>
                      );
                    })}

                    <TableCell className="sticky right-0 !bg-white z-20 group-hover:!bg-slate-50
                     transition-colors shadow-[-2px_0_2px_-2px_#eee] border-l border-gray-100"
                     style={{ width: 120, minWidth: 60, maxWidth: 600 }}>
                      <div className="flex items-center justify-center gap-1.5 py-1">
                        {isEditing ? (
                          <div className="flex gap-1 items-center">
                            <CellTooltip content="Save Changes">
                              <Button size="icon" variant="ghost" onClick={() => handleSave(item._id)}
                               className="h-7 w-7 text-emerald-600 hover:text-emerald-700
hover:bg-emerald-50">
                                <Check className="h-4 w-4" />
                              </Button>
                            </CellTooltip>
                            <CellTooltip content="Cancel">
                              <Button size="icon" variant="ghost" onClick={() => handleCancel(item._id)}
                                className="h-7 w-7 text-slate-400 hover:text-red-500 hover:bg-slate-100">
```

```
                              <X className="h-4 w-4" />
                            </Button>
                          </CellTooltip>
                        </div>
                      ) : (
                        <>
                          <CellTooltip content="Activity Log">
                            <Button  size="icon"  variant="ghost"
                              onClick={(e) => {
                                e.stopPropagation();window.open(`/bugs/${item._id}/activity`, "_blank");
                              }}
                              className="h-7 w-7 text-indigo-500 hover:text-indigo-600 hover:bg-indigo-50"
>
                              <LibraryBig className="h-4 w-4" />
                            </Button>
                          </CellTooltip>
                          {isRowEditable(item) ? ( <BugTableRowMenu onEdit={() => handleEdit(item)} />)
                          : (   <span className="text-xs text-gray-300 select-none px-2">—</span>)}
                        </>
                      )}
                    </div>
                  </TableCell>
                </TableRow>
              </BugRightClickMenu>
            );
          })
        )}
      </TableBody>
    </Table>
  </div>
  {/* Pagination Controls */}
  <div className="flex flex-col sm:flex-row items-center justify-between mt-4 px-2 gap-4">
    <div className="flex flex-col sm:flex-row items-center gap-4">
      <div className="flex items-center gap-2 text-sm text-gray-500">
        <span>Rows per page:</span>
        <select className="border border-gray-200 rounded px-1 py-0.5 bg-white text-gray-700 outline-none"
          value={rowsPerPage} onChange={(e) => {setRowsPerPage(Number(e.target.value));
setCurrentPage(1);}}>
          <option value={10}>10</option>
          <option value={20}>20</option>
          <option value={50}>50</option>
          <option value={100}>100</option>
        </select>
      </div>
      <div className="text-sm text-gray-500">
        Showing {phaseBugs?.length > 0 ? (currentPage - 1) * rowsPerPage + 1 : 0}
        to {Math.min(currentPage * rowsPerPage, phaseBugs?.length || 0)} of {phaseBugs?.length || 0} bugs
      </div>
    </div>
    <div className="flex items-center space-x-2 gap-2">
      <Button  variant="outline" size="sm" onClick={() => setCurrentPage(prev => Math.max(prev - 1, 1))}
        disabled={currentPage === 1}>
        <ChevronLeft className="w-4 h-4 mr-1" /> Previous
```

```
        </Button>
        <div className="text-sm font-medium px-2">Page {currentPage} of {totalPages}</div>
        <Button variant="outline" size="sm" disabled={currentPage >= totalPages}
          onClick={() => setCurrentPage(prev => Math.min(prev + 1, totalPages))} > Next
          <ChevronRight className="w-4 h-4 ml-1" />
        </Button>
      </div>
    </div>
  </div>
 );
}
```

BugTableRowMenu.jsx

```
import { SquarePen } from "lucide-react";
import { Button } from "@/components/ui/button";
function BugTableRowMenu({ onEdit }) {
 return (
   <Button size="icon" variant="ghost"  className="h-7 w-7 text-slate-500 hover:text-cyan-600
hover:bg-cyan-50"
     onClick={onEdit}>
     <SquarePen className="h-4 w-4" />
   </Button>
 );
}
export default BugTableRowMenu;
```

BugViewToggle.jsx

```
import React, { useState } from "react";
import { Search } from "lucide-react";
import BugTable from "./BugTable";
import BugModal from "./BugModal";
export default function BugViewToggle({ phaseBugs, title, actionButton, phaseId, editableColumnKeys,
canEditRow }) {
 const [isModalOpen, setIsModalOpen] = useState(false);
 const [activeBugData, setActiveBugData] = useState(null);
 const [searchQuery, setSearchQuery] = useState("");
 const openModal = (bug) => { setActiveBugData(bug);   setIsModalOpen(true); };
 const closeModal = () => { setIsModalOpen(false);  setActiveBugData(null); };
 const navigateModal = (bug) => { setActiveBugData(bug); };
 const filteredBugs = phaseBugs.filter((bug) => {
   if (!searchQuery) return true;
   const query = searchQuery.toLowerCase();
   const bugId = bug.bugId?.toLowerCase() || "";
   const toolName = bug.phaseI_BugReport?.toolInfo?.toolName?.toLowerCase() || "";
   const desc = bug.phaseI_BugReport?.toolInfo?.bugDescription?.toLowerCase() || "";
   return bugId.includes(query) || toolName.includes(query) || desc.includes(query);
 });
 return (
   <div className="flex flex-col h-full w-full">
     {/* Header Row: Title, Search, Action Button */}
     <div className="flex flex-col md:flex-row items-start md:items-center justify-between mb-4 mt-2 gap-4">
       {title && <h3 className="text-2xl font-bold">{title}</h3>}
```

```
      <div className="flex flex-wrap items-center gap-3 w-full md:w-auto">
        {/* Search Box */}
        <div className="relative flex items-center w-full md:w-auto">
          <Search className="w-4 h-4 absolute left-3 text-gray-400" />
          <input type="text"  placeholder="Search bugs..."   value={searchQuery}
            onChange={(e) => setSearchQuery(e.target.value)}
            className="pl-9 pr-4 py-2 border rounded-lg text-sm focus:outline-none focus:border-cyan-500
              w-full md:w-64 shadow-sm" />
        </div>
        {/* Optional Action Button (New Bug) */}
        {actionButton && <div className="ml-0 md:ml-1">{actionButton}</div>}
      </div>
    </div>
    {/* Always render Table View */}
    <BugTable phaseBugs={filteredBugs} onClickCardToModal={openModal}  phaseId={phaseId}
      editableColumnKeys={editableColumnKeys} canEditRow={canEditRow}/>
    {/* Shared Modal Overlay */}
    <BugModal isOpen={isModalOpen} onClose={closeModal} bugData={activeBugData}  phaseBugs={filteredBugs}
      onNavigate={navigateModal}/>
  </div>
);
}
```

Settings (folder)

AddUser (folder)
Adduser.jsx

```
// AddUser.jsx
import React, { useEffect, useState } from "react";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Asterisk, Upload, Bug, Menu, Loader2, UserRoundPlus,} from "lucide-react";
import { register } from "@/API_Call/Auth";
import { useAuth } from "@/context/AuthContext";
import { DropdownMenu, DropdownMenuContent, DropdownMenuCheckboxItem, DropdownMenuTrigger,
} from "@/components/ui/dropdown-menu";
import { SheetContent, SheetDescription, SheetHeader, SheetTitle, SheetFooter,} from "@/components/ui/sheet";
import { Button } from "@/components/ui/button";
import toast from "react-hot-toast";
function AddUser({ setIsOpen }) {
 const { setAllUsers } = useAuth();
 const [errors, setErrors] = useState({});
 const [isSubmitting, setIsSubmitting] = useState(false);
 const [full_name, setFull_name] = useState("");
 const [email_name, setEmail_name] = useState("");
 const [user_name, setUser_name] = useState("");

 // User role and admin option lists
 const roleOptions_ = ["Admin", "Developer", "Tester", "Bug Reporter"]; //"Manager"
 const adminControlOptions = ["Create", "Edit", "Delete", "View"];
 const adminOptionsList = [   "Share",   "Generate Report",   "Insight View",   "Export", ];
 // Selected multi-select state
 const [roles, setRoles] = useState([]);
 const [adminRights, setAdminRights] = useState([]);
```

```
const [adminOptions, setAdminOptions] = useState([]);
const checkValidation = () => {
  const newErrors = {};
  if (!full_name || !full_name.trim()) newErrors.full_name = "Full Name is required.";
  if (!email_name || !email_name.trim()) newErrors.email_name = "Email is required.";
  else if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email_name))
    newErrors.email_name = "Enter a valid email address.";
  if (!user_name || !user_name.trim()) newErrors.user_name = "Username is required.";
  if (!roles || roles.length === 0) newErrors.roles = "Select at least one role.";
  setErrors(newErrors);
  return {newErrors,};
};

const handleSubmit = async () => {
  const { newErrors } = checkValidation();
  if (Object.keys(newErrors).length > 0) { console.log("❌ Validation Failed:", newErrors);
    return; // stop submit
  }
  // START LOADING
  setIsSubmitting(true);
  const isAdminCheck = roles.includes("Admin");
  let finalAdminRights = [];
  let finalAdminOptions = [];
  if (isAdminCheck) {
    finalAdminRights = Array.isArray(adminRights) ? adminRights.map(v => v.toLowerCase()) : [];
    if (finalAdminRights.some(r => ["create", "edit", "delete"].includes(r)) &&
      !finalAdminRights.includes("view")) {  finalAdminRights.push("view"); }
    finalAdminOptions = Array.isArray(adminOptions)
      ? adminOptions.map(v => v.toLowerCase().replace(" ", "_")) : [];
  }
  const userData = {     name: full_name,     email: email_name,     username: user_name,
    password: "password123", // Default password for new users
    role: roles.includes("Admin") ? "admin" : "user",
    roletype: roles.map(r => r === "Developer" ? "dev" : (r === "Tester" ? "tester" : "bugreporter")),
    adminControl: finalAdminRights,     adminOption: finalAdminOptions,
  };
  try {
    const res = await register(userData);
    if (res.success && res.user) {
      // Map the backend DB structure locally
      const mappedUser = { ...res.user, id: res.user._id };
      // Push globally so table immediately updates without reloading DB
      setAllUsers(prev => [...prev, mappedUser]);
      toast.success("User created successfully", { position: "top-center" });
      resetForm();  setIsOpen(false);
    } else { toast.error(res.message || "Create user failed", { position: "top-center" });}
  } catch (err) {
    console.error("API Submit Error:", err);
    toast.error(err.message || "Create user failed", {  position: "top-center", });
  } finally {
    // STOP LOADING
    setIsSubmitting(false);
  }
```

```
};
const resetForm = () => {setErrors({}); setUser_name("");
  // reset user related fields
  setRoles([]); setAdminRights([]);  setAdminOptions([]);  setFull_name(""); setEmail_name("");
};

const handleCancel = () => { setIsOpen(false); };
return (
  <SheetContent className="justify-between w-[96%] sm:!max-w-none sm:w-[390px] lg:w-[390px] overflow-y-auto
      rounded-xl bg-white shadow-lg border border-gray-100 my-[3.2%] mx-[1.8%] sm:m-[1.2%] h-[97%] p-6">
    <SheetHeader>
      <div className="flex items-center gap-3 pb-4 border-b">
        <div className="w-12 h-12 bg-gradient-to-br from-blue-500 to-sky-200 rounded-lg flex items-center
            justify-center shadow-md">
          <UserRoundPlus className="w-6 h-6 text-white" />
        </div>
        <div>
          <h3 className="font-semibold text-xl">Create New User</h3>
          <p className="text-xs sm:text-sm text-gray-500"> Add user details and permissions below. </p>
        </div>
      </div>
      <div className="hidden">
        <SheetTitle>  <Bug className="w-6 h-6 text-white" /> Create New Bug </SheetTitle>
        <SheetDescription>  Fill in the details below to report a new bug. </SheetDescription>
      </div>
    </SheetHeader>
    {/* Bug Form Component */}
    <div className="flex flex-col gap-4 h-[92%]">
      {/* bug form start here */}
      <div className="space-y-6 py-4 rounded-md shadow-sm">
        {/* Tool Name */}
        <div className="space-y-2">
          <Label  htmlFor="tool"  className="text-sm font-medium flex items-center">
            <Asterisk size={12} /> Full Name
          </Label>

          <Input id="full_name" type="text" placeholder="ex: John Doe" value={full_name}
            onChange={(e) => {
              setFull_name(e.target.value);
              if (errors.full_name) {
                setErrors((prev) => {  const n = { ...prev };  delete n.full_name;  return n;
                });
              }
            }}
            className={`border-2 rounded-lg h-10 px-3 ${
              errors.full_name ? "border-red-300" : "border-gray-200"
            } focus:outline-none focus:ring-2 focus:ring-blue-200`}
          />
          {errors.full_name && ( <p className="text-red-400 text-xs">{errors.full_name}</p>  )}
        </div>
        {/* Assign Tester */}
        <div className="grid grid-cols-1 gap-4">
          <div className="space-y-2">
```

```
            <Label htmlFor="tester" className="text-sm font-medium flex gap-0 items-center" >
              <Asterisk size={12} /> Email Address
            </Label>
            <Input id="email_name" type="email" placeholder="ex: your@email.com" value={email_name}
              onChange={(e) => {
                setEmail_name(e.target.value);
                if (errors.email_name) {
                  setErrors((prev) => {  const n = { ...prev };    delete n.email_name;  return n;
                  });
                }
              }}
              className={`border-2 rounded-lg h-10 px-3 ${
                errors.email_name ? "border-red-300" : "border-gray-200"
              } focus:outline-none focus:ring-2 focus:ring-blue-200`}
            />
            {errors.email_name && ( <p className="text-red-400 text-xs">{errors.email_name}</p> )}
          </div>
          <div className="space-y-2">
            <Label  htmlFor="priority" className="flex text-sm font-medium items-center">
              <Asterisk size={12} /> Username
            </Label>
            <Input id="user_name" type="text" placeholder="ex: param18" value={user_name}
              onChange={(e) => {
                setUser_name(e.target.value);
                if (errors.user_name) {
                     setErrors((prev) => {   const n = { ...prev }; delete n.user_name; return n;});
                }
              }}
              className={`border-2 rounded-lg h-10 px-3 ${
                errors.user_name ? "border-red-300" : "border-gray-200"
              } focus:outline-none focus:ring-2 focus:ring-blue-200`}
            />
            {errors.user_name && ( <p className="text-red-400 text-xs">{errors.user_name}</p> )}
          </div>
        </div>
        {/* Roles & Admin controls */}
        <div className="grid grid-cols-1 gap-4">
          <div>
            <Label htmlFor="priority"className="flex text-sm font-medium items-center" >
              <Asterisk size={12} />  Roles
            </Label>
            <DropdownMenu>
              <DropdownMenuTrigger asChild>
                <Button variant="outline"
                  className="h-10 w-full mt-2 text-left flex items-center justify-between">
                  <span className="truncate"> {roles.length > 0 ? roles.join(", ") : "Select Roles"}
</span>
                  {roles.length > 0 && (
                    <span className="ml-3 bg-blue-50 text-blue-700 rounded-full px-2 py-0.5 text-xs">
                      {roles.length}
                    </span>
                  )}
                </Button>
```

```
            </DropdownMenuTrigger>
            <DropdownMenuContent className="w-56">
              {roleOptions_.map((option) => (
                <DropdownMenuCheckboxItem key={option} checked={roles.includes(option)}
                  onClick={(e) => { e.preventDefault(); setRoles((prev) =>
                      prev.includes(option) ? prev.filter((r) => r !== option)  : [...prev, option]
                    );
                  }} >
                  {option}
                </DropdownMenuCheckboxItem>
              ))}
            </DropdownMenuContent>
          </DropdownMenu>
          {errors.roles && ( <p className="text-red-400 text-xs mt-1">{errors.roles}</p>  )}
        </div>
        <div>
          <Label className="text-sm font-medium">Admin Right</Label>
          <div className="w-full" title={!roles.includes("Admin") ? "This is admin rights" : ""}>
            <DropdownMenu>
              <DropdownMenuTrigger asChild>
                <Button variant="outline"  disabled={!roles.includes("Admin")}
                  className="h-10 w-full mt-2 text-left flex items-center justify-between">
                  <span className="truncate">
                    {adminRights.length > 0  ? adminRights.join(", ") : "Select Rights"}
                  </span>
                  {adminRights.length > 0 && (
                    <span className="ml-3 bg-blue-50 text-blue-700 rounded-full px-2 py-0.5 text-xs">
                      {adminRights.length}
                    </span>
                  )}
                </Button>
              </DropdownMenuTrigger>
              <DropdownMenuContent className="w-56">
                {adminControlOptions.map((option) => (
                  <DropdownMenuCheckboxItem key={option} checked={adminRights.includes(option)}
                    onClick={(e) => {
                      e.preventDefault();
                      setAdminRights((prev) =>
                        prev.includes(option) ? prev.filter((r) => r !== option): [...prev, option]
                      );
                    }} >
                    {option}
                  </DropdownMenuCheckboxItem>
                ))}
              </DropdownMenuContent>
            </DropdownMenu>
          </div>
        </div>
      </div>
      <div className="grid grid-cols-1 gap-4">
        <div>
          <Label className="text-sm font-medium">Admin Option</Label>
          <div className="w-full" title={!roles.includes("Admin") ? "This is admin rights" : ""}>
```

```
              <DropdownMenu>
                <DropdownMenuTrigger asChild>
                  <Button  variant="outline" disabled={!roles.includes("Admin")}
                    className="h-10 w-full mt-2 text-left flex items-center justify-between" >
                    <span className="truncate">
                      {adminOptions.length > 0 ? adminOptions.join(", "): "Select Options"}
                    </span>
                    {adminOptions.length > 0 && (
                      <span className="ml-3 bg-blue-50 text-blue-700 rounded-full px-2 py-0.5 text-xs">
                        {adminOptions.length}
                      </span>
                    )}
                  </Button>
                </DropdownMenuTrigger>
                <DropdownMenuContent className="w-56">
                  {adminOptionsList.map((option) => (
                    <DropdownMenuCheckboxItem key={option}  checked={adminOptions.includes(option)}
                      onClick={(e) => {  e.preventDefault();
                        setAdminOptions((prev) =>
                          prev.includes(option) ? prev.filter((r) => r !== option)  : [...prev, option]
                        );
                      }} >
                      {option}
                    </DropdownMenuCheckboxItem>
                  ))}
                </DropdownMenuContent>
              </DropdownMenu>
            </div>
          </div>
        </div>
      </div>
      {/* Add User end here */}
      <SheetFooter className="gap-3 py-2">
        <Button variant="outline" onClick={handleCancel}> Cancel </Button>
        <Button disabled={isSubmitting}  onClick={handleSubmit} > // className="w-full"
          {isSubmitting ? (
            <div className="flex items-center gap-2">  <Loader2 className="animate-spin w-4 h-4" />
              Adding User...
            </div>
          ) : ( "Add User")}
        </Button>
      </SheetFooter>
    </div>
  </SheetContent>
);
}
export default AddUser;
```

Tools (folder)
AddTool.jsx

```
import React, { useState } from "react";
import { Input } from "@/components/ui/input";
```

```
import { Label } from "@/components/ui/label";
import { Asterisk, DraftingCompass, Loader2 } from "lucide-react";
import { SheetContent, SheetHeader, SheetTitle, SheetDescription, SheetFooter,} from "@/components/ui/sheet";
import { Button } from "@/components/ui/button";
import toast from "react-hot-toast";
import { Textarea } from "@/components/ui/textarea";
import { updateTool } from "@/API_Call/Tool";
import { useAuth } from "@/context/AuthContext";
function AddTool({ setIsOpen, onToolAdded }) {
 const { stacksList, setToolList, allUsers } = useAuth();
 const [errors, setErrors] = useState({});
 const [isSubmitting, setIsSubmitting] = useState(false);
 // Tool Schema States
 const [toolName, setToolName] = useState("");
 const [toolDescription, setToolDescription] = useState("");
 const [libraryName, setLibraryName] = useState("");
 const [htmlVersion, setHtmlVersion] = useState("");
 const [selectedStack, setSelectedStack] = useState("");
 const [testerId, setTesterId] = useState("");
 const [devId, setDevId] = useState("");
 const [SOP, setSOP] = useState("");
 const [ReleaseNotes, setReleaseNotes] = useState("");
 const checkValidation = () => {
   const newErrors = {};
   if (!toolName || !toolName.trim()) {  newErrors.toolName = "Tool Name is required."; }
   setErrors(newErrors);
   return { newErrors };
 };

 const handleSubmit = async () => {
   const { newErrors } = checkValidation();
   if (Object.keys(newErrors).length > 0) { return; }
   setIsSubmitting(true);
   const toolData = { toolName, toolDescription, libraryName, htmlVersion,
     stack: selectedStack ? [selectedStack] : [],  testerId, devId, SOP, ReleaseNotes, };

   try {
     const response = await updateTool(toolData);
     if (response.success) {
       toast.success("Tool added successfully", { position: "top-center" });
       if (onToolAdded) onToolAdded();
       // Push globally so table immediately updates without reloading DB
       if (response.data && response.data.results && response.data.results.length > 0) {
          const newToolBackend = response.data.results[0].tool;
          if (newToolBackend) {  const mappedTool = { ...newToolBackend, id: newToolBackend._id };
            setToolList(prev => [...prev, mappedTool]);
          }
       }
       resetForm();
       setIsOpen(false);
     } else { toast.error(response.message || "Failed to create tool"); }
   } catch (err) {
     console.error("API Submit Error:", err);
```

```
    toast.error(err.message || "Failed to create tool", { position: "top-center" });
  } finally {  setIsSubmitting(false); }
};

const resetForm = () => {
  setErrors({}); setToolName(""); setToolDescription(""); setLibraryName(""); setHtmlVersion("");
  setSelectedStack(""); setTesterId(""); setDevId(""); setSOP("");  setReleaseNotes("");
};
const handleCancel = () => { setIsOpen(false);};
return (
  <SheetContent className="justify-between w-[96%] sm:!max-w-none sm:w-[500px] lg:w-[500px] overflow-y-auto
      rounded-xl bg-white shadow-lg border border-gray-100 my-[3.2%] mx-[1.8%] sm:m-[1.2%] h-[97%] p-6">
    <SheetHeader>
      <div className="flex items-center gap-3 pb-4 border-b">
        <div className="w-12 h-12 bg-gradient-to-br from-green-500 to-blue-200 rounded-lg flex items-center
            justify-center shadow-md">
          <DraftingCompass className="w-6 h-6 text-white" />
        </div>
        <div>
          <h3 className="font-semibold text-xl">Create New Tool</h3>
          <p className="text-xs sm:text-sm text-gray-500">  Add tool details and specs below.</p>
        </div>
      </div>
      <div className="hidden">
        <SheetTitle>Create New Tool</SheetTitle> <SheetDescription>Form to add a tool</SheetDescription>
      </div>
    </SheetHeader>
    <div className="flex flex-col gap-4 h-[92%]">
      <div className="space-y-6 py-4 rounded-md shadow-sm">
        {/* Tool Name */}
        <div className="space-y-2">
          <Label htmlFor="toolName" className="text-sm font-medium flex items-center">
            <Asterisk size={12} className="text-red-500" /> Tool Name
          </Label>
          <Input id="toolName"  type="text"  placeholder="" value={toolName}
            onChange={(e) => {
              setToolName(e.target.value);
              if (errors.toolName) {
                setErrors((prev) => {  const n = { ...prev };  delete n.toolName;  return n; });
              }
            }}
            className={`border-2 rounded-lg h-10 px-3 ${
              errors.toolName ? "border-red-300" : "border-gray-200"
            } focus:outline-none focus:ring-2 focus:ring-blue-200`}
          />
          {errors.toolName &&  (  <p className="text-red-400 text-xs">{errors.toolName}</p> )}
        </div>
        {/* Description */}
        <div className="space-y-2">
          <Label htmlFor="toolDescription" className="text-sm font-medium"> Description </Label>
          <Textarea id="toolDescription" placeholder="Brief description of the tool"
            value={toolDescription} onChange={(e) => setToolDescription(e.target.value)}
            className="text-sm border-2 rounded-lg border-gray-200" rows={2} />
```

```
        </div>
        {/* Stack */}
        <div className="space-y-2">
          <Label htmlFor="stack" className="text-sm font-medium">  Stack </Label>
          <select id="stack" value={selectedStack} onChange={(e) => setSelectedStack(e.target.value)}
            className="w-full border-2 rounded-lg h-10 px-3 border-gray-200 bg-white text-sm
             focus:outline-none focus:ring-2 focus:ring-blue-200">
            <option value="" disabled>Select a stack to add...</option>
            {stacksList && stacksList.map(stackOpt => (
               <option key={stackOpt} value={stackOpt}>{stackOpt}</option>
            ))}
          </select>
        </div>
                  {/* Library & HTML Details */}
        {selectedStack === "Script" && (
          <div className="grid grid-cols-2 gap-4">
            <div className="space-y-2">
              <Label htmlFor="libraryName" className="text-sm font-medium">  Library Version </Label>
              <Input  id="libraryName"  placeholder="" value={libraryName}
                onChange={(e) => setLibraryName(e.target.value)}
                className="border-2 rounded-lg h-10 px-3 border-gray-200"
              />
            </div>
            <div className="space-y-2">
              <Label htmlFor="htmlVersion" className="text-sm font-medium"> HTML Version </Label>
              <Input id="htmlVersion" placeholder="" value={htmlVersion}
                onChange={(e) => setHtmlVersion(e.target.value)}
                className="border-2 rounded-lg h-10 px-3 border-gray-200"
              />
            </div>
          </div>
        )}
        <div className="grid grid-cols-2 gap-4">
          <div className="space-y-2">
            <Label htmlFor="testerId" className="text-sm font-medium">   Responsible Tester </Label>
            <select id="testerId" value={testerId}  onChange={(e) => setTesterId(e.target.value)}
              className="w-full border-2 rounded-lg h-10 px-3 border-gray-200 bg-white text-sm
               focus:outline-none focus:ring-2 focus:ring-blue-200" >
              <option value="">None Assigned</option>
              {allUsers && allUsers.filter(u => Array.isArray(u.roletype) ? u.roletype.includes("tester")
              : u.roletype === "tester").map(u => (
                <option key={u.id} value={u.id}>{u.name || u.username || u.email}</option>
              ))}
            </select>
          </div>
          <div className="space-y-2">
            <Label htmlFor="devId" className="text-sm font-medium">  Responsible Dev </Label>
            <select id="devId" value={devId} onChange={(e) => setDevId(e.target.value)}
              className="w-full border-2 rounded-lg h-10 px-3 border-gray-200 bg-white text-sm
               focus:outline-none focus:ring-2 focus:ring-blue-200" >
              <option value="">None Assigned</option>
              {allUsers && allUsers.filter(u => Array.isArray(u.roletype) ? u.roletype.includes("dev")
              : u.roletype === "dev").map(u => (
```

```
                <option key={u.id} value={u.id}>{u.name || u.username || u.email}</option>
              ))}
            </select>
          </div>
        </div>
        {/* SOP */}
        <div className="space-y-2">
          <Label htmlFor="sop" className="text-sm font-medium"> SOP Document Link / Text </Label>
          <Textarea id="sop"  placeholder="SOP guidelines" value={SOP} rows={2}
           onChange={(e) => setSOP(e.target.value)} className="text-sm border-2 rounded-lg border-gray-200"
/>
        </div>
        {/* Release Notes */}
        <div className="space-y-2">
          <Label htmlFor="releaseNotes" className="text-sm font-medium"> Release Notes</Label>
          <Textarea id="releaseNotes" placeholder="Latest release details" value={ReleaseNotes}
            onChange={(e) => setReleaseNotes(e.target.value)} rows={2}
            className="text-sm border-2 rounded-lg border-gray-200"  />
        </div>
      </div>
      <SheetFooter className="gap-3 py-2">
        <Button variant="outline" onClick={handleCancel}>  Cancel </Button>
        <Button disabled={isSubmitting} onClick={handleSubmit}>
          {isSubmitting ? (
            <div className="flex items-center gap-2">
              <Loader2 className="animate-spin w-4 h-4" /> Adding Tool...
            </div>
          ) : ( "Add Tool" )}
        </Button>
      </SheetFooter>
    </div>
  </SheetContent>
 );
}
export default AddTool;
```

TableTool.jsx

```
import React, { useState, useEffect, useRef } from "react";
import { Upload, ArrowUp, ArrowDown, Pin, ChevronLeft, ChevronRight, ChevronDown } from "lucide-react";
import { Button } from "@/components/ui/button";
import { Checkbox } from "@/components/ui/checkbox";
import { Input } from "@/components/ui/input";
import { Textarea } from "@/components/ui/textarea";
import toast from "react-hot-toast";
import { updateTool } from "@/API_Call/Tool";
import { useAuth } from "@/context/AuthContext";

import { DropdownMenu, DropdownMenuContent, DropdownMenuItem, DropdownMenuTrigger,
} from "@/components/ui/dropdown-menu";
import { Table, TableBody, TableCell, TableHead, TableHeader, TableRow,} from "@/components/ui/table";
import TableContextMenu from "../RightClickMenu";
import TableRowMenu from "../TableMenu";
const tableDetails = [
```

```
 { key: "toolName", label: "Tool Name", type: "text", width: 180 },
 { key: "toolDescription", label: "Description", type: "text", width: 220 },
 { key: "stack", label: "Stack", type: "text", width: 200 },
 { key: "testerId", label: "Responsible Tester", type: "userSelect", width: 180 },
 { key: "devId", label: "Responsible Dev", type: "userSelect", width: 180 },
 { key: "SOP", label: "SOP Document", type: "textarea", width: 220 },
 { key: "ReleaseNotes", label: "Release Notes", type: "textarea", width: 220 },
 { key: "libraryName", label: "Library Version", type: "text", width: 120 },
 { key: "htmlVersion", label: "HTML Version", type: "text", width: 120 },
];
const ResizableHeader = ({ children, columnKey, colWidths, setColWidths }) => {
 const startX = useRef(0);
 const startWidth = useRef(0);
 const handleMouseDown = (e) => {
   e.preventDefault(); startX.current = e.clientX;
   startWidth.current = colWidths[columnKey];
   document.addEventListener("mousemove", handleMouseMove);
   document.addEventListener("mouseup", handleMouseUp);
 };

 const handleMouseMove = (e) => {
   const diff = e.clientX - startX.current;
   setColWidths((prev) => ({ ...prev, [columnKey]: Math.max(60, startWidth.current + diff), }));
 };

 const handleMouseUp = () => {
   document.removeEventListener("mousemove", handleMouseMove);
   document.removeEventListener("mouseup", handleMouseUp);
 };

 return (
   <div className="flex items-center group select-none" style={{ width: colWidths[columnKey], minWidth: 48
}}>
     <div className="flex-1">{children}</div>
     <div onMouseDown={handleMouseDown}
       className="w-4 h-6 cursor-col-resize ml-1 flex items-center justify-center text-gray-300
        hover:text-gray-500">|
     </div>
   </div>
 );
};
const ColumnHeader = ({ children, onPin, isPinned, columnKey }) => {
 return (
   <DropdownMenu>
     <DropdownMenuTrigger asChild>
       <div className="flex items-center gap-2 cursor-pointer group">
         <span>{children}</span> {isPinned && <Pin className="w-4 h-4 text-blue-500" />}
       </div>
     </DropdownMenuTrigger>
     <DropdownMenuContent align="start">
       <DropdownMenuItem onClick={() => onPin(columnKey)}>
         <Pin className="w-4 h-4 mr-2" /> {isPinned ? "Unpin Column" : "Pin Column"}
       </DropdownMenuItem>
```

```
      </DropdownMenuContent>
    </DropdownMenu>
  );
};
const TableTool = ({ isOpen, setIsOpen }) => {
  const { toolList, setToolList, allUsers, user: authUser } = useAuth();
  const currentUser = allUsers?.find(u => u._id === authUser?.id || u.id === authUser?.id) || authUser;
  const canCreate = currentUser?.role === "superadmin" || currentUser?.adminControl?.includes("create");
  const canEdit = currentUser?.role === "superadmin" || currentUser?.adminControl?.includes("edit");
  const canDelete = currentUser?.role === "superadmin" || currentUser?.adminControl?.includes("delete");
  const [editingIds, setEditingIds] = useState([]);
  const [editData, setEditData] = useState({});
  const [pinnedColumns, setPinnedColumns] = useState([]);
  const [colWidths, setColWidths] = useState(
    Object.fromEntries(tableDetails.map((col) => [col.key, col.width || 160]))
  );
  const [selectedRows, setSelectedRows] = useState([]);
  const [currentPage, setCurrentPage] = useState(1);
  const [rowsPerPage, setRowsPerPage] = useState(10);
   const totalPages = Math.max(1, Math.ceil((toolList?.length || 0) / rowsPerPage));
  const paginatedData = toolList ? toolList.slice((currentPage - 1) * rowsPerPage, currentPage * rowsPerPage)
: [];

  const handlePinColumn = (columnKey) => {
    setPinnedColumns((prev) =>
      prev.includes(columnKey) ? prev.filter((col) => col !== columnKey) : [...prev, columnKey]
    );
  };

  const getPinnedStyle = (columnKey) => {
    if (!pinnedColumns.includes(columnKey) && columnKey !== "checkbox") return {};
    let left = colWidths.checkbox || 48;
    for (const col of tableDetails) {
      if (col.key === columnKey) break;
      if (pinnedColumns.includes(col.key)) { left += colWidths[col.key] || 160; }
    }
    if (columnKey === "checkbox") left = 0;
    return {  position: "sticky", left: `${left}px`, zIndex: 10,
              background: "white",  boxShadow: "2px 0 2px -2px #eee",
    };
  };
  const handleSave = async (id) => {
    const updatingData = editData[id];
    if (!updatingData) return;
    // Convert stack back to array if modified as string
    if (typeof updatingData.stack === "string") {
      updatingData.stack = updatingData.stack.split(",").map(i => i.trim()).filter(Boolean);
    }
    // update to API
    const res = await updateTool({ ...updatingData, toolName: updatingData.toolName });
    if (res.success) {
      toast.success("Tool updated!");
      // Update global context cache instantly
```

```
    setToolList((prev) => prev.map((t) => (t.id === id ? { ...t, ...updatingData } : t)) );
    handleCancel(id);
  } else { toast.error("Update failed."); }
};
const handleCancel = (id) => {
  setEditingIds((prev) => prev.filter((eid) => eid !== id));
  setEditData((prev) => { const n = { ...prev }; delete n[id]; return n; });
};
const handleEditSelected = () => {
  if (selectedRows.length > 0) {
    setEditingIds(selectedRows);
    const newEditData = {};
    selectedRows.forEach((id) => {
      const item = toolList.find((i) => i.id === id || i._id === id);
      if (item) newEditData[id] = { ...item };
    });
    setEditData(newEditData);
  }
};
const handleDeleteSelected = async () => {
  setToolList(prev => prev.filter(it => !selectedRows.includes(it.id) && !selectedRows.includes(it._id)));
  setSelectedRows([]); toast.success("Selected tools deleted!");
};
const handleDeleteSingle = async (id) => {
  setToolList(prev => prev.filter(it => it.id !== id && it._id !== id)); toast.success("Tool deleted!");
};
const handleSaveAll = async () => {
  for (const id of editingIds) {  if (editData[id]) await handleSave(id);}
  setSelectedRows([]);
};
const handleCancelAll = () => { setEditingIds([]); setEditData({});  setSelectedRows([]);};
const handleRowSelect = (id) => {
  setSelectedRows((prev) =>  prev.includes(id) ? prev.filter((r) => r !== id) : [...prev, id]  );
};
const renderCell = (col, item, isEditing, displayData) => {
  if (col.type === "userSelect") {
    const currentUserId = displayData[col.key] || "";
    const currentObj = typeof currentUserId === "object" ? currentUserId
         : allUsers?.find(u => u.id === currentUserId || u._id === currentUserId);
    const displayVal = currentObj?.name || "Unassigned";

    if (isEditing) {
      return (
        <DropdownMenu>
          <DropdownMenuTrigger asChild>
            <Button variant="outline" className="h-8 w-full justify-between text-left shrink-0
             text-xs px-2" onClick={(e) => e.stopPropagation()}>
              <span className="truncate">{displayVal}</span>
              <ChevronDown className="ml-1 h-3 w-3 opacity-50 flex-shrink-0" />
            </Button>
          </DropdownMenuTrigger>
          <DropdownMenuContent className="w-48 z-50 max-h-60 overflow-y-auto">
            <DropdownMenuItem className="text-xs py-1.5 cursor-pointer"
```

```
              onClick={(e) => {
                e.stopPropagation();
                setEditData((prev) => ({ ...prev, [item.id]: { ...prev[item.id], [col.key]: null }}));
              }}>
              Unassigned
            </DropdownMenuItem>
            {allUsers?.filter(u => {
              const role = Array.isArray(u.roletype) ? u.roletype.join(", ").toLowerCase()
              : (u.roletype?.toLowerCase() || "");
              if (col.key === "testerId") return role === "tester";
              if (col.key === "devId") return role === "developer" || role === "dev";
              return true;
            }).map((u) => (
               <DropdownMenuItem key={u.id || u._id}   className="text-xs py-1.5 cursor-pointer"
                 onClick={(e) => {
                   e.stopPropagation();
                   setEditData((prev) => ({
                     ...prev, [item.id]: { ...prev[item.id], [col.key]: u._id || u.id }
                   }));
                 }} >
                 {u.name}
               </DropdownMenuItem>
            ))}
          </DropdownMenuContent>
        </DropdownMenu>
      );
    } else {
      const val = item[col.key];
      const disp = val?.name || (typeof val === "object" ? val?.name : (allUsers?.find(u => u.id === val
            || u._id === val)?.name)) || "Unassigned";
      return <span className="text-sm truncate block text-gray-700" title={val?.email || ""}>{disp}</span>;
    }
  }
  if (col.type === "textarea") {
    const val = displayData[col.key] || "";
    if (isEditing) {
      return (
        <Textarea value={val} onChange={(e) =>
            setEditData((prev) => ({
              ...prev, [item.id]: { ...prev[item.id], [col.key]: e.target.value },
            }))
          }
          className="w-full min-h-[60px] text-xs resize-y" />
      );
    } else {
      const displayVal = item[col.key] || "";
      return <div className="text-sm max-h-[80px] overflow-y-auto whitespace-pre-wrap
            text-gray-700">{displayVal || "-"}</div>;
    }
  }
  if (isEditing) {
    const val = displayData[col.key] || "";
    const displayVal = Array.isArray(val) ? val.join(", ") : val;
```

```
    return (
      <Input value={displayVal} className="h-8"
        onChange={(e) =>
          setEditData((prev) => ({  ...prev, [item.id]: { ...prev[item.id], [col.key]: e.target.value }, }))
        }
      />
    );
  } else {
    const val = item[col.key] || "";
    const displayVal = Array.isArray(val) ? val.join(", ") : val;
    return <span className="text-sm truncate block" title={displayVal}>{displayVal || "-"}</span>;
  }
};

return (
  <div className="w-full">
    <div className="[&>div]:rounded-sm [&>div]:border rounded-xl border border-gray-200 overflow-x-auto
        overflow-y-auto h-[calc(100vh-280px)] bg-white shadow-sm">
      <Table className="min-w-full">
        <TableHeader>
          <TableRow className="hover:bg-transparent bg-slate-50">
            <TableHead
             className="p-0 flex items-center justify-center h-12 sticky left-0 z-30 border-r
border-gray-200"
              style={{ width: 48, minWidth: 48, background: "#f8fafc", top: 0 }}>
              <Checkbox
                checked={paginatedData?.length > 0 && selectedRows.length === paginatedData.length}
                onCheckedChange={(checked) => {
                  if (checked) { setSelectedRows(paginatedData.map(item => item.id || item._id)); }
                  else {  setSelectedRows([]); }
                }}
              />
            </TableHead>
            {tableDetails.map((col) => {
              const isPinned = pinnedColumns.includes(col.key);
              const pinStyle = getPinnedStyle(col.key);
              return (
              <TableHead  key={col.key} className="h-12 border-gray-200 font-semibold text-gray-700 sticky"
                style={{ ...pinStyle,  width: colWidths[col.key], backgroundColor: "#f8fafc",
                  top: 0, zIndex: isPinned ? 30 : 20
                }} >
                <ResizableHeader columnKey={col.key} colWidths={colWidths} setColWidths={setColWidths}>
                  <ColumnHeader columnKey={col.key} onPin={handlePinColumn} isPinned={isPinned}>
                    {col.label}
                  </ColumnHeader>
                </ResizableHeader>
              </TableHead>
              );
            })}
            <TableHead className="sticky right-0 bg-slate-50 text-center"
             style={{ width: 96, top: 0, zIndex: 30 }}>
              Actions
            </TableHead>
```

```
          </TableRow>
        </TableHeader>
        <TableBody>
          {!paginatedData || paginatedData.length === 0 ? (
            <TableRow>
              <TableCell colSpan={tableDetails.length + 2} className="h-32 text-center text-gray-400">
                No tools found. Add a tool to get started.
              </TableCell>
            </TableRow>
          ) : (
            paginatedData.map((item) => {
              const isEditing = editingIds.includes(item.id);
              const displayData = isEditing && editData[item.id] ? editData[item.id] : item;
              return (
                <TableContextMenu key={item.id || item._id} item={item}
                  onEdit={() => {
                    setEditingIds([item.id]); setEditData(prev => ({ ...prev, [item.id]: { ...item } }));
                  }}
                  onDelete={() => handleDeleteSingle(item.id || item._id)}
                  onSaveAll={handleSaveAll}  onEditSelected={handleEditSelected}
                  onDeleteSelected={handleDeleteSelected} onCancelAll={handleCancelAll} >
                  <TableRow className="cursor-pointer hover:bg-slate-50/80 transition-colors border-b">
                    <TableCell className="p-0 flex items-center justify-center h-14 sticky left-0 z-10
                       bg-white border-r border-gray-100"  style={{ width: 48, minWidth: 48, left: 0 }} >
                      <Checkbox checked={selectedRows.includes(item.id)}
                        onCheckedChange={() => handleRowSelect(item.id)} />
                    </TableCell>
                    {tableDetails.map((col) => (
                      <TableCell key={col.key} style={{ ...getPinnedStyle(col.key), width:
colWidths[col.key],
                           maxWidth: colWidths[col.key] }}> {renderCell(col, item, isEditing, displayData)}
                      </TableCell>
                    ))}
                    <TableCell className="sticky right-0 bg-white z-10 w-24 px-2"
                      style={{ boxShadow: "-2px 0 2px -2px #eee" }}>
                      {isEditing ? (
                        <div className="flex gap-1 justify-center">
                          <Button size="sm" onClick={() => handleSave(item.id)} className="h-7 px-2
                            text-xs">Save</Button>
                          <Button size="sm" variant="outline" onClick={() => handleCancel(item.id)}
                            className="h-7 px-2 text-xs">Cancel</Button>
                        </div>
                      ) : (() => {
                        if (!canEdit && !canDelete) return <span className="text-gray-400 text-xs">-</span>;
                        return (
                          <TableRowMenu item={item}
                            onEdit={canEdit ? () => {
                              setEditingIds([item.id]);
                              setEditData(prev => ({ ...prev, [item.id]: { ...item } }));
                            } : undefined}
                            onDelete={canDelete ? () => handleDeleteSingle(item.id || item._id) : undefined}
                            onArchive={() => toast.success("Archive tool")} />
                        );
```

```
                        })()}
                      </TableCell>
                    </TableRow>
                  </TableContextMenu>
                );
              })
            )}
          </TableBody>
        </Table>
      </div>
      {/* Pagination Controls */}
      <div className="flex flex-col sm:flex-row items-center justify-between mt-4 px-2 gap-4">
        <div className="flex flex-col sm:flex-row items-center gap-4">
          <div className="flex items-center gap-2 text-sm text-gray-500">
            <span>Rows per page:</span>
            <select  value={rowsPerPage}
              className="border border-gray-200 rounded px-1 py-0.5 bg-white text-gray-700 outline-none"
              onChange={(e) => {setRowsPerPage(Number(e.target.value));  setCurrentPage(1);  }}>
              <option value={10}>10</option>
              <option value={20}>20</option>
              <option value={50}>50</option>
              <option value={100}>100</option>
            </select>
          </div>
          <div className="text-sm text-gray-500">
            Showing {toolList?.length > 0 ? (currentPage - 1) * rowsPerPage + 1 : 0}
            to {Math.min(currentPage * rowsPerPage, toolList?.length || 0)} of {toolList?.length || 0} tools
          </div>
        </div>
        <div className="flex items-center space-x-2 gap-2">
          <Button variant="outline" size="sm" onClick={() => setCurrentPage(prev => Math.max(prev - 1, 1))}
            disabled={currentPage === 1}>
            <ChevronLeft className="w-4 h-4 mr-1" />  Previous
          </Button>
          <div className="text-sm font-medium px-2"> Page {currentPage} of {totalPages} </div>
          <Button variant="outline" size="sm" disabled={currentPage >= totalPages}
            onClick={() => setCurrentPage(prev => Math.min(prev + 1, totalPages))}>
            Next
            <ChevronRight className="w-4 h-4 ml-1" />
          </Button>
        </div>
      </div>
    </div>
  );
};
export default TableTool;
```

Tools.jsx

```
import React from "react";
import { Sheet } from "@/components/ui/sheet";
import TableTool from "./TableTool";
import AddTool from "./AddTool";
function ToolManagment({ isOpen, setIsOpen }) {
```

```
 return (
   <div>
     <TableTool isOpen={isOpen} setIsOpen={setIsOpen} />
     <Sheet open={isOpen} onOpenChange={setIsOpen}>
       <AddTool setIsOpen={setIsOpen} onToolAdded={() => {}} />
     </Sheet>
   </div>
 );
}
export default ToolManagment;
```

RightClickMenu.jsx

```
import { ContextMenu, ContextMenuContent, ContextMenuItem, ContextMenuSeparator, ContextMenuTrigger,
} from "@/components/ui/context-menu";
import { Edit2, Trash2, Archive, CheckSquare, Settings } from "lucide-react";
export default function TableContextMenu({ children, item, onSaveAll, onEdit, onEditSelected, onDelete,
 onDeleteSelected, onCancelAll,}) {
 return (
   <ContextMenu>
     <ContextMenuTrigger asChild>{children}</ContextMenuTrigger>
     <ContextMenuContent className="w-56">
       <ContextMenuItem onClick={() => onEdit(item)}> <Edit2 className="w-4 h-4 mr-2"
/>Edit</ContextMenuItem>
       <ContextMenuItem className="text-red-600 focus:text-red-600 focus:bg-red-50"
         onClick={() => onDelete(item)}>
         <Trash2 className="w-4 h-4 mr-2" />  Delete
       </ContextMenuItem>
       <ContextMenuSeparator />
       <div className="px-2 py-1.5 text-xs font-semibold text-gray-500"> Bulk Actions </div>
       <ContextMenuItem onClick={() => onEditSelected()}>
         <CheckSquare className="w-4 h-4 mr-2" /> Edit Selected
       </ContextMenuItem>
       <ContextMenuItem className="text-red-600 focus:text-red-600 focus:bg-red-50"
          onClick={() => onDeleteSelected()}>
         <Trash2 className="w-4 h-4 mr-2" />  Delete Selected </ContextMenuItem>
       <ContextMenuSeparator />
       <ContextMenuItem onClick={() => onSaveAll(item)} className="text-green-600 focus:text-green-600
         focus:bg-green-50">
         <Settings className="w-4 h-4 mr-2" />  Save All Changes
       </ContextMenuItem>
       <ContextMenuItem onClick={() => onCancelAll()}>
         <Archive className="w-4 h-4 mr-2" /> Discard Changes
       </ContextMenuItem>
     </ContextMenuContent>
   </ContextMenu>
 );
}
```

TableCreation.jsx

```
import React, { useState, useRef } from "react";
import { Upload, ArrowUp, ArrowDown, Pin, ChevronLeft, ChevronRight } from "lucide-react";
```

```
import { Avatar, AvatarFallback, AvatarImage } from "@/components/ui/avatar";
import { Button } from "@/components/ui/button";
import { Checkbox } from "@/components/ui/checkbox";
import { Input } from "@/components/ui/input";
import { DropdownMenu, DropdownMenuContent, DropdownMenuCheckboxItem, DropdownMenuTrigger, DropdownMenuItem,
} from "@/components/ui/dropdown-menu";
import { Table, TableBody, TableCell, TableHead, TableHeader, TableRow,} from "@/components/ui/table";
import TableContextMenu from "./RightClickMenu";
import TableRowMenu from "./TableMenu";
import { useAuth } from "@/context/AuthContext";
import { updateUser, deleteUser } from "@/API_Call/User";
import toast from "react-hot-toast";
// Dynamic column config
const tableDetails = [
 {
   key: "personDetails", label: "Person Details", type: "multipleInfo", width: 180,
   fields: { photo: true,title: true,subheader: true, },
 },
 {key: "email", label: "Email",type: "email",width: 180,},
 {key: "username",label: "Username",type: "text", width: 160,},
 {key: "roles",label: "Roles",type: "multiselect",width: 180,},
 {key: "adminRight",label: "Admin Right", type: "multiselect",width: 180,},
 { key: "adminOption", label: "Admin Option",type: "multiselect", width: 200,},
 {key: "createdAt",label: "Created Date", type: "date", width: 140,}
];
const roleOptions = ["Admin", "Developer", "Tester", "Bug Reporter"];
const adminControlOptions = ["Create", "Edit", "Delete", "View"];
const adminOptionsList = ["Share", "Generate Report", "Insight View", "Export"];
const toProperCase = (str) => {
 if (!str) return "";
 const s = str.toLowerCase();
 if (s === "generate_report" || s === "generate report") return "Generate Report";
 if (s === "insight_view" || s === "insight view") return "Insight View";
 if (s === "bugreporter" || s === "bug reporter") return "Bug Reporter";
 if (s === "dev" || s === "developer") return "Developer";
 if (s === "tester") return "Tester";
 if (s === "admin") return "Admin";
  // Format words automatically
 return s.split(' ').map(word => word.charAt(0).toUpperCase() + word.slice(1)).join(' ');
};
// Resizable header component
const ResizableHeader = ({ children, columnKey, colWidths, setColWidths, showResizeIcon = false,}) => {
 const startX = useRef(0); const startWidth = useRef(0);
 const handleMouseDown = (e) => {
   e.preventDefault();
   startX.current = e.clientX; startWidth.current = colWidths[columnKey];
   document.addEventListener("mousemove", handleMouseMove);
   document.addEventListener("mouseup", handleMouseUp);
 };
 const handleMouseMove = (e) => {
   const diff = e.clientX - startX.current;
   setColWidths((prev) => ({ ...prev, [columnKey]: Math.max(60, startWidth.current + diff), }));
 };
```

```
 const handleMouseUp = () => {
   document.removeEventListener("mousemove", handleMouseMove);
   document.removeEventListener("mouseup", handleMouseUp);
 };
 return (
   <div  className="flex items-center group" style={{ width: colWidths[columnKey], minWidth: 48 }}>
     <div className="flex-1">{children}</div>
     <div onMouseDown={handleMouseDown}
       className={`w-4 h-6 cursor-col-resize ml-1 flex items-center justify-center `}
       title="Resize" style={{ fontSize: "18px", color: "#cbc8c8" }} > |
     </div>
   </div>
 );
};
const ColumnHeader = ({ children, onPin, isPinned, columnKey }) => {
 const [sortOrder, setSortOrder] = useState(null);
 return (
   <DropdownMenu>
     <DropdownMenuTrigger asChild>
       <div className="flex items-center gap-2 cursor-pointer group select-none">
         <span>{children}</span>
         {isPinned && ( <Pin className="w-4 h-4 text-blue-500" fill="currentColor" />)}
         <div className="opacity-0 group-hover:opacity-100 transition-opacity">
           <svg className="w-4 h-4" fill="none" stroke="currentColor"  viewBox="0 0 24 24" >
             <path strokeLinecap="round" strokeLinejoin="round"  strokeWidth={2}
               d="M19 13l-7 7-7-7m0-6l7-7 7 7" />
           </svg>
         </div>
       </div>
     </DropdownMenuTrigger>
     <DropdownMenuContent align="start">
       <DropdownMenuItem onClick={() => setSortOrder(sortOrder === "asc" ? null : "asc")} >
         <ArrowUp className="w-4 h-4 mr-2" /> Sort Ascending
       </DropdownMenuItem>
       <DropdownMenuItem  onClick={() => setSortOrder(sortOrder === "desc" ? null : "desc")} >
         <ArrowDown className="w-4 h-4 mr-2" />  Sort Descending
       </DropdownMenuItem>
       <DropdownMenuItem onClick={() => onPin(columnKey)}>
         <Pin className="w-4 h-4 mr-2" />  {isPinned ? "Unpin" : "Pin"}
       </DropdownMenuItem>
     </DropdownMenuContent>
   </DropdownMenu>
 );
};
const TableCreation = ({ isOpen, setIsOpen, searchQuery = "" }) => {
 const { allUsers, setAllUsers, user: currentUser } = useAuth();
 const [editingIds, setEditingIds] = useState([]); // Array of editing row ids
 const [editData, setEditData] = useState({}); // { [id]: rowData }
 const [pinnedColumns, setPinnedColumns] = useState([]);
 const [colWidths, setColWidths] = useState(
   Object.fromEntries(tableDetails.map((col) => [col.key, col.width || 160]))
 );
 const [selectedRows, setSelectedRows] = useState([]);
```

```
const [currentPage, setCurrentPage] = useState(1);
const [rowsPerPage, setRowsPerPage] = useState(10);
 // Filter data based on search query
const filteredUsers = allUsers?.filter(user => {
  if (!searchQuery) return true;
  const query = searchQuery.toLowerCase();
  return (
    user.name?.toLowerCase().includes(query) || user.email?.toLowerCase().includes(query) ||
    user.username?.toLowerCase().includes(query) ||
    (Array.isArray(user.roletype) ? user.roletype.join(" ") : user.roletype)?.toLowerCase().includes(query)
  );
}) || [];
const totalPages = Math.max(1, Math.ceil(filteredUsers.length / rowsPerPage));
const paginatedData = filteredUsers.slice((currentPage - 1) * rowsPerPage, currentPage * rowsPerPage);
const handlePinColumn = (columnKey) => {
  setPinnedColumns((prev) =>
    prev.includes(columnKey) ? prev.filter((col) => col !== columnKey) : [...prev, columnKey] );
};
const getPinnedStyle = (columnKey) => {
  // Checkbox column is always sticky at left: 0
  if (!pinnedColumns.includes(columnKey)) return {};
  let left = colWidths.checkbox || 48; // Start after checkbox column
  for (const col of tableDetails) {
    if (col.key === columnKey) break;
    if (pinnedColumns.includes(col.key)) { left += colWidths[col.key] || 160; }
  }
  // If this is the checkbox column, left should be 0
  if (columnKey === "checkbox") left = 0;
  return {
    position: "sticky", left: `${left}px`, zIndex: 10,  background: "white", boxShadow: "2px 0 2px -2px
#eee",
  };
};
const handleEdit = (item) => {  setEditingIds([item.id]);  setEditData({ [item.id]: { ...item } });};
const handleSave = async (id) => {
  // Sync backend...
  const dataToSave = editData[id];
  let finalRole = dataToSave.roles !== undefined ? dataToSave.roles.map(r => {
      const rc = r.toLowerCase();
      if (rc === "developer") return "dev";
      if (rc === "bug reporter" || rc === "bugreporter") return "bugreporter";
      if (rc === "admin") return "admin";
      return rc;
  }) : (Array.isArray(dataToSave.roletype) ? dataToSave.roletype : [dataToSave.roletype || "bugreporter"]);
  const isAdminFinal = finalRole.includes("admin") || finalRole.includes("superadmin");
  let finalAdminControl = [];
  let finalAdminOption = [];
  if (isAdminFinal) {
      finalAdminControl = (dataToSave.adminRight !== undefined ? dataToSave.adminRight
      : (dataToSave.adminControl || [])).map(v => v.toLowerCase());
      if (finalAdminControl.some(r => ["create", "edit", "delete"].includes(r)) &&
      !finalAdminControl.includes("view")) { finalAdminControl.push("view"); }
      finalAdminOption = (dataToSave.adminOption || []).map(v => v.toLowerCase().replace(" ", "_"));
```

```
  }
  const updatePayload = {
    name: dataToSave.name, email: dataToSave.email,username: dataToSave.username,
    roletype: finalRole,adminControl: finalAdminControl,adminOption: finalAdminOption
  };
  try {
    const res = await updateUser(id, updatePayload);
    if (res.success) {
      // Sync frontend context array
      setAllUsers((prev) => prev.map((it) => (it.id === id ? { ...it, ...updatePayload } : it)) );
      toast.success("User updated!");
    } else {toast.error(res.message);}
  } catch(err) { toast.error("Failed to update user.");}

  setEditingIds((prev) => prev.filter((eid) => eid !== id));
  setEditData((prev) => {  const newData = { ...prev };  delete newData[id];  return newData;});
};
const handleCancel = (id) => {
  setEditingIds((prev) => prev.filter((eid) => eid !== id));
  setEditData((prev) => {  const newData = { ...prev }; delete newData[id];  return newData; });};

const handleDeleteSelected = async () => {
  // Ideally map deletes over Promise.all
  for(const id of selectedRows) { await deleteUser(id);}
  setAllUsers((prev) => prev.filter((it) => !selectedRows.includes(it.id)));
  setSelectedRows([]); toast.success("Users deleted!");
};
const handleDeleteSingle = async (id) => {
   try {
     const res = await deleteUser(id);
     if (res.success) {
        setAllUsers(prev => prev.filter(it => it.id !== id));  toast.success("User deleted!");
     } else {   toast.error(res.message);  }
   } catch(err) {  toast.error("Delete failed!"); }
};
const handleEditSelected = () => {
  if (selectedRows.length > 0) {
    setEditingIds(selectedRows);
    // Prepare editData for all selected
    const newEditData = {};
    selectedRows.forEach((id) => {
      const item = allUsers.find((i) => i.id === id);  if (item) newEditData[id] = { ...item };
    });
    setEditData(newEditData);
  }
};
const handleSaveAll = async () => {
  for (const id of editingIds) {  if (editData[id]) await handleSave(id);}
  setEditingIds([]); setEditData({}); setSelectedRows([]); // Unselect all checkboxes
};
const handleRowSelect = (id) => {
  setSelectedRows((prev) =>  prev.includes(id) ? prev.filter((rowId) => rowId !== id) : [...prev, id]);
};
```

```
const handleCancelAll = () => {
  setEditingIds([]); setEditData({});  setSelectedRows([]); // Unselect all checkboxes
};
// Cell renderers by type
const renderCell = (col, item, isEditing, displayData, setRowEditData) => {
  if (col.type === "multipleInfo") {
    return (
      <div className="flex items-center gap-3">
        {col.fields.photo && (
          <Avatar className="rounded-full w-12 h-12 border bg-gray-50 flex justify-center items-center">
            <AvatarFallback className="text-xs bg-transparent">
              {item.name ? item.name.substring(0,2).toUpperCase() : "U"}
            </AvatarFallback>
          </Avatar>
        )}
        <div>
          {col.fields.title && (
            <div className="font-medium">
              {isEditing ? (
                <Input  value={displayData.name ?? ""}  onChange={(e) =>
                    setRowEditData({ name: e.target.value }) }  className="h-8 mb-1"/>
              ) : (  item.name || item.username )}
            </div>
          )}
          {col.fields.subheader && !isEditing && (
            <span className="text-muted-foreground mt-0.5 text-[14px]"> {item.role || "user"} </span>
          )}
        </div>
      </div>
    );
  }
  if (col.type === "date") {
     const dateVal = item.createdAt ? new Date(item.createdAt).toLocaleString('en-GB',
      { day: '2-digit', month: 'short', year: 'numeric' }) : "-";
     return <span className="text-gray-500 whitespace-nowrap text-sm">{dateVal}</span>;
  }

  if (col.type === "email") {
    return isEditing ? (
      <Input type="email" value={displayData.email ?? ""}
        onChange={(e) => setRowEditData({ email: e.target.value })} className="h-8" />
    ) : ( item.email );
  }
  if (col.type === "text") {
    return isEditing ? (
      <Input  value={displayData[col.key] ?? ""}
        onChange={(e) => setRowEditData({ [col.key]: e.target.value })} className="h-8" />
    ) : ( item[col.key] );
  }
  if (col.type === "multiselect") {
    const options =
      col.key === "roles" ? ["Admin", "Developer", "Tester", "Bug Reporter"]: col.key === "adminRight"
```

```
        ? adminControlOptions : adminOptionsList;
    // Extract raw arrays representing options based off new Mongo structure dynamically
    let currentValRaw = [];
    if (col.key === "roles") {
      currentValRaw = displayData.roles !== undefined ? displayData.roles
      :(Array.isArray(displayData.roletype) ? displayData.roletype : (displayData.roletype
      ? [displayData.roletype] : []));
    } else if (col.key === "adminRight") {
      currentValRaw = displayData.adminRight !== undefined ? displayData.adminRight
      : (displayData.adminControl || []);
    } else if (col.key === "adminOption") { currentValRaw = displayData.adminOption || []; }
    currentValRaw = currentValRaw.map(v => toProperCase(v));
    const currentRolesRaw = displayData.roles !== undefined ? displayData.roles :
      (Array.isArray(displayData.roletype) ? displayData.roletype : (displayData.roletype
      ? [displayData.roletype] : []));
    const isAdmin = currentRolesRaw.some(r => r.toLowerCase() === "admin");
    const isDisabled = !isAdmin && (col.key === "adminRight" || col.key === "adminOption");
    return isEditing ? (
      <div className="w-full" title={isDisabled ? "This is admin rights" : ""}>
        <DropdownMenu>
          <DropdownMenuTrigger asChild>
            <Button variant="outline" className="h-8 w-full justify-start text-left" disabled={isDisabled}>
              <span className="truncate">
                {currentValRaw.length > 0 ? currentValRaw.join(", ") : "Select"}
              </span>
            </Button>
          </DropdownMenuTrigger>
          <DropdownMenuContent className="w-56">
          {options.map((option) => (
            <DropdownMenuCheckboxItem key={option} checked={currentValRaw.includes(option)}
              onClick={(event) => {  event.preventDefault();
                // Standard multi-select toggle for all options
                const newArray = currentValRaw.includes(option)
                    ? currentValRaw.filter((item) => item !== option) : [...currentValRaw, option];
                if (col.key === "roles") { setRowEditData({ roles: newArray });}
                else if (col.key === "adminRight") { setRowEditData({ adminRight: newArray });}
                else { setRowEditData({ adminOption: newArray });}
              }} >
              {option}
            </DropdownMenuCheckboxItem>
          ))}
        </DropdownMenuContent>
      </DropdownMenu>
      </div>
    ) : (
      <div className="max-w-[180px]">
        <span className="text-sm truncate block" title={currentValRaw.join(", ")} >
          {currentValRaw.join(", ")}
        </span>
      </div>
    );
  }
  return null;
```

```
 };
 const canCreate = allUsers && (useAuth().user?.role === "superadmin" ||
useAuth().user?.adminControl?.includes("create"));

 return (
   <div className="w-full">
     <div className="[&>div]:rounded-sm [&>div]:border overflow-x-auto overflow-y-auto
h-[calc(100vh-280px)]">
       <Table className="min-w-full">
         <TableHeader>
           <TableRow className="hover:bg-transparent">
             <TableHead
               className="p-0 flex items-center justify-center h-16 sticky left-0 z-30 bg-white"
               style={{  width: colWidths.checkbox, minWidth: 48,  maxWidth: 120, left: 0, top: 0,
                 zIndex: 30, background: "white",}}>
               <Checkbox
                 checked={paginatedData?.length > 0 && selectedRows.length === paginatedData.length}
                 onCheckedChange={(checked) => {
                   if (checked) {setSelectedRows(paginatedData.map(item => item.id || item._id));}
                   else {setSelectedRows([]);}
                 }} />
             </TableHead>
             {tableDetails.map((col) => {
               const isPinned = pinnedColumns.includes(col.key);
               const pinStyle = getPinnedStyle(col.key);
               return (
               <TableHead  key={col.key} className="sticky"
                 style={{...pinStyle, width: colWidths[col.key], minWidth: 60, maxWidth: 600,
                   top: 0,zIndex: isPinned ? 30 : 20, backgroundColor: 'white'
                 }}>
                 <ResizableHeader columnKey={col.key} colWidths={colWidths} setColWidths={setColWidths} >
                   <ColumnHeader columnKey={col.key} onPin={handlePinColumn} isPinned={isPinned} >
                     {col.label}
                   </ColumnHeader>
                 </ResizableHeader>
               </TableHead>
               );
             })}
             <TableHeadclassName="sticky right-0 top-0 bg-white"
               style={{ width: 120, minWidth: 60, maxWidth: 600, zIndex: 30 }}> Actions
             </TableHead>
           </TableRow>
         </TableHeader>
         <TableBody>
           {(!paginatedData || paginatedData.length === 0) ? (
             <TableRow>
                <TableCell colSpan={tableDetails.length + 2} className="text-center py-10 text-gray-500">
                   No users found.</TableCell>
             </TableRow>
           ) : (
             paginatedData.map((item) => {
             const isEditing = editingIds.includes(item.id);
             const displayData =isEditing && editData[item.id] ? editData[item.id] : item;
```

```
            return (
              <TableContextMenu key={item.id} item={item} onEdit={() => setEditingIds([item.id])}
                onDelete={() => handleDeleteSingle(item.id)} onSaveAll={handleSaveAll}
                onEditSelected={handleEditSelected} onDeleteSelected={handleDeleteSelected}
                onCancelAll={handleCancelAll} >
                <TableRow className="cursor-pointer hover:bg-gray-50">
                  {/* Row checkbox */}
                  <TableCell background: "white",}}
                    className="p-0 flex items-center justify-center h-16 sticky left-0 z-10 bg-white"
                    style={{  width: colWidths.checkbox,minWidth: 48,maxWidth: 120, left: 0, zIndex: 20,>
                    <Checkbox checked={selectedRows.includes(item.id)}
                      onCheckedChange={() => handleRowSelect(item.id)}id={`checkbox-${item.id}`}/>
                  </TableCell>
                  {/* Dynamic columns */}
                  {tableDetails.map((col) => (
                    <TableCell key={col.key} style={{
                        ...getPinnedStyle(col.key), width: colWidths[col.key],minWidth: 60,  maxWidth: 600,
                      }} >
                      {renderCell(col, item, isEditing, displayData, (val) =>
                        setEditData((prev) => ({...prev,[item.id]: { ...(prev[item.id] || item), ...val
},}))

                      )}
                    </TableCell>
                  ))}
                  {/* Actions */}
                  <TableCell className="sticky right-0 bg-white z-10"
                    style={{ width: 120, minWidth: 60, maxWidth: 600 }}>
                    {isEditing ? (
                      <div className="flex gap-2">
                        <Button size="sm"onClick={() => handleSave(item.id)}className="h-8 px-3" > Save
                        </Button>
                        <Button size="sm"variant="outline"onClick={() => handleCancel(item.id)}
                          className="h-8 px-3"> Cancel</Button>
                      </div>
                    ) : (() => {
                       const isSuperAdminEditBlocked = item.role === "superadmin"
                            && currentUser?.role !== "superadmin";
                       const canEdit = currentUser?.role === "superadmin" ||
                             currentUser?.adminControl?.includes("edit");
                       const canDelete = currentUser?.role === "superadmin" ||
                            currentUser?.adminControl?.includes("delete");
                       if (isSuperAdminEditBlocked) return
                             <span className="text-gray-400 text-xs italic">Protected</span>;
                       if (!canEdit && !canDelete) return null;
                       return (
                          <TableRowMenu item={item} onEdit={canEdit ? () => handleEdit(item) : undefined}
                            onDelete={canDelete ? () => handleDeleteSingle(item.id) : undefined}
                            onArchive={() => alert("Archive " + item.id)}  />
                       );
                    })()}
                  </TableCell>
                </TableRow>
              </TableContextMenu>
```

```
            );
          }))}
        </TableBody>
      </Table>
    </div>
    {/* Pagination Controls */}
    <div className="flex flex-col sm:flex-row items-center justify-between mt-4 px-2 gap-4">
      <div className="flex flex-col sm:flex-row items-center gap-4">
        <div className="flex items-center gap-2 text-sm text-gray-500"><span>Rows per page:</span>
          <select  value={rowsPerPage}
             className="border border-gray-200 rounded px-1 py-0.5 bg-white text-gray-700 outline-none"
             onChange={(e) => { setRowsPerPage(Number(e.target.value));setCurrentPage(1);}} >
            <option value={10}>10</option><option value={20}>20</option><option value={50}>50</option>
            <option value={100}>100</option>
          </select>
        </div>
        <div className="text-sm text-gray-500">
          Showing {allUsers?.length > 0 ? (currentPage - 1) * rowsPerPage + 1 : 0}
          to {Math.min(currentPage * rowsPerPage, allUsers?.length || 0)} of {allUsers?.length || 0} users
        </div>
      </div>
      <div className="flex items-center space-x-2 gap-2">
        <Buttonvariant="outline" size="sm"
          onClick={() => setCurrentPage(prev => Math.max(prev - 1, 1))} disabled={currentPage === 1}>
          <ChevronLeft className="w-4 h-4 mr-1" /> Previous</Button>
        <div className="text-sm font-medium px-2">Page {currentPage} of {totalPages}</div>
        <Button variant="outline"size="sm"
          onClick={() => setCurrentPage(prev => Math.min(prev + 1, totalPages))}
          disabled={currentPage >= totalPages} >Next<ChevronRight className="w-4 h-4 ml-1" />
        </Button>
      </div>
    </div>
  </div>
);
};
export default TableCreation;
```

TableMenu.jsx

```
import { Copy, Edit2, Archive, Trash2, MoreHorizontal } from "lucide-react";
import {DropdownMenu,DropdownMenuContent, DropdownMenuItem,DropdownMenuSeparator,DropdownMenuTrigger,
} from "@/components/ui/dropdown-menu";
import { Button } from "@/components/ui/button";
const TableRowMenu = ({ item, onEdit, onDelete, onArchive }) => {
 return (
   <DropdownMenu>
     <DropdownMenuTrigger asChild>
       <Button variant="ghost" size="icon" className="h-8 w-8"><MoreHorizontal className="h-4 w-4"
/></Button>
     </DropdownMenuTrigger>
     <DropdownMenuContent align="end" className="w-40">
       <DropdownMenuItem onClick={() => navigator.clipboard.writeText(item.id || item._id)}>
         <Copy className="h-4 w-4 mr-2" />Copy ID </DropdownMenuItem>
       <DropdownMenuItem onClick={() => onEdit(item)}><Edit2 className="h-4 w-4 mr-2" />Edit
```

```
        </DropdownMenuItem>
        <DropdownMenuSeparator />
        {onArchive && (
          <DropdownMenuItem onClick={() => onArchive(item)}> <Archive className="h-4 w-4 mr-2" /> Archive
          </DropdownMenuItem>
        )}
        <DropdownMenuItem onClick={() => onDelete(item)} className="text-red-500 hover:text-red-600
focus:text-red-600 focus:bg-red-50"> <Trash2 className="h-4 w-4 mr-2" />Delete
        </DropdownMenuItem>
      </DropdownMenuContent>
    </DropdownMenu>
 );
};
export default TableRowMenu;
```

UserManagment.jsx

```
import React from "react";
import TableCreation from "./TableCreation";
import { Sheet } from "@/components/ui/sheet";
import AddUser from "./AddUser/Adduser";
function UserManagment({ searchQuery, isSheetOpen, setIsSheetOpen }) {
 return (
   <div className="flex flex-col gap-4 h-full">
     <TableCreation isOpen={isSheetOpen}  setIsOpen={setIsSheetOpen}  searchQuery={searchQuery}/>
     <Sheet open={isSheetOpen} onOpenChange={setIsSheetOpen}> <AddUser setIsOpen={setIsSheetOpen} /> </Sheet>
   </div>
 );
}
export default UserManagment;
```

Shared (Folder)
AttachmentGallery.jsx

```
import React, { useState } from "react";
import { FileText, X, Plus, Trash2, Eye, Loader2 } from "lucide-react";
import { Button } from "@/components/ui/button";
import FileUploader from "./FileUploader";

export default function AttachmentGallery({ files = [], isEditable = false, onUpdate, label = "Attachments"})
{
 const [showUploader, setShowUploader] = useState(false);
 const safeFiles = Array.isArray(files) ? files : [];
 const isImage = (url) => {
   if (!url) return false;
   const cleanUrl = url.split('?')[0].toLowerCase();
   return cleanUrl.match(/\.(jpeg|jpg|gif|png|webp|avif|svg)$/i) ||  url.includes("cloudinary.com") ||
          url.includes("res.cloudinary.com");
 };
 const handleDelete = (id) => {
   const newFiles = safeFiles.filter(f => (f.id || f.url) !== id);
   onUpdate(newFiles);
 };
 if (isEditable && (showUploader || safeFiles.length === 0)) {
   return (
```

```
    <div className="min-w-[220px] p-2 bg-white rounded-xl border border-cyan-100 shadow-xl animate-in
fade-in
        slide-in-from-top-1 duration-200 z-50">
      <div className="flex justify-between items-center mb-2 px-1">
        <span className="text-[10px] font-bold text-cyan-700 uppercase tracking-tight">{label}</span>
        {safeFiles.length > 0 && (
          <button onClick={() => setShowUploader(false)}
            className="p-1 hover:bg-gray-100 rounded-full transition-colors">
            <X className="w-3.5 h-3.5 text-gray-400" />
          </button>
        )}
      </div>
      <FileUploader files={[]}  label="" compact={true}
        onChange={(newBatch) => { onUpdate([...safeFiles, ...newBatch]); setShowUploader(false); }} />
      {safeFiles.length > 0 && (
         <Button variant="ghost" size="sm"  onClick={() => setShowUploader(false)}
           className="w-full mt-2 h-7 text-[10px] text-gray-500 hover:bg-gray-50" >
           Cancel
         </Button>
      )}
    </div>
  );
}
return (
  <div className="flex flex-wrap gap-2.5 items-center py-2 min-h-[52px]">
    {safeFiles.map((file, idx) => {
      const fileId = file.id || file.url || `file-${idx}`;
      const fileName = file.name || file.fileName || "File";
      const fileUrl = file.url;
      const isImg = isImage(fileUrl);
      return (
        <div  key={fileId} className="relative group w-12 h-12 rounded-xl bg-white shadow-sm border
             border-gray-100 hover:border-cyan-400 hover:shadow-md transition-all duration-300" >
          {/* Thumbnail Container */}
          <div className="w-full h-full rounded-xl overflow-hidden">
            {isImg ? (
              <img src={fileUrl} alt={fileName} className="w-full h-full object-cover transition-transform
                    duration-500 group-hover:scale-110 group-hover:blur-[1px]"
                onError={(e) => {
                   e.target.onerror = null; e.target.src = "https://placehold.co/48x48?text=File";
                }}
              />
            ) : (
              <div className="w-full h-full flex items-center justify-center bg-slate-50">
                <FileText className="w-6 h-6 text-slate-300" />
              </div>
            )}
          </div>
          {/* Corner Badges: TOP-RIGHT (VIEW) */}
          <a href={fileUrl} target="_blank" rel="noreferrer"  onClick={(e) => e.stopPropagation()}
            className="absolute -top-1.5 -right-1.5 w-6 h-6 flex items-center justify-center rounded-full
            bg-cyan-600 text-white shadow-lg border-2 border-white scale-0 opacity-0 group-hover:scale-100
             group-hover:opacity-100 transition-all duration-200 hover:bg-cyan-700 z-10"
```

```
                title="View Full Size">
              <Eye className="w-3 h-3" />
            </a>

            {/* Corner Badges: TOP-LEFT (DELETE) - Only if editable */}
            {isEditable && (
              <button onClick={(e) => {  e.stopPropagation(); handleDelete(fileId);}}  title="Delete"
                 className="absolute -top-1.5 -left-1.5 w-6 h-6 flex items-center justify-center rounded-full
                bg-red-500 text-white shadow-lg border-2 border-white scale-0 opacity-0 group-hover:scale-100
                group-hover:opacity-100 transition-all duration-200 delay-75 hover:bg-red-600 z-10" >
                 <Trash2 className="w-3 h-3" />
              </button>
            )}
            {/* File Extension Tag (Bottom) */}
            <div className="absolute bottom-1 right-1 px-1 py-0.5 rounded-md bg-black/40 text-[7px] text-white
                 font-bold opacity-0 group-hover:opacity-100 transition-opacity uppercase
pointer-events-none">
              {fileName.split('.').pop()}
            </div>
          </div>
        );
      })}
      {isEditable && safeFiles.length < 5 && (
        <button onClick={() => setShowUploader(true)}  title="Add New Attachment"
          className="w-12 h-12 rounded-xl border-2 border-dashed border-gray-200 bg-slate-50 flex items-center
          justify-center text-gray-400 hover:text-cyan-600 hover:border-cyan-400 hover:bg-cyan-50/50
          transition-all shadow-sm group" >
          <Plus className="w-6 h-6 transition-transform duration-300 group-hover:rotate-90" />
        </button>
      )}
      {safeFiles.length === 0 && !isEditable && (
        <span className="text-slate-400 text-[10px] font-medium italic px-2 py-1.5 rounded-lg bg-slate-50
border
          border-slate-100 flex items-center gap-1.5 select-none">
          <FileText className="w-3.5 h-3.5" />  No files
        </span>
      )}
    </div>
  );
}
```

FileUploader.jsx

```
import React, { useState, useRef } from "react";
import { Upload, X, FileText, AlertCircle, Loader2 } from "lucide-react";
import { Button } from "@/components/ui/button";
import toast from "react-hot-toast";
import { uploadFiles } from "@/API_Call/Bug";
const MAX_FILES = 5;
const MAX_TOTAL_SIZE_MB = 50;
const MAX_TOTAL_SIZE_BYTES = MAX_TOTAL_SIZE_MB * 1024 * 1024;
export default function FileUploader({ files = [], onChange, label = "Attachments", compact = false }) {
 const fileInputRef = useRef(null);
 const [isUploading, setIsUploading] = useState(false);
 const handleFileChange = async (e) => {
```

```
  const selectedFiles = Array.from(e.target.files);
  if (!selectedFiles.length) return;
  if (files.length + selectedFiles.length > MAX_FILES) {
    toast.error(`You can only upload up to ${MAX_FILES} files.`); return;
  };
  const currentTotalSize = files.reduce((acc, f) => acc + (f.size || 0), 0);
  const newFilesTotalSize = selectedFiles.reduce((acc, f) => acc + f.size, 0);
  if (currentTotalSize + newFilesTotalSize > MAX_TOTAL_SIZE_BYTES) {
    toast.error(`Total file size must not exceed ${MAX_TOTAL_SIZE_MB}MB.`);return;
  }
  setIsUploading(true);
  const uploadToast = toast.loading("Uploading to Cloudinary...");

  try {
    const res = await uploadFiles(selectedFiles);
    if (res.success && res.data.results) {
      const newUploadedFiles = res.data.results.map(file => ({
        id: file.publicId || Math.random().toString(36).substr(2, 9), name: file.fileName,
        size: file.size,  url: file.url, uploadedAt: file.uploadedAt
      }));
      onChange([...files, ...newUploadedFiles]);
      toast.success("Files uploaded successfully", { id: uploadToast });
    } else {  toast.error(res.message || "Failed to upload files", { id: uploadToast });}
  } catch (error) {
    toast.error("An error occurred during upload", { id: uploadToast });
  } finally {
    setIsUploading(false);
    if (fileInputRef.current) fileInputRef.current.value = "";
  }
};
const removeFile = (id) => { onChange(files.filter(f => (f.id || f.url) !== id));};
const formatSize = (bytes) => {
  if (bytes === 0) return '0 Bytes';
  const k = 1024;
  const sizes = ['Bytes', 'KB', 'MB', 'GB'];
  const i = Math.floor(Math.log(bytes) / Math.log(k));
  return parseFloat((bytes / Math.pow(k, i)).toFixed(2)) + ' ' + sizes[i];
};
const totalSize = files.reduce((acc, f) => acc + (f.size || 0), 0);
const sizePercentage = (totalSize / MAX_TOTAL_SIZE_BYTES) * 100;
return (
  <div className={`${compact ? 'space-y-1' : 'space-y-3'} w-full`}>
    <div className="flex items-center justify-between gap-2">
      {!compact && (
        <label className="text-sm font-semibold text-gray-700 flex items-center gap-2">
          {label}
          <span className="text-[10px] font-normal text-gray-400"> ({files.length}/{MAX_FILES}) </span>
        </label>
      )}
      {files.length < MAX_FILES && (
        <Button type="button" variant="outline" size="sm" disabled={isUploading}
          className={`${compact ? 'h-7 px-2 text-[10px]' : 'h-8 px-3 text-xs'} w-full gap-2 border-dashed
```

```
          border-cyan-200 bg-cyan-50/30 text-cyan-700 hover:bg-cyan-50 hover:border-cyan-300
transition-all`}
          onClick={() => fileInputRef.current?.click()}>
          {isUploading ? <Loader2 className="w-3.5 h-3.5 animate-spin" /> : <Upload className="w-3.5 h-3.5"
/>}
          {isUploading ? "Uploading..." : "Add Files"}
        </Button>
      )}
    </div>
    <input type="file" ref={fileInputRef} onChange={handleFileChange} multipleclassName="hidden"
      accept="image/*,video/*,application/pdf,.doc,.docx,.xls,.xlsx,.zip"
    />
    {files.length > 0 && (
      <div className={`${compact ? 'space-y-1' : 'space-y-2'}`}>
        <div className="grid grid-cols-1 gap-1.5">
          {files.map((file) => (
            <div  key={file.id || file.url} className={`flex items-center justify-between
              ${compact ? 'p-1.5' : 'p-2'} rounded-lg bg-gray-50 border border-gray-100 group
              hover:border-cyan-200 transition-all`}>
              <div className="flex items-center gap-2 overflow-hidden">
                <div className={`${compact ? 'w-6 h-6' : 'w-8 h-8'} rounded bg-white border border-gray-100
                      flex items-center justify-center shrink-0`}>
                  <FileText className={`${compact ? 'w-3 h-3' : 'w-4 h-4'} text-gray-400`} />
                </div>
                <div className="flex flex-col overflow-hidden text-left">
                  <span className="text-[10px] font-medium text-gray-700 truncate max-w-[120px]">
                    {file.name || file.fileName}</span>
                  <span className="text-[9px] text-gray-400">{file.size ? formatSize(file.size)
                    : "Cloud File"}</span>
                </div>
              </div>
              <div className="flex items-center gap-1">
                <button type="button" onClick={() => removeFile(file.id || file.url)}
                  className="p-1 rounded-md text-gray-400 hover:text-red-500 hover:bg-red-50
transition-all">
                  <X className="w-3.5 h-3.5" />
                </button>
              </div>
            </div>
          ))}
        </div>
        <div className="h-1 w-full bg-gray-100 rounded-full overflow-hidden mt-1">
          <div className={`h-full transition-all duration-500 ${
              sizePercentage > 90 ? 'bg-red-500' : sizePercentage > 70 ? 'bg-orange-400' : 'bg-cyan-500'
            }`} style={{ width: `${Math.min(sizePercentage, 100)}%` }} />
        </div>
      </div>
    )}
    {files.length === 0 && !isUploading && !compact && (
      <div  className="border-2 border-dashed border-gray-100 rounded-xl p-6 flex flex-col items-center
        justify-center text-gray-400 cursor-pointer hover:border-cyan-100 hover:bg-cyan-50/10
transition-all"
        onClick={() => fileInputRef.current?.click()}>
```

```
        <div className="w-10 h-10 rounded-full bg-gray-50 flex items-center justify-center mb-2">
          <Upload className="w-5 h-5 text-gray-300" />
        </div>
        <p className="text-xs font-medium text-gray-500">Click to upload attachments</p>
        <p className="text-[10px] mt-1 italic">Images, Videos, PDFs (Max 50MB total)</p>
      </div>
    )}
    {isUploading && files.length === 0 && (
       <div className={`border-2 border-dashed border-cyan-100 bg-cyan-50/10 rounded-xl ${compact ? 'p-3' :
'p-6'} flex flex-col items-center justify-center text-cyan-400`}>
          <Loader2 className={`${compact ? 'w-4 h-4' : 'w-8 h-8'} animate-spin mb-1`} />
          <p className="text-[10px] font-medium">Uploading...</p>
        </div>
    )}
  </div>
 );
}
```

app-sidebar.jsx

```
import * as React from "react";
import { Bug, BugPlay, CircleCheckBig, CloudCheck, LayoutDashboard, Microscope, Rocket, Settings, Star,
 GalleryVerticalEnd,} from "lucide-react";
import { useAuth } from "@/context/AuthContext";
import { NavProjects } from "@/components/nav-projects";
import { NavUser } from "@/components/nav-user";
import { TeamSwitcher } from "@/components/team-switcher";
import { Sidebar, SidebarContent, SidebarFooter, SidebarHeader, SidebarSeparator, SidebarRail,
} from "@/components/ui/sidebar";
const data = { teams: [{ name: "Rohit",logo: GalleryVerticalEnd, plan: "Bug Bunters",},],
 systemMenu: [
   { name: "Dashboard", url: "/",     icon: LayoutDashboard },{ name: "Starred",   url: "/star", icon: Star
},
 ],
 bugPhase: [
   { name: "New Bug",          url: "/create-bug",       icon: Bug },
   { name: "Bug Testing",      url: "/bug-testing",      icon: BugPlay },
   { name: "Analyze Bug",      url: "/analyze-bug",      icon: Microscope },
   { name: "Ready To Testing", url: "/ready-to-testing", icon: CloudCheck },
   { name: "Ready To Deploy",  url: "/ready-to-deploy",  icon: Rocket },
   { name: "Deployed",         url: "/deployed",         icon: CircleCheckBig },
 ],
 adminSetting: [{ name: "Settings", url: "/settings", icon: Settings }, ],
};
export function AppSidebar({ ...props }) {
 const { user } = useAuth();
 const hasSettingsAccess = user?.role === "superadmin" || (Array.isArray(user?.adminControl) &&
user.adminControl.some(r => ["create", "edit", "view", "delete"].includes(r?.toLowerCase())));

 return (
   <Sidebar collapsible="icon" {...props}> <SidebarHeader><TeamSwitcher teams={data.teams} />
</SidebarHeader>
     <SidebarContent>
       <NavProjects projects={data.systemMenu} label="Overview" />
```

```
      <SidebarSeparator className="opacity-20" />
      <NavProjects projects={data.bugPhase}   label="Bug Phases" />
      {hasSettingsAccess && (
        <>
          <SidebarSeparator className="opacity-20" />
          <NavProjects projects={data.adminSetting} label="Admin" />
        </>
      )}
    </SidebarContent>
    <SidebarFooter>
      <NavUser />
    </SidebarFooter>
    <SidebarRail />
  </Sidebar>
);
}
```

BugHunterSidebar.jsx

```
import * as React from "react";
import { Home, Bug, TestTube, Settings, ChevronRight } from "lucide-react";
import { useNavigate, useLocation } from "react-router-dom";
import { useAuth } from "@/context/AuthContext";
import { Sidebar, SidebarContent, SidebarFooter, SidebarHeader, SidebarRail, SidebarMenu,
 SidebarMenuItem, SidebarMenuButton,} from "@/components/ui/sidebar";
import { NavUser } from "@/components/nav-user";
const navItems = [
 {   title: "Dashboard",   url: "/dashboard",   icon: Home, },
 {   title: "Assign Bug",   url: "/dashboard/assign-bug",   icon: Bug, },
 {   title: "Testing",   url: "/dashboard/testing",   icon: TestTube, },
 {   title: "Settings",   url: "/dashboard/settings",   icon: Settings, },
];
export function BugHunterSidebar({ ...props }) {
 const navigate = useNavigate();
 const location = useLocation();
 const { user } = useAuth();
 const hasSettingsAccess = user?.role === "superadmin" || (Array.isArray(user?.adminControl) &&
       user.adminControl.some(r => ["create", "edit", "view", "delete"].includes(r?.toLowerCase())));
 const userData = {
   name: user?.name || "User",email: user?.email || "user@example.com", avatar: "/avatars/shadcn.jpg",
 };
 return (
   <Sidebar collapsible="icon" {...props}>
     <SidebarHeader>
       <div className="flex items-center gap-2 px-4 py-2">
         <Bug className="h-6 w-6 text-primary" /><span className="font-bold text-lg">Bug Tracker</span>
       </div>
     </SidebarHeader>
     <SidebarContent>
       <SidebarMenu>
         {navItems.map((item) => {
           if (item.title === "Settings" && !hasSettingsAccess) return null;
           return (
           <SidebarMenuItem key={item.title}>
             <SidebarMenuButton onClick={() => navigate(item.url)} isActive={location.pathname === item.url}
```

```
              tooltip={item.title}>
              <item.icon className="h-4 w-4" />
              <span>{item.title}</span>
            </SidebarMenuButton>
          </SidebarMenuItem>
          );
        })}
      </SidebarMenu>
    </SidebarContent>
    <SidebarFooter> <NavUser user={userData} /> </SidebarFooter>
    <SidebarRail />
  </Sidebar>
);
}
```

nav-projects.jsx

```
import { useLocation } from "react-router-dom";
import { SidebarGroup, SidebarGroupLabel, SidebarMenu, SidebarMenuButton, SidebarMenuItem,} from
"@/components/ui/sidebar";
export function NavProjects({ projects, label }) {
 const { pathname } = useLocation();
 return (
   <SidebarGroup>
     {label && (
       <SidebarGroupLabel className="text-[10px] font-bold tracking-widest uppercase px-3 mb-1"
         style={{ color: "rgba(180,230,240,0.45)", letterSpacing: "0.12em" }}>{label}
       </SidebarGroupLabel>
     )}
     <SidebarMenu>
       {projects.map((item) => {
         const isActive = pathname === item.url;
         return (
           <SidebarMenuItem key={item.name}>
             <SidebarMenuButton asChild>
               <a href={item.url} className={`flex items-center gap-3 px-3 py-2 rounded-lg text-sm
font-medium
                  transition-all duration-150 ${
                   isActive ? "!bg-cyan-500/20 !text-cyan-300 font-semibold"
                     : "hover:!bg-white/5 !text-sidebar-foreground/80 hover:!text-white"
                 }`} >
                 <span className={`flex-shrink-0 ${isActive ? "text-cyan-400" : "opacity-70"}`}>
                   <item.icon size={16} />
                 </span>
                 <span>{item.name}</span>
                 {isActive && (
                   <span className="ml-auto w-1.5 h-1.5 rounded-full bg-cyan-400 flex-shrink-0" />
                 )}
               </a>
             </SidebarMenuButton>
           </SidebarMenuItem>
         );
       })}
     </SidebarMenu>
```

```
    </SidebarGroup>
  );
}
```

nav-user.jsx

```
"use client";
import { BadgeCheck, ChevronsUpDown, LogOut } from "lucide-react";
import { Avatar, AvatarFallback, AvatarImage } from "@/components/ui/avatar";
import { DropdownMenu, DropdownMenuContent, DropdownMenuGroup, DropdownMenuItem,
 DropdownMenuLabel, DropdownMenuSeparator, DropdownMenuTrigger,} from "@/components/ui/dropdown-menu";
import { SidebarMenu, SidebarMenuButton, SidebarMenuItem, useSidebar,} from "@/components/ui/sidebar";
import { logout } from "@/API_Call/Auth";
import toast from "react-hot-toast";
import { useNavigate } from "react-router-dom";
import { useAuth } from "@/context/AuthContext";
export function NavUser() {
 const { isMobile } = useSidebar();
 const navigate = useNavigate();
 const { user, setUser, loading, setLoading } = useAuth();
 const handleLogOut = async (e) => {
   e.preventDefault();
   setLoading(true);
   const logOut_check = await logout();
   console.log("Logout response:", logOut_check);
   if (logOut_check.success) {setUser(""); }
   else {  toast.error(logOut_check.message); setLoading(false); return;}
   toast.success("Logout successfully"); setLoading(false);  navigate("/login");
 };
 return (
   <SidebarMenu>
     <SidebarMenuItem>
       <DropdownMenu>
         <DropdownMenuTrigger asChild>
           <SidebarMenuButton size="lg"
             className="data-[state=open]:bg-sidebar-accent data-[state=open]:text-sidebar-accent-foreground"
>
             <Avatar className="h-8 w-8 rounded-lg">
               <AvatarImage src={user.photo} alt={user.name} />
               <AvatarFallback className="rounded-lg">
                 {user?.name?.charAt(0)?.toUpperCase() || user?.username?.charAt(0)?.toUpperCase() ||
                   user?.email?.charAt(0)?.toUpperCase() || "U"}
               </AvatarFallback>
             </Avatar>
             <div className="grid flex-1 text-left text-sm leading-tight">
               <span className="truncate font-semibold">{user.name}</span>
               <span className="truncate text-xs">
                 {user?.email ||
                   (user?.username ? user.username.charAt(0).toUpperCase() +
                       user.username.slice(1).toLowerCase() : "")}
               </span>
             </div>
             <ChevronsUpDown className="ml-auto size-4" />
           </SidebarMenuButton>
         </DropdownMenuTrigger>
```

```
        <DropdownMenuContent className="w-[--radix-dropdown-menu-trigger-width] min-w-56 rounded-lg"
          side={isMobile ? "bottom" : "right"} align="end" sideOffset={4}>
          <DropdownMenuLabel className="p-0 font-normal">
            <div className="flex items-center gap-2 px-1 py-1.5 text-left text-sm">
              <Avatar className="h-8 w-8 rounded-lg">
                <AvatarImage src={user.photo} alt={user.name} />
                <AvatarFallback className="rounded-lg">
                  {user?.name?.charAt(0)?.toUpperCase() || user?.username?.charAt(0)?.toUpperCase() ||
                    user?.email?.charAt(0)?.toUpperCase() || "U"}
                </AvatarFallback>
              </Avatar>
              <div className="grid flex-1 text-left text-sm leading-tight">
                <span className="truncate font-semibold">{user.name}</span>
                <span className="truncate text-xs">{user.email}</span>
              </div>
            </div>
          </DropdownMenuLabel>
          <DropdownMenuSeparator />
           <DropdownMenuSeparator />
          <DropdownMenuGroup>
            <DropdownMenuItem onClick={() => navigate("/account")}> <BadgeCheck />
Account</DropdownMenuItem>
          </DropdownMenuGroup>
          <DropdownMenuSeparator />
          <DropdownMenuItem onClick={handleLogOut}>  <LogOut /> Log out </DropdownMenuItem>
        </DropdownMenuContent>
      </DropdownMenu>
    </SidebarMenuItem>
  </SidebarMenu>
 );
}
```

ProtectedRoute.jsx

```
import { Navigate } from "react-router-dom";
import { useAuth } from "@/context/AuthContext";
function ProtectedRoute({ children }) {
 const { user, loading } = useAuth();
 if (loading) {
   return (
     <div className="flex items-center justify-center h-screen">
       <div className="animate-spin rounded-full h-12 w-12 border-b-2 border-primary"></div>
     </div>
   );
 }
 if (!user) { return <Navigate to="/login" replace />; } return children;
}
export default ProtectedRoute;
```

team-switcher.jsx

```
import * as React from "react"
import { ChevronsUpDown, Plus } from "lucide-react"
import { DropdownMenu, DropdownMenuContent, DropdownMenuItem, DropdownMenuLabel,
 DropdownMenuSeparator, DropdownMenuShortcut, DropdownMenuTrigger,} from "@/components/ui/dropdown-menu"
import { SidebarMenu, SidebarMenuButton, SidebarMenuItem, useSidebar,} from "@/components/ui/sidebar"
```

```
export function TeamSwitcher({ teams}) {
 const { isMobile } = useSidebar()
 const [activeTeam, setActiveTeam] = React.useState(teams[0])
 if (!activeTeam) {   return null }
 return (
   <SidebarMenu>
     <SidebarMenuItem>
       <DropdownMenu>
         <DropdownMenuTrigger asChild>
           <SidebarMenuButton size="lg"
             className="data-[state=open]:bg-sidebar-accent
data-[state=open]:text-sidebar-accent-foreground">
             <div className="flex aspect-square size-8 items-center justify-center rounded-lg
                bg-sidebar-primary text-sidebar-primary-foreground">
               <activeTeam.logo className="size-4" />
             </div>
             <div className="grid flex-1 text-left text-sm leading-tight">
               <span className="truncate font-semibold"> {activeTeam.name}</span>
               <span className="truncate text-xs">{activeTeam.plan}</span>
             </div>
             <ChevronsUpDown className="ml-auto" />
           </SidebarMenuButton>
         </DropdownMenuTrigger>
         <DropdownMenuContent className="w-[--radix-dropdown-menu-trigger-width] min-w-56 rounded-lg"
           align="start" side={isMobile ? "bottom" : "right"}  sideOffset={4}>
           <DropdownMenuLabel className="text-xs text-muted-foreground"> Teams </DropdownMenuLabel>
           {teams.map((team, index) => (
             <DropdownMenuItem key={team.name} onClick={() => setActiveTeam(team)} className="gap-2 p-2">
               <div className="flex size-6 items-center justify-center rounded-sm border">
                 <team.logo className="size-4 shrink-0" />
               </div>
               {team.name}
               <DropdownMenuShortcut>⌘{index + 1}</DropdownMenuShortcut>
             </DropdownMenuItem>
           ))}
           <DropdownMenuSeparator />
           <DropdownMenuItem className="gap-2 p-2">
             <div className="flex size-6 items-center justify-center rounded-md border bg-background">
               <Plus className="size-4" />
             </div>
             <div className="font-medium text-muted-foreground">Add team</div>
           </DropdownMenuItem>
         </DropdownMenuContent>
       </DropdownMenu>
     </SidebarMenuItem>
   </SidebarMenu>
 );
}
```

AuthContext.jsx

```
import { checkAuth } from "@/API_Call/Auth";
import { systemConfig } from "@/API_Call/Config";
import { getTools } from "@/API_Call/Tool";
import { getAllUsers } from "@/API_Call/User";
```

```
import { getBugs } from "@/API_Call/Bug";
import { createContext, useContext, useState, useEffect } from "react";
const AuthContext = createContext();
console.log("AuthContext MODULE EVALUATED. Context instance:", AuthContext);
export function AuthProvider({ children }) {
 let apiLink = "http://localhost:8000/api";
 if (document.location.href.includes("localhost")) {   apiLink = "http://localhost:8000/api"; }
 else if (document.location.href.includes("103.209.144.223")) { apiLink = "//103.209.144.223:6969/api"; }
 else {   apiLink = "https://bug.ceoitbox.com/api"; }
 const [user, setUser] = useState(null);
 const [loading, setLoading] = useState(true);
 const [headers, setHeaders] = useState({   "Content-Type": "application/json", });
 useEffect(() => {
   const fetchGlobalData = async () => {
     try {
       const [userData, configData, toolsData, usersData, bugsData] = await Promise.all([
         checkAuth(), systemConfig(), getTools(), getAllUsers(), getBugs() ]);
       setUser(userData);
       if (configData.success && configData.data) {  setStacksList(configData.data.stacksList || []); }
       if (toolsData.success && toolsData.data) {
         const mappedTools = toolsData.data.map(t => ({ ...t, id: t._id })); setToolList(mappedTools);
       }
       if (usersData.success && usersData.data) {
         const mappedUsers = usersData.data.map(u => ({ ...u, id: u._id }));  setAllUsers(mappedUsers);
       }
       if (bugsData.success && bugsData.data) {
         // Adjust based on the actual API wrapper if necessary.
         // The Bug.js API layer directly maps the response properly for us here.
         setBugsList(bugsData.data.results || bugsData.data || []);
       }
     } catch (err) {  console.log("Global data load failed:", err);}
     finally {  setLoading(false);}
   };
   fetchGlobalData();
 }, []);
 const [stacksList, setStacksList] = useState([]);
 const [allStages, setAllStages] = useState([]);
 const [allStatus, setAllStatus] = useState([]);
 const [priority,setPriority] = useState([]);
 const [toolList, setToolList] = useState([]);
 const [allUsers, setAllUsers] = useState([]);
 const [bugsList, setBugsList] = useState([]);
 const value = {
   apiLink,   headers,   setHeaders,   user,   setUser,   loading,   setLoading,    stacksList,setBugsList
   allStages,   allStatus,   priority,   toolList,   setToolList,   allUsers,   setAllUsers,   bugsList,
 };
 return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
}

export function useAuth() {
 const context = useContext(AuthContext);
 if (!context) { throw new Error("useAuth must be used within AuthProvider");}
 return context;
```

```
}
```

NotificationContext.jsx

```
"use client"
import { createContext, useContext } from "react"
export const NotificationContext = createContext()
export function useNotification() {
 const context = useContext(NotificationContext)
 if (!context) { throw new Error("useNotification must be used within NotificationContext.Provider")}
 return context
}
```

hooks (folder)
Use-mobile.jsx

```
import * as React from "react"
const MOBILE_BREAKPOINT = 768
export function useIsMobile() {
 const [isMobile, setIsMobile] = React.useState(undefined)
 React.useEffect(() => {
   const mql = window.matchMedia(`(max-width: ${MOBILE_BREAKPOINT - 1}px)`)
   const onChange = () => {      setIsMobile(window.innerWidth < MOBILE_BREAKPOINT)    }
   mql.addEventListener("change", onChange)
   setIsMobile(window.innerWidth < MOBILE_BREAKPOINT)
   return () => mql.removeEventListener("change", onChange);
 }, [])
 return !!isMobile
}
```

useNotification.jsx

```
import { useNotification as useNotificationContext } from "../context/NotificationContext";
export function useNotification() { return useNotificationContext();
```

Pages (folder)

```
import { useState } from "react";
import General from "@/components/My_Account/General";
import AdminControl from "@/components/My_Account/Admin_Control";
import PasswordControl from "@/components/My_Account/Password";
import { Bolt, User, Lock, Bell } from "lucide-react";
import { Tooltip, TooltipContent, TooltipProvider, TooltipTrigger,} from "@/components/ui/tooltip";
function Account() {
 const [activeTab, setActiveTab] = useState("general");
 const menuItems = [
   { id: "general", label: "General", icon: Bolt },
   { id: "admin", label: "Admin", icon: User },
   { id: "password", label: "Password", icon: Lock },
 ];
 return (
   <div className="w-full h-full bg-white rounded-2xl shadow-sm border border-gray-100 overflow-hidden flex
        flex-col">
     {/* ── Header with Icon Tabs ── */}
     <header className="px-6 py-4 border-b border-gray-100 bg-gray-50/30 flex items-center justify-between
       gap-4">
       <div>
         <h4 className="text-xl font-bold text-gray-900">My Account</h4>
         <p className="text-xs text-gray-500 mt-0.5 capitalize">{activeTab} Settings</p>
```

```
      </div>
      {/* Icon Navigation */}
      <TooltipProvider>
        <nav className="flex items-center bg-gray-100/50 p-1 rounded-xl">
          {menuItems.map((item) => {
            const Icon = item.icon;
            const isActive = activeTab === item.id;
            return (
              <Tooltip key={item.id} delayDuration={0}>
                <TooltipTrigger asChild>
                  <button onClick={() => setActiveTab(item.id)} className={`flex items-center justify-center
                     w-10 h-10 rounded-lg transition-all duration-200 ${
                      isActive ? "bg-white text-cyan-600 shadow-sm"
                        : "text-gray-500 hover:text-gray-700 hover:bg-gray-200/50"
                    }`}>
                    <Icon size={20} className={isActive ? "text-cyan-600" : "text-gray-400"} />
                  </button>
                </TooltipTrigger>
                <TooltipContent side="bottom">  <p>{item.label}</p> </TooltipContent>
              </Tooltip>
            );
          })}
        </nav>
      </TooltipProvider>
    </header>
    {/* ── Content Area ── */}
    <main className="flex-1 overflow-auto p-6 bg-white">
      <div className="max-w-4xl">
        {activeTab === "general" && <General />}
        {activeTab === "admin" && <AdminControl />}
        {activeTab === "password" && <PasswordControl />}
      </div>
    </main>
  </div>
 );
}
export default Account;
```

BugActivityLog.jsx

```
import React, { useState, useEffect, useMemo } from "react";
import { useParams, useNavigate } from "react-router-dom";
import { ChevronLeft, History, Clock, AlertCircle, ArrowRight, User, Filter, Info, Bug as BugIcon,
 Search, Download, Calendar, Flag} from "lucide-react";
import { Button } from "@/components/ui/button";
import { Card } from "@/components/ui/card";
import { Skeleton } from "@/components/ui/skeleton";
import { Table, TableBody, TableCell, TableHead, TableHeader, TableRow} from "@/components/ui/table";
import { getBugLogs, getBugById } from "@/API_Call/Bug";
import { format } from "date-fns";
import { Badge } from "@/components/ui/badge";
const ALL_COLUMNS = [
 { key: "logTimestamp", label: "Date & Time", width: 180 },
 { key: "bugId", label: "Bug ID", width: 140 },
 { key: "toolName", label: "Tool Name", width: 180 },
```

```
 { key: "priority", label: "Priority", width: 120 },
 { key: "stack", label: "Stack", width: 120 },
 { key: "reportedBy", label: "Bug Raiser", width: 150 },
 { key: "issueFacedBy", label: "Issue Faced By", width: 160 },
 { key: "assignedTester", label: "Assigned Tester", width: 150 },
 { key: "assignedDev", label: "Assigned Dev", width: 150 },
 { key: "sopFollowedRaiser", label: "SOP by Raiser", width: 220 },
 { key: "testerSop", label: "Tester SOP", width: 220 },
 { key: "bugDescription", label: "Bug Description", width: 300 },
 { key: "status", label: "Bug Status", width: 150 },
 { key: "expectedResult", label: "Expected Result", width: 250 },
 { key: "actualResult", label: "Actual Result", width: 250 },
 { key: "rootCause", label: "Root Cause", width: 200 },
 { key: "remarks", label: "Testing Remark", width: 200 },
 { key: "analyzeRemark", label: "Analyse Remark", width: 200 },
 { key: "sopForSolution", label: "SOP for Solution", width: 200 },
 { key: "delayedReason", label: "Delayed Reason", width: 200 },
 { key: "testingFlag", label: "Testing Flag", width: 170 },
 { key: "finalTestingRemark", label: "Final Testing Remark", width: 200 },
 { key: "deploymentStatus", label: "Deployment Status", width: 170 },
 { key: "deployRemark", label: "Deploy Remark", width: 200 },
 { key: "finalSop", label: "Final SOP", width: 200 },
 { key: "reportedDate", label: "Reported At", width: 180 },
 { key: "logAction", label: "Action", width: 150 },
 { key: "logUser", label: "Performed By", width: 180 },
 { key: "currentPhase", label: "Current Phase", width: 160 },
];
export default function BugActivityLog() {
 const { id } = useParams();
 const [snapshots, setSnapshots] = useState([]);
 const [loading, setLoading] = useState(true);
 const [bugDetails, setBugDetails] = useState(null);

 const flattenBug = (bug) => {
   if (!bug) return {};
   const p1 = bug.phaseI_BugReport || {};
   const tool = p1.toolInfo || {};
   const client = p1.clientContext || {};
   // Helper to get status from current or fallback phases
   const getStatus = () => {
      return bug.phaseIV_Maintenance?.maintenanceInfo?.status ||
             bug.phaseIII_BugAnalysis?.analysisInfo?.status ||
             bug.phaseII_BugConfirmation?.testingInfo?.status ||
             "-";
   };
   // Format SOP Raiser Checklist to string
   const getSopRaiser = () => {
     const checklist = client.sopChecklist;
     if (!checklist) return "-";
     return Object.entries(checklist).filter(([_, checked]) => checked).map(([text]) => text).join(", ") ||
"-";
   };
   const getName = (user) => {
```

```
    if (!user) return "-";
    if (typeof user === 'object') return user.name || user.fullName || user.label || "-";
    return user;
  };
  return {  bugId: bug.bugId, toolName: tool.toolName,  priority: tool.priority,  stack: tool.stack,
    reportedBy: getName(p1.reportedBy), issueFacedBy: client.facedByClient ? client.clientName : 'Self',
    assignedTester: getName(p1.assignedTester),
    assignedDev: getName(bug.phaseII_BugConfirmation?.assignedDeveloper), sopFollowedRaiser: getSopRaiser(),
    testerSop: Array.isArray(bug.phaseII_BugConfirmation?.testingInfo?.sopFollowed)
      ? bug.phaseII_BugConfirmation.testingInfo.sopFollowed.join(", ")
      : bug.phaseII_BugConfirmation?.testingInfo?.sopFollowed || "-",
    bugDescription: tool.bugDescription,  status: getStatus(), currentPhase: bug.currentPhase,
    expectedResult: tool.expectedResult,  actualResult: tool.actualResult,
    rootCause: bug.phaseIII_BugAnalysis?.analysisInfo?.rootCause,
    remarks: bug.phaseII_BugConfirmation?.testingInfo?.remarks || "-",
    analyzeRemark: bug.phaseIII_BugAnalysis?.analysisInfo?.remarks || "-",
    sopForSolution: Array.isArray(bug.phaseIII_BugAnalysis?.analysisInfo?.sopProvided)
      ? bug.phaseIII_BugAnalysis.analysisInfo.sopProvided.join(", ")
      : bug.phaseIII_BugAnalysis?.analysisInfo?.sopProvided || "-",
    delayedReason: bug.phaseIII_BugAnalysis?.analysisInfo?.delayedReason,
    testingFlag: bug.phaseIV_Maintenance?.maintenanceInfo?.testingFlag || "-",
    finalTestingRemark: bug.phaseIV_Maintenance?.maintenanceInfo?.remarks || "-",
    deploymentStatus: bug.phaseV_FinalTesting?.testingInfo?.deploymentStatus,
    deployRemark: bug.phaseV_FinalTesting?.testingInfo?.deployRemark,
    finalSop: bug.phaseV_FinalTesting?.testingInfo?.finalSop, reportedDate: bug.createdAt
  };
};
useEffect(() => {
  const fetchData = async () => {
    try {
      setLoading(true);
      const [logRes, bugRes] = await Promise.all([  getBugLogs({ id }),  getBugById({ id }) ]);
      if (logRes.success && bugRes.success) {
        const bug = bugRes.data.result;
        setBugDetails(bug);
        const rawLogs = logRes.data.results || [];
        const sortedLogs = [...rawLogs].sort((a, b) => new Date(b.timestamp) - new Date(a.timestamp));
        let timeline = [];
        let tempState = flattenBug(bug);
        sortedLogs.forEach((log) => {
          try {
            const details = JSON.parse(log.details);
            let changedFields = [];
            const snapshotState = { ...tempState };
            if (log.type === 'creation') {
              changedFields = Object.keys(details);
              tempState = {};
            } else if (Array.isArray(details)) {
              details.forEach(diff => {
                if (diff && diff.field) {
                  // Precise mapping from MongoDB path to flattened key
                  const pathMap = {
                    'phaseII_BugConfirmation.testingInfo.remarks': 'remarks',
```

```
                    'testingInfo.remarks': 'remarks',
                    'phaseIII_BugAnalysis.analysisInfo.remarks': 'analyzeRemark',
                    'analysisInfo.remarks': 'analyzeRemark',
                    'phaseIII_BugAnalysis.analysisInfo.sopProvided': 'sopForSolution',
                    'analysisInfo.sopProvided': 'sopForSolution',
                    'sopProvided': 'sopForSolution',
                    'phaseIII_BugAnalysis.analysisInfo.delayedReason': 'delayedReason',
                    'phaseIV_Maintenance.maintenanceInfo.testingFlag': 'testingFlag',
                    'maintenanceInfo.testingFlag': 'testingFlag',
                    'phaseIV_Maintenance.maintenanceInfo.remarks': 'finalTestingRemark',
                    'maintenanceInfo.remarks': 'finalTestingRemark',
                    'phaseV_FinalTesting.testingInfo.deploymentStatus': 'deploymentStatus',
                    'testingInfo.deploymentStatus': 'deploymentStatus',
                    'phaseII_BugConfirmation.assignedDeveloper': 'assignedDev',
                    'phaseI_Reported.assignedTester': 'assignedTester',
                    'phaseII_BugConfirmation.testingInfo.sopFollowed': 'testerSop',
                    'testingInfo.sopFollowed': 'testerSop',
                    'sopFollowed': 'testerSop',
                    'phaseI_Reported.sopChecklist': 'sopFollowedRaiser',
                  };
                  let field = pathMap[diff.field] || diff.field.split('.').pop();
                  if (diff.field.includes('status') && !pathMap[diff.field]) field = 'status';
                  const getCompareValue = (val) => {
                    if (!val) return "";
                    if (typeof val === 'object') return String(val._id || val.id || val.name || "");
                    return String(val).trim();
                  };
                  const oldV = getCompareValue(diff.oldValue);
                  const newV = getCompareValue(diff.newValue);
                  if (oldV !== newV) { changedFields.push(field); }
                  tempState[field] = diff.oldValue;
                }
              });
            }
            // Add log-specific metadata to the state for the row
            const finalState = { ...snapshotState,  logTimestamp: log.timestamp,
              logAction: log.action, logUser: log.performedBy?.name || "System"
            };
            timeline.push({...log, state: finalState,changedFields
            });
          } catch (e) { console.error("Error parsing log details", e);}
        });
        setSnapshots(timeline);
      }
    } catch (error) {console.error("Error fetching activity logs:", error); }
     finally { setLoading(false);}
  };
  fetchData();
}, [id]);

const getPriorityColor = (p) => {
  const priority = String(p || '').toLowerCase();
  if (priority === 'high' || priority === 'critical') return "bg-rose-50 text-rose-700 border-rose-100";
```

```
  if (priority === 'medium') return "bg-amber-50 text-amber-700 border-amber-100";
  return "bg-slate-50 text-slate-700 border-slate-100";
};

const getStatusColor = (s) => {
  const status = String(s || '').toLowerCase();
  if (status.includes('resolved') || status.includes('closed') || status.includes('minor')
      || status.includes('fix')) return "bg-emerald-50 text-emerald-700 border-emerald-100";
  if (status.includes('pending') || status.includes('progress') || status.includes('testing')) return
      "bg-blue-50 text-blue-700 border-blue-100";
  if (status.includes('reopen') || status.includes('critical') || status.includes('rejected')) return
      "bg-rose-50 text-rose-700 border-rose-100";
  return "bg-slate-50 text-slate-700 border-slate-100";
};

const renderCellContent = (key, value) => {
  if (key === 'logTimestamp') {
    try { return <span className="text-xs font-bold text-slate-900">{format(new Date(value),
       "dd/MM/yyyy HH:mm:ss")}</span>;
    } catch (e) { return <span className="text-xs font-bold text-slate-400">Invalid Date</span>; }
  }

  if (key === 'logAction') { return <span className="text-[10px] font-black uppercase tracking-widest
      text-indigo-600 bg-indigo-50 px-2 py-0.5 rounded">{String(value)}</span>;}
  if (key === 'logUser') {  return <span className="text-xs font-bold
text-slate-700">{String(value)}</span>; }
  if (value === null || value === undefined || value === "" || value === "-") return
      <span className="text-slate-300">—</span>;
  if (key === 'priority') {
    return ( <Badge variant="outline" className={`font-black uppercase text-[10px] tracking-widest
       ${getPriorityColor(value)}`}>{value}</Badge>
    );
  }
  if (key === 'status') {
    return (
      <Badge variant="outline" className={`font-bold text-[11px] ${getStatusColor(value)}`}>{value}</Badge>
    );
  }
  if (key === 'testingFlag') {
    const getFlagColor = (val) => {
      if (val === 'Go back to dev' || val === 'Red flag') return 'text-red-500';
      if (val === 'Test by bug raiser' || val === 'Blue Flag') return 'text-blue-500';
      if (val === 'Ready for rollout' || val === 'Green Flag') return 'text-green-500';
      return 'text-slate-300';
    };
    return (
      <div className="flex items-center gap-1.5">
        <Flag className={`w-3.5 h-3.5 shrink-0 ${getFlagColor(value)}`} fill="currentColor" />
        <span className="text-xs font-medium text-slate-700">{value}</span>
      </div>
    );
  }
  if (key === 'reportedDate') {
```

```
      return <span className="text-[11px] font-medium text-slate-500">{format(new Date(value),
        "MMM d, yyyy")}</span>;
    }
    if (Array.isArray(value)) {
      return <span className="text-xs font-medium text-slate-700">{value.join(", ")}</span>;
    }
    if (typeof value === 'object' && value !== null) {
      if (value.name) return <span className="text-xs font-medium text-slate-700">{value.name}</span>;
      if (value.label) return <span className="text-xs font-medium text-slate-700">{value.label}</span>;
      return <span className="text-[10px] font-mono text-slate-400">{JSON.stringify(value)}</span>;
    }
    return <span className="text-xs font-medium text-slate-700 line-clamp-2"
        title={String(value)}>{String(value)}</span>;
  };

  return (
    <div className="min-h-screen bg-slate-100/30 p-4 sm:p-8 flex flex-col gap-6">
      <div className="flex flex-col md:flex-row md:items-center justify-between gap-4 bg-white p-6
        rounded-[2rem] border border-slate-200 shadow-sm relative overflow-hidden">
        <div className="absolute top-0 right-0 w-64 h-64 bg-indigo-500/5 -mr-32 -mt-32 rounded-full" />
        <div className="flex items-center gap-6 relative z-10">
          <Button variant="ghost" size="icon" onClick={() => window.close()}
            className="rounded-2xl h-11 w-11 bg-slate-50 hover:bg-white hover:shadow-md transition-all
              border border-slate-100" >
            <ChevronLeft className="w-5 h-5 text-slate-600" />
          </Button>
          <div className="flex items-center gap-4">
            <div className="bg-indigo-600 p-2.5 rounded-2xl shadow-lg shadow-indigo-200">
              <History className="w-6 h-6 text-white" />
            </div>
            <div>
              <h1 className="text-2xl font-black text-slate-900 tracking-tight">Version Timeline</h1>
              <p className="text-slate-500 text-xs font-bold uppercase tracking-widest mt-1 opacity-60">
                Full Snapshot History</p>
            </div>
          </div>
        </div>
        {bugDetails && (
          <div className="flex items-center gap-3 bg-indigo-50/50 px-4 py-2.5 rounded-2xl border
              border-indigo-100 relative z-10">
            <div className="flex flex-col">
              <span className="text-[9px] font-black text-indigo-400 uppercase tracking-widest">
                Currently Viewing</span>
              <div className="flex items-center gap-2">
                <span className="text-sm font-black text-indigo-700">{bugDetails.bugId}</span>
                <div className="w-1 h-1 rounded-full bg-indigo-200" />
                <span className="text-xs font-bold text-slate-600 max-w-[200px] truncate">
                  {bugDetails.phaseI_BugReport?.toolInfo?.toolName}</span>
              </div>
            </div>
          </div>
        )}
      </div>
```

```
    {/* Main Table Container */}
    <Card className="flex-1 flex flex-col bg-white border-slate-200 shadow-xl rounded-[2.5rem]
        overflow-hidden">
      {/* Table Header Info */}
      <div className="p-6 border-b border-slate-100 bg-slate-50/50 flex items-center justify-between">
         <div className="flex items-center gap-4">
            <div className="flex items-center gap-2 px-3 py-1.5 bg-white border border-slate-200 rounded-xl
               shadow-sm">
               <Calendar className="w-3.5 h-3.5 text-slate-400" />
               <span className="text-[10px] font-black text-slate-500 uppercase tracking-widest">
               Recorded Versions: {snapshots.length}</span>
            </div>
         </div>
         <div className="flex items-center gap-2 text-[10px] font-bold text-slate-400">
            <div className="w-2.5 h-2.5 bg-amber-400 rounded-full animate-pulse
              shadow-[0_0_8px_rgba(251,191,36,0.5)]" />
            <span>Highlighted cells indicate changes at that timestamp</span>
         </div>
      </div>
      <div className="flex-1 overflow-auto relative custom-scrollbar">
        {loading ? (
           <div className="p-8 space-y-4">
              {[1, 2, 3, 4, 5].map(i => <Skeleton key={i} className="h-16 w-full rounded-2xl bg-slate-50"
/>)}
           </div>
        ) : (
          <Table className="border-collapse min-w-max">
            <TableHeader className="sticky top-0 z-30 bg-white/95 backdrop-blur-md shadow-sm">
              <TableRow className="hover:bg-transparent border-b border-slate-100">
                {ALL_COLUMNS.map((col) => (
                  <TableHead key={col.key} className="p-6 text-[10px] font-black text-slate-400 uppercase
                     tracking-widest border-r border-slate-50"style={{ width: col.width }}>{col.label}
                  </TableHead>
                ))}
              </TableRow>
            </TableHeader>
            <TableBody>
              {snapshots.length > 0 ? (
                snapshots.map((snap, sIdx) => (
                  <TableRow key={snap._id} className="group hover:bg-slate-50/30 transition-all border-b
                     border-slate-50 last:border-0">
                    {/* Data Cells */}
                    {ALL_COLUMNS.map((col) => {
                      const isChanged = snap.changedFields.includes(col.key);
                      return (
                        <TableCell key={col.key} className={`p-4 border-r border-slate-200 transition-all ${
                            isChanged ? 'bg-yellow-100/80 !text-black shadow-inner border-y-yellow-400/30' :
''
                        }`}>
                          <div className={isChanged ? 'font-bold' : ''}>
                            {renderCellContent(col.key, snap.state[col.key])}
                          </div>
                        </TableCell>
```

```
                      );
                    })}
                  </TableRow>
                ))
              ) : (
                <TableRow>
                  <TableCell colSpan={ALL_COLUMNS.length} className="h-48 text-center bg-slate-50/50">
                    <div className="flex flex-col items-center gap-3">
                      <div className="p-3 bg-white rounded-2xl shadow-sm border border-slate-100">
                        <Info className="w-6 h-6 text-slate-300" />
                      </div>
                      <div>
                        <p className="text-sm font-black text-slate-400 tracking-tight">
                          No Log History Found</p>
                        <p className="text-[10px] font-bold text-slate-400 uppercase tracking-widest
mt-1">
                          This bug has not been updated yet</p>
                      </div>
                    </div>
                  </TableCell>
                </TableRow>
              )}
            </TableBody>
          </Table>
        )}
      </div>
    </Card>

    <style dangerouslySetInnerHTML={{ __html: `
      .custom-scrollbar::-webkit-scrollbar { width: 8px; height: 8px;}
      .custom-scrollbar::-webkit-scrollbar-track {background: #f8fafc; border-radius: 10px;}
      .custom-scrollbar::-webkit-scrollbar-thumb { background: #e2e8f0; border-radius: 10px;
        border: 2px solid #f8fafc;
      }
      .custom-scrollbar::-webkit-scrollbar-thumb:hover { background: #cbd5e1;}
    `}} />
  </div>
 );
}
```

BugsPage.jsx

```
import { useState } from "react";
import { useAuth } from "@/context/AuthContext";
import { filterBugsForRaiser, filterBugsForTester, filterBugsForDev,} from "@/utils/bugRoleFilter";
import CreateBug from "../components/Phases/PhaseI/CreateBug";
import BugTesting from "../components/Phases/PhaseII/BugTesting";
import AnalyzeBug from "../components/Phases/PhaseIII/BugAnalyze";
import Maintenance from "../components/Phases/PhaseIV/Maintenance";
import ReadyToTest from "../components/Phases/PhaseV/ReadyToTesting";
import ReadyToDeploy from "../components/Phases/PhaseVI/ReadyToDeploy";
import Deployed from "../components/Phases/PhaseVII/Deployed";
import AllBugsStage from "../components/Phases/Shared/AllBugsStage";
import { LayoutGrid, Bug as BugIconLucide, TestTube2, Microscope, Wrench, CheckCircle2,
 Rocket, BadgeCheck as DeployedIcon,} from "lucide-react";
```

```
const RAW_PHASES = [
 {id: "all",   label: "All Bugs",   shortLabel: "All",   icon: LayoutGrid,   color: "#3b82f6",
   component: AllBugsStage,   phaseStr: "All",},
 {id: "create",   label: "New Bug",   shortLabel: "New",   icon: BugIconLucide,   color: "#06b6d4",
   component: CreateBug,   phaseStr: "Bug Reported",},
 { id: "testing",   label: "Bug Testing",   shortLabel: "Testing",   icon: TestTube2,   color: "#6366f1",
   component: BugTesting,   phaseStr: "Bug Testing",},
 {id: "analyze",   label: "Analyze Bug",   shortLabel: "Analyze",   icon: Microscope,   color: "#8b5cf6",
   component: AnalyzeBug,   phaseStr: "Bug Analysis", },
 { id: "maintenance",   label: "Maintenance",   shortLabel: "Maintain",   icon: Wrench,   color: "#ec4899",
   component: Maintenance,   phaseStr: "Maintenance", },
 {id: "rtt",   label: "Ready to Test",   shortLabel: "Rdy Test",   icon: CheckCircle2,   color: "#f59e0b",
   component: ReadyToTest,   phaseStr: "Ready to Test", },
 {id: "rtd",   label: "Ready to Deploy",   shortLabel: "Rdy Deploy",   icon: Rocket,   color: "#f97316",
   component: ReadyToDeploy,   phaseStr: "Ready to Deploy", },
 {id: "deployed",   label: "Deployed",   shortLabel: "Live",   icon: DeployedIcon,   color: "#10b981",
   component: Deployed,   phaseStr: "Closure", },
];
function getUserRoleFlags(user) {
 if (!user)
   return {     isAdmin: false,     isBugRaiser: false,     isTester: false,     isDev: false,   };
 const isAdmin = user.role === "admin" || user.role === "superadmin";
 const roleArr = Array.isArray(user.roletype)? user.roletype : user.roletype ? [user.roletype]: [];
 const isBugRaiser = roleArr.includes("bugreporter");
 const isTester = roleArr.includes("tester");
 const isDev = roleArr.includes("dev");
 return { isAdmin, isBugRaiser, isTester, isDev };
}
function getVisiblePhases(user) {
 const { isAdmin, isBugRaiser, isTester, isDev } = getUserRoleFlags(user);
 if (isAdmin) return RAW_PHASES;
 const allowedIds = new Set(["all"]);
 if (isBugRaiser) {   ["create", "deployed"].forEach((id) => allowedIds.add(id)); }
 if (isTester) {   ["create", "testing", "rtt", "deployed"].forEach((id) => allowedIds.add(id),); }
 if (isDev) {   ["analyze", "maintenance", "rtd", "deployed"].forEach((id) => allowedIds.add(id)); }
 return RAW_PHASES.filter((p) => allowedIds.has(p.id));
}
function getDefaultActiveTab(user, visiblePhases) {
 const { isAdmin, isBugRaiser, isTester, isDev } = getUserRoleFlags(user);
 const ids = visiblePhases.map((p) => p.id);
 if (isAdmin || isTester) return ids.includes("all") ? "all" : ids[0];
 if (isBugRaiser && !isTester && !isDev)   return ids.includes("create") ? "create" : ids[0];
 if (isDev && !isBugRaiser && !isTester)   return ids.includes("analyze") ? "analyze" : ids[0];
 return ids[0];
}
export default function BugsPage() {
 const { bugsList, user } = useAuth();
 const PHASES = getVisiblePhases(user);
 const defaultTab = getDefaultActiveTab(user, PHASES);
 const [active, setActive] = useState(defaultTab);
 const current = PHASES.find((p) => p.id === active) || PHASES[0];
 const ActiveComponent = current?.component;
 const getPhaseCount = (phaseStr) => {
```

```
  if (!bugsList) return 0;
  if (phaseStr === "All") return bugsList.length;
  let phaseBugs;
  if (phaseStr === "Bug Reported") { phaseBugs = bugsList.filter((b) => b.currentPhase === "Bug Reported");
    return filterBugsForRaiser(phaseBugs, user).length;
  }
  if (phaseStr === "Bug Testing") {
    phaseBugs = bugsList.filter(
      (b) => b.currentPhase === "Bug Testing" || b.currentPhase === "Bug Confirmation",
    );
    return filterBugsForTester(phaseBugs, user).length;
  }
  if (phaseStr === "Bug Analysis") {
    phaseBugs = bugsList.filter((b) => b.currentPhase === "Bug Analysis");
    return filterBugsForDev(phaseBugs, user).length;
  }
  if (phaseStr === "Ready to Test") {
    phaseBugs = bugsList.filter((b) => b.currentPhase === "Ready to Test");
    return filterBugsForTester(phaseBugs, user).length;
  }
  if (phaseStr === "Ready to Deploy") {
    phaseBugs = bugsList.filter(
      (b) => b.currentPhase === "Ready to Deploy" || b.currentPhase === "Final Testing",
    );
    return filterBugsForDev(phaseBugs, user).length;
  }
  return bugsList.filter((b) => b.currentPhase === phaseStr).length;
};
return (
  <div className="flex flex-col min-h-full gap-4">
    {/* ═══ Page title row ═══ */}
    <div className="flex items-center justify-between flex-wrap gap-2">
      <div> <h1 className="text-2xl font-bold text-gray-900 leading-tight"> Bugs</h1> </div>
    </div>
    {/* ═══ Horizontal scrollable phase tab bar ═══ */}
    <div className="flex items-stretch gap-2 overflow-x-auto pb-1 flex-shrink-0"
      style={{ scrollbarWidth: "none", WebkitOverflowScrolling: "touch" }}>
      {PHASES.map((phase) => {
        const isActive = phase.id === active;
        return (
          <button key={phase.id} onClick={() => setActive(phase.id)}
            className="relative flex items-center gap-2 px-4 py-3 rounded-2xl text-sm font-semibold
             whitespace-nowrap flex-shrink-0 transition-all duration-200 group overflow-hidden"
            style={
              isActive
                ? {background: `linear-gradient(135deg, ${phase.color}dd, ${phase.color}aa)`,color: "#fff",
                    boxShadow: `0 6px 20px ${phase.color}40`, transform: "translateY(-1px)", }
                : {background: "#fff",color: "#64748b",border: "1.5px solid #e2e8f0", }
            }>
            <span
              className="min-w-[20px] h-5 px-1.5 rounded-lg text-[10px] font-bold flex items-center
             justify-center flex-shrink-0 transition-colors"
              style={
```

```
                isActive ? { background: "rgba(255,255,255,0.25)", color: "#fff" }
                  : { background: "#f1f5f9", color: "#94a3b8" }
              } >
              {getPhaseCount(phase.phaseStr)}
            </span>
            <phase.icon size={18} className="flex-shrink-0" />
            <span className="hidden sm:inline">{phase.label}</span>
            <span className="sm:hidden">{phase.shortLabel}</span>
            {isActive && (
              <span style={{ background: "rgba(255,255,255,0.5)" }}
                className="absolute bottom-0 left-1/2 -translate-x-1/2 w-8 h-0.5 rounded-full" />
            )}
          </button>
        );
      })}
    </div>
    {/* ═══ Content card ═══ */}
    <div className="flex-1 bg-white rounded-2xl border border-gray-100 shadow-sm overflow-hidden flex
      flex-col">
      <div className="p-5 overflow-auto flex-1">{ActiveComponent && <ActiveComponent />}</div>
      {active === "deployed" && (
        <div className="py-2.5 text-center bg-red-50/30 border-t border-red-100/50">
          <p className="text-[10px] text-red-500 font-semibold italic uppercase tracking-wider">
            Note: Data will no longer be visible here after 15 days of deployment.
          </p>
        </div>
      )}
    </div>
  </div>
);
}
```

Dashboard.jsx

```
import { useMemo } from "react";
import { useAuth } from "@/context/AuthContext";
import { Bug, Settings, CheckCircle2, User as UserIcon, Rocket, AlertCircle, Clock, LayoutGrid,
 ChevronRight, TrendingUp, Activity} from "lucide-react";
import { Link } from "react-router-dom";
import { AreaChart, Area, XAxis, YAxis, CartesianGrid, Tooltip as RechartsTooltip, ResponsiveContainer,
 PieChart, Pie, Cell, Legend} from "recharts";
function Dashboard() {
 const { bugsList, allUsers, user } = useAuth();
 const today = new Date().toLocaleDateString("en-IN", { weekday: "long", day: "numeric", month: "long", year:
"numeric" });
 const stats = useMemo(() => {
   if (!bugsList) return { total: 0, inProgress: 0, resolved: 0, urgent: 0, unassigned: 0, myTasks: 0, today:
0 };
   const todayStart = new Date();
   todayStart.setHours(0, 0, 0, 0);
   return {
     total: bugsList.length,
     inProgress: bugsList.filter(b => ["Bug Testing", "Bug Analysis", "Ready to Test",
"Maintenance"].includes(b.currentPhase)).length,
```

```
    resolved: bugsList.filter(b => ["Ready to Deploy", "Final Testing",
"Closure"].includes(b.currentPhase)).length,
    urgent: bugsList.filter(b => b.phaseI_BugReport?.toolInfo?.priority === "Urgent").length,
    unassigned: bugsList.filter(b => !b.phaseII_BugConfirmation?.assignedDeveloper &&
!b.phaseI_BugReport?.assignedTester).length,
    myTasks: bugsList.filter(b =>
      (b.phaseII_BugConfirmation?.assignedDeveloper === user?._id) ||
      (b.phaseI_BugReport?.reportedBy === user?._id && b.currentPhase === "Bug Reported")
    ).length,
    today: bugsList.filter(b => new Date(b.createdAt) >= todayStart).length
  };
}, [bugsList, user]);
const stackStats = useMemo(() => {
  if (!bugsList) return [];
  const counts = {};
  bugsList.forEach(b => {
    const s = b.phaseI_BugReport?.toolInfo?.stack || "Other"; counts[s] = (counts[s] || 0) + 1;
  });
  return Object.entries(counts)
    .map(([name, count]) => ({ name, count, pct: (count / bugsList.length) * 100 }))
    .sort((a, b) => b.count - a.count) .slice(0, 5);
}, [bugsList]);
const toolStats = useMemo(() => {
  if (!bugsList) return [];
  const counts = {};
  bugsList.forEach(b => {
    const t = b.phaseI_BugReport?.toolInfo?.toolName || "Unknown"; counts[t] = (counts[t] || 0) + 1;
  });
  return Object.entries(counts).map(([name, count]) => ({ name, count })).sort((a, b) => b.count - a.count)
    .slice(0, 6);
}, [bugsList]);
const pipeline = useMemo(() => {
  if (!bugsList) return [];
  const phases = [
    { id: "reported", label: "Reported", color: "bg-cyan-500", key: "Bug Reported" },
    { id: "testing", label: "Testing", color: "bg-indigo-500", key: "Bug Testing" },
    { id: "analysis", label: "Analysis", color: "bg-violet-500", key: "Bug Analysis" },
    { id: "rtt", label: "Ready to Test", color: "bg-orange-500", key: "Ready to Test" },
    { id: "rtd", label: "Ready to Deploy", color: "bg-amber-500", key: "Ready to Deploy" },
    { id: "closure", label: "Closure", color: "bg-emerald-500", key: "Closure" }
  ];
  return phases.map(p => {
    const count = bugsList.filter(b => b.currentPhase === p.key).length;
    return { ...p, count, pct: (count / (bugsList.length || 1)) * 100 };
  });
}, [bugsList]);
const priorities = useMemo(() => {
  if (!bugsList) return [];
  const levels = [
    { label: "Urgent", color: "bg-rose-500", text: "text-rose-500" },
    { label: "High", color: "bg-orange-500", text: "text-orange-500" },
    { label: "Medium", color: "bg-amber-500", text: "text-amber-500" },
    { label: "Low", color: "bg-emerald-500", text: "text-emerald-500" }
```

```
  ];
  return levels.map(l => {
    const count = bugsList.filter(b => b.phaseI_BugReport?.toolInfo?.priority === l.label).length;
    return { ...l, count, pct: (count / (bugsList.length || 1)) * 100 };
  });
}, [bugsList]);
const recentActivity = useMemo(() => {
  if (!bugsList) return [];
  return [...bugsList].sort((a, b) => new Date(b.updatedAt) - new Date(a.updatedAt)).slice(0, 6);
}, [bugsList]);
const teamStats = useMemo(() => {
  if (!allUsers || !bugsList) return [];
  return allUsers
    .map(u => {
      const assigned = bugsList.filter(b =>
        b.phaseII_BugConfirmation?.assignedDeveloper === u.id ||b.phaseI_BugReport?.assignedTester === u.id
      ).length;
      const resolved = bugsList.filter(b =>
        (b.phaseVII_Closure?.closedBy === u.id) ||
        (b.currentPhase === "Closure" && b.phaseII_BugConfirmation?.assignedDeveloper === u.id)
      ).length;
      return { ...u, assigned, resolved };
    })
    .filter(u => u.assigned > 0).sort((a, b) => b.assigned - a.assigned).slice(0, 5);
}, [allUsers, bugsList]);
const canReport = useMemo(() => {
  if (!user) return false;
  const isAdmin = user.role === "admin" || user.role === "superadmin";
  const roleArr = Array.isArray(user.roletype) ? user.roletype : (user.roletype ? [user.roletype] : []);
  return roleArr.includes("bugreporter") || isAdmin;
}, [user]);
const trendData = useMemo(() => {
  if (!bugsList) return [];
  const days = [];
  for (let i = 6; i >= 0; i--) {
    const d = new Date();
    d.setDate(d.getDate() - i);
    days.push({ date: d.toLocaleDateString("en-US", { month: "short", day: "numeric" }),
      rawDate: new Date(d.setHours(0,0,0,0)), reported: 0,resolved: 0
    });
  }
  bugsList.forEach(b => {
    const bDate = new Date(b.createdAt);bDate.setHours(0,0,0,0);
    const day = days.find(d => d.rawDate.getTime() === bDate.getTime());
    if (day) day.reported += 1;
    if (["Ready to Deploy", "Final Testing", "Closure"].includes(b.currentPhase)) {
      const rDate = new Date(b.updatedAt);rDate.setHours(0,0,0,0);
      const rDay = days.find(d => d.rawDate.getTime() === rDate.getTime());
      if (rDay) rDay.resolved += 1;
    }
  });
  return days;
}, [bugsList]);
```

```
const statusPieData = useMemo(() => {
  if (!bugsList) return [];
  const counts = {
    "Open": bugsList.filter(b => ["Bug Reported"].includes(b.currentPhase)).length,
    "In Progress": bugsList.filter(b => ["Bug Testing", "Bug Analysis", "Ready to Test", "Maintenance"]
      .includes(b.currentPhase)).length,
    "Resolved": bugsList.filter(b => ["Ready to Deploy", "Final Testing", "Closure"]
      .includes(b.currentPhase)).length
  };
  return Object.entries(counts).filter(([_, v]) => v > 0).map(([name, value]) => ({ name, value }));
}, [bugsList]);
const PIE_COLORS = ['#ef4444', '#f59e0b', '#10b981']; // Red, Amber, Emerald
return (
  <div className="space-y-6 pb-8 animate-in fade-in duration-500">
    {/* ── Page Header ── */}
    <div className="flex flex-col md:flex-row md:items-center justify-between gap-4">
      <div className="space-y-1">
        <div className="flex items-center gap-2">
          <h1 className="text-3xl font-extrabold text-slate-900 tracking-tight">Dashboard</h1>
          <span className="px-2.5 py-1 rounded-full bg-indigo-50 text-indigo-600 text-[10px] font-bold
              uppercase tracking-wider border border-indigo-100 flex items-center gap-1.5">
            <Activity size={12} /> Live Updates
          </span>
        </div>
        <p className="text-sm text-slate-400 font-medium flex items-center gap-2">
          <Clock size={14} className="text-slate-300" />{today}</p>
      </div>
      <div className="flex items-center gap-3">
        <Link to="/bugs" className="flex items-center gap-2 px-5 py-2.5 rounded-2xl text-sm font-bold
            text-slate-600 bg-white border border-slate-200 hover:bg-slate-50 transition-all shadow-sm
            active:scale-95">
          <LayoutGrid size={18} /> View All Bugs
        </Link>
        {canReport && (
          <Link to="/bugs" className="flex items-center gap-2 px-5 py-2.5 rounded-2xl text-sm font-bold
              text-white transition-all shadow-lg shadow-indigo-200 hover:shadow-indigo-300
              hover:-translate-y-0.5 active:translate-y-0 active:scale-95"
            style={{ background: "linear-gradient(135deg, #6366f1, #4f46e5)" }}>
            <span className="text-lg">+</span> New Bug
          </Link>
        )}
      </div>
    </div>

    {/* ── KPI Stat Grid ── */}
    <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 xl:grid-cols-6 gap-4">
      {[
        { label: "Total Bugs", value: stats.total, icon: Bug, color: "text-blue-600", bg: "bg-blue-50",
            border: "border-blue-100", delta: `${stats.today} new today` },
        { label: "In Progress", value: stats.inProgress, icon: Settings, color: "text-indigo-600",
            bg: "bg-indigo-50", border: "border-indigo-100", delta: "Active cycle" },
        { label: "Resolved", value: stats.resolved, icon: CheckCircle2, color: "text-emerald-600",
            bg: "bg-emerald-50", border: "border-emerald-100", delta: "Master record" },
```

```
        { label: "Urgent", value: stats.urgent, icon: AlertCircle, color: "text-rose-600",
              bg: "bg-rose-50", border: "border-rose-100", delta: "Immediate focus" },
        { label: "Unassigned", value: stats.unassigned, icon: UserIcon, color: "text-amber-600",
              bg: "bg-amber-50", border: "border-amber-100", delta: "Pending triage" },
        { label: "My Tasks", value: stats.myTasks, icon: Rocket, color: "text-violet-600",
             bg: "bg-violet-50", border: "border-violet-100", delta: "Current workload" },
      ].map((item, idx) => (
        <div key={idx} className={`bg-white rounded-[24px] border ${item.border}
              p-5 shadow-sm hover:shadow-md transition-all group overflow-hidden relative`}>
           <div className="absolute -right-4 -top-4 w-24 h-24 rounded-full bg-slate-50/50
              group-hover:scale-110 transition-transform duration-500" />
           <div className="relative z-10 flex flex-col h-full justify-between gap-4">
             <div className={`w-11 h-11 rounded-2xl ${item.bg} flex items-center justify-center
              ${item.color} shadow-sm border border-white/50`}><item.icon size={22} strokeWidth={2.5} />
             </div>
             <div>
               <p className="text-3xl font-black text-slate-800 tracking-tight">{item.value}</p>
               <div className="flex flex-col">
                 <p className="text-[11px] font-bold text-slate-400 uppercase tracking-widest mt-1">
                     {item.label}</p>
                 <p className="text-[10px] font-semibold text-slate-300 italic mt-0.5">{item.delta}</p>
               </div>
             </div>
           </div>
        </div>
      ))}
    </div>
    {/* ── Advanced Insights Row ── */}
    <div className="grid grid-cols-1 lg:grid-cols-2 gap-6">
       {/* Tech Stack Distribution */}
       <div className="bg-white rounded-[32px] border border-slate-100 shadow-sm p-7">
          <h2 className="text-lg font-bold text-slate-800 mb-6 flex items-center gap-2">
             <div className="w-1.5 h-6 bg-indigo-500 rounded-full" /> Tech Stack Distribution
          </h2>
          <div className="grid grid-cols-1 sm:grid-cols-2 gap-4">
             {stackStats.map((s, i) => (
               <div key={i} className="p-4 rounded-[20px] bg-slate-50/50 border border-slate-100
                    hover:border-indigo-200 transition-colors">
                  <div className="flex justify-between items-center mb-2">
                     <span className="text-xs font-black text-slate-600 uppercase tracking-tight">
                     {s.name}</span>
                     <span className="text-xs font-black text-indigo-600">{s.count}</span>
                  </div>
                  <div className="h-2 bg-white rounded-full overflow-hidden p-0.5 border border-slate-200">
                 <div className="h-full bg-indigo-500 rounded-full shadow-sm" style={{ width: `${s.pct}%` }}
/>
                  </div>
               </div>
             ))}
          </div>
       </div>
       {/* Project Hotspots */}
       <div className="bg-white rounded-[32px] border border-slate-100 shadow-sm p-7">
```

```
          <h2 className="text-lg font-bold text-slate-800 mb-6 flex items-center gap-2">
             <div className="w-1.5 h-6 bg-rose-500 rounded-full" />Project Hotspots
          </h2>
          <div className="flex flex-wrap gap-2">
             {toolStats.map((t, i) => (
               <div key={i} className="px-4 py-3 rounded-2xl bg-white border border-slate-200 flex
items-center
                    gap-3 shadow-sm hover:shadow-md transition-all cursor-default group">
                  <div className={`w-8 h-8 rounded-xl flex items-center justify-center font-black
text-[10px]
                    ${ i === 0 ? "bg-rose-50 text-rose-600 border border-rose-100"
                    : "bg-slate-50 text-slate-400 border border-slate-100"}`}>#{i + 1}
                  </div>
                  <div><p className="text-xs font-bold text-slate-700">{t.name}</p>
                     <p className="text-[10px] font-black text-slate-400 uppercase tracking-tighter
                       group-hover:text-rose-500 transition-colors">{t.count} Active Bugs</p>
                  </div>
               </div>
             ))}
          </div>
       </div>
    </div>
    {/* ── Charts & Trends Row ── */}
    <div className="grid grid-cols-1 lg:grid-cols-3 gap-6">
       {/* 7-Day Trend Chart */}
       <div className="lg:col-span-2 bg-white rounded-[32px] border border-slate-100 shadow-sm p-7">
          <div className="flex items-center justify-between mb-6">
            <h2 className="text-lg font-bold text-slate-800 flex items-center gap-2">
               <div className="w-1.5 h-6 bg-blue-500 rounded-full" />7-Day Reporting Trend
            </h2>
          </div>
          <div className="h-72">
            <ResponsiveContainer width="100%" height="100%">
              <AreaChart data={trendData} margin={{ top: 10, right: 10, left: -20, bottom: 0 }}>
                <defs>
                  <linearGradient id="colorReported" x1="0" y1="0" x2="0" y2="1">
                    <stop offset="5%" stopColor="#6366f1" stopOpacity={0.3}/>
                    <stop offset="95%" stopColor="#6366f1" stopOpacity={0}/>
                  </linearGradient>
                  <linearGradient id="colorResolved" x1="0" y1="0" x2="0" y2="1">
                    <stop offset="5%" stopColor="#10b981" stopOpacity={0.3}/>
                    <stop offset="95%" stopColor="#10b981" stopOpacity={0}/>
                  </linearGradient>
                </defs>
                <CartesianGrid strokeDasharray="3 3" vertical={false} stroke="#f1f5f9" />
                <XAxis dataKey="date" axisLine={false} tickLine={false} tick={{ fontSize: 12,
                     fill: '#94a3b8' }} dy={10} />
                <YAxis axisLine={false} tickLine={false} tick={{ fontSize: 12, fill: '#94a3b8' }} />
                <RechartsTooltip contentStyle={{ borderRadius: '16px', border: 'none',
                     boxShadow: '0 10px 15px -3px rgb(0 0 0 / 0.1)' }}
                />
                <Legend iconType="circle" wrapperStyle={{ fontSize: '12px', paddingTop: '20px' }} />
                <Area type="monotone" name="Bugs Reported" dataKey="reported" stroke="#6366f1"
```

```
                    strokeWidth={3} fillOpacity={1} fill="url(#colorReported)" />
                <Area type="monotone" name="Bugs Resolved" dataKey="resolved" stroke="#10b981"
                    strokeWidth={3} fillOpacity={1} fill="url(#colorResolved)" />
              </AreaChart>
            </ResponsiveContainer>
          </div>
        </div>

        {/* Status Distribution Pie Chart */}
        <div className="bg-white rounded-[32px] border border-slate-100 shadow-sm p-7 flex flex-col">
          <h2 className="text-lg font-bold text-slate-800 mb-2 flex items-center gap-2">
            <div className="w-1.5 h-6 bg-amber-500 rounded-full" /> Status Overview
          </h2>
          <div className="flex-1 flex items-center justify-center -mt-4">
            <ResponsiveContainer width="100%" height={240}>
              <PieChart>
                <Pie data={statusPieData}cx="50% "cy="50%" innerRadius={65}  outerRadius={85}
                  paddingAngle={5} dataKey="value" stroke="none">
                  {statusPieData.map((entry, index) => (
                    <Cell key={`cell-${index}`} fill={PIE_COLORS[index % PIE_COLORS.length]} />
                  ))}
                </Pie>
                <RechartsTooltip contentStyle={{ borderRadius: '12px', border: 'none',
                    boxShadow: '0 4px 6px -1px rgb(0 0 0 / 0.1)' }}
                  itemStyle={{ color: '#1e293b', fontWeight: 'bold' }}/>
                <Legend verticalAlign="bottom" height={36} iconType="circle"
                    wrapperStyle={{ fontSize: '12px' }} />
              </PieChart>
            </ResponsiveContainer>
          </div>
        </div>
      </div>
      {/* ── Main Content Area ── */}
      <div className="grid grid-cols-1 xl:grid-cols-3 gap-6">
        {/* Bug Pipeline Analysis */}
        <div className="xl:col-span-2 space-y-6">
          <div className="bg-white rounded-[32px] border border-slate-100 shadow-sm p-7">
            <div className="flex items-center justify-between mb-8">
              <div> <h2 className="text-lg font-bold text-slate-800">Lifecycle Pipeline</h2>
                <p className="text-xs text-slate-400 mt-1 font-medium">
                Tracking bugs across operational phases</p>
              </div>
              <div className="flex items-center gap-1 text-[10px] font-bold text-slate-400 uppercase
                tracking-tighter">
                <TrendingUp size={14} className="text-emerald-500" />Real-time Distribution
              </div>
            </div>
            <div className="space-y-6">
              {pipeline.map((p) => (
                <div key={p.id} className="group cursor-default">
                  <div className="flex items-center justify-between mb-2">
                    <span className="text-sm font-bold text-slate-600 group-hover:text-slate-900
                      transition-colors">{p.label}</span>
```

```
                    <div className="flex items-center gap-2">
                      <span className="text-xs font-black text-slate-900">{p.count}</span>
                      <span className="text-[10px] font-bold text-slate-300">{Math.round(p.pct)}%</span>
                    </div>
                  </div>
                  <div className="h-3.5 bg-slate-50 rounded-full overflow-hidden border border-slate-100
p-0.5">
                    <div className={`h-full rounded-full ${p.color} shadow-sm transition-all duration-1000
                       ease-out`}style={{ width: `${p.pct}%` }}/>
                  </div>
                </div>
              ))}
            </div>
          </div>
          <div className="grid grid-cols-1 md:grid-cols-2 gap-6">
            {/* Priority Breakdown */}
            <div className="bg-white rounded-[32px] border border-slate-100 shadow-sm p-7">
              <h2 className="text-lg font-bold text-slate-800 mb-6">Priority Mix</h2>
              <div className="flex h-12 gap-1 rounded-2xl overflow-hidden mb-8 border-4 border-slate-50">
                {priorities.map((l, i) => l.count > 0 && (
                  <div key={i} className={`${l.color} h-full transition-all duration-700 hover:opacity-80`}
                    style={{ width: `${l.pct}%` }} title={`${l.label}: ${l.count} bugs`}/>
                ))}
              </div>
              <div className="grid grid-cols-2 gap-4">
                {priorities.map((l, i) => (
                  <div key={i} className="flex items-center gap-3 p-3 rounded-2xl border border-slate-50
                     bg-slate-50/30">
                    <div className={`w-2 h-8 rounded-full ${l.color}`} />
                    <div>   <p className="text-xs font-black text-slate-800">{l.count}</p>
                      <p className={`text-[10px] font-bold uppercase tracking-tight
${l.text}`}>{l.label}</p>
                    </div>
                  </div>
                ))}
              </div>
            </div>
            {/* Team Spotlight */}
            <div className="bg-white rounded-[32px] border border-slate-100 shadow-sm p-7">
              <h2 className="text-lg font-bold text-slate-800 mb-6">Resource Allocation</h2>
              <div className="space-y-4">
                {teamStats.length > 0 ? teamStats.map((member, i) => (
                  <div key={i} className="flex items-center gap-4 group">
                    <div className="w-10 h-10 rounded-2xl bg-slate-100 flex items-center justify-center
                         text-slate-500 font-bold text-sm border-2 border-white shadow-sm overflow-hidden">
                      {member.avatar ? ( <span className="bg-indigo-500 text-white w-full h-full flex
                           items-center justify-center uppercase">{member.name.charAt(0)}</span>
                      ) : (<UserIcon size={18} /> )}
                    </div>
                    <div className="flex-1 min-w-0">
                      <p className="text-sm font-bold text-slate-700 truncate">{member.name}</p>
                      <div className="flex items-center gap-2 mt-0.5">
                        <div className="flex items-center gap-1">
```

```
                          <span className="w-1.5 h-1.5 rounded-full bg-indigo-500" />
                          <span className="text-[10px] font-bold text-slate-400">{member.assigned}
                           Assigned</span>
                        </div>
                        <div className="flex items-center gap-1">
                          <span className="w-1.5 h-1.5 rounded-full bg-emerald-500" />
                          <span className="text-[10px] font-bold
text-slate-400">{member.resolved}Fixed</span>
                        </div>
                      </div>
                    </div>
                    <ChevronRight size={16} className="text-slate-200 group-hover:text-slate-400
                     transition-colors" />
                  </div>
                )) : (
                  <div className="flex flex-col items-center justify-center h-48 text-center
text-slate-300">
                    <UserIcon size={32} className="mb-2 opacity-20" />
                    <p className="text-xs font-medium">No active assignments</p>
                  </div>
                )}
              </div>
            </div>
          </div>
        </div>
        {/* Recent Activity Feed */}
        <div className="bg-white rounded-[32px] border border-slate-100 shadow-sm flex flex-col">
          <div className="p-7 border-b border-slate-50">
            <h2 className="text-lg font-bold text-slate-800">Live Activity</h2>
            <p className="text-xs text-slate-400 mt-1 font-medium">Latest forensic updates from the field</p>
          </div>
          <div className="flex-1 overflow-auto p-4 space-y-1">
            {recentActivity.length > 0 ? recentActivity.map((activity, idx) => (
              <div key={idx} className="flex items-start gap-4 p-4 rounded-3xl hover:bg-slate-50
                transition-colors group cursor-pointer">
                <div className="relative">
                  <div className="w-11 h-11 rounded-2xl bg-indigo-50 flex items-center justify-center
                      text-indigo-500 border border-indigo-100"><Bug size={20} />
                  </div>
                  <div className="absolute -bottom-1 -right-1 w-5 h-5 rounded-full bg-white border-2
                      border-white shadow-sm flex items-center justify-center">
                    <div className={`w-2.5 h-2.5 rounded-full ${
                      activity.phaseI_BugReport?.toolInfo?.priority === "Urgent" ? "bg-rose-500" :
                      activity.phaseI_BugReport?.toolInfo?.priority === "High" ? "bg-orange-500" :
"bg-blue-400"
                    }`} />
                  </div>
                </div>
                <div className="flex-1 min-w-0">
                  <div className="flex items-center justify-between">
                    <span className="text-[10px] font-black text-indigo-500 uppercase tracking-tighter">
                      {activity.bugId}</span>
                    <span className="text-[10px] font-bold text-slate-300 uppercase italic">
```

```
                    {new Date(activity.updatedAt).toLocaleTimeString([], { hour: '2-digit',
                    minute: '2-digit' })}
                  </span>
                </div>
                <p className="text-sm font-bold text-slate-700 mt-0.5 truncate group-hover:text-indigo-600
                    transition-colors"> {activity.phaseI_BugReport?.toolInfo?.bugDescription}
                </p>
                <div className="flex items-center gap-2 mt-1.5">
                  <span className="px-2 py-0.5 rounded-lg bg-slate-100 text-slate-500 text-[10px]
font-bold">
                    {activity.currentPhase}
                  </span>
                  <span className="text-[10px] font-bold text-slate-300">•</span>
                  <span className="text-[10px] font-bold text-slate-400 truncate max-w-[80px]">
                    {activity.phaseI_BugReport?.toolInfo?.toolName}
                  </span>
                </div>
              </div>
            </div>
          )) : ( <div className="flex flex-col items-center justify-center h-96 text-center text-slate-300">
                        <Activity size={48} className="mb-4 opacity-10" />
                    <p className="text-sm font-bold">No recent activities</p>
                    <p className="text-xs mt-1">Updates will appear as bugs are processed</p>
              </div>
          )}
        </div>
        <div className="p-5 bg-slate-50/50 border-t border-slate-100 rounded-b-[32px]">
           <Link to="/bugs" className="flex items-center justify-center gap-2 text-xs font-black
               text-indigo-600 hover:text-indigo-700 transition-colors uppercase tracking-widest">
             Explore Full Master List <ChevronRight size={14} />
           </Link>
        </div>
      </div>
    </div>
  </div>
 );
}
export default Dashboard;
```

Settings.jsx

```
import { useState } from "react";
import { useParams } from "react-router-dom";
import { Shield, Search, UserPlus, Wrench } from "lucide-react";
import UserManagment from "@/components/Settings/UserManagment";
import ToolManagment from "@/components/Settings/Tools/Tools";
import ReportControl from "@/components/Settings/Report";
import { useAuth } from "@/context/AuthContext";
import { Input } from "@/components/ui/input";
import { Button } from "@/components/ui/button";
function Settings() {
 const { user, allUsers } = useAuth();
 const { tab } = useParams();
 const activeTab = tab || "users";
 const [searchQuery, setSearchQuery] = useState("");
 const [isUserSheetOpen, setIsUserSheetOpen] = useState(false);
```

```
 const [isToolSheetOpen, setIsToolSheetOpen] = useState(false);
 const hasSettingsAccess = user?.role === "superadmin" || (Array.isArray(user?.adminControl)
       && user.adminControl.some(r => ["create", "edit", "view", "delete"].includes(r?.toLowerCase())));
 const canCreateUser = allUsers && (user?.role === "superadmin" || user?.adminControl?.includes("create"));
 if (!hasSettingsAccess) {
   return (
     <div className="flex flex-col items-center justify-center h-full text-center p-12">
       <Shield size={48} className="text-gray-200 mb-4" />
       <h2 className="text-xl font-bold text-gray-800">Access Denied</h2>
       <p className="text-gray-500">You don't have permission to access these settings.</p>
     </div>
   );
 }
 return (
   <div className="flex flex-col h-full bg-white rounded-2xl border border-gray-100 shadow-sm
overflow-hidden">
     {/* ── Header ── */}
     <div className="px-6 py-4 border-b border-gray-100 flex flex-col lg:flex-row lg:items-center
justify-between gap-4">
       <div>   <h1 className="text-xl font-bold text-gray-900 capitalize leading-tight">
           {activeTab === "reports" ? "System Reports" : `${activeTab} Management`}
         </h1>
         <p className="text-[10px] text-gray-400 uppercase font-bold tracking-wider mt-0.5">
           Settings / {activeTab}
         </p>
       </div>
       {activeTab === "users" && (
         <div className="flex items-center gap-3 w-full lg:w-auto">
           <div className="relative w-full lg:w-64">
             <Search className="absolute left-3 top-1/2 -translate-y-1/2 h-4 w-4 text-gray-400" />
             <Input placeholder="Search users..." onChange={(e) => setSearchQuery(e.target.value)}
             className="pl-9 h-9 border-gray-200 focus:border-cyan-500 focus:ring-cyan-50"
value={searchQuery}/>
           </div>
           {canCreateUser && (
             <Button onClick={() => setIsUserSheetOpen(true)}
               className="h-9 gap-2 bg-cyan-600 hover:bg-cyan-700 whitespace-nowrap">
               <UserPlus size={16} /> <span className="hidden sm:inline">Add User</span>
             </Button>
           )}
         </div>
       )}
       {activeTab === "tools" && canCreateUser && (
         <Button onClick={() => setIsToolSheetOpen(true)}
           className="h-9 gap-2 bg-indigo-600 hover:bg-indigo-700 whitespace-nowrap" >
           <Wrench size={16} />  <span className="hidden sm:inline">Add Tool</span>
         </Button>
       )}
     </div>
     {/* ── Content ── */}
     <div className="flex-1 overflow-auto p-6">
       {activeTab === "users" && (
         <UserManagment searchQuery={searchQuery} isSheetOpen={isUserSheetOpen}
```

```
            setIsSheetOpen={setIsUserSheetOpen}/>
        )}
        {activeTab === "tools" && <ToolManagment isOpen={isToolSheetOpen} setIsOpen={setIsToolSheetOpen} />}
        {activeTab === "reports" && <ReportControl />}
      </div>
    </div>
  );
}
export default Settings;
```

Util (folder)

bugRoleFilter.js

```
/*** Shared role-based bug filtering utilities.*/
function extractId(val) {
 if (!val) return null;
 if (typeof val === "string") return val; return val._id || val.id || null;
}
/*** Parse role flags from the logged-in user object.*/
export function getUserRoleFlags(user) {
 if (!user) return { isAdmin: false, isBugRaiser: false, isTester: false, isDev: false };
 const isAdmin = user.role === "admin" || user.role === "superadmin";
 const roleArr = Array.isArray(user.roletype) ? user.roletype: user.roletype? [user.roletype]: [];
 return { isAdmin, isBugRaiser: roleArr.includes("bugreporter"),isTester: roleArr.includes("tester"),
   isDev: roleArr.includes("dev"),};
}
/*** Returns the current user's string ID (tries _id then id).*/
export function getUserId(user) {
 if (!user) return null; return user._id || user.id || null;
}
/*** Returns true if the given bug was raised by userId.*/
export function isBugRaisedBy(bug, userId) {
 const reportedBy = bug.phaseI_BugReport?.reportedBy;return extractId(reportedBy) === userId;
}
/*** Returns true if the given bug has userId assigned as tester.*/
export function isBugAssignedToTester(bug, userId) {
 const tester = bug.phaseI_BugReport?.assignedTester;return extractId(tester) === userId;
}
/*** Returns true if the given bug has userId assigned as developer.*/
export function isBugAssignedToDev(bug, userId) {
 const dev = bug.phaseII_BugConfirmation?.assignedDeveloper; return extractId(dev) === userId;
}
/*** Returns true if the user is involved in the bug in any capacity*/
export function isUserInvolvedInBug(bug, user) {
 const { isAdmin } = getUserRoleFlags(user);
 if (isAdmin) return true;
 const uid = getUserId(user);
 if (!uid) return false;
 return ( isBugRaisedBy(bug, uid) || isBugAssignedToTester(bug, uid) || isBugAssignedToDev(bug, uid) );
}
/*** Filters a bug list to only bugs the user can SEE in the New Bug tab* (bugs they personally raised).
* Admins see all.*/
export function filterBugsForRaiser(bugs, user) {
 const { isAdmin } = getUserRoleFlags(user);
 if (isAdmin) return bugs;
```

```
 const uid = getUserId(user);
 if (!uid) return [];
 // Show bugs user raised OR bugs user is assigned to as tester
 return bugs.filter((b) => isBugRaisedBy(b, uid) || isBugAssignedToTester(b, uid));
}
/*** Filters a bug list to only bugs assigned to the logged-in tester.* Admins see all.*/
export function filterBugsForTester(bugs, user) {
 const { isAdmin } = getUserRoleFlags(user);
 if (isAdmin) return bugs;
 const uid = getUserId(user);
 if (!uid) return []; return bugs.filter((b) => isBugAssignedToTester(b, uid));
}
/*** Filters a bug list to only bugs assigned to the logged-in developer.* Admins see all.*/
export function filterBugsForDev(bugs, user) {
 const { isAdmin } = getUserRoleFlags(user);
 if (isAdmin) return bugs;
 const uid = getUserId(user);
 if (!uid) return []; return bugs.filter((b) => isBugAssignedToDev(b, uid));
}
```

Backend Codes :

.env (example)

```
DB_NAME=bugs
JWT_SECRET=your_super_secret_key_here
PORT=8000
MONGO_URI = 'mongodb+srv://Param_eswaran:xxxx%xxxx@cluster0.as4he.mongodb.net/bug_tracker'
# for email services use these credentials
MAIL_ID=rohit@xxxxxxx.in
MAIL_PASSWORD=maln xxx clal cnsi
# coludinay creditionals
Cloudinary_name=dmsl7x8mu
Cloudinary_API_Key=381561479814879
Cloudinary_API_Secret=WchBtGEv21EHntUPa6KBBzbKD9I
# create and use folder name Bug_Tracker
```

Tsconfig.json

```
{
 "compilerOptions": {
   "target": "es6", "module": "commonjs",  "moduleResolution": "node", "allowJs": true,
   "esModuleInterop": true, "strict": true, "skipLibCheck": true, "outDir": "./dist", "rootDir": "./src",
   "declaration": true, "declarationMap": true, "sourceMap": true,"lib": ["es2022", "DOM"],
   "resolveJsonModule": true, "isolatedModules": true, "noUnusedLocals": true, "noUnusedParameters": true,
   "noImplicitReturns": true, "noFallthroughCasesInSwitch": true
 },
 "include": ["src/**/*"], "exclude": ["node_modules", "dist"],
 "ts-node": { "transpileOnly": true, "files": true,"compilerOptions": { "module": "commonjs"}
 }
}
```

Package.json

```
{  "name": "@repo/backend",
 "scripts": { "dev": "nodemon --watch src --exec ts-node src/server.ts", "build": "tsc",
```

```
   "start": "node dist/server.js","test": "jest"
 },
 "dependencies": { "bcrypt": "^6.0.0", "cloudinary": "^2.9.0", "cookie-parser": "^1.4.7", "cors": "^2.8.5",
   "dotenv": "^16.5.0", "express": "^5.1.0", "express-rate-limit": "^8.4.1", "jsonwebtoken": "^9.0.2",
   "mongoose": "^8.16.0",  "multer": "^2.1.1",  "nodemailer": "^8.0.5", "winston": "^3.17.0"
 },
 "devDependencies": { "@types/bcrypt": "^5.0.2", "@types/cookie-parser": "^1.4.10",  "@types/cors":
"^2.8.19",
   "@types/express": "^5.0.3", "@types/jest": "^29.5.14", "@types/jsonwebtoken": "^9.0.10",
   "@types/multer": "^2.1.0", "@types/nodemailer": "^8.0.0", "@types/stack-trace": "^0.0.33",
   "@types/supertest": "^6.0.3", "jest": "^29.7.0", "root": "*", "supertest": "^6.3.4",
   "ts-jest": "^29.4.9", "ts-node": "^10.9.2", "typescript": "^5.8.3"
 }
}
```

jest.config.js (jest testing)

```
module.exports = { preset: 'ts-jest', testEnvironment: 'node', roots: ['<rootDir>/src/tests'],
 moduleFileExtensions: ['ts', 'js'], transform: { '^.+\\.ts$': 'ts-jest',}, forceExit: true,
};
```

/src (folder)
controllers (folder)

auth.controller.ts

```
import User from "../models/user.model";
import { asyncHandler } from "../utils/asyncHandler";
import bcrypt from "bcrypt";
import jwt from "jsonwebtoken";
import * as dotenv from "dotenv";
import { HttpStatusCodes } from "../utils/errorCodes";
dotenv.config();
const JWT_SECRET = process.env.JWT_SECRET || "secret";
export default class AuthController { // Method to register a new user with only email....
 static registerUser = asyncHandler(async (req, res): Promise<void> => {
   const { email, password } = req.body ?? {};
   if (!email || !password) {     // Validate required fields
     res.status(HttpStatusCodes.BAD_REQUEST).json({ message: "Email and password are required",});return;
   }
   // Check if user already exists
   const existingUser = await User.findOne({ email }).lean();
   if (existingUser) {
     res.status(HttpStatusCodes.BAD_REQUEST).json({ message: "Email already exists", }); return;
   }
   // Hash password and create new user
   const hashedPassword = await bcrypt.hash(password, 10);
   await new User({ email, password: hashedPassword, role: "user", resetPass: false, roletype: "bugreporter",
   }).save();
   res.status(HttpStatusCodes.CREATED).json({message: "User registered successfully", });
 });
 // Method to login user
 static loginUser = asyncHandler(async (req, res): Promise<void> => {
   const { username_email, password, keepMeSignedIn } = req.body || {};
   if (!username_email || !password) {
```

```
      res.status(HttpStatusCodes.BAD_REQUEST).json({ message: "Username/email and password are required" });
      return;
    }
    // Special case for initial superadmin creation/login
    if (username_email === "admin" && password === "password") {
      let user = await User.findOne({ role: "superadmin" }).lean();
      if (!user) {
        const hashedPassword = await bcrypt.hash(password, 10);
        const newUser = new User({
          username: username_email, password: hashedPassword, role: "superadmin", resetPass: false,
          roletype: ["admin", "dev", "tester", "bugreporter"],
          adminControl: ["create", "edit", "view", "delete"],
          adminOption: ["share", "generate_report", "insight_view", "export"],
        });
        const savedUser = await newUser.save();
        const expiresIn = keepMeSignedIn ? "7d" : "1d";
        const maxAge = keepMeSignedIn ? 7 * 24 * 60 * 60 * 1000 : 24 * 60 * 60 * 1000;
        const token = jwt.sign({ userId: savedUser._id }, JWT_SECRET, { expiresIn, });
        const userObj = savedUser.toObject();
        const { password: _, ...userWithoutPassword } = userObj;
        const isProduction = process.env.NODE_ENV === "production";
        res.cookie("bb_token", token, {  httpOnly: true,  secure: isProduction,
          sameSite: isProduction ? "none" : "lax", maxAge,  path: "/",
        });
        res.status(HttpStatusCodes.CREATED).json({
          message: "Superadmin created and logged in successfully",  user: userWithoutPassword,
        });return;
      }
    }
    // Check if the user exists
    const user = await User.findOne({ $or: [{ email: username_email }, { username: username_email }],
}).lean();
    if (!user) {  res.status(HttpStatusCodes.NOT_FOUND).json({ message: "User not found" }); return; }
    // Compare the password
    const isMatch = await bcrypt.compare(password, user.password);
    if (!isMatch) { res.status(HttpStatusCodes.BAD_REQUEST).json({  message: "Invalid credentials" }); return;
}
    const expiresIn = keepMeSignedIn ? "7d" : "1d";
    const maxAge = keepMeSignedIn ? 7 * 24 * 60 * 60 * 1000 : 24 * 60 * 60 * 1000;
    const token = jwt.sign({ userId: user._id }, JWT_SECRET, { expiresIn,}); // Generate JWT
    const isProduction = process.env.NODE_ENV === "production";
    res.cookie("bb_token", token, {  httpOnly: true,  secure: isProduction,
      sameSite: isProduction ? "none" : "lax", maxAge, path: "/",
    });
    const { password: _, ...userWithoutPassword } = user;
    res.status(HttpStatusCodes.OK).json({  message: "Login successful",  user: userWithoutPassword,});
  })
  // Method to logout user
  static logout = asyncHandler(async (_req, res): Promise<void> => {
    const isProduction = process.env.NODE_ENV === "production";
    res.clearCookie("bb_token", {  path: "/", sameSite: isProduction ? "none" : "lax",  secure:
isProduction,});
    res.status(HttpStatusCodes.OK).json({  message: "Logged out",});
```

```
 });
}
```

authExtend.controller.ts

```
import User from "../models/user.model";
import { asyncHandler } from "../utils/asyncHandler";
import bcrypt from "bcrypt";
import * as dotenv from "dotenv";
import { HttpStatusCodes } from "../utils/errorCodes";
dotenv.config();
export default class AuthExtendController {
 static Me = asyncHandler(async (req: any, res): Promise<void> => {  // Method to register a new user []
   // console.log("Param handle", req.user);
   if (!req.user?._id) {res.status(HttpStatusCodes.UNAUTHORIZED).json({  message: "Unauthorized",}); return;}
   const user = await User.findById(req.user._id).lean();
   if (!user) { res.status(HttpStatusCodes.NOT_FOUND).json({ message: "User not found",});return;}
   const { password, ...safeUser } = user;
   res.status(HttpStatusCodes.OK).json({ user: safeUser,});
 });
 static registerUserWithToken = asyncHandler(
   async (req, res): Promise<void> => {
     const userData = req.body;
     const {username = "", email = "", password, role = "", name, designation, profile, roletype,
       adminControl,adminOption,} = userData;
     if ((!email?.trim() && !username?.trim()) || !password?.trim()) {
       res.status(HttpStatusCodes.BAD_REQUEST).json({message: "Username/email and password are required",});
       return;
     } // Check for existing email/username
     const orConditions: any[] = [];
     if (email?.trim()) {orConditions.push({ email: email.trim() });}
     if (username?.trim()) { orConditions.push({ username: username.trim() });}
     const existingUser = await User.findOne({$or: orConditions,}).lean();
     if (existingUser) { let message = "User already exists";
       if (email && existingUser.email === email.trim() && username &&
        existingUser.username === username.trim()) { message = "Email and username already exist"; }
       else if (email && existingUser.email === email.trim()) {message = "Email already exists";}
       else if (username && existingUser.username === username.trim()) {message = "Username already
exists";}
       res.status(HttpStatusCodes.BAD_REQUEST).json({ message });return;
     }
     const hashedPassword = await bcrypt.hash(password, 10);
     const newUserObj: any = { password: hashedPassword, defaultPassword: password, role, resetPass: true,};
     if (username?.trim()) newUserObj.username = username.trim();
     if (email?.trim()) newUserObj.email = email.trim();
     if (roletype) newUserObj.roletype = roletype;
     if (name) newUserObj.name = name;
     if (designation) newUserObj.designation = designation;
     if (profile) newUserObj.profile = profile;
     if (Array.isArray(adminControl) && adminControl.length > 0) { newUserObj.adminControl = adminControl;}
     if (Array.isArray(adminOption) && adminOption.length > 0) { newUserObj.adminOption = adminOption;}
     const newUser = new User(newUserObj);
     const savedUser = await newUser.save();
     const userObj = savedUser.toObject();
```

```
    const { password: _, ...userWithoutPassword } = userObj;
    res.status(HttpStatusCodes.CREATED).json({message: "User registered successfully",
      user: userWithoutPassword});
  }
);
 static UpdateUserWithToken = asyncHandler( async (_req, _res): Promise<void> => {});
}
```

bug.controller.ts

```
import mongoose from "mongoose";
import { Request, Response } from "express";
import { AuthenticatedRequest } from "../middlewares/authMiddleware";
import { asyncHandler } from "../utils/asyncHandler";
import { HttpStatusCodes } from "../utils/errorCodes";
import BugModel from "../models/bug.models";
import { createNotification } from "../utils/notification";
import { uploadOnCloudinary } from "../utils/cloudinary";
import LogModel from "../models/logs.models";
import { logBugActivity, captureBugChanges } from "../utils/bugActivity";
export default class BugController {
 static BugRaise = asyncHandler( // PHASE I: CREATE / RAISE BUG(S)
   async (req: AuthenticatedRequest, res: Response): Promise<void> => {
     const payload = Array.isArray(req.body) ? req.body : [req.body];
     const results: any[] = [];
     const user = req.user; // Retrieved from Auth Middleware
     if (!user) {res.status(HttpStatusCodes.UNAUTHORIZED).json({ message: "Authentication required" });
return;}
     // Pre-fetch all potentially matching bugs to prevent N+1 Queries
     const toolIdsAndDescriptions = payload
       .filter((b) => b.toolInfo?.toolId && b.toolInfo?.bugDescription &&
           b.toolInfo?.toolName && b.toolInfo?.toolDescription && b.toolInfo?.priority &&
           b.toolInfo?.expectedResult && b.toolInfo?.actualResult
       ).map((b) => ({
             "phaseI_BugReport.toolInfo.toolId": b.toolInfo.toolId,
             "phaseI_BugReport.toolInfo.bugDescription": b.toolInfo.bugDescription, isActive: true,
       }));
     let existingBugs: any[] = [];
     if (toolIdsAndDescriptions.length > 0) { // Find existing matches to prevent duplicates
         existingBugs = await BugModel.find({ $or: toolIdsAndDescriptions }).lean();
     }
     // Check existence based on cached results (memory filtering vs repetitive DB trips)
     const isDuplicate = (toolId: string, bugDesc: string) => {
       return existingBugs.find(
         (eb) => eb.phaseI_BugReport.toolInfo.toolId.toString() === toolId &&
           eb.phaseI_BugReport.toolInfo.bugDescription === bugDesc
       );
     };
     for (const bugData of payload) {
       try {const { toolInfo, assignedTester } = bugData;  // 1. Validate required fields structurally
         const requiredInfo = ["toolId","toolName","toolDescription","priority","bugDescription",
           "expectedResult","actualResult",];
         const missingFields = requiredInfo.filter((field) => !toolInfo || !toolInfo[field]);
```

```
        if (missingFields.length > 0) {
          results.push({status: "error",
            message: `Missing required toolInfo fields: ${missingFields.join(", ")}`,data: bugData,
          });continue;
        }
        // 2. Check for Duplicates (from memory check, saving network round trips!)
        const existingMatch = isDuplicate(toolInfo.toolId,toolInfo.bugDescription);
        if (existingMatch) {
          results.push({status: "skipped",message: `Similar bug for '${toolInfo.toolName}' already exists`,
            bugId: existingMatch.bugId, mongoId: existingMatch._id,});continue;
        }
        // 3. Save Bug with secure assigned reporter (User token auto-assign)
        const newBug = new BugModel({
          phaseI_BugReport: {
            toolInfo: {
              toolId: toolInfo.toolId,toolName: toolInfo.toolName,
              toolDescription: toolInfo.toolDescription,  stack: toolInfo.stack || "",
              priority: toolInfo.priority, libraryName: toolInfo.libraryName || "",
              bugDescription: toolInfo.bugDescription, stepsToReproduce: toolInfo.stepsToReproduce || "",
              expectedResult: toolInfo.expectedResult,actualResult: toolInfo.actualResult,
              attachments: toolInfo.attachments?.map((att: any) => ({
               url:att.url,fileName:att.fileName || "",uploadedAt:att.uploadedAt || new Date(),})) || [], },
            reportedBy: user._id, // Assign to the logged-in user automatically
            assignedTester: assignedTester?.userId || undefined,clientContext: bugData.clientContext || {},
            reportedAt: new Date(),
          },
          bugPhaseNo: bugData.bugPhaseNo || 1,currentPhase: bugData.currentPhase || "Bug Reported",
          isActive: true,tags: bugData.tags || [],
        });
        // Await save sequentially to guarantee sequential custom ID creation (BUG-XXXX hook)
        const savedBug = await newBug.save();
        // Trigger notification for assigned tester
        if (assignedTester?.userId) {
          await createNotification({
            recipient: assignedTester.userId,sender: user._id as string,type: "assignment",
            title: "New Bug Assigned", link: `/bugs/${savedBug._id}`,
            message: `You have been assigned as a tester for bug: ${savedBug.bugId}`,
            });
        }
        // Log Bug Creation
        await logBugActivity(savedBug._id,user._id,"Bug Created",
          JSON.stringify({toolName: toolInfo.toolName,bugId: savedBug.bugId,
            stack: toolInfo.stack || "Not Specified",priority: toolInfo.priority,
            bugDescription: toolInfo.bugDescription,expectedResult: toolInfo.expectedResult,
            actualResult: toolInfo.actualResult,assignedTester: assignedTester?.name || "None",
            currentPhase: savedBug.currentPhase,reportedBy: user.name || user.email
          }),"creation");
        results.push({
          status: "created", message: "Bug raised successfully",bugId: savedBug.bugId,
          mongoId: savedBug._id, toolName: toolInfo.toolName, stack: toolInfo.stack || "Not Specified",
          priority: toolInfo.priority, bug: savedBug,
        });
      } catch (error: any) {
```

```
        results.push({status: "error",message: error.message,data: bugData,});
      }
    }
    const summary = {
      total: payload.length,created: results.filter((r) => r.status === "created").length,
      skipped: results.filter((r) => r.status === "skipped").length,
      failed: results.filter((r) => r.status === "error").length,
    };
    const statusCode =summary.failed === payload.length  ? HttpStatusCodes.BAD_REQUEST  :
HttpStatusCodes.OK;
    // Adjust to what Axios expects to map directly
    res.status(statusCode).json({ success: statusCode === HttpStatusCodes.OK,
      message: "Bug operation completed", summary, results,});
  }
);
//  PHASE II: BUG CONFIRMATION
static ConfirmBug = asyncHandler(
  async (req: Request, res: Response): Promise<void> => {
    const {id,testingInfo,testedBy,assignedDeveloper,bugPhaseNo,currentPhase,} = req.body;
    const bug = await BugController.findBugById(id);
    if (!bug) {res.status(HttpStatusCodes.NOT_FOUND).json({ message: "Bug not found",}); return; }
    // Validate required fields
    if (!testingInfo?.status) {res.status(HttpStatusCodes.BAD_REQUEST).json({ message: "status is required",
      }); return;}
    if (testingInfo.isConfirmed === undefined ||testingInfo.canReproduce === undefined)
      {res.status(HttpStatusCodes.BAD_REQUEST).json({
        message: "isConfirmed and canReproduce flags are required",});return;}
    // Update Phase II
    bug.phaseII_BugConfirmation = {
      testingInfo: {
        status: testingInfo.status,isConfirmed: testingInfo.isConfirmed,
        canReproduce: testingInfo.canReproduce,forwardedToDev: testingInfo.forwardedToDev || false,
        remarks: testingInfo.remarks || "", attachments:testingInfo.attachments?.map((att: any) => ({
            url: att.url,fileName: att.fileName || "", uploadedAt: att.uploadedAt || new Date(),})) || [],
        sopFollowed: testingInfo.sopFollowed || [],
      },
      testedBy: testedBy?.userId || undefined,
      assignedDeveloper: assignedDeveloper?.userId || undefined,testedAt: new Date(),
    };
    // Update phase and phase number from frontend
    if (bugPhaseNo !== undefined) {bug.bugPhaseNo = bugPhaseNo; }
    if (currentPhase) {bug.currentPhase = currentPhase;}
    const oldBug = bug.toObject();
    const updatedBug = await bug.save();
    // Log activity
    const changes = captureBugChanges(oldBug, updatedBug.toObject());
    if (changes.length > 0) {
      await logBugActivity(updatedBug._id,(req as any).user?._id,"Bug Confirmation Updated",
        JSON.stringify(changes));
    }
    if (assignedDeveloper?.userId) {  // Trigger notification for assigned developer
      await createNotification({
        recipient: assignedDeveloper.userId,sender: (req as any).user?._id as string,type: "assignment",
```

```
        title: "Bug Assigned for Development",link: `/bugs/${updatedBug._id}`,
        message: `You have been assigned as a developer for bug: ${updatedBug.bugId}`,});
    }
    res.status(HttpStatusCodes.OK).json({
      message: "Bug confirmation completed",bugId: updatedBug.bugId,currentPhase: updatedBug.currentPhase,
      result: updatedBug,});}
);
//  PHASE III: BUG ANALYSIS
static AnalyzeBug = asyncHandler(
  async (req: Request, res: Response): Promise<void> => {
    const { id, analysisInfo, analyzedBy, bugPhaseNo, currentPhase } =req.body;
    const bug = await BugController.findBugById(id);
    if (!bug) {res.status(HttpStatusCodes.NOT_FOUND).json({  message: "Bug not found",});return;}
    // Validate required fields
    if (!analysisInfo?.status) { res.status(HttpStatusCodes.BAD_REQUEST).json({
        message: "status is required",});return; }
    bug.phaseIII_BugAnalysis = { // Update Phase III
      analysisInfo: {status: analysisInfo.status,rootCause: analysisInfo.rootCause || "",
        estimatedEffort: analysisInfo.estimatedEffort || "",
        affectedModules: analysisInfo.affectedModules || [],remarks: analysisInfo.remarks || "",
        attachments:
          analysisInfo.attachments?.map((att: any) => ({
            url: att.url,fileName: att.fileName || "",uploadedAt: att.uploadedAt || new Date(),})) || [],
        sopProvided: analysisInfo.sopProvided || [],
      },
      analyzedBy: analyzedBy?.userId || undefined,analyzedAt: new Date(),
    };
    // Update phase and phase number from frontend
    if (bugPhaseNo !== undefined) { bug.bugPhaseNo = bugPhaseNo;}
    if (currentPhase) {bug.currentPhase = currentPhase;}
    const oldBug = bug.toObject();
    const updatedBug = await bug.save();
    // Log activity
    const changes = captureBugChanges(oldBug, updatedBug.toObject());
    if (changes.length > 0) {
      await logBugActivity(updatedBug._id,(req as any).user?._id,"Bug Analysis Updated",
        JSON.stringify(changes) );}
    res.status(HttpStatusCodes.OK).json({ message: "Bug analysis completed", bugId: updatedBug.bugId,
      currentPhase: updatedBug.currentPhase, result: updatedBug,});
  }
);
//  PHASE IV: MAINTENANCE (FIX BUG)
static FixBug = asyncHandler(
  async (req: Request, res: Response): Promise<void> => {
    const { id, maintenanceInfo, fixedBy, bugPhaseNo, currentPhase } =req.body;
    const bug = await BugController.findBugById(id);
    if (!bug) {res.status(HttpStatusCodes.NOT_FOUND).json({message: "Bug not found",});return;}
    // Validate required fields
    if (!maintenanceInfo?.status) {res.status(HttpStatusCodes.BAD_REQUEST).json({
        message: "status is required",});return;}
    // Update Phase IV
    bug.phaseIV_Maintenance = {
      maintenanceInfo: {status: maintenanceInfo.status,
```

```
        fixDescription: maintenanceInfo.fixDescription || "",codeChanges: maintenanceInfo.codeChanges || "",
        remarks: maintenanceInfo.remarks || "",
        attachments:
          maintenanceInfo.attachments?.map((att: any) => ({
            url: att.url, fileName: att.fileName || "", uploadedAt: att.uploadedAt || new Date(),})) || [],
        sopProvided: maintenanceInfo.sopProvided || [],
      },
      fixedBy: fixedBy?.userId || undefined,fixedAt: new Date(),
    };
    // Update phase and phase number from frontend
    if (bugPhaseNo !== undefined) { bug.bugPhaseNo = bugPhaseNo;}
    if (currentPhase) {bug.currentPhase = currentPhase;}
    const oldBug = bug.toObject();
    const updatedBug = await bug.save();
    // Log activity
    const changes = captureBugChanges(oldBug, updatedBug.toObject());
    if (changes.length > 0) {
      await logBugActivity(
        updatedBug._id,(req as any).user?._id,"Bug Maintenance Updated",JSON.stringify(changes));
    }
    res.status(HttpStatusCodes.OK).json({
      message: "Bug fix completed",bugId: updatedBug.bugId,currentPhase: updatedBug.currentPhase,
      result: updatedBug,});
  }
);
// PHASE V: FINAL TESTING
static FinalTestBug = asyncHandler(
  async (req: Request, res: Response): Promise<void> => {
    const {id, testingInfo, testedBy, approvedBy, bugPhaseNo, currentPhase,} = req.body;
    const bug = await BugController.findBugById(id);
    if (!bug) { res.status(HttpStatusCodes.NOT_FOUND).json({message: "Bug not found",});return;}
    // Validate required fields
    if (!testingInfo?.status) {
      res.status(HttpStatusCodes.BAD_REQUEST).json({ message: "status is required",});return;
    }
    if (testingInfo.isFixed === undefined) {res.status(HttpStatusCodes.BAD_REQUEST).json({
        message: "isFixed flag is required",});return;
    }
    // Update Phase V
    bug.phaseV_FinalTesting = {
      testingInfo: {
        status: testingInfo.status,isFixed: testingInfo.isFixed,
       regressionTested: testingInfo.regressionTested || false,remarks: testingInfo.remarks || "",
        attachments:
          testingInfo.attachments?.map((att: any) => ({url: att.url,fileName: att.fileName || "",
            uploadedAt: att.uploadedAt || new Date(),
          })) || [],
      },
      testedBy: testedBy?.userId || undefined,approvedBy: approvedBy?.userId || undefined,
      testedAt: new Date(),approvedAt: approvedBy ? new Date() : undefined,
    };
    // Update phase and phase number from frontend
    if (bugPhaseNo !== undefined) {bug.bugPhaseNo = bugPhaseNo;}
```

```
    if (currentPhase) {bug.currentPhase = currentPhase;}
    const oldBug = bug.toObject();
    const updatedBug = await bug.save();
    // Log activity
    const changes = captureBugChanges(oldBug, updatedBug.toObject());
    if (changes.length > 0) {
      await logBugActivity(updatedBug._id,(req as any).user?._id,"Final Testing Updated",
        JSON.stringify(changes));
    }
    res.status(HttpStatusCodes.OK).json({message: "Final testing completed",bugId: updatedBug.bugId,
      currentPhase: updatedBug.currentPhase,result: updatedBug,});
  }
);
// PHASE VI: DEPLOYMENT
static DeployBug = asyncHandler(
  async (req: Request, res: Response): Promise<void> => {
    const { id, deploymentInfo, deployedBy, bugPhaseNo, currentPhase } =req.body;
    const bug = await BugController.findBugById(id);
    if (!bug) {res.status(HttpStatusCodes.NOT_FOUND).json({ message: "Bug not found",}); return;}
    // Validate required fields
    if (!deploymentInfo?.environment) {
      res.status(HttpStatusCodes.BAD_REQUEST).json({message: "environment is required",}); return;
    }
    if (!deploymentInfo?.deploymentType) {
      res.status(HttpStatusCodes.BAD_REQUEST).json({message: "deploymentType is required",});return;
    }
    // Update Phase VI
    bug.phaseVI_Deployment = {
      deploymentInfo: {
        environment: deploymentInfo.environment,deploymentType: deploymentInfo.deploymentType,
        remarks: deploymentInfo.remarks || "", sopFollowed: deploymentInfo.sopFollowed || [],
      },
      deployedBy: deployedBy?.userId || undefined,deployedAt: new Date(),
    };
    // Update phase and phase number from frontend
    if (bugPhaseNo !== undefined) {bug.bugPhaseNo = bugPhaseNo;}
    if (currentPhase) {bug.currentPhase = currentPhase;}
    const oldBug = bug.toObject();
    const updatedBug = await bug.save();
    // Log activity
    const changes = captureBugChanges(oldBug, updatedBug.toObject());
    if (changes.length > 0) {
      await logBugActivity(updatedBug._id,(req as any).user?._id,"Bug Deployed",JSON.stringify(changes));
    }
    res.status(HttpStatusCodes.OK).json({message: "Bug fix deployed successfully",
      bugId: updatedBug.bugId,currentPhase: updatedBug.currentPhase,result: updatedBug,});
  }
);
// PHASE VII: CLOSE BUG
static CloseBug = asyncHandler(
  async (req: Request, res: Response): Promise<void> => {
    const { id, closureInfo, closedBy, bugPhaseNo, currentPhase } = req.body;
    const bug = await BugController.findBugById(id);
```

```
    if (!bug) {res.status(HttpStatusCodes.NOT_FOUND).json({ message: "Bug not found",}); return;}
    // Update Phase VII
    bug.phaseVII_Closure = {
      closureInfo: {resolutionSummary: closureInfo?.resolutionSummary || "",
        lessonsLearned: closureInfo?.lessonsLearned || "",remarks: closureInfo?.remarks || "",
      },closedBy: closedBy?.userId || undefined,closedAt: new Date(),
    };
    // Update phase and phase number from frontend
    if (bugPhaseNo !== undefined) {bug.bugPhaseNo = bugPhaseNo;}
    if (currentPhase) {bug.currentPhase = currentPhase;}
    bug.isActive = false;
    const oldBug = bug.toObject();
    const updatedBug = await bug.save();
    // Log activity
    const changes = captureBugChanges(oldBug, updatedBug.toObject());
    if (changes.length > 0) {
      await logBugActivity( updatedBug._id,(req as any).user?._id,"Bug Closed",JSON.stringify(changes) );
    }
    res.status(HttpStatusCodes.OK).json({ message: "Bug closed successfully", bugId: updatedBug.bugId,
      currentPhase: updatedBug.currentPhase, result: updatedBug,});
  }
);
//  GET BUG LIST (ALL OR FILTERED)
static GetBugs = asyncHandler(
  async (_req: Request, res: Response): Promise<void> => {
    const {currentPhase, toolId,toolName, stack, priority, isActive, reportedBy, bugId, tags,
    } = _req.query as any;
    const filter: any = {};
    if (currentPhase) {filter.currentPhase = currentPhase;}
    // Filter by tool ID
    if (toolId) {filter["phaseI_BugReport.toolInfo.toolId"] = toolId;}
    // Filter by tool name
    if (toolName) {filter["phaseI_BugReport.toolInfo.toolName"] = { $regex: toolName,$options: "i",};}
    if (stack) {filter["phaseI_BugReport.toolInfo.stack"] = stack;}
    if (priority) {filter["phaseI_BugReport.toolInfo.priority"] = priority;}
    if (isActive !== undefined) {filter.isActive = isActive === "true";}
    if (reportedBy) {filter["phaseI_BugReport.reportedBy"] = reportedBy; }
    if (bugId) {filter.bugId = {$regex: bugId,$options: "i",};}
    if (tags) {filter.tags = { $in: Array.isArray(tags) ? tags : [tags] };}
    const bugs = await BugModel.find(filter)
      .populate("phaseI_BugReport.reportedBy", "name email")
      .populate("phaseI_BugReport.assignedTester", "name email")
      .populate("phaseII_BugConfirmation.assignedDeveloper", "name email").sort({ createdAt: -1 }).lean();
    res.status(HttpStatusCodes.OK).json({message: "Bug list fetched successfully",count: bugs.length,
      filters: filter,results: bugs,
    });
  }
);
//  GET SINGLE BUG BY ID
static GetBugById = asyncHandler(
  async (req: Request, res: Response): Promise<void> => {
    const id = req.params.id || req.body.id || req.body._id;
    const bug = await BugController.findBugById(id);
```

```
    if (!bug) { res.status(HttpStatusCodes.NOT_FOUND).json({ message: "Bug not found",bugId: id,});return;}
    res.status(HttpStatusCodes.OK).json({message: "Bug fetched successfully", result: bug,});
  }
);
//  GET BUG ACTIVITY LOGS
static GetBugLogs = asyncHandler(
  async (req: Request, res: Response): Promise<void> => {
    const { id } = req.body;
    console.log("Fetching logs for ID:", id);
    let mongoId: any = null;
    if (mongoose.Types.ObjectId.isValid(id)) {mongoId = new mongoose.Types.ObjectId(id);} else {
      const bug = await BugModel.findOne({ bugId: id.toUpperCase() });
      if (bug) {mongoId = bug._id;}
    }
    if (!mongoId) {
      console.log("No valid bug found for ID:", id);
      res.status(HttpStatusCodes.OK).json({success: true,message: "No bug found for identifier",results: [],
      });
      return;
    }
    const logs = await LogModel.find({$or: [{ bugId: mongoId },{ bugId: mongoId.toString() }]})
      .populate("performedBy", "name email").sort({ timestamp: -1 }).lean();
    res.status(HttpStatusCodes.OK).json({success: true,message: "Bug logs fetched successfully",
      results: logs,});
  }
);
// GET BUG STATISTICS
static GetBugStatistics = asyncHandler(
  async (_req: Request, res: Response): Promise<void> => {
    const totalBugs = await BugModel.countDocuments();
    const activeBugs = await BugModel.countDocuments({ isActive: true });
    const closedBugs = await BugModel.countDocuments({ isActive: false });
    const phaseDistribution = await BugModel.aggregate([  // Count by phase
      {$group: {_id: "$currentPhase", count: { $sum: 1 },},},
    ]);
    const priorityDistribution = await BugModel.aggregate([ // Count by priority
      {$group: {_id: "$phaseI_BugReport.toolInfo.priority", count: { $sum: 1 },},},
    ]);
    const stackDistribution = await BugModel.aggregate([  // Count by stack
      {$group: {_id: "$phaseI_BugReport.toolInfo.stack",count: { $sum: 1 },},},
    ]);
    res.status(HttpStatusCodes.OK).json({
      message: "Bug statistics fetched successfully",statistics: {total: totalBugs,active: activeBugs,
        closed: closedBugs,byPhase: phaseDistribution,byPriority: priorityDistribution,
        byStack: stackDistribution,
      },
    });
  }
);
//UPDATE BUG DETAILS (Generic Update)
static UpdateBug = asyncHandler(
  async (req: AuthenticatedRequest, res: Response): Promise<void> => {
    const id = req.body.id || req.body._id;
```

```
    const updateData = req.body;
    const user = req.user;
    const bug = await BugController.findBugById(id);
    if (!bug) {res.status(HttpStatusCodes.NOT_FOUND).json({message: "Bug not found",bugId: id,});return;}
    // Track old assignments and phase to detect changes
    const oldPhase = bug.currentPhase;
    const oldTesterId = bug.phaseI_BugReport?.assignedTester?.toString();
    const oldDevId = bug.phaseII_BugConfirmation?.assignedDeveloper?.toString();
    // Update bug with new data
    const oldBug = bug.toObject();
    bug.set(updateData);
    const updatedBug = await bug.save();
    // Log activity
    const changes = captureBugChanges(oldBug, updatedBug.toObject());
    if (changes.length > 0) {
      await logBugActivity(updatedBug._id,user?._id,"Bug Updated",JSON.stringify(changes));
    }
    // Trigger notifications if assignments or phases changed
    const newPhase = updatedBug.currentPhase;
    const newTesterId = updatedBug.phaseI_BugReport?.assignedTester?.toString();
    const newDevId = updatedBug.phaseII_BugConfirmation?.assignedDeveloper?.toString();
    const raiserId = updatedBug.phaseI_BugReport?.bugRaiser?.id;
    const senderId = user?._id as string;
    // 1. Assignment Notifications (If IDs changed but phase might not have)
    if (newTesterId && newTesterId !== oldTesterId) {
      await createNotification({
        recipient: newTesterId,sender: senderId,type: "assignment",title: "Bug Assigned to You",
        message: `You have been assigned as a tester for bug ${updatedBug.bugId}`,
        link: `/bugs/${updatedBug._id}`,
      });
    }
    if (newDevId && newDevId !== oldDevId) {
      await createNotification({
        recipient: newDevId,sender: senderId,type: "assignment",title: "Development Assignment",
        message: `You have been assigned to analyze bug ${updatedBug.bugId}`,link:
`/bugs/${updatedBug._id}`,
      });
    }
    // 2. Phase Transition Notifications
    if (newPhase !== oldPhase) {
      // Bug Reported -> Bug Testing
      if (oldPhase === "Bug Reported" && newPhase === "Bug Testing" && newTesterId) {
        await createNotification({
          recipient: newTesterId,sender: senderId,type: "status_change",title: "New Testing Task",
          message: `Bug ${updatedBug.bugId} moved to Testing phase. Please verify.`,
          link: `/bugs/${updatedBug._id}`,
        });
      }
      // Bug Testing -> Bug Analysis
      if (oldPhase === "Bug Testing" && newPhase === "Bug Analysis" && newDevId) {
        await createNotification({
          recipient: newDevId,sender: senderId,type: "status_change",title: "Bug for Analysis",
          message: `Bug ${updatedBug.bugId} confirmed and moved to Analysis phase.`,
```

```
          link: `/bugs/${updatedBug._id}`,
        });
      }
      // Bug Analysis -> Ready to Test
      if (oldPhase === "Bug Analysis" && newPhase === "Ready to Test" && newTesterId) {
        await createNotification({
          recipient: newTesterId,sender: senderId,type: "status_change",title: "Ready for Testing",
          message: `Fix for bug ${updatedBug.bugId} is ready to be tested.`,link: `/bugs/${updatedBug._id}`,
        });
      }
      // Ready to Test -> Bug Analysis (Re-opened/Failed)
      if (oldPhase === "Ready to Test" && newPhase === "Bug Analysis" && newDevId) {
        await createNotification({
          recipient: newDevId,sender: senderId,type: "status_change",title: "Bug Re-Analysis",
          message: `Testing failed for bug ${updatedBug.bugId}. Please re-analyze.`,
          link: `/bugs/${updatedBug._id}`,
        });
      }
      // Ready to Test -> Ready to Deploy
      if (oldPhase === "Ready to Test" && newPhase === "Ready to Deploy" && newDevId) {
        await createNotification({recipient: newDevId,sender: senderId,type: "status_change",
          title: "Ready for Deployment",link: `/bugs/${updatedBug._id}`,
          message: `Bug ${updatedBug.bugId} passed testing and is ready for deploy.`,
        });
      }
      // Ready to Deploy -> Closure
      if (oldPhase === "Ready to Deploy" && newPhase === "Closure") {
        const notifyList = [raiserId, newTesterId, newDevId].filter(Boolean);
        for (const recipientId of notifyList) {
          await createNotification({
            recipient: recipientId as string,sender: senderId,type: "status_change", title: "Bug Resolved",
            message: `Bug ${updatedBug.bugId} has been successfully deployed and closed.`,
           link: `/bugs/${updatedBug._id}`,
          });
        }
      }
    }
    res.status(HttpStatusCodes.OK).json({message: "Bug updated successfully",result: updatedBug,});
  }
);
// DELETE BUG (SOFT DELETE)
static DeleteBug = asyncHandler(
  async (req: Request, res: Response): Promise<void> => {
    const id = req.body.id || req.body._id;
    const bug = await BugController.findBugById(id);
    if (!bug) {res.status(HttpStatusCodes.NOT_FOUND).json({ message: "Bug not found", bugId: id,});return;}
    bug.isActive = false; await bug.save();
    res.status(HttpStatusCodes.OK).json({ message: "Bug deleted (soft delete) successfully",
      bugId: bug.bugId,});
  }
);
// FILE UPLOAD (CLOUDINARY)
static UploadFiles = asyncHandler(
```

```
  async (req: Request, res: Response): Promise<void> => {
    const files = req.files as Express.Multer.File[];
    if (!files || files.length === 0) {res.status(HttpStatusCodes.BAD_REQUEST)
      .json({message: "No files provided",});return;
    }
    const uploadResults = [];
    for (const file of files) {
      const result = await uploadOnCloudinary(file.path);
      if (result) {
        uploadResults.push({url: result.secure_url,fileName: file.originalname, publicId: result.public_id,
          size: file.size,uploadedAt: new Date(),});
      }
    }
    if (uploadResults.length === 0) {res.status(HttpStatusCodes.INTERNAL_SERVER_ERROR).json({
        message: "Failed to upload files to Cloudinary",});return;
    }
    res.status(HttpStatusCodes.OK).json({ message: "Files uploaded successfully",results: uploadResults,});
  }
);
//  HELPER: FIND BUG BY ID OR BUGID
private static async findBugById(id: string) {
  const query = mongoose.Types.ObjectId.isValid(id) ? BugModel.findById(id)
    : BugModel.findOne({ bugId: id.toUpperCase() });
  return await query .populate("phaseI_BugReport.reportedBy", "name email")
    .populate("phaseI_BugReport.assignedTester", "name email")
    .populate("phaseII_BugConfirmation.assignedDeveloper", "name email")
    .populate("phaseIII_BugAnalysis.analyzedBy", "name email")
    .populate("phaseIV_Maintenance.fixedBy", "name email")
    .populate("phaseV_FinalTesting.testedBy", "name email")
    .populate("phaseV_FinalTesting.approvedBy", "name email");
}
}
```

credential.controller.ts

```
import User from "../models/user.model";
import { asyncHandler } from "../utils/asyncHandler";
import bcrypt from "bcrypt";
import jwt from "jsonwebtoken";
import * as dotenv from "dotenv";
import { HttpStatusCodes } from "../utils/errorCodes";
dotenv.config();
export default class CredentialController {
 static ChangePassword = asyncHandler(async (req: any, res): Promise<void> => {
  const { _id } = req.user;
  const { existingPassword, newPassword } = req.body ?? {};
    if (!existingPassword || !newPassword) { //1 Validate required fields
   res.status(HttpStatusCodes.BAD_REQUEST).json({ message: "Existing and new password are required",});
return;
  }
  // 2. Ensure new password is not same as old one
  if (existingPassword === newPassword) {
    res.status(HttpStatusCodes.BAD_REQUEST).json({
      message: "New password cannot be the same as the current password",
    });
```

```
    return;
  }
  const user = await User.findById(_id); // 3. Find user (no .lean() because we need to update)
  if (!user) {  res.status(HttpStatusCodes.NOT_FOUND).json({ message: "User not found" }); return; }

  // 4. Compare old password
  const isMatch = await bcrypt.compare(existingPassword, user.password);
  if (!isMatch) {
    res.status(HttpStatusCodes.BAD_REQUEST).json({  message: "Current password is incorrect",}); return;
  }
  // 5. Extra security check: enforce password policy
  if (newPassword.length < 8) {
    res.status(HttpStatusCodes.BAD_REQUEST).json({
      message: "New password must be at least 8 characters long",
    });
    return;
  }
  const hashedPassword = await bcrypt.hash(newPassword, 10);     // 6. Hash new password
    await User.updateOne( // 7. Update user password + unset defaultPassword
    { _id },
    {$set: { password: hashedPassword, resetPass: false },$unset: { defaultPassword: "" },}
  );
  res.status(HttpStatusCodes.OK).json({  message: "Password changed successfully",});
});

// POST /auth/reset-password
static ResetPassword = asyncHandler(async (req: any, res) => {
  const { resetToken, newPassword } = req.body;
  if (!resetToken || !newPassword) {
    res.status(400) .json({ message: "Reset token and new password required" }); return;
  }
  let payload: any;
  try { payload = jwt.verify(resetToken, process.env.JWT_SECRET!);}
  catch {  res.status(400).json({ message: "Invalid or expired reset token" });return;}
  const user = await User.findById(payload.userId);
  if (!user) {  res.status(404).json({ message: "User not found" }); return; }
  // Hash & update password
  user.password = await bcrypt.hash(newPassword, 10);user.resetPass = false;await user.save();
  res.json({ message: "Password reset successful" });
});
}
```

logs.controller.ts

```
import { Request, Response } from "express";
import { asyncHandler } from "../utils/asyncHandler";
import { HttpStatusCodes } from "../utils/errorCodes";
import Log from "../models/logs.models";
import mongoose from "mongoose";
class LogController {
 // Get timeline logs for a specific bug or tool
 static getTimeline = asyncHandler(async (req: Request, res: Response) => {
   const { bugId, toolId, limit = 50, skip = 0 } = req.body;
    if (!bugId && !toolId) {  // Validate that at least one ID is provided
```

```
      res.status(HttpStatusCodes.BAD_REQUEST).json({success: false,
        message: "Either bugId or toolId is required",}); return;
    }
    const filter: any = {}; // Build query filter
    if (bugId) filter.bugId = new mongoose.Types.ObjectId(bugId);
    if (toolId) filter.toolId = new mongoose.Types.ObjectId(toolId);
    // Fetch logs with population
    const logs = await Log.find(filter) .populate("performedBy", "name email")
      .populate("bugId", "title status").populate("toolId", "name").sort({ timestamp: -1 })
      .limit(Number(limit)).skip(Number(skip));
    // Get total count for pagination
    const totalCount = await Log.countDocuments(filter);
    res.status(HttpStatusCodes.OK).json({
      success: true,message: "Timeline fetched successfully",
      data: {logs, pagination: {total: totalCount,limit: Number(limit),skip: Number(skip),
          hasMore: totalCount > Number(skip) + logs.length,
        },
      },
    });
  });
  //  Delete timeline logs
  static deleteTimeline = asyncHandler(async (req: Request, res: Response) => {
    const { logIds, bugId, toolId, deleteAll } = req.body;
    // Option 1: Delete specific log entries by their IDs
    if (logIds && Array.isArray(logIds) && logIds.length > 0) {
      const result = await Log.deleteMany({_id:{$in:logIds.map((id) => new mongoose.Types.ObjectId(id))},});
      res.status(HttpStatusCodes.OK).json({ success: true,
        message: `${result.deletedCount} log(s) deleted successfully`,data:{deletedCount:result.deletedCount
},
      });return;
    }
    // Option 2: Delete all logs for a specific bug or tool
    if (deleteAll && (bugId || toolId)) {
      const filter: any = {};
      if (bugId) filter.bugId = new mongoose.Types.ObjectId(bugId);
      if (toolId) filter.toolId = new mongoose.Types.ObjectId(toolId);
      const result = await Log.deleteMany(filter);
      res.status(HttpStatusCodes.OK).json({ success: true, message: `All logs deleted successfully`,
        data: { deletedCount: result.deletedCount },});return;
    }
    // No valid deletion criteria provided
    res.status(HttpStatusCodes.BAD_REQUEST)
        .json({success: false,message: "Please provide logIds or set deleteAll with bugId/toolId",
    });
  });
}
export default new LogController();
```

notification.controller.ts

```
import { Response } from "express";
import { AuthenticatedRequest } from "../middlewares/authMiddleware";
import { asyncHandler } from "../utils/asyncHandler";
import { HttpStatusCodes } from "../utils/errorCodes";
import Notification from "../models/notification.models";
```

```
export default class NotificationController {
 // Get all notifications for the logged-in user
 static GetNotifications = asyncHandler(async (req: AuthenticatedRequest, res: Response) => {
   const userId = req.user?._id;
   if (!userId) {res.status(HttpStatusCodes.UNAUTHORIZED).json({ message: "Unauthorized" }); return;}
   const notifications = await Notification.find({ recipient: userId }).sort({ createdAt: -1 })
     .populate("sender", "name email photo");
   const unreadCount = await Notification.countDocuments({ recipient: userId, isRead: false });
   console.log(`Found ${notifications.length} notifications (${unreadCount} unread)`);
   res.status(HttpStatusCodes.OK).json({notifications,unreadCount,});
 });
 // Mark a specific notification as read
 static MarkAsRead = asyncHandler(async (req: AuthenticatedRequest, res: Response) => {
   const { id } = req.params;
   const userId = req.user?._id;
   if (!userId) { res.status(HttpStatusCodes.UNAUTHORIZED).json({ message: "Unauthorized" });return;}
   const notification = await Notification.findOneAndUpdate(
     { _id: id, recipient: userId },{ isRead: true },{ new: true }
   );
   if (!notification){res.status(HttpStatusCodes.NOT_FOUND)
       .json({ message: "Notification not found" }); return;}
   res.status(HttpStatusCodes.OK).json({ message: "Notification marked as read" });
 });
 // Mark all notifications as read
 static MarkAllAsRead = asyncHandler(async (req: AuthenticatedRequest, res: Response) => {
   const userId = req.user?._id;
   if (!userId) {res.status(HttpStatusCodes.UNAUTHORIZED).json({ message: "Unauthorized" });return;}
   await Notification.updateMany({ recipient: userId, isRead: false }, { isRead: true });
   res.status(HttpStatusCodes.OK).json({ message: "All notifications marked as read" });
 });
 // Delete a notification
 static DeleteNotification = asyncHandler(async (req: AuthenticatedRequest, res: Response) => {
   const { id } = req.params;
   const userId = req.user?._id;
   if (!userId) {  res.status(HttpStatusCodes.UNAUTHORIZED).json({ message: "Unauthorized" });return;}
   await Notification.findOneAndDelete({ _id: id, recipient: userId });
   res.status(HttpStatusCodes.OK).json({ message: "Notification deleted" });
 });
}
```

resetpass.controller.ts

```
import User from "../models/user.model";
import { asyncHandler } from "../utils/asyncHandler";
import { createHash } from "crypto";
import bcrypt from "bcrypt";
import jwt from "jsonwebtoken";
import * as dotenv from "dotenv";
import { HttpStatusCodes } from "../utils/errorCodes";
import { sendEmail } from "../utils/mail";
import { passwordResetOtpTemplate } from "../utils/emailTemplates";
dotenv.config();
export default class ResetPasswordController {
 static RequestReset = asyncHandler(async (req: any, res) => {
```

```
  const { username_email } = req.body; // email or username
  if (!username_email) {res.status(HttpStatusCodes.BAD_REQUEST)
       .json({ message: "Email or Username is required" }); return;
  }
  // Find user by email or username
  const user = await User.findOne({ $or: [{ email: username_email }, { username: username_email }],});
  if (!user) { res.status(HttpStatusCodes.NOT_FOUND).json({ message: "User not found" }); return;}
  // Decide where to send OTP
  let targetEmail = "rohit@ceoitbox.in"; //user.email;
  // Case 1: User has email → send to their email
  if (!targetEmail) {
    // Case 2: No email → try superadmin
    const superAdmin = await User.findOne({ role: "superadmin" }).lean();
    if (superAdmin?.email) { targetEmail = superAdmin.email;} else {
      // Case 3: No email available at all
      res.status(HttpStatusCodes.BAD_REQUEST).json({
        message: "No email found for OTP send. Please contact admin.",
      });
      return;
    }
  }
  // Generate and hash OTP
  const otp = Math.floor(100000 + Math.random() * 900000).toString();
  const hashedOtp = createHash("sha256").update(otp).digest("hex");
  // Save OTP & expiry in DB
  user.resetOtp = hashedOtp;
  user.resetOtpExpires = new Date(Date.now() + 1000 * 60 * 5); // 5 min
  await user.save();
  // Send email
  await sendEmail( targetEmail, "Your Password Reset OTP",`Your OTP is: ${otp}. It is valid for 5 minutes.`,
    passwordResetOtpTemplate(otp)
  );
  res.status(HttpStatusCodes.OK).json({ message: `OTP sent successfully to ${targetEmail}` });
});
// POST /auth/verify-otp
static VerifyOtp = asyncHandler(async (req: any, res) => {
  const { username_email, otp } = req.body; // can be email or username
  if (!username_email || !otp) {res.status(HttpStatusCodes.BAD_REQUEST)
       .json({ message: "Email/Username and OTP are required" });return;
  }
   const hashedOtp = createHash("sha256").update(otp).digest("hex");// Hash the OTP for comparison
  // Find user by email OR username and check OTP + expiry
  const user = await User.findOne({$or: [{ email: username_email }, { username: username_email }],
       resetOtp: hashedOtp,resetOtpExpires: { $gt: new Date() }, // must not be expired
  });
  if (!user) { res.status(HttpStatusCodes.BAD_REQUEST).json({ message: "Invalid or expired OTP" });return;}
  // Clear OTP once verified (prevent reuse)
  user.resetOtp = undefined;
  user.resetOtpExpires = undefined;
  await user.save();
  // Generate a short-lived reset token (10 min)
  const resetToken = jwt.sign( { userId: user._id },process.env.JWT_SECRET || "secret",{ expiresIn: "10m"}
);
```

```
    res.status(HttpStatusCodes.OK).json({ message: "OTP verified successfully", resetToken });
  });
  // POST /auth/reset-password
  static ResetPassword = asyncHandler(async (req: any, res) => {
    const { resetToken, newPassword } = req.body;
    if (!resetToken || !newPassword) {res.status(HttpStatusCodes.BAD_REQUEST)
        .json({ message: "Reset token and new password are required" });
      return;
    }
    let payload: any;
    try { payload = jwt.verify(resetToken, process.env.JWT_SECRET!);} catch {
      res.status(HttpStatusCodes.UNAUTHORIZED).json({ message: "Invalid or expired reset token" });
      return;
    }
    const user = await User.findById(payload.userId);
    if (!user) { res.status(HttpStatusCodes.NOT_FOUND).json({ message: "User not found" });
      return;
    }
    // Prevent using the old password again
    const isSamePassword = await bcrypt.compare(newPassword, user.password);
    if (isSamePassword) { res.status(HttpStatusCodes.BAD_REQUEST)
        .json({ message: "New password cannot be the same as the old password",}); return;
    }
    //  Hash the new password
    user.password = await bcrypt.hash(newPassword, 10);
    user.resetPass = false; // mark reset complete
    user.resetOtp = undefined; // cleanup
    user.resetOtpExpires = undefined;
    await user.save();
    res.status(HttpStatusCodes.OK).json({ message: "Password reset successful" }); return;
  });
}
```

stack.controller.ts

```
import { Request, Response } from "express";
import { asyncHandler } from "../utils/asyncHandler";
import { HttpStatusCodes } from "../utils/errorCodes";
import StackModel from "../models/stack.models";
export default class StackController {
  // Add or Update Tool Stacks
  static UpdateStacks = asyncHandler(
    async (req: Request, res: Response): Promise<void> => {
      const { stacksList } = req.body;
      if (!stacksList || !stacksList.length) {
        res.status(HttpStatusCodes.BAD_REQUEST).json({ message: "Stack list is required" });
        return;
      }
      // Ensure stacksList is always an array
      const stacksArray = Array.isArray(stacksList) ? stacksList : [stacksList];
      // Check if a stack document already exists
      let existingStack = await StackModel.findOne();
      if (existingStack) {
        // Update existing stacksList
        existingStack.stacksList = stacksArray;
```

```
      await existingStack.save(); res.status(HttpStatusCodes.OK).json({
        message: "Tool stacks updated successfully", stack: existingStack, });
    } else {
      // Create a new stack document
      const newStack = await StackModel.create({ stacksList: stacksArray });
      res.status(HttpStatusCodes.CREATED).json({ message: "Tool stacks added successfully",
        stack: newStack, });
    }
  }
);
// Get Global System Configuration (Stack List)
static GetSystemConfig = asyncHandler(
  async (_req: Request, res: Response): Promise<void> => {
    try { const config = await StackModel.findOne();
      if (!config) {res.status(HttpStatusCodes.OK).json({ stacksList: [] });return;}
      res.status(HttpStatusCodes.OK).json(config);
    } catch (error) {res.status(HttpStatusCodes.INTERNAL_SERVER_ERROR)
      .json({ message: "Failed to fetch system config", error });
    }
  }
);
}
```

tool.controller.ts

```
import mongoose from "mongoose";
import { Request, Response } from "express";
import { asyncHandler } from "../utils/asyncHandler";
import { HttpStatusCodes } from "../utils/errorCodes";
import Tool from "../models/tool.models";
interface ToolPayload { id?: string; _id?: string; toolName?: string; toolDescription?: string;
 testerId?: string; devId?: string; stack?: string[] | string; libraryName?: string;
 htmlVersion?: string; lastLibraryUpdate?: Date | string | null; lastHtmlUpdate?: Date | string | null;
 lastResolvedDev?: Date | string | null;lastResolvedTester?: Date | string | null;
 lastBugReport?: Date | string | null; SOP?: string; ReleaseNotes?: string; [key: string]: any;
}
type BatchStatus = "created" | "updated" | "skipped" | "error";
interface BatchResult {toolName?: string; status: BatchStatus; tool?: any; error?: any; details?: any;}
export default class ToolController {
 static UpdateTool = asyncHandler(
   async (req: Request, res: Response): Promise<void> => {
     // Normalize input: prefer body.tools, else the body itself (array or single)
     const body = req.body ?? {};
     const payloadCandidate = body.tools ?? body;
     const toolItems: ToolPayload[] = Array.isArray(payloadCandidate) ? payloadCandidate :
[payloadCandidate];
    if (!toolItems.length){res.status(HttpStatusCodes.BAD_REQUEST).json({ message: "No tool data provided"
});
       return;
     }
     // mergeStacks flag default true; set mergeStacks=false to replace stack arrays
     const mergeStacksFlag =  String(req.query.mergeStacks ?? "true") !== "false";
     // helper to sanitize ObjectId fields
     const safeObjectId = (id: any): string | null | undefined => {
```

```
    if (id === null || id === "") return null;
    if (id === undefined) return undefined;
    if (typeof id === "string" && mongoose.isValidObjectId(id)) return id;
    if ( id && typeof id === "object" && (id as any)._id) return (id as any)._id.toString();
    if ( id && typeof id === "object" && (id as any)._bsontype === "ObjectID")return id;
    return undefined;
  };
  const results: BatchResult[] = [];
  // Pre-fetch existing tools (by IDs present) to minimize DB roundtrips
  const idsToFetch = toolItems.map((t) => safeObjectId(t.id || t._id)).filter(Boolean) as string[];
  const existingTools = idsToFetch.length ? await Tool.find({ _id: { $in: idsToFetch } }).lean() : [];
  const existingMap = new Map<string, any>();
  existingTools.forEach((t) => existingMap.set(t._id.toString(), t));
  // Process sequentially for deterministic merging
  for (const item of toolItems) {
    const toolName = item?.toolName;
    if (!toolName || typeof toolName !== "string") {
      results.push({ status: "error", error: "toolName is required", details: item,}); continue;
    }
    const itemId = safeObjectId(item.id || item._id);
    try {
      const existing = itemId ? existingMap.get(itemId) ?? (await Tool.findById(itemId)) : null;
      // Build setOps for non-stack fields
      const setOps: Record<string, any> = {};
      if (typeof item.toolDescription !== "undefined")
        setOps.toolDescription = item.toolDescription;
      // sanitize testerId and devId
      const tester = safeObjectId(item.testerId);
      if (tester !== undefined) setOps.testerId = tester;
      const dev = safeObjectId(item.devId);
      if (dev !== undefined) setOps.devId = dev;
      if (typeof item.libraryName !== "undefined")
        setOps.libraryName = item.libraryName;
      if (typeof item.htmlVersion !== "undefined")
        setOps.htmlVersion = item.htmlVersion;
      if (typeof item.lastLibraryUpdate !== "undefined")
        setOps.lastLibraryUpdate = item.lastLibraryUpdate;
      if (typeof item.lastHtmlUpdate !== "undefined")
        setOps.lastHtmlUpdate = item.lastHtmlUpdate;
      if (typeof item.lastResolvedDev !== "undefined")
        setOps.lastResolvedDev = item.lastResolvedDev;
      if (typeof item.lastResolvedTester !== "undefined")
        setOps.lastResolvedTester = item.lastResolvedTester;
      if (typeof item.lastBugReport !== "undefined")
        setOps.lastBugReport = item.lastBugReport;
      if (typeof item.SOP !== "undefined") setOps.SOP = item.SOP;
      if (typeof item.ReleaseNotes !== "undefined")
        setOps.ReleaseNotes = item.ReleaseNotes;
      // Normalize incoming stack to an array of trimmed strings if provided
      let incomingStack: string[] | undefined;
      if (typeof item.stack !== "undefined") {
        incomingStack = Array.isArray(item.stack) ? item.stack.slice() : [item.stack];
        incomingStack = incomingStack .map((s) => (typeof s === "string" ? s.trim() : s))
```

```
        .filter(Boolean) as string[];
      incomingStack = Array.from(new Set(incomingStack)); // dedupe
    }

    if (existing) {
      // Build updateOps combining $set for non-stack fields and either $addToSet or $set for stack
      const existingStackRaw = Array.isArray(existing.stack)? existing.stack: [];
      const existingStackNormalized = existingStackRaw
        .map((s: any) => (typeof s === "string" ? s.trim() : s)) .filter(Boolean) as string[];
      // create a case-insensitive set for quick checking (lowercased)
      const existingSetLower = new Set( existingStackNormalized.map((s) => s.toLowerCase()) );
      // Normalize incomingStack already computed earlier as `incomingStack`
      const incomingStackNormalized = Array.isArray(incomingStack) ? incomingStack : [];
      // For merge mode: only add items that are not already present (case-insensitive)
      let itemsToAdd: string[] = [];
      if (mergeStacksFlag && incomingStackNormalized.length) {
        itemsToAdd = incomingStackNormalized.filter( (s) => !existingSetLower.has(s.toLowerCase()));
      }
      // Build updateOps
      const updateOps: Record<string, any> = {};
      if (Object.keys(setOps).length) updateOps.$set = setOps;

      if (mergeStacksFlag) {
        if (itemsToAdd.length) {updateOps.$addToSet = { stack: { $each: itemsToAdd } }; }
      } else {
        // replace mode: always set the stack to the incoming normalized array
        updateOps.$set = { ...(updateOps.$set || {}),    stack: incomingStackNormalized, };
      }
      if (Object.keys(updateOps).length === 0) {
        results.push({ toolName, status: "skipped", tool: existing });   // nothing to change
        continue;
      }
      const updatedDoc = await Tool.findByIdAndUpdate(
        itemId, updateOps, { new: true, runValidators: true,  }
      ).lean();

      if (updatedDoc) existingMap.set(itemId as string, updatedDoc);
      results.push({ toolName, status: "updated", tool: updatedDoc });
    } else {
      // Create new tool payload (schema defaults will fill missing fields)
      const createPayload: any = { toolName };
      if (typeof item.toolDescription !== "undefined")
        createPayload.toolDescription = item.toolDescription;
      if (tester) createPayload.testerId = tester;
      if (dev) createPayload.devId = dev;
      if (typeof item.libraryName !== "undefined")
        createPayload.libraryName = item.libraryName;
      if (typeof item.htmlVersion !== "undefined")
        createPayload.htmlVersion = item.htmlVersion;
      if (typeof item.lastLibraryUpdate !== "undefined")
        createPayload.lastLibraryUpdate = item.lastLibraryUpdate;
      if (typeof item.lastHtmlUpdate !== "undefined")
        createPayload.lastHtmlUpdate = item.lastHtmlUpdate;
```

```
          if (typeof item.lastResolvedDev !== "undefined")
            createPayload.lastResolvedDev = item.lastResolvedDev;
          if (typeof item.lastResolvedTester !== "undefined")
            createPayload.lastResolvedTester = item.lastResolvedTester;
          if (typeof item.lastBugReport !== "undefined")
            createPayload.lastBugReport = item.lastBugReport;
          if (typeof item.SOP !== "undefined") createPayload.SOP = item.SOP;
          if (typeof item.ReleaseNotes !== "undefined")
            createPayload.ReleaseNotes = item.ReleaseNotes;
          if (incomingStack) createPayload.stack = incomingStack;
          const newDoc = new Tool(createPayload);
          const saved = await newDoc.save();
          existingMap.set((saved as any)._id.toString(), saved);
          results.push({ toolName, status: "created", tool: saved });
        }
      } catch (err: any) {
        if (err && err.code === 11000) {
          results.push({toolName, status: "error", error: "Duplicate key error", details: err.keyValue, });
        } else {
          results.push({toolName, status: "error", error: err?.message ?? err,  details: err, });
        }
      }
    }
    // Build summary
    const summary = results.reduce(
      (acc, r) => {
        acc.total += 1;
        if (r.status === "created") acc.created += 1;
        if (r.status === "updated") acc.updated += 1;
        if (r.status === "error") acc.errors += 1;
        return acc;
      },
      { total: 0, created: 0, updated: 0, errors: 0 }
    );
    res.status(HttpStatusCodes.OK).json({ message: "Batch tool operation completed", summary, results, });
  }
);
// Get Tool_list
static GetTools = asyncHandler(
  async (_: Request, res: Response): Promise<void> => {
    try {
      const tools = await Tool.find() .populate("testerId", "name email").populate("devId", "name email");
      res.status(HttpStatusCodes.OK).json(tools);
    } catch (error) {
      res.status(HttpStatusCodes.INTERNAL_SERVER_ERROR).json({ message: "Error fetching tool_list", error
});
    }
  }
);
}
```

user.controller.ts

```
import { asyncHandler } from "../utils/asyncHandler";
import { HttpStatusCodes } from "../utils/errorCodes";
import User from "../models/user.model";
```

```
import bcrypt from "bcrypt";
export default class UserController {
 // Method to get user data with jwt token
 static UpdateUserWithToken = asyncHandler( async (_req, _res): Promise<void> => {} );
 static getUserDataWithToken = asyncHandler(
   async (req, res): Promise<void> => {
     console.log("User Data:", req);  res.status(HttpStatusCodes.OK).send(req.user);
   }
 );
 static changePassword = asyncHandler(
   async (req, res): Promise<void> => {
     const { id } = req.params;
     const { oldPassword, newPassword } = req.body;
     if (!oldPassword || !newPassword) {
       res.status(HttpStatusCodes.BAD_REQUEST)
       .json({ success: false, message: "Old password and new password are required" });
       return;
     }
     if (newPassword.length < 6) {
       res.status(HttpStatusCodes.BAD_REQUEST)
       .json({ success: false, message: "New password must be at least 6 characters" });
       return;
     }
     // Fetch user WITH password field
     const user = await User.findById(id).select("+password");
     if (!user) { res.status(HttpStatusCodes.NOT_FOUND).json({ success: false, message: "User not found" });
       return;
     }
     // Verify old password
     const isMatch = await bcrypt.compare(oldPassword, user.password);
     if (!isMatch) {
       res.status(HttpStatusCodes.BAD_REQUEST).json({ success: false, message: "Old password is incorrect"
});
       return;
     }
     // Hash and save new password
     const hashed = await bcrypt.hash(newPassword, 10); user.password = hashed;
     await user.save();
     res.status(HttpStatusCodes.OK).json({ success: true, message: "Password changed successfully" });
   }
 );
 static getAllUsers = asyncHandler(
   async (_req, res): Promise<void> => {
     const users = await User.find().select("-password").lean();
     res.status(HttpStatusCodes.OK).json({ success: true, data: users });
   }
 );
 static updateUser = asyncHandler(
   async (req, res): Promise<void> => {
     const { id } = req.params;
     const updatingData = req.body;
     const updatedUser =
```

```
        await User.findByIdAndUpdate(id, updatingData, { new: true, runValidators: true
}).select("-password");
     if (!updatedUser) {
       res.status(HttpStatusCodes.NOT_FOUND).json({ success: false, message: "User not found" });
       return;
     }
     res.status(HttpStatusCodes.OK)
        .json({ success: true, data: updatedUser, message: "User updated successfully" });
   }
 );
 static deleteUser = asyncHandler(
   async (req, res): Promise<void> => {
     const { id } = req.params;
     const userToDrop = await User.findById(id);
    if (!userToDrop) {
       res.status(HttpStatusCodes.NOT_FOUND).json({ success: false, message: "User not found" });
       return;
     }
    if (userToDrop.role === "superadmin") {
    res.status(HttpStatusCodes.BAD_REQUEST).json({ success: false, message: "Superadmin cannot be deleted"
});
       return;
     }
     await User.findByIdAndDelete(id);
     res.status(HttpStatusCodes.OK).json({ success: true, message: "User deleted successfully" });
   }
 );
}
```

middlewares (folder)
authMiddleware.ts

```
import { Request, Response, NextFunction } from "express";
import jwt from "jsonwebtoken";
import User from "../models/user.model";// Extend the request to include the `user` property
export interface AuthenticatedRequest extends Request {user?: any;}
export const authenticateToken = async ( req: AuthenticatedRequest, res: Response, next: NextFunction)
: Promise<void> => {
 const SECRET_KEY = process.env.JWT_SECRET || "secret";
 // const token = authHeader && authHeader.split(" ")[1];
 const token = req.cookies?.bb_token;
 console.log("Token from cookie:", token); // Debugging line to check the token from cookies
 if (!token) { res.status(401).json({ message: "Access token is missing or invalid" });return; }
 try {
   const decoded = jwt.verify(token, SECRET_KEY) as { userId: string };
   const user = await User.findById(decoded.userId, {
     password: 0, googleRefreshToken: 0, isApproved: 0,
   }).lean();
   console.log("user", user);
   if (!user) { res.status(403).json({ message: "User not found" }); return; }
   req.user = user; // Attach decoded token to request
   next(); // Proceed to the next middleware or route
 } catch (error) { res.status(403).json({ message: "Invalid or expired token" }); }
};
```

multer.ts

```
import multer from "multer";
import path from "path";
import fs from "fs";
// Ensure public/temp directory exists
const tempDir = path.join(process.cwd(), "public", "temp");
if (!fs.existsSync(tempDir)) { fs.mkdirSync(tempDir, { recursive: true });}
const storage = multer.diskStorage({
 destination: function (req, file, cb) { cb(null, tempDir);},
 filename: function (req, file, cb) {
   const uniqueSuffix = Date.now() + "-" + Math.round(Math.random() * 1e9);
   cb(null, file.fieldname + "-" + uniqueSuffix + path.extname(file.originalname));
 },
});
export const upload = multer({   storage,limits: { fileSize: 50 * 1024 * 1024 // 50MB limit}});
```

rateLimiter.ts

```
import rateLimit from "express-rate-limit";
export const authRateLimiter = rateLimit({
 windowMs: 15 * 60 * 1000, // 15 minutes; max: 5, // Limit each IP to 5 requests per windowMs
 message: {message: "Too many login attempts, please try again after 15 minutes",},
 standardHeaders: true,legacyHeaders: false,
});
```

Models (folder)

bug.models.ts

```
import { Schema, Document, model, Types } from "mongoose";
// INTERFACES
interface IAttachment {url: string; fileName?: string; uploadedAt?: Date;}
interface IPhaseI_BugReport {
 toolInfo: {
   toolId: Types.ObjectId; toolName: string; toolDescription: string; stack?: string; priority: string;
   libraryName?: string; bugDescription: string; stepsToReproduce?: string; expectedResult: string;
   actualResult: string; attachments?: IAttachment[];
 };
 clientContext?: {
   facedByMe: boolean; facedByClient: boolean; clientName?: string; companyName?: string;
   sopChecklist?: Record<string, boolean>;
 };
 reportedBy: Types.ObjectId; assignedTester?: Types.ObjectId; reportedAt: Date;
}
interface IPhaseII_BugConfirmation {
 testingInfo: {
   status: string; isConfirmed: boolean; canReproduce: boolean; forwardedToDev: boolean;
   remarks?: string; attachments?: IAttachment[];sopFollowed?: string[];
 };
 testedBy?: Types.ObjectId; assignedDeveloper?: Types.ObjectId; testedAt?: Date;
}

interface IPhaseIII_BugAnalysis {
 analysisInfo: { status: string;  estimatedEffort?: string; delayedReason?: string;
  affectedModules?: string[]; remarks?: string; attachments?: IAttachment[]; sopProvided?: string[];
```

```
 };
 analyzedBy?: Types.ObjectId;  analyzedAt?: Date;
}
interface IPhaseIV_Maintenance {
 maintenanceInfo: {
   status: string; fixDescription?: string; codeChanges?: string; remarks?: string; testingFlag?: string;
   attachments?: IAttachment[]; sopProvided?: string[];
 };
 fixedBy?: Types.ObjectId; fixedAt?: Date;
}
interface IPhaseV_FinalTesting {
 testingInfo: {
   status: string; isFixed: boolean;regressionTested: boolean; remarks?: string; deploymentStatus?: string;
   deployRemark?: string;  finalSop?: string; attachments?: IAttachment[];
 };
 testedBy?: Types.ObjectId; approvedBy?: Types.ObjectId; testedAt?: Date;approvedAt?: Date;
}
interface IPhaseVI_Deployment {
 deploymentInfo: {environment: string; deploymentType: string; remarks?: string; sopFollowed?: string[];};
 deployedBy?: Types.ObjectId;deployedAt?: Date;
}
interface IPhaseVII_Closure {
 closureInfo: { resolutionSummary?: string; lessonsLearned?: string; remarks?: string;};
 closedBy?: Types.ObjectId; closedAt?: Date;
}
// MAIN INTERFACE
export interface IBug extends Document {
 bugId: string; currentPhase: string; bugPhaseNo: number;
 phaseI_BugReport: IPhaseI_BugReport;
 phaseII_BugConfirmation?: IPhaseII_BugConfirmation;
 phaseIII_BugAnalysis?: IPhaseIII_BugAnalysis;
 phaseIV_Maintenance?: IPhaseIV_Maintenance;
 phaseV_FinalTesting?: IPhaseV_FinalTesting;
 phaseVI_Deployment?: IPhaseVI_Deployment;
 phaseVII_Closure?: IPhaseVII_Closure;
 isActive: boolean; tags?: string[];createdAt: Date; updatedAt: Date;
}
// SUB-SCHEMAS
const attachmentSchema = new Schema<IAttachment>(
 {url: { type: String, required: true }, fileName: String, uploadedAt: { type: Date, default: Date.now },},
 { _id: false }
);
// MAIN SCHEMA
const bugSchema = new Schema<IBug>(
 {
   bugId: {type: String, unique: true, index: true, },
   currentPhase: {type: String,  required: true, index: true,}, bugPhaseNo: {type: Number,default: 1,},
   // Phase I: Bug Report (Required)
   phaseI_BugReport: {
     type: {
       toolInfo: {
         toolId: { type: Schema.Types.ObjectId, ref: "Tool", required: true },
         toolName: { type: String, required: true }, stack: String,
```

```
      toolDescription: { type: String, required: true },
      descriptionType: { type: String, enum: ["simple", "detailed"],default: "simple", },
      priority: { type: String, required: true },
      libraryName: String,bugDescription: { type: String, required: true },stepsToReproduce: String,
      expectedResult: { type: String, required: true },actualResult: { type: String, required: true },
      attachments: [attachmentSchema],
    },
    clientContext: {
      facedByMe: { type: Boolean, default: false }, facedByClient: { type: Boolean, default: false },
      clientName: { type: String, default: "" }, companyName: { type: String, default: "" },
      sopChecklist: { type: Schema.Types.Mixed, default: {} },
    },
    reportedBy: {  type: Schema.Types.ObjectId, ref: "user", required: true,},
    assignedTester: { type: Schema.Types.ObjectId, ref: "user" },
    reportedAt: { type: Date, default: Date.now, required: true },
  },
  required: true,
},
// Phase II: Bug Confirmation
phaseII_BugConfirmation: {
  testingInfo: {
    status: String, isConfirmed: Boolean, canReproduce: Boolean,forwardedToDev: Boolean,
    remarks: String, attachments: [attachmentSchema], sopFollowed: [String],
  },
  testedBy: { type: Schema.Types.ObjectId, ref: "user" },
  assignedDeveloper: { type: Schema.Types.ObjectId, ref: "user" }, testedAt: Date,
},
// Phase III: Bug Analysis
phaseIII_BugAnalysis: {
  analysisInfo: {
    status: String, estimatedEffort: String, delayedReason: String, affectedModules: [String],
    remarks: String,attachments: [attachmentSchema],sopProvided: [String],
  },
  analyzedBy: { type: Schema.Types.ObjectId, ref: "user" }, analyzedAt: Date,
},
// Phase IV: Maintenance
phaseIV_Maintenance: {
  maintenanceInfo: {status: String,fixDescription: String, codeChanges: String, remarks: String,
    testingFlag: String,attachments: [attachmentSchema], sopProvided: [String],
  },
  fixedBy: { type: Schema.Types.ObjectId, ref: "user" },fixedAt: Date,
},
// Phase V: Final Testing
phaseV_FinalTesting: {
  testingInfo: {
    status: String,isFixed: Boolean,regressionTested: Boolean,remarks: String, deploymentStatus: String,
    deployRemark: String, finalSop: String, attachments: [attachmentSchema],},
  testedBy: { type: Schema.Types.ObjectId, ref: "user" },
  approvedBy: { type: Schema.Types.ObjectId, ref: "user" },
  testedAt: Date,approvedAt: Date,
},
// Phase VI: Deployment
phaseVI_Deployment: {
```

```
    deploymentInfo: { environment: String,deploymentType: String, remarks: String,sopFollowed: [String],},
    deployedBy: { type: Schema.Types.ObjectId, ref: "user" }, deployedAt: Date,
  },
  // Phase VII: Closure
  phaseVII_Closure: {
    closureInfo: { resolutionSummary: String,lessonsLearned: String, remarks: String,},
    closedBy: { type: Schema.Types.ObjectId, ref: "user" }, closedAt: Date,
  },
  isActive: { type: Boolean, default: true, index: true,}, tags: [String],// Metadata
 },
 { timestamps: true,collection: "bugs",}
);
// INDEXES
bugSchema.index({ "phaseI_BugReport.toolInfo.toolId": 1 });
bugSchema.index({ "phaseI_BugReport.reportedBy": 1 });
bugSchema.index({ "phaseI_BugReport.toolInfo.priority": 1 });
bugSchema.index({ currentPhase: 1, isActive: 1 });
bugSchema.index({ createdAt: -1 });
// METHODS
bugSchema.methods.moveToPhase = function (phase: string, phaseNo?: number) {
 this.currentPhase = phase;
 if (phaseNo !== undefined) {this.bugPhaseNo = phaseNo;} return this.save();
};
// STATIC METHODS
bugSchema.statics.generateBugId = async function (): Promise<string> {
 const count = await this.countDocuments({ bugId: new RegExp(`^BUG-`), });
 return `BUG-${String(count + 1).padStart(4, "0")}`;
};
// PRE-SAVE HOOK
bugSchema.pre("save", async function (next) {
 if (this.isNew && !this.bugId) {   this.bugId = await (this.constructor as any).generateBugId(); } next();
});
// MODEL EXPORT
const BugModel = model<IBug>("bug", bugSchema);
export default BugModel;
```

logs.models.ts

```
import { Schema, Document, model, Types } from "mongoose";
// Define an interface for the Bug Log document
interface LogInterface extends Document {
 type: String; bugId: Types.ObjectId; // reference to Bug model
 toolId: Types.ObjectId; // reference to Bug model
 performedBy?: Types.ObjectId; // reference to User model
 action: string; details?: string; timestamp?: Date;
}
// Define the schema
const LogSchema = new Schema<LogInterface>(
 {
   type: { type: String, required: true, trim: true },
   bugId: { type: Schema.Types.ObjectId, ref: "bug", required: false },
   toolId: { type: Schema.Types.ObjectId, ref: "tool", required: false },
   performedBy: { type: Schema.Types.ObjectId, ref: "user" },
   action: { type: String, required: false, trim: false },
```

```
   details: { type: String, trim: true },
 },
 { timestamps: { createdAt: "timestamp", updatedAt: false } }
);
// Define and export the model
const LogModel = model<LogInterface>("log", LogSchema);
export default LogModel;
export type { LogInterface };
```

notification.models.ts

```
import { Schema, Document, model, Types } from "mongoose";
export interface INotification extends Document {
 recipient: Types.ObjectId; sender: Types.ObjectId;
 type: "assignment" | "status_change" | "comment" | "other";
 title: string; message: string; link?: string; // e.g., link to the bug
 isRead: boolean; createdAt: Date; updatedAt: Date;
}
const notificationSchema = new Schema<INotification>(
 {
   recipient: { type: Schema.Types.ObjectId, ref: "user", required: true, index: true },
   sender: { type: Schema.Types.ObjectId, ref: "user", required: true },
   type: {type: String, enum: ["assignment", "status_change", "comment", "other"],default: "other",  },
   title: { type: String, required: true },   message: { type: String, required: true },
   link: { type: String },   isRead: { type: Boolean, default: false, index: true },
 },
 { timestamps: true }
);
const Notification = model<INotification>("notification", notificationSchema);
export default Notification;
```

stack.models.ts

```
import { Schema, Document, model } from "mongoose";
// Define an interface for the Tool document
interface StackInterface extends Document {stacksList: string[];}
// Define the schema
const stackSchema: Schema<StackInterface> = new Schema(
 {   stacksList: { type: [String], required: true, unique: true, trim: true }, },
 { timestamps: true }
);
// Define and export the model
const StackModel = model<StackInterface>("stack", stackSchema);
export default StackModel;
export type { StackInterface };
```

tool.models.ts

```
import { Schema, Document, model } from "mongoose";
// Define an interface for the Tool document
interface ToolInterface extends Document {
 toolName: string; toolDescription: string; testerId?: string; devId?: string; stack: string[];
 libraryName: string; htmlVersion: string; lastLibraryUpdate?: Date; lastHtmlUpdate?: Date;
 lastResolvedDev?: Date; lastResolvedTester?: Date; lastBugReport?: Date; SOP: string;
 ReleaseNotes: string; defaultStack: string;
```

```
}
// Define the schema
const toolSchema: Schema<ToolInterface> = new Schema(
 {
   toolName: { type: String, required: true, trim: true },
   toolDescription: { type: String, required: false, default: "" },
   testerId: {type: Schema.Types.ObjectId, ref: "user", required: false,  default: null, },
   devId: {type: Schema.Types.ObjectId, ref: "user", required: false, default: null, },
   stack: { type: [String], required: false, default: [] },
   libraryName: { type: String, required: false, default: "" },
   htmlVersion: { type: String, required: false, default: "" },
   lastLibraryUpdate: { type: Date, required: false, default: null },
   lastHtmlUpdate: { type: Date, required: false, default: null },
   lastResolvedDev: { type: Date, required: false, default: null },
   lastResolvedTester: { type: Date, required: false, default: null },
   lastBugReport: { type: Date, required: false, default: null },
   SOP: { type: String, required: false, default: "" },
   ReleaseNotes: { type: String, required: false, default: "" },
   defaultStack: { type: String, required: false, default: "" },
 },
 { timestamps: true }
);
// Define and export the model
const ToolModel = model<ToolInterface>("tool", toolSchema);
export default ToolModel;
export type { ToolInterface };
```

user.model.ts

```
import mongoose, { Schema, Document } from "mongoose";
export interface UserInterface {
 name: string; username?: string; email?: string; password: string; defaultPassword: string;
 photo: string; role: "user" | "admin" | "superadmin"; phone: string; resetPass: boolean;
 roletype: string | string[]; resetOtp?: string; resetOtpExpires?: Date;
 adminControl?: ("create" | "edit" | "view" | "delete")[];
 adminOption?: ("share" | "generate_report" | "insight_view")[];
}
const userSchema: Schema = new Schema(
 {
   name: { type: String, required: false, default: "" },
   username: { type: String, unique: true,  sparse: true, // prevents duplicate index error if missing},
   email: { type: String, unique: true,  sparse: true,   },
   password: { type: String, required: true },
   defaultPassword: { type: String, required: false },
   photo: { type: String, default: "" },
   role: {     type: String,     enum: ["user", "admin", "superadmin"], default: "user",   },
   phone: { type: String, default: "" },
   resetPass: { type: Boolean, default: false },
   roletype: { type: [String], enum: ["bugreporter", "tester", "dev", "admin"], required: false },
   resetOtp: { type: String, required: false },
   resetOtpExpires: { type: Date, required: false },
   // ✅ Multi-select arrays with enum validation
   adminControl: {type: [String], enum: ["create", "edit", "view", "delete"], required: false,   },
   adminOption: {
```

```
        type: [String], enum: ["share", "generate_report", "insight_view","export"], required: false,
    },
 },
 { timestamps: true }
);
// Param Custom validator: Require at least one of username or email
userSchema.pre("validate", function (next) {
 if (!this.username && !this.email) {       return next(new Error("Either username or email is required"));
}next();
});
const User = mongoose.model<UserInterface & Document>("user", userSchema);
export default User;
```

routes (folder)

auth.routes.ts

```
import { Router } from "express";
import AuthController from "../controllers/auth.controller";
import { authRateLimiter } from "../middlewares/rateLimiter";
const router = Router();
router.use(authRateLimiter);
router.post("/register", AuthController.registerUser);
router.post("/login", AuthController.loginUser);
router.post("/logout", AuthController.logout);
export default router;
```

authExtend.routes.ts

```
import { Router } from "express";
import AuthExtendController from "../controllers/authExtend.controller";
const router = Router();
router.get("/me", AuthExtendController.Me);
router.post("/register", AuthExtendController.registerUserWithToken);
router.post("/update_user", AuthExtendController.UpdateUserWithToken);
export default router;
```

bug.routes.ts

```
import { Router } from "express";
import BugController from "../controllers/bug.controller";
import { upload } from "../middlewares/multer";
const router = Router();
router.post("/upload", upload.array("files", 5), BugController.UploadFiles);
router.post("/bug_create", BugController.BugRaise);
router.post("/bug_confirm", BugController.ConfirmBug);
router.post("/bug_Analyze", BugController.AnalyzeBug);
router.post("/bug_Fix", BugController.FixBug);
router.post("/bug_FinalTest", BugController.FinalTestBug);
router.post("/bug_Deploy", BugController.DeployBug);
router.post("/bug_Close", BugController.CloseBug);
router.post("/get_bugs", BugController.GetBugs);
router.post("/get_BugById", BugController.GetBugById);
router.post("/get_BugStatistics", BugController.GetBugStatistics);
router.post("/update_Bug", BugController.UpdateBug);
router.post("/delete_Bug", BugController.DeleteBug);
router.post("/get_BugLogs", BugController.GetBugLogs);
export default router;
```

credential.routes.ts

```
import { Router } from "express";
import CredentialController from "../controllers/credential.controller";
const router = Router();
router.post("/cpass", CredentialController.ChangePassword);
export default router;
```

index.ts

```
import { Router } from "express";
import userRoutes from "./user.routes";
import toolRoutes from "./tool.routes";
import bugRoutes from "./bug.routes";
import stackRoutes from "./stack.routes";
import authRoutes from "./auth.routes";
import authExtend from "./authExtend.routes";
import credential from "./credential.routes";
import resetPassRoutes from "./resetpass.routes";
import notificationRoutes from "./notification.routes";
import { authenticateToken } from "../middlewares/authMiddleware";
const router = Router();
// Routes that don't require authentication
router.use("/auth", authRoutes); // Register and login routes
router.use("/reset", resetPassRoutes); // Register and reset password with token...
// Middleware that checks for a valid token (authentication)
router.use(authenticateToken);
router.use("/authextend", authExtend); // Register new user/amdin with token...
router.use("/credential", credential); // Register and reset password with token...
router.use("/notifications", notificationRoutes);
// Routes that require authentication
router.use("/user", userRoutes);
router.use("/stack", stackRoutes); // Stack routes
router.use("/tools", toolRoutes); // Tool routes
router.use("/bug", bugRoutes); // Tool routes
export default router;
```

logs.routes.ts

```
import { Router } from "express";
import ToolController from "../controllers/tool.controller";
const router = Router();
router.post("/get_timeline", ToolController.UpdateTool);
router.post("/delete_timeline", ToolController.UpdateTool);
export default router;
```

notification.routes.ts

```
import { Router } from "express";
import NotificationController from "../controllers/notification.controller";
import { authenticateToken } from "../middlewares/authMiddleware";
const router = Router();
router.use(authenticateToken);
router.get("/", NotificationController.GetNotifications);
router.patch("/:id/read", NotificationController.MarkAsRead);
router.patch("/read-all", NotificationController.MarkAllAsRead);
router.delete("/:id", NotificationController.DeleteNotification);
export default router;
```

resetpass.routes.ts

```
import { Router } from "express";
import ResetPasswordController from "../controllers/resetpass.controller";
import { authRateLimiter } from "../middlewares/rateLimiter";
const router = Router();
router.use(authRateLimiter);
router.post("/request", ResetPasswordController.RequestReset);
router.post("/verify", ResetPasswordController.VerifyOtp);
router.post("/password", ResetPasswordController.ResetPassword);
export default router;
```

stack.routes.ts

```
import { Router } from "express";
import StackController from "../controllers/stack.controller";
const router = Router();
router.post("/update_stack", StackController.UpdateStacks);
router.get("/system_config", StackController.GetSystemConfig);
export default router;
```

tool.routes.ts

```
import { Router } from "express";
import ToolController from "../controllers/tool.controller";
const router = Router();
router.post("/update_tool", ToolController.UpdateTool);
router.get("/getTools", ToolController.GetTools);
export default router;
```

user.routes.ts

```
import { Router } from "express";
import UserController from "../controllers/user.controller";
const router = Router();
router.get("/update_user", UserController.UpdateUserWithToken);
router.get("/get_all_users", UserController.getAllUsers);
router.put("/update_user/:id", UserController.updateUser);
router.post("/change_password/:id", UserController.changePassword);
router.delete("/delete_user/:id", UserController.deleteUser);
export default router;
```

app.ts

```
import dotenv from "dotenv";
dotenv.config();
import express, { Application } from "express";
import routes from "./routes/c";
import cors from "cors";
import cookieParser from "cookie-parser";
import { errorHandler } from "./utils/errorHandler";
const app: Application = express();
// Middleware
app.use(express.urlencoded({ extended: true }));
app.use(express.json());
const allowedOrigins = ["http://localhost:5173", "http://127.0.0.1:5173"];
```

```
app.use(
 cors({
   origin: (origin, callback) => {
     if (!origin || allowedOrigins.includes(origin)) {callback(null, true);}
     else { callback(new Error("Not allowed by CORS"));}
   },
   credentials: true,
 })
);
console.log("CORS and middleware configured");
// Routes
app.use(cookieParser());
app.use("/api", routes);
// Error handling middleware
app.use(errorHandler);
export default app;
```

server.ts

```
import dotenv from "dotenv";
dotenv.config();
import app from "./app";
import mongoose from "mongoose";
const PORT = parseInt(process.env.PORT || "5500", 10); // Ensure PORT is a number
const MONGO_URI = `${process.env.MONGO_URI}` || "";
app.get('/test', (_req, res) => { res.json({ message: 'Server is working!' });});
try {
   app.listen(PORT, "0.0.0.0" , async () => { await mongoose.connect(MONGO_URI);
       console.log("Connected to MongoDB");
       console.log(`Server running on port ${PORT}`);
   });
} catch (error) { console.error("MongoDB connection failed:", error);}
```

Utils

asyncHandler.ts

```
import { Request, Response, NextFunction } from "express";
import { logger } from "./logger"; // Assuming you have a logger set up
import { UserInterface } from "../models/user.model";
type RequestWithUser = Request & { [key: string]: any, user?: UserInterface };
export const asyncHandler = ( fn: (req: RequestWithUser, res: Response, next: NextFunction) => Promise<void>)
=> {
 return (req: RequestWithUser, res: Response, next: NextFunction) => {
   fn(req, res, next).catch(async (error) => {
     try {
          const stackTrace = (await import("stack-trace")).default;// Dynamically import stack-trace
       // Parse the error stack trace
       const trace = stackTrace.parse(error);
       const firstFrame = trace[0];
       const { fileName, lineNumber, columnNumber } = firstFrame ? {
           fileName: firstFrame.getFileName(),lineNumber: firstFrame.getLineNumber(),
           columnNumber: firstFrame.getColumnNumber()
       } : {};
       // Log the error details, including file name, line number, column number
       logger.error({ message: error.message,stack: error.stack, fileName,lineNumber,columnNumber, });
```

```
    } catch (err) { logger.error({ message: "Failed to capture stack trace",error: err, });}
      next(error);
  });
 };
};
```

bugActivity.ts

```
import LogModel from "../models/logs.models";
import { Types } from "mongoose";
/*** Logs an activity for a bug.*/
export async function logBugActivity(
 bugId: Types.ObjectId | string, performedBy: Types.ObjectId | string | undefined,
 action: string, details: string, type: string = "bug_activity"
) {
 try {
   await LogModel.create({bugId,     performedBy,     action,     details,     type,   });
 } catch (error) {console.error("Error logging bug activity:", error); }
}
/*** Compares two bug objects and returns a list of changes.* This is a simplified deep comparison.*/
export function captureBugChanges(oldData: any, newData: any, prefix: string = ""): any[] {
 const changes: any[] = [];
  // Fields to ignore (metadata, internal mongoose fields)
 const ignoreFields = ["_id", "__v", "updatedAt", "createdAt", "bugId", "reportedAt", "testedAt",
"analyzedAt", "fixedAt", "approvedAt", "deployedAt", "closedAt"];
 // Normalize data (convert mongoose documents to plain objects if needed)
 const oldObj = oldData && typeof oldData.toObject === 'function' ? oldData.toObject() : oldData;
 const newObj = newData && typeof newData.toObject === 'function' ? newData.toObject() : newData;
 for (const key in newObj) {
   if (ignoreFields.includes(key)) continue;
   const oldVal = oldObj ? oldObj[key] : undefined;
   const newVal = newObj[key];
   const fullKey = prefix ? `${prefix}.${key}` : key;
   // Skip if both are empty/null/undefined
   if (!oldVal && !newVal && oldVal !== false && newVal !== false && oldVal !== 0 && newVal !== 0) continue;
   if (newVal && typeof newVal === 'object' && !(newVal instanceof Date) &&
       !(newVal instanceof Types.ObjectId) && !Array.isArray(newVal)) {
     // Recurse for nested objects
     const nestedChanges = captureBugChanges(oldVal || {}, newVal, fullKey);
     changes.push(...nestedChanges);
   } else {
     // Comparison for primitives, arrays, Dates, ObjectIds
     let isDifferent = false;
     if (Array.isArray(newVal)) {
       isDifferent = JSON.stringify(oldVal) !== JSON.stringify(newVal);
     } else if (newVal instanceof Date || oldVal instanceof Date) {
       isDifferent = new Date(oldVal).getTime() !== new Date(newVal).getTime();
     } else if (newVal instanceof Types.ObjectId || oldVal instanceof Types.ObjectId) {
       isDifferent = oldVal?.toString() !== newVal?.toString();
     } else {isDifferent = oldVal !== newVal;}
     if (isDifferent) {changes.push({ field: fullKey,oldValue: oldVal, newValue: newVal });}
   }
 }
 return changes;
}
```

cloudinary.ts

```
import { v2 as cloudinary } from "cloudinary";
import fs from "fs";
import dotenv from "dotenv";
dotenv.config();
cloudinary.config({ cloud_name: process.env.Cloudinary_name, api_key: process.env.Cloudinary_API_Key,
 api_secret: process.env.Cloudinary_API_Secret,
});
export const uploadOnCloudinary = async (localFilePath: string) => {
 try {
   if (!localFilePath) return null;
    const response = await cloudinary.uploader.upload(localFilePath, {  // Upload the file on cloudinary
     resource_type: "auto",
     folder: "Bug_Tracker", // As requested by user
   });
   console.log("File is uploaded on cloudinary", response.url); // File has been uploaded successfully
   // Remove the locally saved temporary file
   if (fs.existsSync(localFilePath)) { fs.unlinkSync(localFilePath);   }  return response;
 } catch (error) {   console.error("Cloudinary upload error:", error);
   // Remove the locally saved temporary file as the upload operation failed
   if (fs.existsSync(localFilePath)) { fs.unlinkSync(localFilePath);}    return null;
 }
};
```

cron.ts

```
import cron from "node-cron";
import BugModel from "../models/bug.models";
import User from "../models/user.model";
import { sendEmail } from "./mail";
export const sendDailyReport = async () => {
 try { console.log("Running Daily Bug Report Cron Job...");
     const bugs = await BugModel.find({ // Get bugs that are NOT in Closure
     currentPhase: { $nin: ["Closure", "Closed"] },     isActive: true
   }).populate("phaseI_BugReport.reportedBy", "name email");
   if (bugs.length === 0) {     console.log("No pending bugs to report."); return;   }
   // Create email content
   let htmlContent = `
     <h2>Daily Bug Report</h2>
     <p>Here is the list of bugs that are created, updated, or pending (excluding closed/closure bugs) as of
8:00 PM today:</p>
     <table border="1" cellpadding="8" style="border-collapse: collapse; width: 100%;">
       <thead>
         <tr style="background-color: #f3f4f6;">
           <th style="text-align: left;">Bug ID</th><th style="text-align: left;">Phase</th>
           <th style="text-align: left;">Priority</th><th style="text-align: left;">Reported By</th>
           <th style="text-align: left;">Tool</th>
         </tr>
       </thead>
       <tbody>
   `;
   bugs.forEach(bug => {
     const reporter = bug.phaseI_BugReport?.reportedBy as any;
```

```
    const reporterName = reporter?.name || "Unknown";
    const priority = bug.phaseI_BugReport?.toolInfo?.priority || "N/A";
    const toolName = bug.phaseI_BugReport?.toolInfo?.toolName || "N/A";
    htmlContent += `
      <tr>
        <td><strong>${bug.bugId}</strong></td><td>${bug.currentPhase}</td><td>${priority}</td>
        <td>${reporterName}</td><td>${toolName}</td>
      </tr>
    `;
  });
  htmlContent += `
      </tbody>
    </table>
    <p>Please review these bugs in the system dashboard.</p>
  `;
  // Find all admins & superadmins
  const admins = await User.find({ role: { $in: ["admin", "superadmin"] } });
  const adminEmails = admins.map(admin => admin.email).filter(Boolean) as string[];
  if (adminEmails.length > 0) {
    await sendEmail(
      adminEmails.join(","), "Daily Bug Report (Created, Updated & Pending)",
      "Daily Report of Pending Bugs", htmlContent
    );
    console.log(`Daily bug report sent to: ${adminEmails.join(", ")}`);
  } else { console.log("No admins found with email addresses to send the report."); }
 } catch (error) {console.error("Error in daily bug report cron job:", error); }
};
export const setupCronJobs = () => { cron.schedule("0 20 * * *", sendDailyReport);};
// Run every day at 8:00 PM (20:00)
```

emailTemplates.ts

```
export const passwordResetOtpTemplate = (otp: string) => `
<!DOCTYPE html>
<html>
<head>
  <style>
      .container { font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif; max-width: 600px; margin: 0
auto; padding: 20px; border: 1px solid #e0e0e0; border-radius: 10px; }
      .header { background-color: #ff5722; color: white; padding: 20px; text-align: center; border-radius:
10px 10px 0 0; }
      .content { padding: 30px; text-align: center; color: #333; }
      .otp { font-size: 32px; font-weight: bold; color: #ff5722; letter-spacing: 5px; margin: 20px 0;
padding: 15px; background-color: #fff5f0; border-radius: 5px; display: inline-block; }
      .footer { font-size: 12px; color: #777; text-align: center; margin-top: 20px; }
  </style>
</head>
<body>
  <div class="container">
      <div class="header"> <h1>Bug Tracker</h1> </div>
      <div class="content">
          <h2>Password Reset Request</h2><p>Hello,</p>
          <p>We received a request to reset your Bug Tracker password. Use the following OTP to proceed.
This code is valid for 5 minutes.</p>
          <div class="otp">${otp}</div>
```

```
          <p>If you didn't request a password reset, you can safely ignore this email.</p>
      </div>
      <div class="footer"><p>&copy; 2026 Bug Tracker. All rights reserved.</p></div>
   </div>
</body>
</html>
`;
```

errorCodes.ts

```
export enum HttpStatusCodes {
   // Informational Responses
   CONTINUE = 100,SWITCHING_PROTOCOLS = 101 PROCESSING = 102,EARLY_HINTS = 103,
    // Successful Responses
   OK = 200, CREATED = 201,ACCEPTED = 202,NON_AUTHORITATIVE_INFORMATION = 203,NO_CONTENT = 204,
   RESET_CONTENT = 205,PARTIAL_CONTENT = 206, MULTI_STATUS = 207, ALREADY_REPORTED = 208,IM_USED = 226,
    // Redirection Messages
   MULTIPLE_CHOICES = 300,MOVED_PERMANENTLY = 301,FOUND = 302,SEE_OTHER = 303,NOT_MODIFIED = 304,
   USE_PROXY = 305,TEMPORARY_REDIRECT = 307,PERMANENT_REDIRECT = 308,
    // Client Error Responses
   BAD_REQUEST = 400,UNAUTHORIZED = 401, PAYMENT_REQUIRED = 402,  FORBIDDEN = 403, NOT_FOUND = 404,
   METHOD_NOT_ALLOWED = 405,NOT_ACCEPTABLE = 406,PROXY_AUTHENTICATION_REQUIRED = 407, REQUEST_TIMEOUT = 408,
   CONFLICT = 409,GONE = 410,LENGTH_REQUIRED = 411, PRECONDITION_FAILED = 412,PAYLOAD_TOO_LARGE = 413,
   URI_TOO_LONG = 414,UNSUPPORTED_MEDIA_TYPE = 415, RANGE_NOT_SATISFIABLE = 416,EXPECTATION_FAILED = 417,
   I_M_A_TEAPOT = 418, // Yes, this is a real HTTP status code!
   MISDIRECTED_REQUEST = 421, UNPROCESSABLE_ENTITY = 422, LOCKED = 423, FAILED_DEPENDENCY = 424,
   TOO_EARLY = 425,UPGRADE_REQUIRED = 426,PRECONDITION_REQUIRED = 428, TOO_MANY_REQUESTS = 429,
   REQUEST_HEADER_FIELDS_TOO_LARGE = 431,UNAVAILABLE_FOR_LEGAL_REASONS = 451,
    // Server Error Responses
   INTERNAL_SERVER_ERROR = 500,NOT_IMPLEMENTED = 501,BAD_GATEWAY = 502,SERVICE_UNAVAILABLE = 503,
   GATEWAY_TIMEOUT = 504,HTTP_VERSION_NOT_SUPPORTED = 505,VARIANT_ALSO_NEGOTIATES = 506,
   INSUFFICIENT_STORAGE = 507,LOOP_DETECTED = 508,NOT_EXTENDED = 510, NETWORK_AUTHENTICATION_REQUIRED = 511,
}
```

errorHandler.ts

```
import { Request, Response, NextFunction } from "express";
import { logger } from "./logger";
import { HttpStatusCodes } from "./errorCodes";
export const errorHandler = ( err: Error, req: Request, res: Response, _next: NextFunction) => {
 logger.error({message: err.message,stack: err.stack,url: req.originalUrl, method: req.method,});
 res.status(HttpStatusCodes.INTERNAL_SERVER_ERROR).json({
   message: "Internal Server Error",error: process.env.NODE_ENV === "production" ? undefined : err.message,
 });
};
```

logger.ts

```
import winston from "winston";
export const logger = winston.createLogger({
 level: "error",
 format: winston.format.combine(
   winston.format.timestamp(),   winston.format.printf(({ timestamp, level, message, stack }) => {
     return `${timestamp} [${level}]: ${message} - ${stack}`;
   })
 ),
```

```
 transports: [new winston.transports.Console(),new winston.transports.File({ filename: "logs/error.log" }) ],
});
```

mail.ts

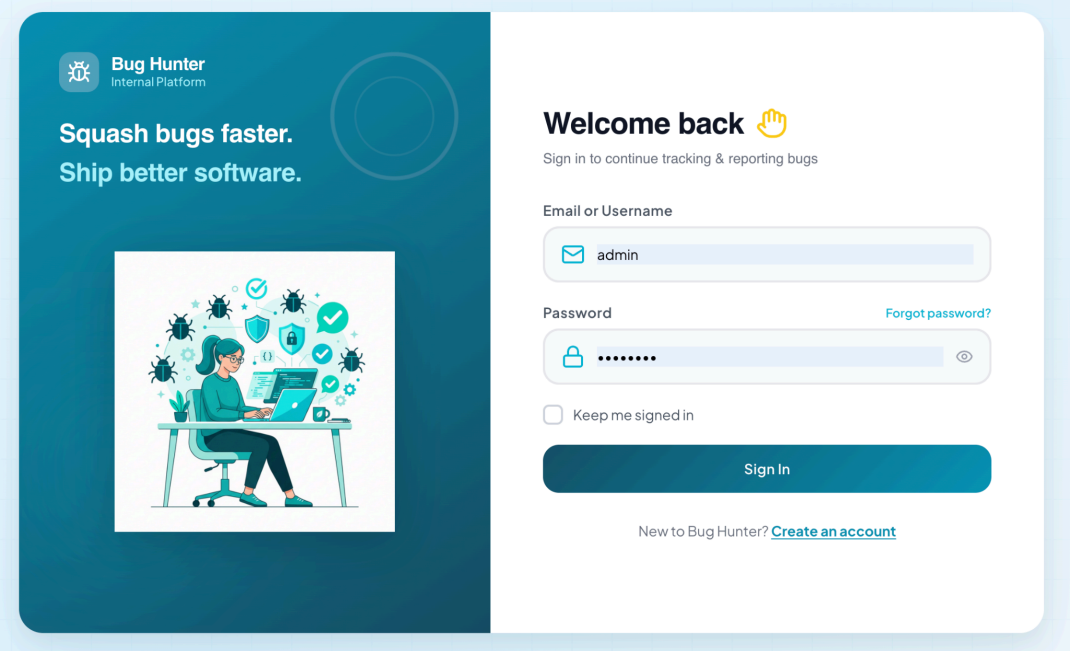
```
import nodemailer from "nodemailer";
import * as dotenv from "dotenv";
dotenv.config();
const transporter = nodemailer.createTransport({
 service: "gmail", auth: {user: process.env.MAIL_ID,   pass: process.env.MAIL_PASSWORD, },
});
export const sendEmail = async (to: string, subject: string, text: string, html?: string) => {
 try {
   const mailOptions = {from: process.env.MAIL_ID, to,subject,text,html,};
   const info = await transporter.sendMail(mailOptions);
   console.log("Email sent: " + info.response);
   return true;
 } catch (error) {
   console.error("Error sending email: ", error);
   return false;
 }
};
```

notification.ts

```
import Notification from "../models/notification.models";
import { Types } from "mongoose";
export const createNotification = async (data: {
 recipient: Types.ObjectId | string; sender: Types.ObjectId | string;
 type: "assignment" | "status_change" | "comment" | "other";
 title: string; message: string; link?: string;}) => {
 try {
   console.log("Creating notification in DB...", {
     recipient: data.recipient,title: data.title, type: data.type
   });
   const notification = new Notification({
     recipient: new Types.ObjectId(data.recipient), sender: new Types.ObjectId(data.sender),
     type: data.type, title: data.title, message: data.message, link: data.link,
   });
   const saved = await notification.save();
   console.log("Notification saved successfully ID:", saved._id);
   return saved;
 } catch (error) {
   console.error("CRITICAL: Error creating notification:", error);
   return null;
 }
};
```

10. System Output screen, Input Screen

1. Authentication Interface — User Login Screen



1. Authentication Interface — User Login Screen

Overview: The Login Screen serves as the primary gateway to the Bug Hunter platform. It features a modern, split-pane design that balances aesthetic branding with a high-fidelity, user-centric authentication flow.

Key Features:

Dynamic Branding Pane (Left): Utilizes a professional teal-gradient aesthetic with the platform's core mission statement: "Squash bugs faster. Ship better software." The high-fidelity vector illustration reinforces the developer-focused nature of the tool.

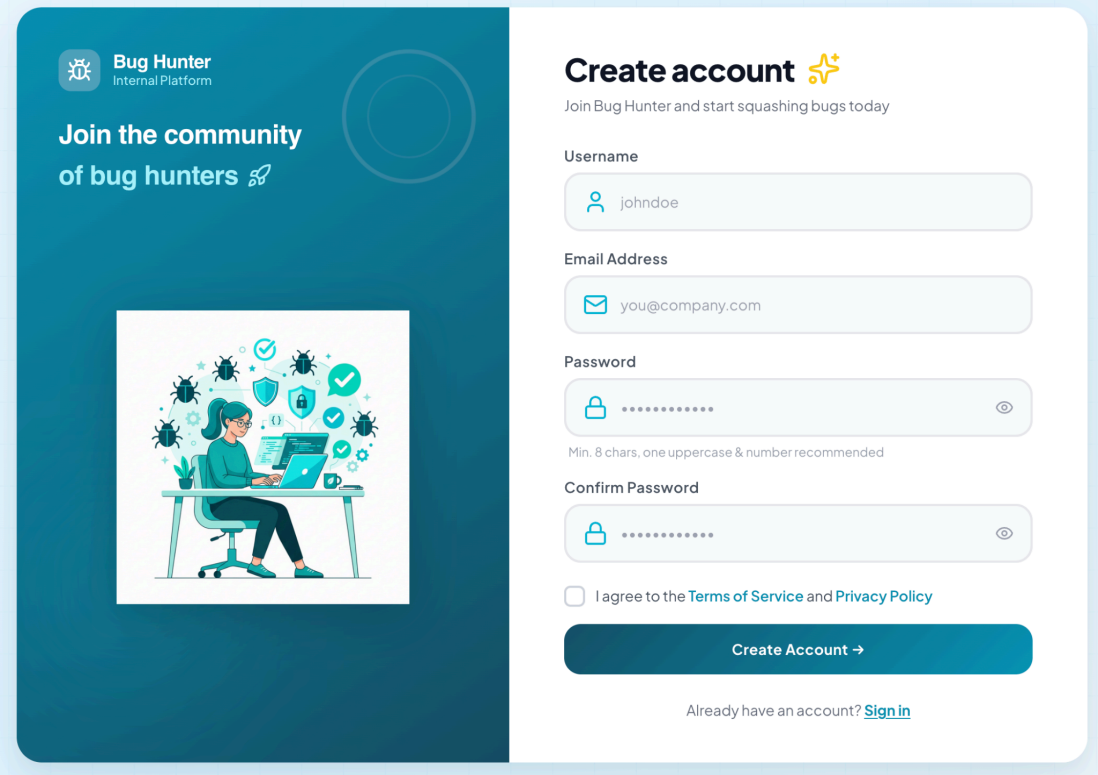
Unified Access Control (Right):Multi-Identifier Support: Allows users to sign in using either their registered Email or a unique Username, providing flexibility in identity management.

Enhanced Security: Includes a password visibility toggle to prevent input errors and a dedicated "Forgot password?" link that triggers the secure OTP-based recovery workflow.

Session Persistence: Features a "Keep me signed in" option, which manages long-lived secure session cookies for improved user experience.

Technical Integration: The interface is fully integrated with the backend AuthController, utilizing JWT (JSON Web Tokens) and HTTP-only secure cookies to ensure all user sessions are protected against unauthorized access and XSS (Cross-Site Scripting) attacks.

2. Registration Interface



2. Registration Interface — User Sign-Up Screen

Overview: The Registration Screen is designed to provide a frictionless onboarding experience for new users. It maintains visual consistency with the login interface while introducing expanded data-entry fields required for secure account provisioning.

Key Features:

Community-Focused Branding (Left): Features a welcoming headline, "Join the community of bug hunters," and continues the high-fidelity visual theme to establish brand trust and professional context.

Secure Onboarding Flow (Right):

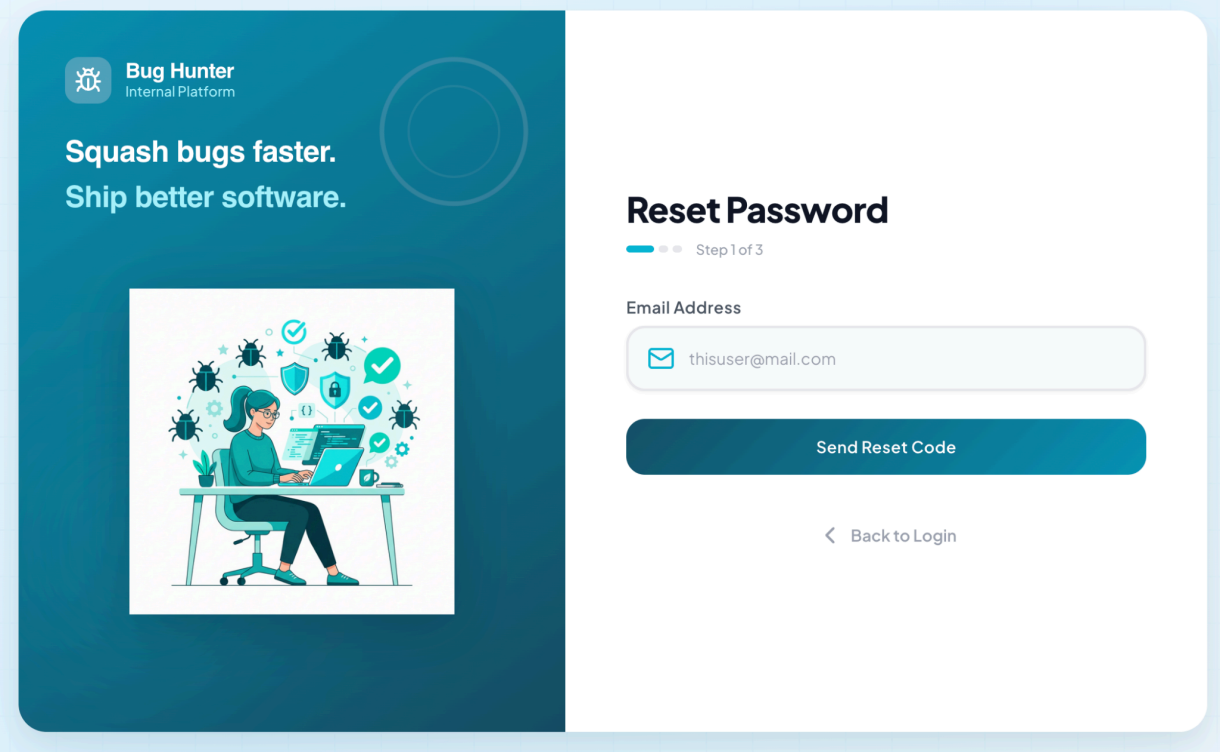
Identity Validation: Captures both a unique Username and a verified Email Address. The system performs real-time backend checks to ensure credential uniqueness across the database.

Advanced Credential Security: Implements a dual-entry password system with a "Confirm Password" field to eliminate typographical errors. Both fields are equipped with visibility toggles to assist the user during complex password entry.

Legal & Procedural Compliance: Includes an integrated agreement checkbox for the platform's Terms of Service and Privacy Policy, ensuring that all new members are aligned with the system's usage guidelines.

Technical Implementation: This interface communicates directly with the AuthController.registerUser endpoint. Upon submission, passwords are encrypted using bcrypt (salted hashing) before storage, and the system automatically initializes the new user profile with default "User" permissions, ready for administrative approval or role assignment.

3. Password Recovery Interface



3. Password Recovery Interface — Step 1: Account Identification

Overview: The Password Recovery system is a secure, multi-stage workflow designed to help users regain access to their accounts while maintaining strict security protocols. This first step focuses on identifying the user and initiating the verification process.

Key Features:

Progressive Disclosure UI: Utilizes a clear "Step 1 of 3" indicator to guide the user through the recovery journey (Identification → OTP Verification → New Password Entry), reducing cognitive load and improving user retention.

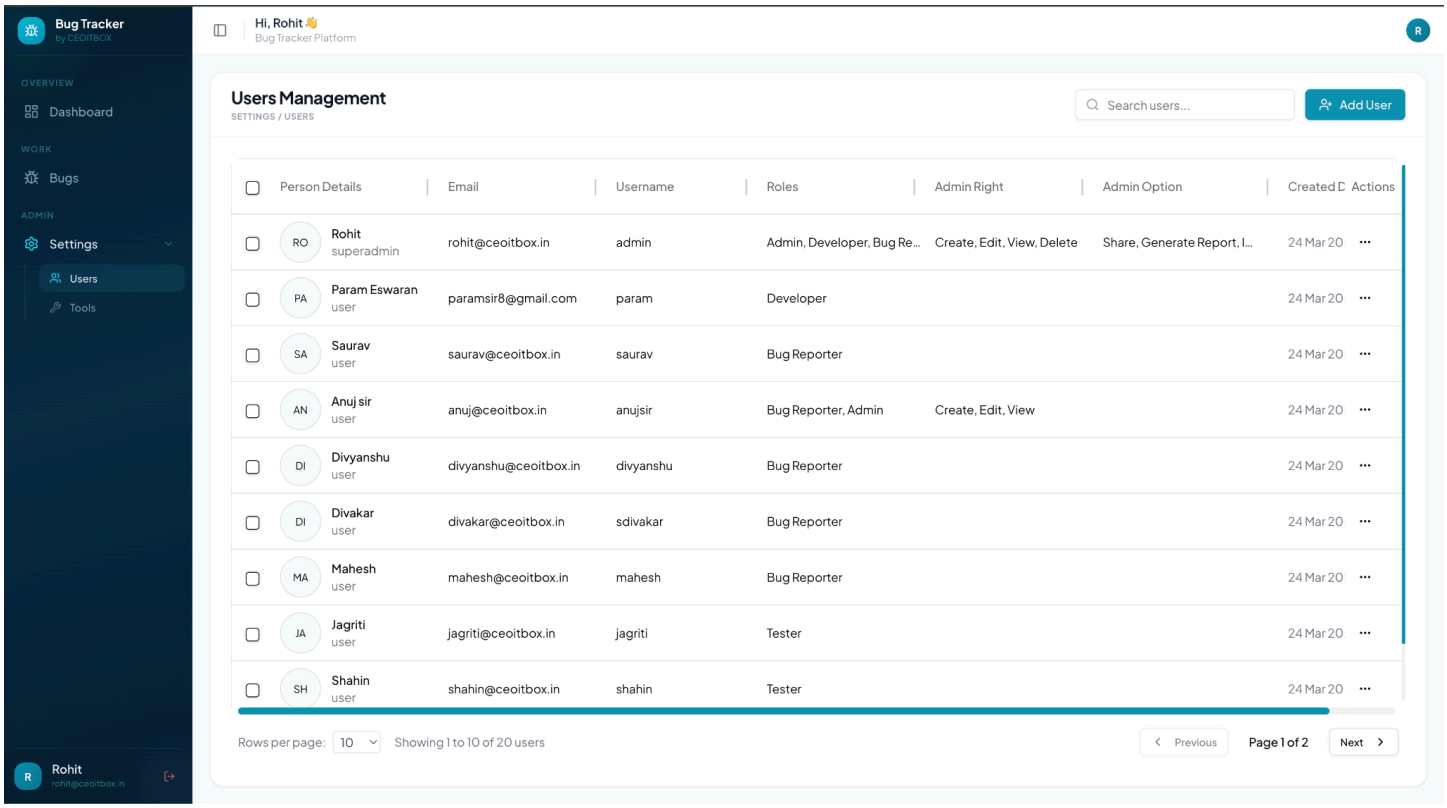
Secure Recovery Trigger:
Email Identification: Requires the user to input their registered email address. This acts as a security anchor, ensuring that recovery instructions are only sent to the verified owner of the account.

Automated OTP Dispatch: Clicking "Send Reset Code" triggers a backend event that generates a unique, time-sensitive One-Time Password (OTP) and dispatches it via a professionally formatted email.

Navigation Continuity: Includes a "Back to Login" action, allowing for seamless navigation if the user recalls their credentials or navigated to the recovery screen in error.

Technical Implementation: This interface connects to the ResetPasswordController.RequestReset endpoint. The backend verifies the account's existence, generates a cryptographically secure 6-digit OTP (hashed using SHA-256 for database storage), and sets a 5-minute expiration window to prevent brute-force recovery attempts.

4. Administrative Dashboard — User Management Console



4. Administrative Dashboard — User Management Console

Overview: The User Management Console is a high-fidelity, centralized command center for system administrators. It provides a comprehensive overview of all platform participants, enabling granular governance of roles, permissions, and account statuses.

Key Features:

High-Density Data Grid:

Rich Profile Visualization: Displays detailed user identities, including high-resolution avatars, contact emails, and unique system usernames for immediate recognition.

Professional Designation Tracking: Clearly maps users to their functional roles (e.g., Bug Reporter, Tester, Developer, Admin), ensuring transparency in the bug resolution workforce.

Granular Permission Matrix: Directly visualizes active "Admin Rights" (e.g., Create, Edit, Delete) and "Admin Options" (e.g., Share, Insight View), allowing for rapid security audits and access control management.

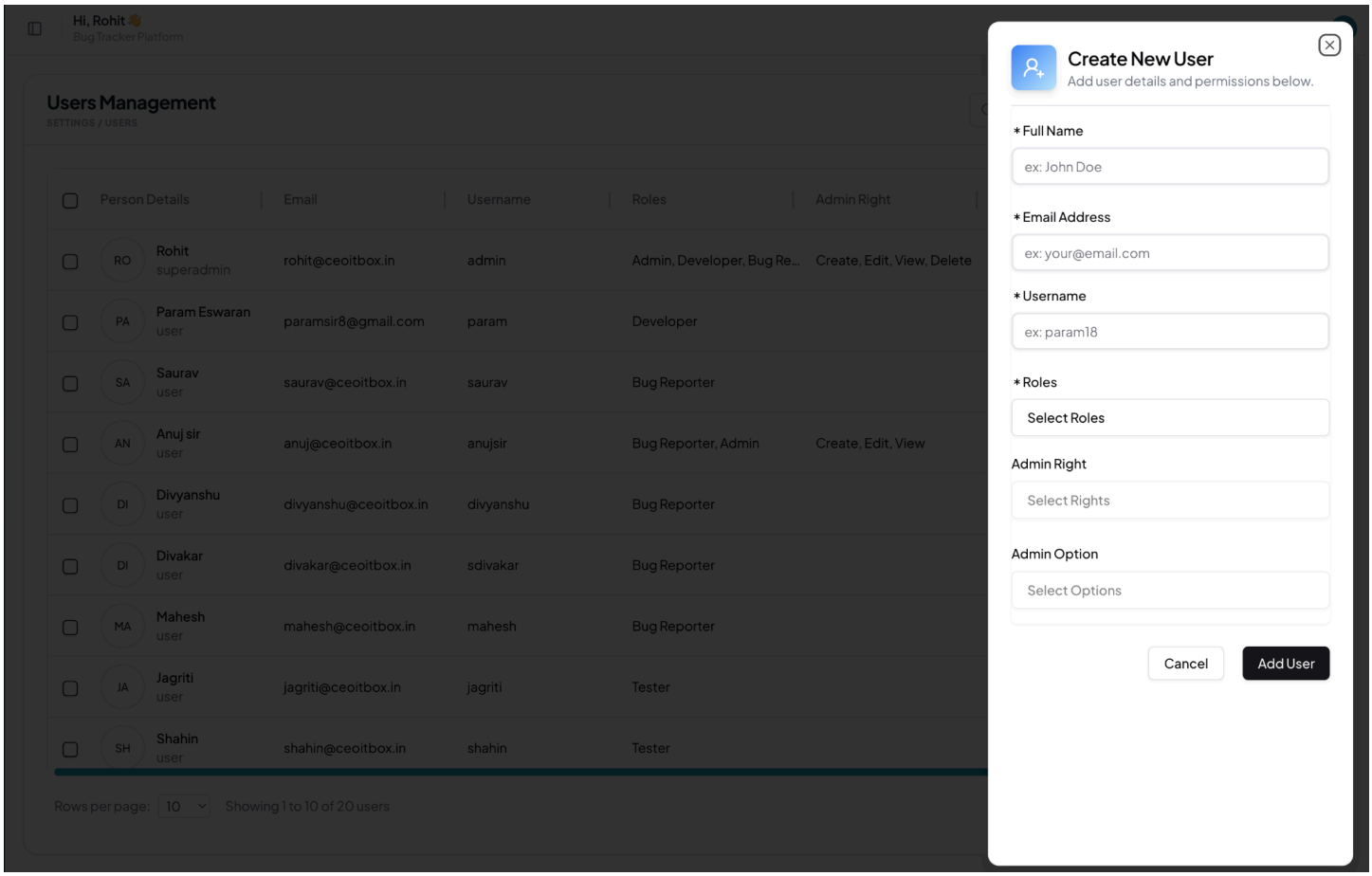
System Operations: Unified Search & Discovery: Includes a global search utility to quickly filter and locate specific users within large datasets.

Account Lifecycle Management: Features an intuitive "Add User" entry point and row-level action menus for editing or removing users.

Optimized Data Handling: Implements professional-grade pagination and data-density controls (e.g., "Showing 1 to 10 of 20 users") to maintain high performance and readability.

Technical Implementation: Behind this interface, the system utilizes the UserController.getAllUsers endpoint. Data is fetched using Mongoose's .lean() method for maximum efficiency, with sensitive fields like passwords explicitly excluded from the response payload to maintain strict security standards.

5. Administrative Operations - Add User



5. Administrative Operations — User Provisioning Interface

Overview: The User Provisioning Interface is a high-fidelity, non-intrusive side-drawer designed for administrators to rapidly onboard new platform members. It focuses on precision, allowing for the simultaneous creation of user identities and the assignment of complex permission matrices.

Key Features:

Structured Data Entry: Core Identification: Captures essential account metadata (Full Name, Email, and Username). Fields marked with an asterisk (*) indicate mandatory requirements enforced by frontend and backend validation.

Multi-Select Role Engine: Features a dynamic role selector that allows administrators to assign one or more professional designations (e.g., Tester, Developer, Bug Reporter) to a single user, reflecting real-world cross-functional responsibilities.

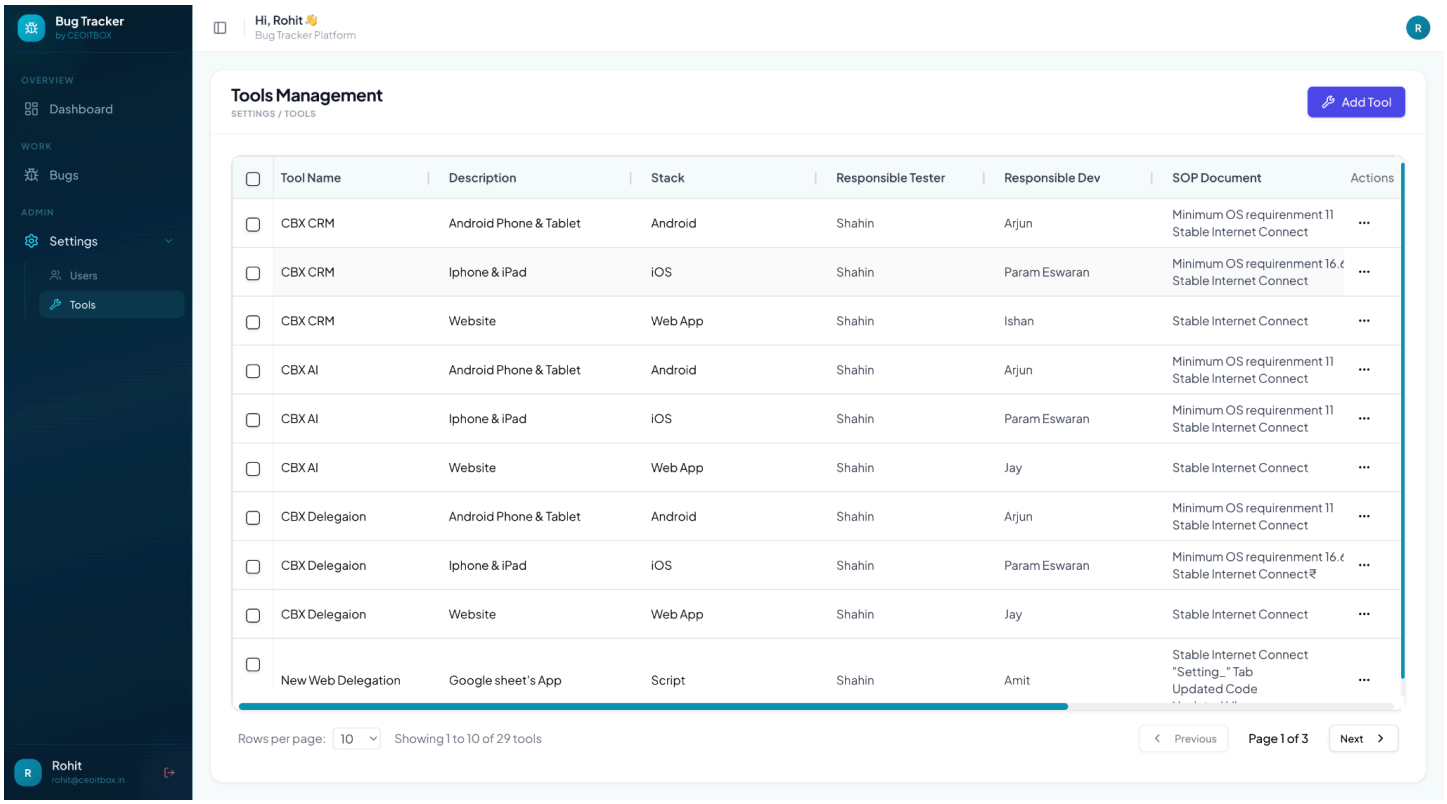
Granular Privilege Delegation: Includes specialized dropdowns for Admin Rights (e.g., Create, Edit, Delete) and Admin Options (e.g., Generate Report, Insight View). This enables precise, "least-privilege" access control tailored to individual staff needs.

Enhanced UX Design: Contextual Overlay: The drawer-based UI allows the administrator to perform provisioning without losing the visual context of the main Users Management dashboard.

Modern Interaction Pattern: Utilizes clean, minimalist input fields and high-contrast action buttons ("Cancel" and "Add User") to streamline the administrative workflow.

Technical Implementation: This interface connects to the authExtend.controller.ts on the backend, specifically the registerUserWithToken endpoint. The system performs multi-layered validation to prevent credential collisions (duplicate emails/usernames) and securely stores the new user's encrypted credentials while initializing their specific permission set in the database.

6.Tools Management



6. Tools Management — Project Tools Configuration Dashboard

Overview: The Tools Management Dashboard serves as the central registry for all software products, platforms, and scripts governed within the Bug Hunter ecosystem. It provides a structured overview of the technology stack and identifies the key personnel responsible for each asset's quality and maintenance.

Key Features:

Asset Categorization & Environment Mapping: Technology Segmentation: Organizes tools based on their specific "Stack" (e.g., Android, iOS, Web App, Script). This allows for precise filtering and ensures that bug reports are triaged to the correct technical environment.

Contextual Descriptions: Provides brief but essential functional summaries for each entry, such as identifying if a tool is for "Phone & Tablet" or a "Google Sheet's App."

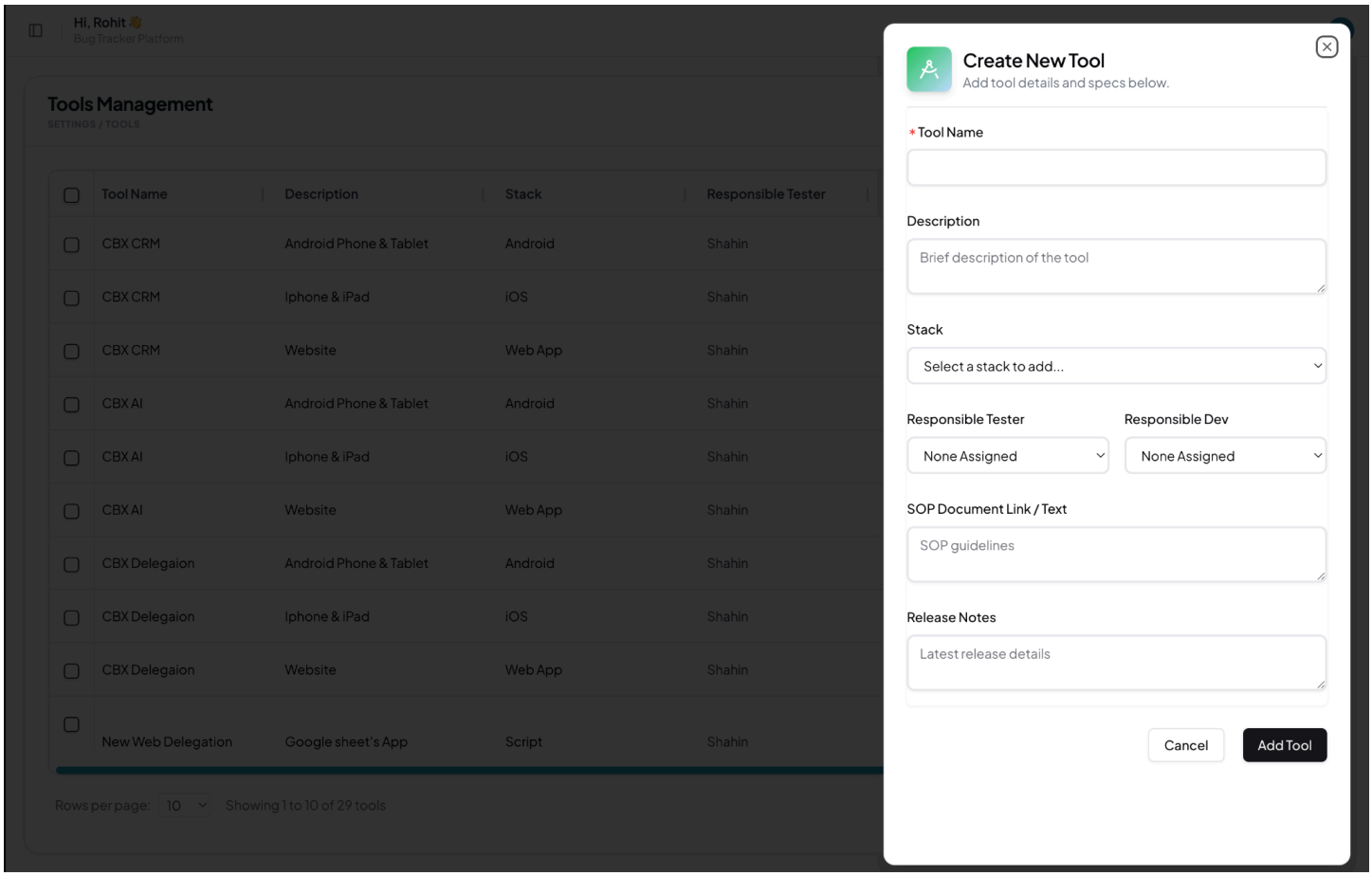
Strategic Responsibility Matrix: Personnel Linkage: Explicitly maps every tool to a Responsible Tester and a Responsible Dev. This data is critical for the platform's automated assignment engine, ensuring that when a bug is raised for a specific tool, the correct staff members are notified immediately.

Operational Documentation: SOP Integration: Includes a dedicated column for SOP Documents, providing immediate access to Standard Operating Procedures and environment prerequisites (e.g., "Minimum OS requirement 16.4" or "Stable Internet Connect"). This ensures that testers follow standardized protocols during verification.

Administrative Oversight: Portfolio Governance: Features a unified "Add Tool" utility and row-level action menus, enabling administrators to manage the project portfolio as the software ecosystem evolves.

Technical Implementation: This interface leverages the ToolController.GetTools endpoint. The backend performs automated population of testerId and devId references, allowing the dashboard to display human-readable names while maintaining a normalized database structure for high-efficiency reporting and assignments.

7. Create Tool - Form



7. Create Tool — Asset Configuration Interface

Overview: The Asset Configuration Interface is a specialized side-drawer designed for administrators to define and register new software assets within the platform's tracking ecosystem. It ensures that every tool is onboarded with complete technical context and clear personnel accountability.

Key Features:

Comprehensive Resource Profiling: Foundational Metadata: Captures the unique Tool Name and a functional Description, establishing the primary record for all future bug tracking and reporting.

Technical Categorization: Utilizes a Stack selector to classify the tool by its environment (e.g., Web, iOS, Android), which is essential for data segmentation and reporting.

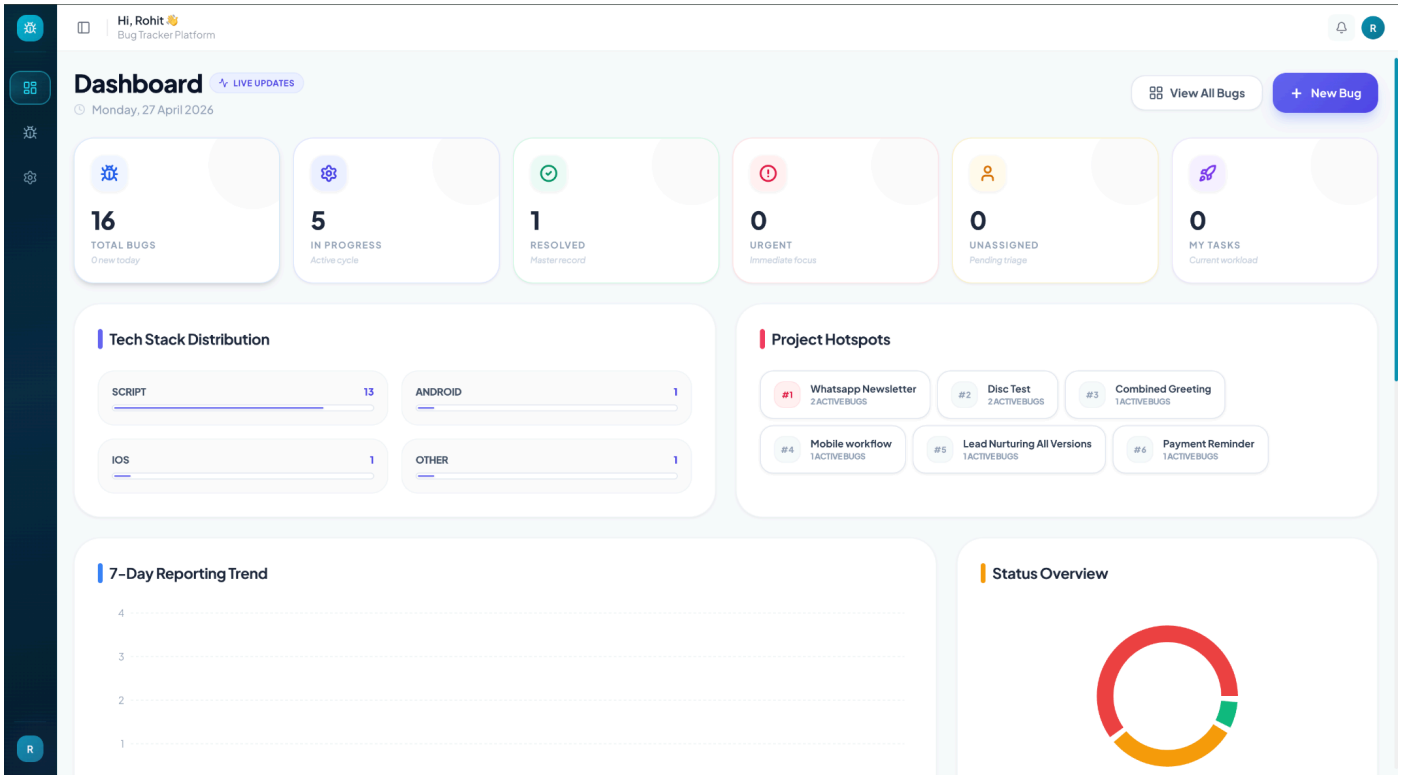
Workforce Alignment & Accountability: Stakeholder Assignment: Enables the direct selection of a Responsible Tester and a Responsible Dev from the platform's user database. This establishes a permanent accountability link that the system uses to automate notifications and triage assignments throughout the bug lifecycle.

Operational Intelligence: Embedded SOP Guidelines: Provides a dedicated area for SOP Document Links or instructional text, ensuring that quality assurance protocols are always accessible to the assigned team.

Change History Tracking: Includes a Release Notes field to document the tool's evolution, giving testers and developers vital context regarding recent updates or known environment constraints.

Technical Implementation: This interface communicates with the ToolController.UpdateTool endpoint. It supports sophisticated data payloads, including multi-line text and relational ID mappings for personnel. The backend performs intelligent merging, ensuring that tool registries are updated without data loss while maintaining strict normalization of assigned staff records.

8. Dashboard — Analytical Summary Interface



8. Dashboard — Analytical Summary Interface

Overview: The main Dashboard provides a real-time, high-level snapshot of the platform's health, aggregating complex data into actionable insights for the entire project team.

Key Features:

Real-Time KPIs: Displays critical metrics at the top, including Total Bugs, In-Progress Tasks, and Resolution Counts, providing an immediate pulse on the current development cycle.

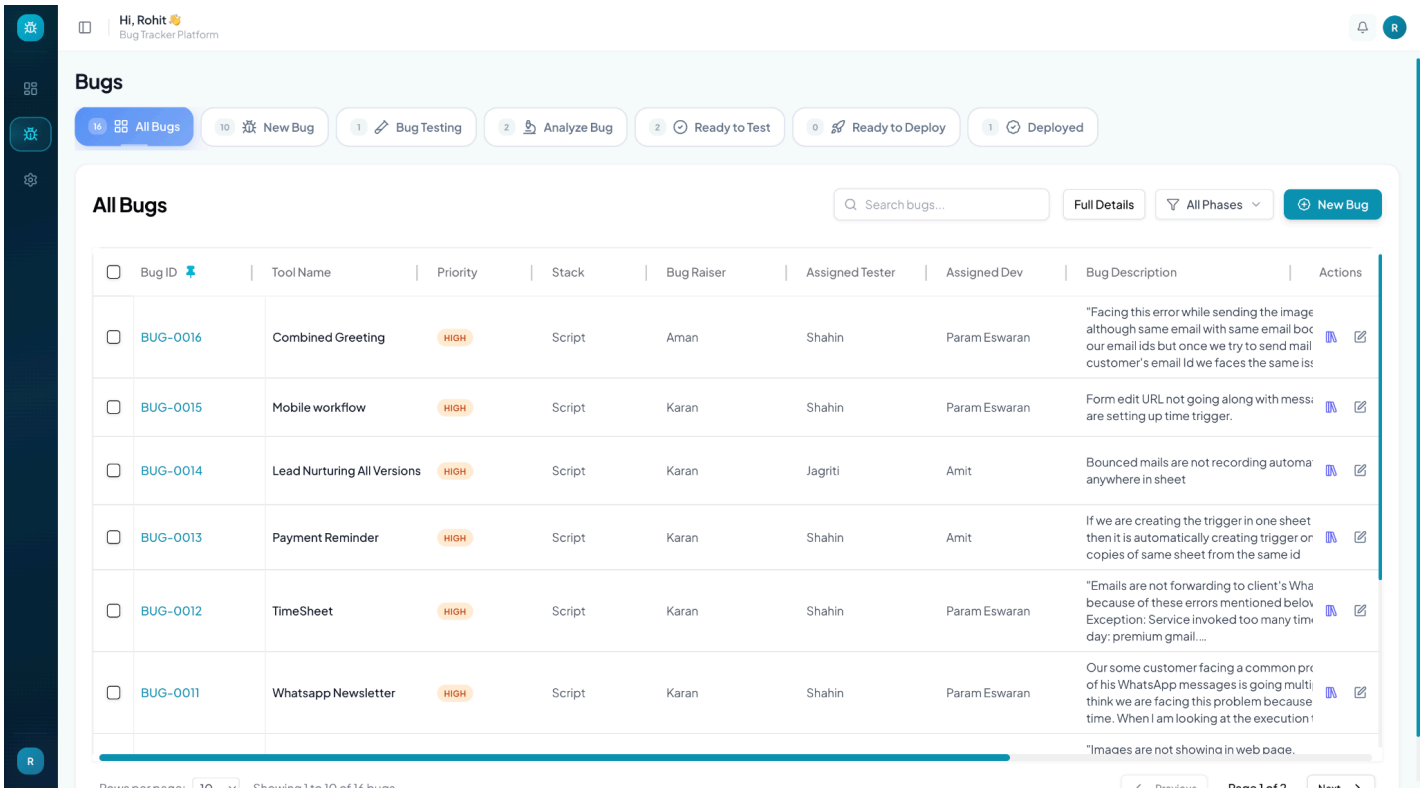
Visual Distribution: Features dynamic charts for Tech Stack Distribution and Status Overview, allowing leads to identify which platforms (e.g., Script, Android) require the most attention.

Project Hotspots: Highlights the most problematic modules (e.g., Whatsapp Newsletter) based on active bug density, enabling teams to prioritize "hot" areas for immediate maintenance.

Strategic Action Center: Provides instantaneous shortcuts to View All Bugs or create a New Bug, ensuring the core workflow is always one click away.

Trend Analytics: Includes a 7-Day Reporting Trend graph to visualize incoming bug rates and track the team's resolution velocity over time.

9. Lifecycle Management — Centralized Issue Tracking Grid



9. Lifecycle Management — Centralized Issue Tracking Grid

Overview: The "All Bugs" view is the primary operational workspace, providing a unified and highly filterable interface for managing the end-to-end lifecycle of every reported issue.

Key Features:

Phase-Based Navigation: Features a dynamic top bar that segments issues into their 7 Lifecycle Phases (e.g., New Bug, Bug Testing, Maintenance). This allows users to drill down into specific workflow stages with a single click.

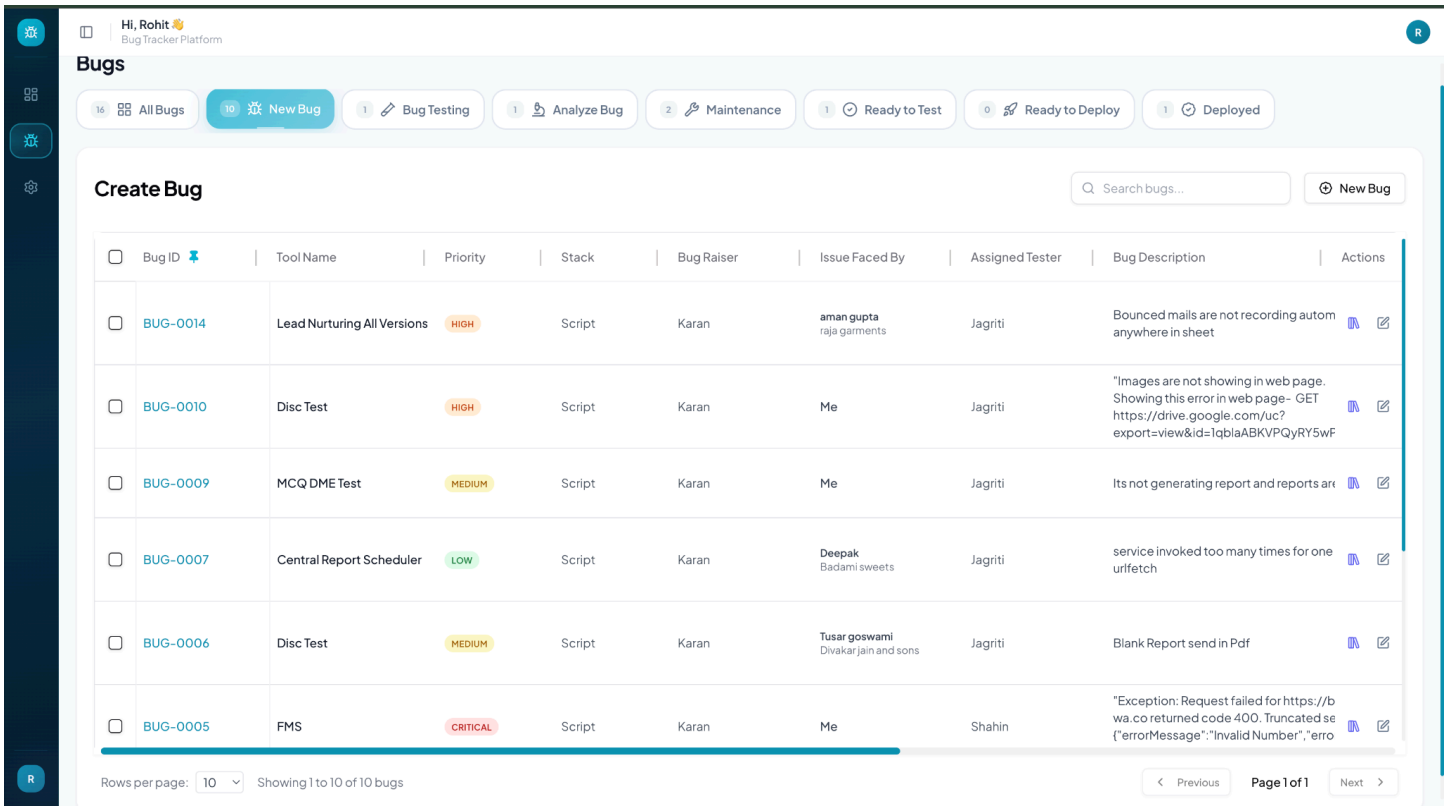
Sequential Identification: Utilizes a professional, human-readable ID system (e.g., BUG-0016), ensuring every issue is uniquely referenceable across technical documentation and team discussions.

Cross-Functional Visibility: Clearly maps each bug to its parent Tool, Tech Stack, and Assigned Personnel (Raiser, Tester, and Developer), facilitating seamless collaboration between departments.

Rich Contextual Previews: Displays concise bug descriptions and priority levels directly in the grid, enabling rapid triage and assessment without opening individual records.

Advanced Discovery Tools: Includes a global search utility and a "Full Details" toggle, allowing power users to switch between high-level overviews and granular technical data instantly.

10. Phase I — New Bug



10. Phase I — New Bug

Overview: The "New Bug" view represents the initial entry and triage stage of the issue lifecycle. It is designed for QA leads and testers to review, prioritize, and assign incoming reports before they proceed to formal verification.

Key Features:

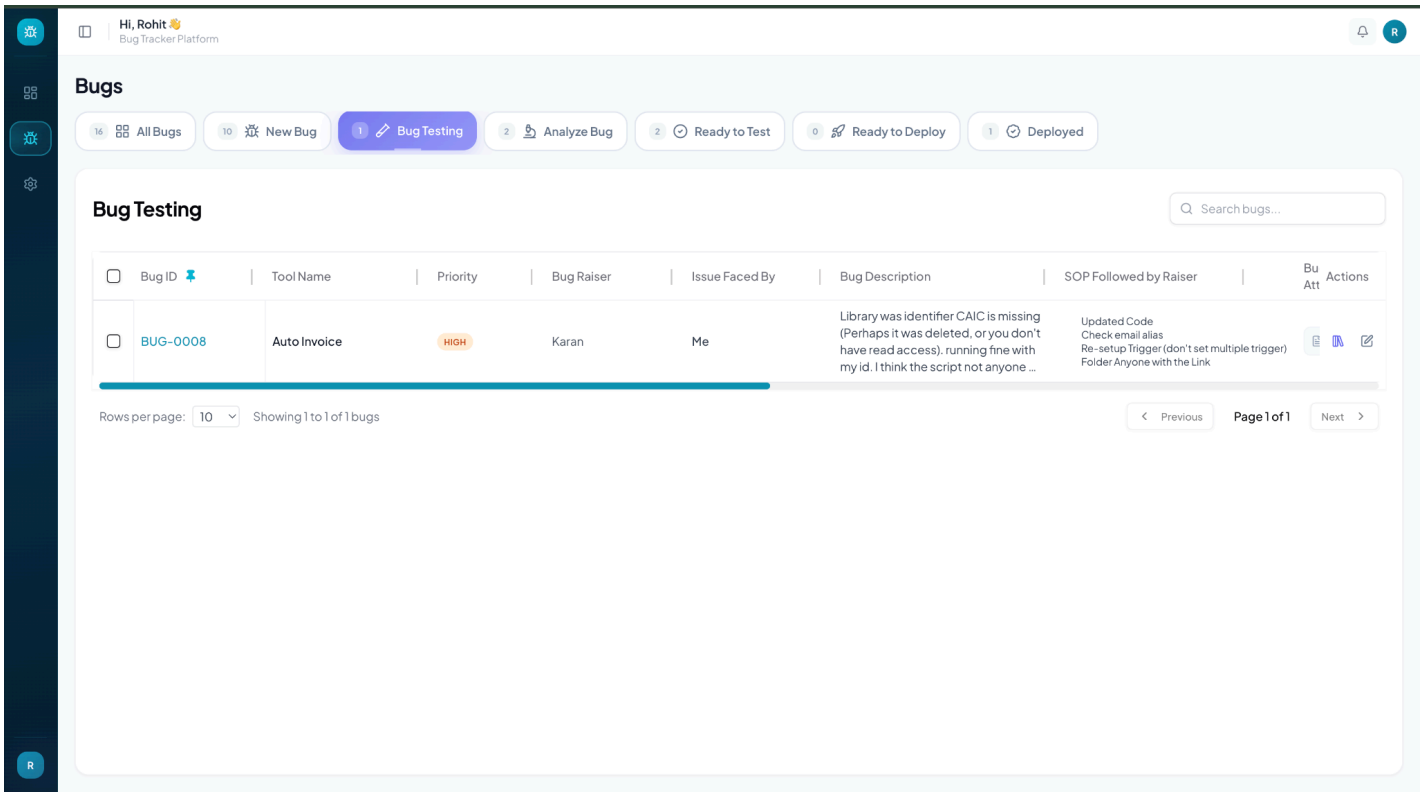
Focused Workflow: Automatically filters the issue database to show only unconfirmed reports, allowing the QA team to focus exclusively on the intake queue without distraction.

Client Context Tracking: Includes the "Issue Faced By" column, which clearly distinguishes between internal discoveries and client-facing disruptions (e.g., Aman Gupta from Raja Garments). This is vital for prioritizing issues that impact external stakeholders. Accountability Mapping: Identifies the Assigned Tester for each new report, ensuring that every submission has a dedicated owner responsible for moving it to the "Testing" phase.

Severity Stratification: Utilizes color-coded priority labels (e.g., CRITICAL, HIGH, LOW) to provide immediate visual cues for urgency and resource allocation.

Technical Implementation: This interface manages the phaseI_BugReport data object within the IBug schema. It is powered by the BugController.BugRaise endpoint, which handles the initial data persistence and triggers the first round of automated notifications to the assigned QA staff.

11. Phase II —Bug Testing



11. Phase II —Bug Testing

Overview: The "Bug Testing" interface is a specialized workspace for the formal verification process. It allows testers to document their findings, confirm bug reproducibility, and prepare the issue for development handover.

Key Features:

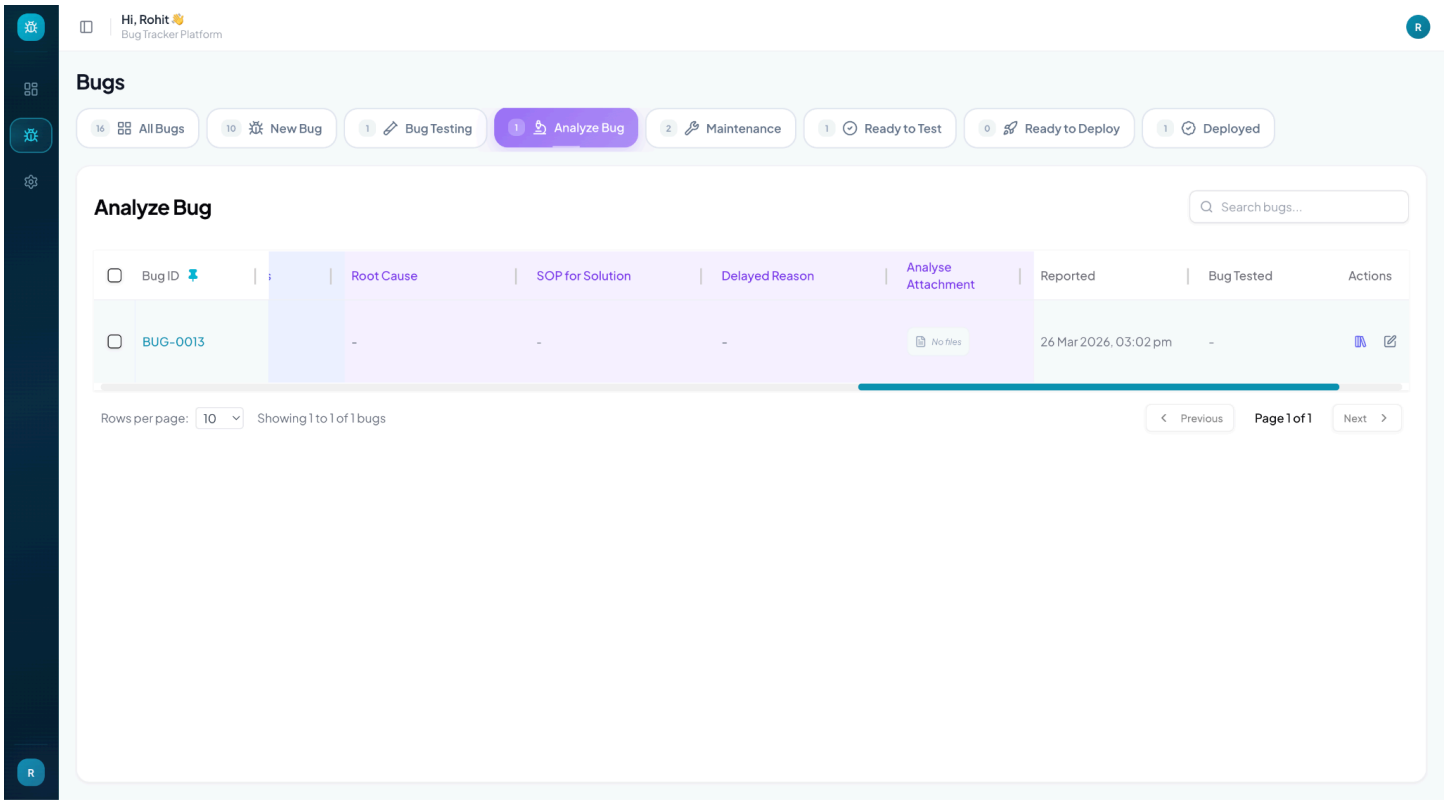
Process Rigor: Specifically tracks "SOP Followed" and "Testing Remarks", ensuring that all verification steps are documented in accordance with standard operating procedures before an issue is escalated.

Evidence Chain: Provides dedicated areas for both original Bug Attachments and new Testing Attachments, maintaining a clear, verifiable history of the bug from report to confirmation.

Handover Automation: Features the "Forward to Developer" field, which identifies the specific engineer responsible for the next stage of resolution. Upon confirmation, the system automatically transitions the bug and notifies the assigned developer.
SLA Tracking: Displays the precise Reported timestamp, allowing the QA team to monitor lead times and ensure that critical bugs are verified within project-defined deadlines.

Technical Implementation: This interface interacts with the phaseII_BugConfirmation data object in the backend. It is powered by the BugController.ConfirmBug endpoint, which captures the verification evidence and manages the state transition from "New" to "Confirmed."

12. Phase III — Root Cause Analysis



12. Phase III — Root Cause Analysis

Overview: The "Analyze Bug" interface is the technical assessment hub where developers perform deep-dive investigations into confirmed issues. This phase focuses on identifying the underlying failure mechanism and designing a standardized remediation strategy.

Key Features:

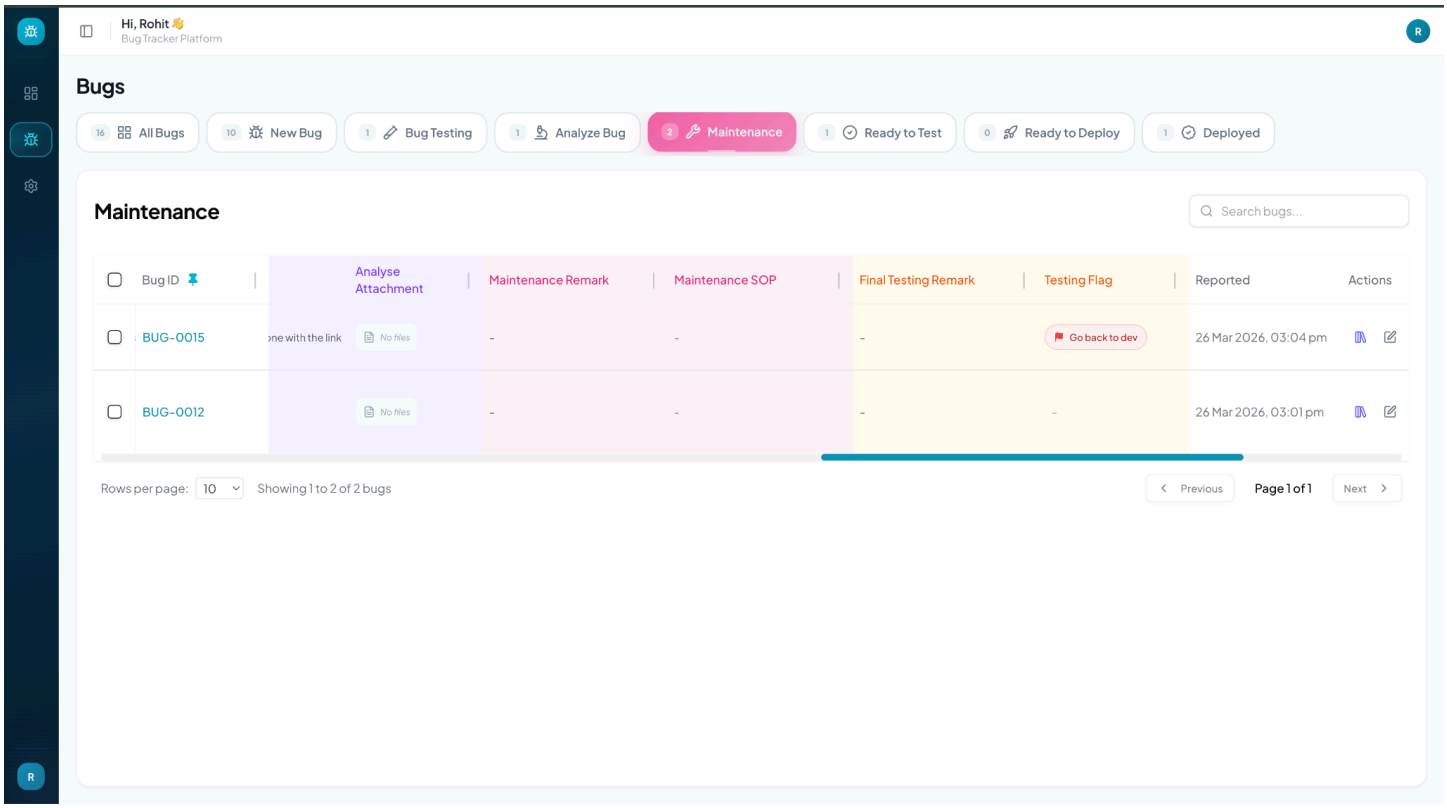
Diagnostic Documentation: Specifically captures the "Root Cause" and "SOP for Solution", ensuring that every fix is evidence-based and follows established implementing patterns to prevent regression.

Operational Transparency: Features a "Delayed Reason" field, providing clear visibility into any external blockers (e.g., Third-party API downtime or Environmental constraints) that may impact the resolution timeline.

Technical Evidence: Supports "Analyse Attachments" for the storage of diagnostic logs, stack traces, or memory dumps discovered during the investigation, serving as a vital resource for future knowledge-sharing.
Lifecycle Awareness: Displays both the "Reported" and "Bug Tested" timestamps, giving developers a complete view of the issue's historical context and its progression through the QA pipeline.

Technical Implementation: This interface manages the phaseIII_BugAnalysis data object. It is powered by the BugController.AnalyzeBug endpoint, which formalizes the technical plan and prepares the issue for the active "Maintenance" (fixing) phase.

13. Phase IV — Active Maintenance & Fix Implementation



13. Phase IV — Active Maintenance & Fix Implementation

Overview: The "Maintenance" interface is the primary execution workspace for developers. It focuses on the implementation of code changes and the documentation of the remediation process, ensuring that every fix is traceable and compliant with technical standards.

Key Features:

Remediation Governance: Specifically captures "Maintenance Remarks" and "Maintenance SOPs", providing a clear record of the specific code changes performed and the standard protocols followed during the fix implementation.

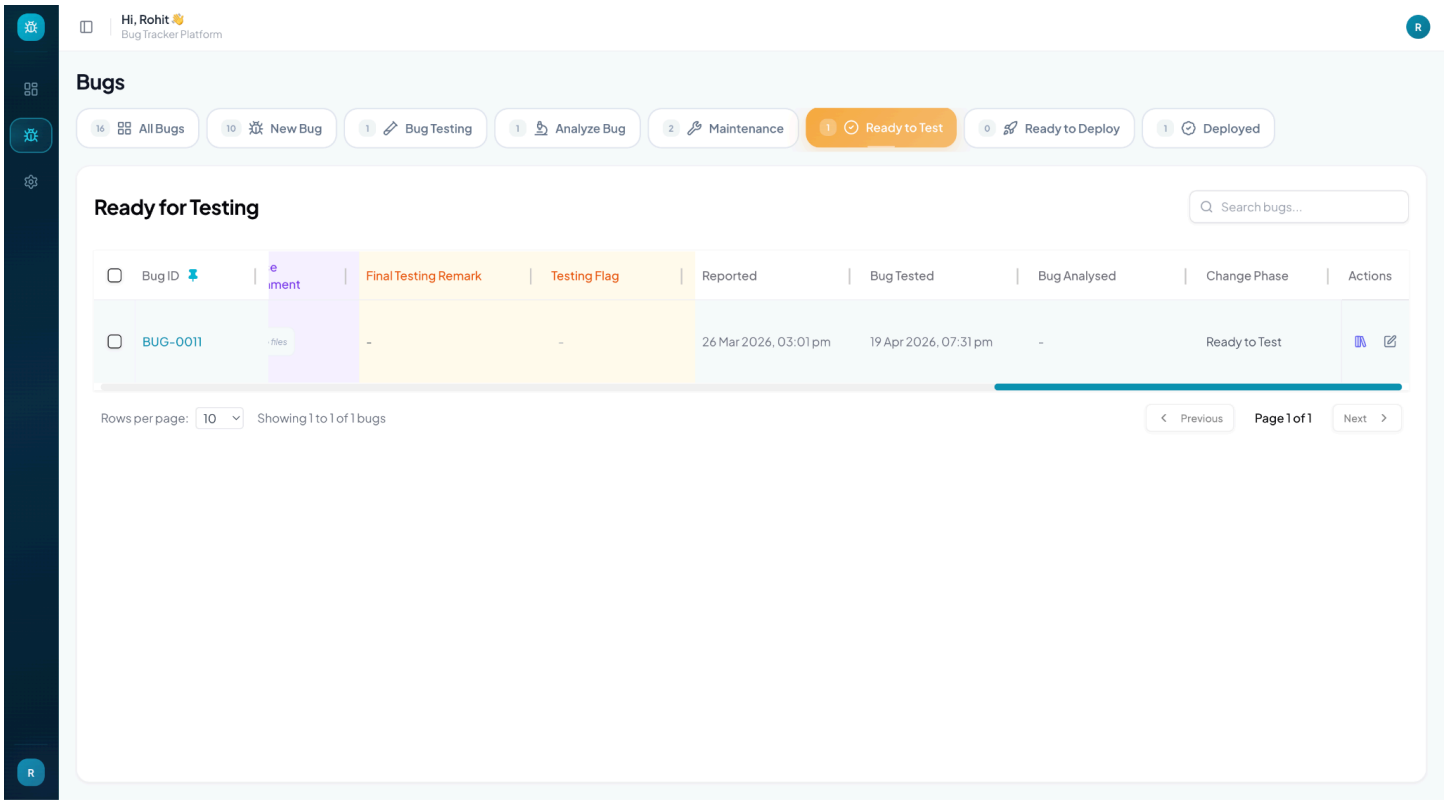
Iterative Feedback Loop: Features a high-visibility "Testing Flag" system. As shown in the screenshot (e.g., "Go back to dev"), this allows testers to flag insufficient fixes, automatically returning them to the developer's queue for further refinement.

Continuous Context: Retains a direct link to the "Analyse Attachment" from the previous phase, ensuring that developers have constant access to the diagnostic data (logs, traces) while they implement the fix.

Validation Readiness: Includes a placeholder for "Final Testing Remarks", facilitating the handoff between development and the final verification team before the bug proceeds to deployment.

Technical Implementation: This interface manages the phaseIV_Maintenance data object. It is powered by the BugController.FixBug endpoint, which archives the resolution details and updates the bug's status to "Ready to Test" upon completion.

14. Phase V — (Final Quality Validation & Pre-Deployment Audit)



14. Phase V — Final Quality Validation & Pre-Deployment Audit

Overview: The "Ready for Testing" interface represents the final quality gate. In this phase, QA engineers perform comprehensive regression testing to ensure that the fix is stable, effective, and has not introduced any secondary issues into the platform.

Key Features:

Final Certification: Specifically focuses on "Final Testing Remarks" and the "Testing Flag", providing a dedicated area for testers to certify that the issue has been successfully resolved according to the project's acceptance criteria.

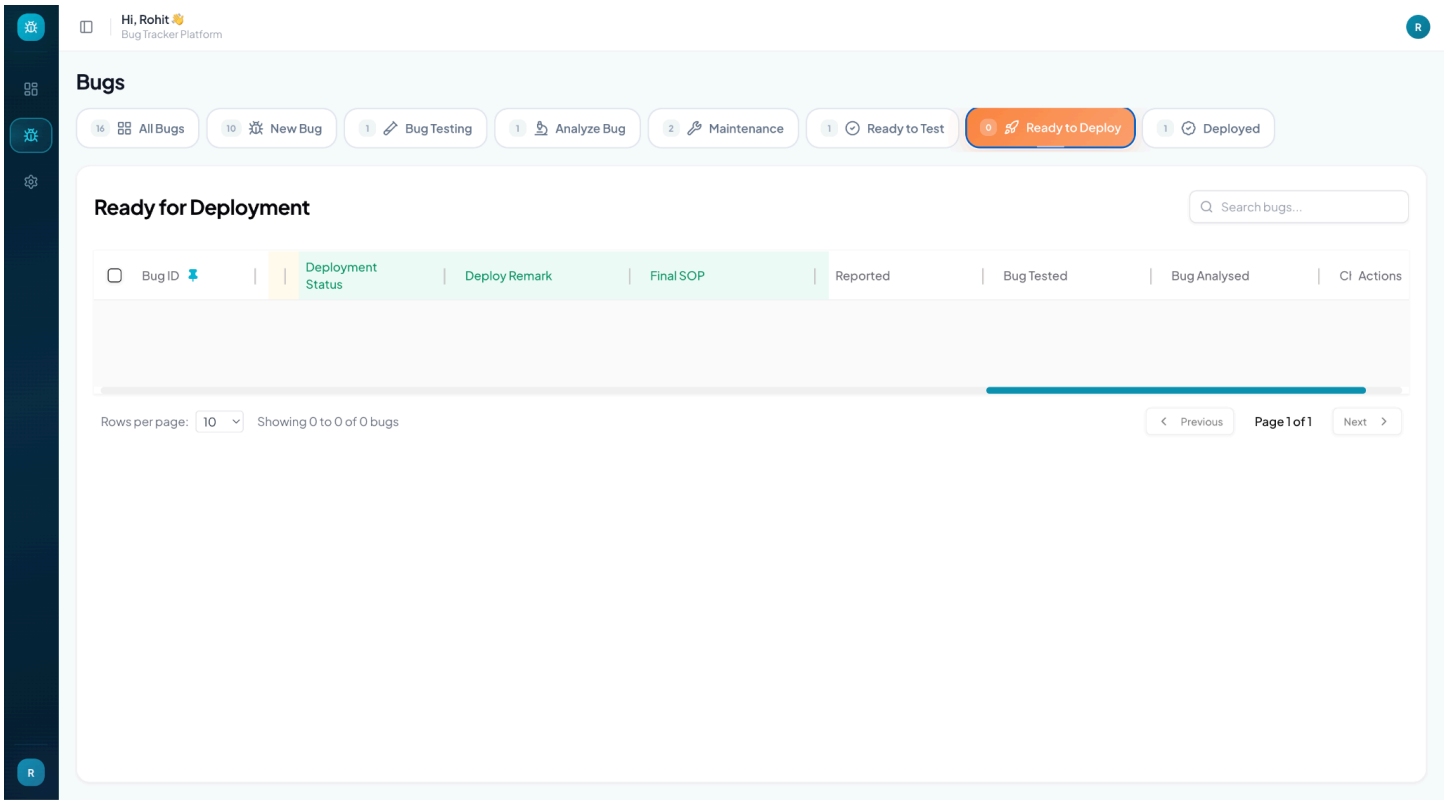
Full Lifecycle Audit: Displays the complete historical timeline of the issue—including Reported, Bug Tested, and Bug Analysed dates. This gives the final verifier a 360-degree view of the resolution journey before granting official approval.

Pipeline Visibility: Highlights the "Change Phase" status as "Ready to Test", clearly communicating the bug's position in the deployment pipeline to all project stakeholders.

Accountability & Sign-off: Facilitates the formal approval process, ensuring that every bug moving toward production has a documented and timestamped validation from a qualified QA engineer.

Technical Implementation: This interface manages the phaseV_FinalTesting data object. It is powered by the BugController.FinalTestBug endpoint, which handles the regression data capture and orchestrates the transition to the "Ready to Deploy" (Phase VI) state upon successful validation.

15. Phase VI — Deployment



15. Phase VI — Deployment

Overview: The "Ready for Deployment" interface serves as the final staging area for verified fixes. It is used by release engineers and DevOps teams to coordinate the transition of resolved bugs from the testing environment to production.

Key Features:

Release Telemetry: Specifically tracks the "Deployment Status" and "Deploy Remark", ensuring that every production release is documented with its specific environmental context and success criteria.

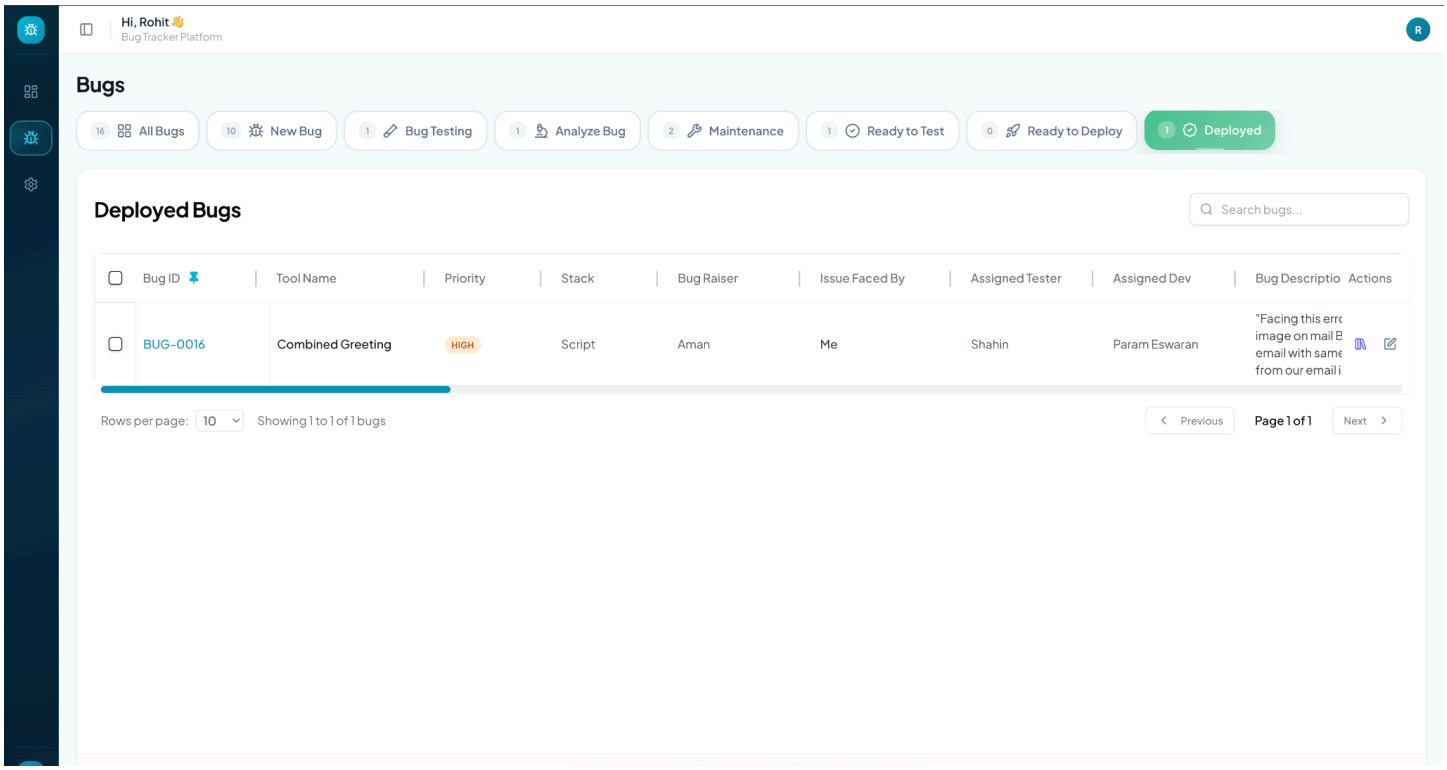
Standardized Implementation: Includes a "Final SOP" field, which provides the deployment team with the exact technical instructions or configuration changes required to implement the fix safely in the live system.

Governance Anchoring: Retains visibility of the "Bug Tested" and "Bug Analysed" timestamps, serving as a final audit point to confirm that all mandatory quality gates were cleared before the release.

Deployment Coordination: Provides a high-clarity, consolidated list of approved fixes, enabling Release Leads to plan deployment cycles with complete visibility into the ready-to-ship backlog.

Technical Implementation: This interface manages the phaseVI_Deployment data object. It is powered by the BugController.DeployBug endpoint, which captures the final deployment logs and transitions the bug to the "Deployed" (Phase VII) state upon successful implementation.

16. Phase VII — Resolution Archive & Knowledge Base



16. Phase VII — Resolution Archive & Knowledge Base

Overview: The "Deployed" view is the final repository for all successfully resolved and implemented bug fixes. It acts as a historical knowledge base, documenting the successful closure of issues after they have cleared all seven stages of the platform's lifecycle.

Key Features:

Finalized Resolution Audit: Provides a consolidated overview of bugs that have been fully integrated into the production environment, marking the formal end of their resolution journey.

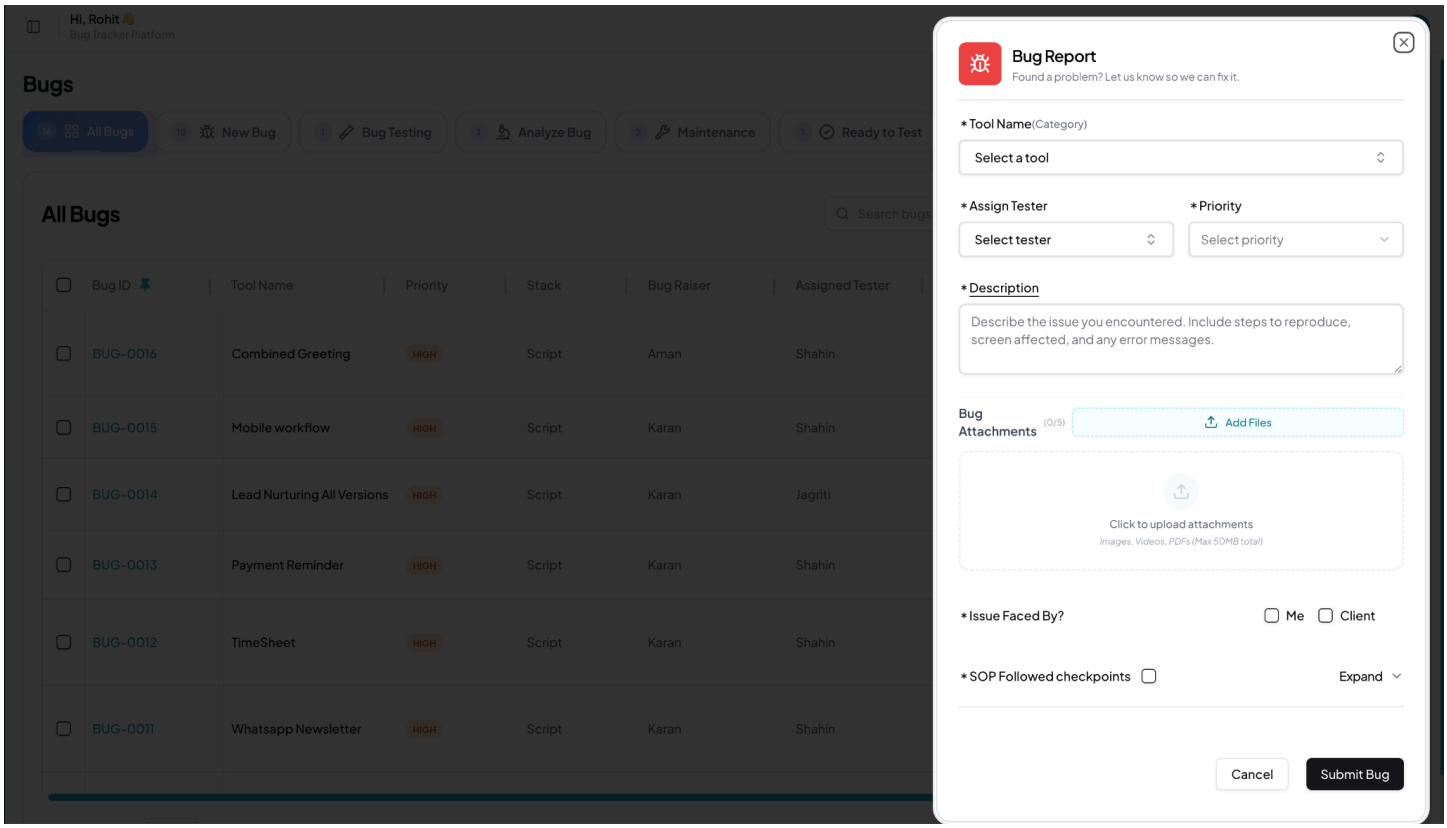
Knowledge Persistence: Maintains a permanent record of the issue's context—including the technical description, tech stack, and the full cross-functional team involved—which is invaluable for future post-mortem analysis and pattern recognition.

System Maturity Metric: The population of this archive serves as a key performance indicator (KPI), reflecting the platform's increasing stability and the team's overall resolution velocity.

Searchable Archive: Enables users to search and review historical fixes, ensuring that lessons learned from past bugs are preserved and can be leveraged to resolve similar future issues more efficiently.

Technical Implementation: This interface retrieves finalized data from the phaseVII_Closure data structure. It is powered by the BugController.GetBugs endpoint, signifying that the issue has reached its terminal state in the database and is no longer part of the active workload.

17. Create/Raise Bug — Bug Reporting Interface



17. Create/Raise Bug — Bug Reporting Interface

Overview: The Bug Reporting Interface is the primary entry point for all platform anomalies. It is meticulously designed to capture high-fidelity data from the moment an issue is discovered, ensuring the QA team has all necessary context for rapid verification.

Key Features:

Structured Data Capture: Contextual Triage: Forces selection of Tool Name and Priority, ensuring the report is instantly categorized within the correct project domain and severity level.

Instant Accountability: Allows the reporter to Assign a Tester during the creation process, which triggers an immediate notification to the assigned staff and eliminates triage delays.

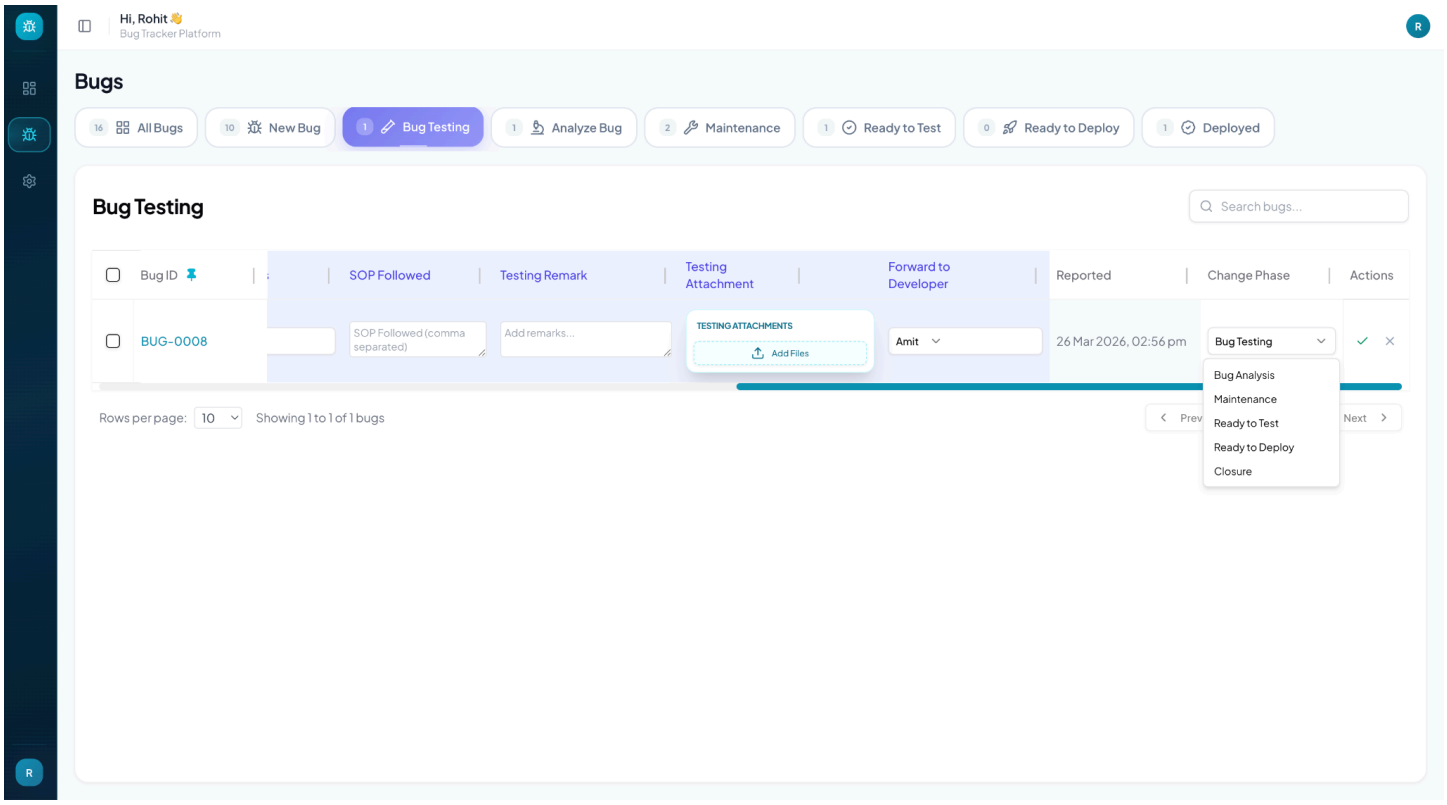
Evidence & Context: High-Volume Attachments: Features a dedicated drag-and-drop zone for Bug Attachments (supporting images, videos, and logs up to 50MB), providing the verification team with indispensable visual evidence.

Stakeholder Identification: Includes the "Issue Faced By?" selector, which distinguishes between internal QA discoveries and live Client disruptions, a key factor in the system's automated priority calculations.

Quality Control: SOP Enforcement: Integrates "SOP Followed" checkpoints to encourage reporters to perform preliminary environmental checks before submission, significantly improving the quality and reproducibility of incoming reports.

Technical Implementation: This interface connects to the BugController.BugRaise endpoint. It utilizes a robust multi-part form data handler (via multer) to process attachments and simultaneously dispatches real-time alerts to the assigned personnel through the Notification utility.

18. Update Bug — Inline Verification & State Management



18. Update Bug — Inline Verification & State Management

Overview: The platform features a high-performance inline editing system that allows QA engineers and developers to update issue records directly within the data grid. This maximizes operational efficiency by eliminating the need to navigate between multiple pages during verification.

Key Features:

Real-Time Data Capture:

Contextual Entry: Enables testers to document SOP Checkpoints, provide Testing Remarks, and upload fresh evidence (Testing Attachments) in a single, unified row-level interaction.

Seamless Handover: Features a dynamic dropdown for selecting the Forward to Developer target, facilitating an immediate and clear transition of responsibility.

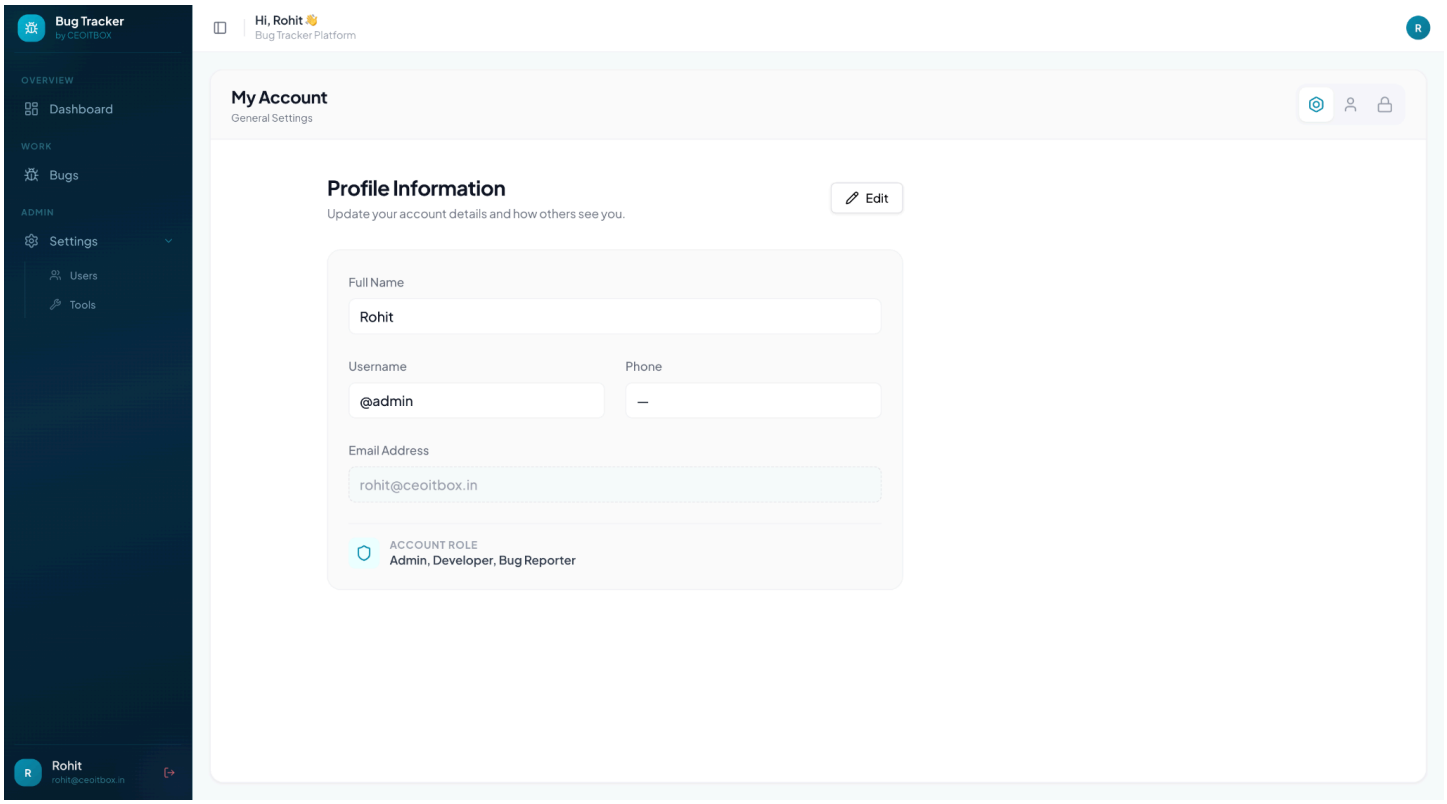
Advanced Lifecycle Orchestration:

State-Transition Control: Includes a "Change Phase" selector that empowers authorized staff to move a bug between any of its seven stages (e.g., from Bug Testing to Maintenance or Closure).

Transactional Integrity: Utilizes high-contrast "Confirm" (Checkmark) and "Revert" (X) controls, ensuring that complex phase transitions and data updates are only committed to the database after final user verification.

Technical Implementation: This interactive interface leverages the BugController.UpdateBug and BugController.ConfirmBug endpoints. It employs React-based state management to provide an "optimistic UI" experience, while the backend ensures that all transitions adhere to the platform's strict sequential logic and trigger the necessary notification events.

19. Personal Profile & Account Settings



19. Personal Profile & Account Settings

Overview: The "My Account" interface provides users with a centralized space to manage their personal identities and review their functional designations within the platform. It is designed for maximum clarity, ensuring users can keep their contact information up to date.

Key Features:

Profile Identity Management:

Unified Credential View: Displays essential account metadata, including Full Name, Username (with a professional "@" prefix), and verified Email Address, allowing for quick verification of the user's primary identifiers.

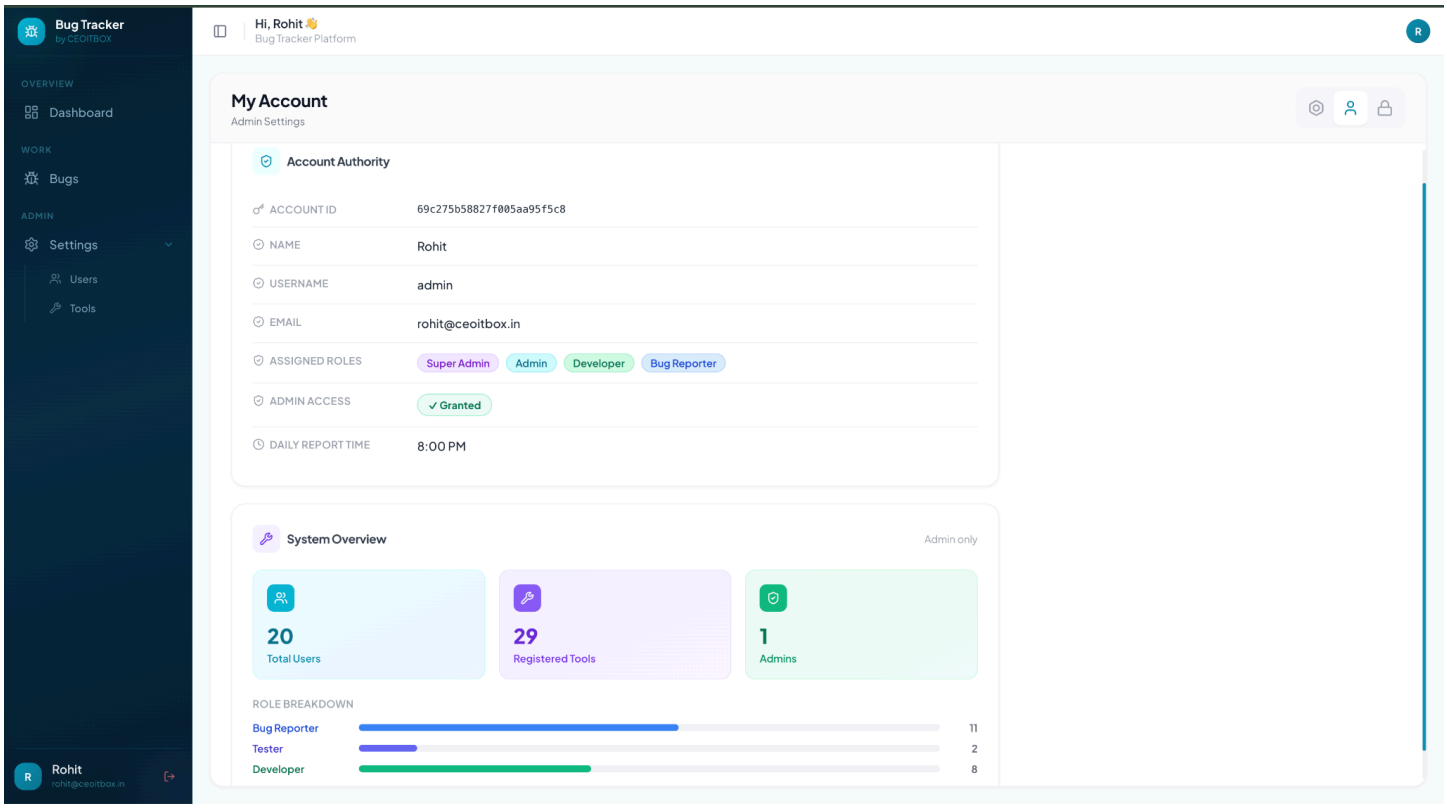
Editable Contact Points: Provides fields for personal data like Phone numbers, ensuring that cross-functional teams have the necessary information for urgent offline collaboration.

Functional Role Transparency: Features an integrated "Account Role" banner that explicitly lists the user's assigned professional designations (e.g., Admin, Developer, Bug Reporter). This keeps the user informed of their specific permissions and responsibilities.

Task-Oriented Navigation: Utilizes a clean, icon-based tab system at the top right to separate General Settings, Advanced Profile Options, and Security/Password Management (Lock icon) into logical, easily accessible sub-modules.

Technical Implementation: This interface is powered by the UserController.getUserDataWithToken endpoint. It leverages the platform's authMiddleware to hydrate the UI with secure, user-specific data from the backend, ensuring that sensitive profile details are only accessible to the authenticated account owner.

20. User Oversight



20. User Oversight

Overview: The "Admin Settings" view within the account profile provides privileged users with a comprehensive summary of their systemic authority and an executive snapshot of the platform's organizational health.

Key Features:

Account Authority Audit: Elevated Identity Verification: Displays unique system identifiers (Account ID) and explicitly confirms "Admin Access" status, providing a clear and transparent audit trail for high-level account privileges.

Permission Mapping: Visualizes the full spectrum of assigned responsibilities through high-contrast labels, identifying the user's strategic roles (e.g., Super Admin, Admin, Developer).

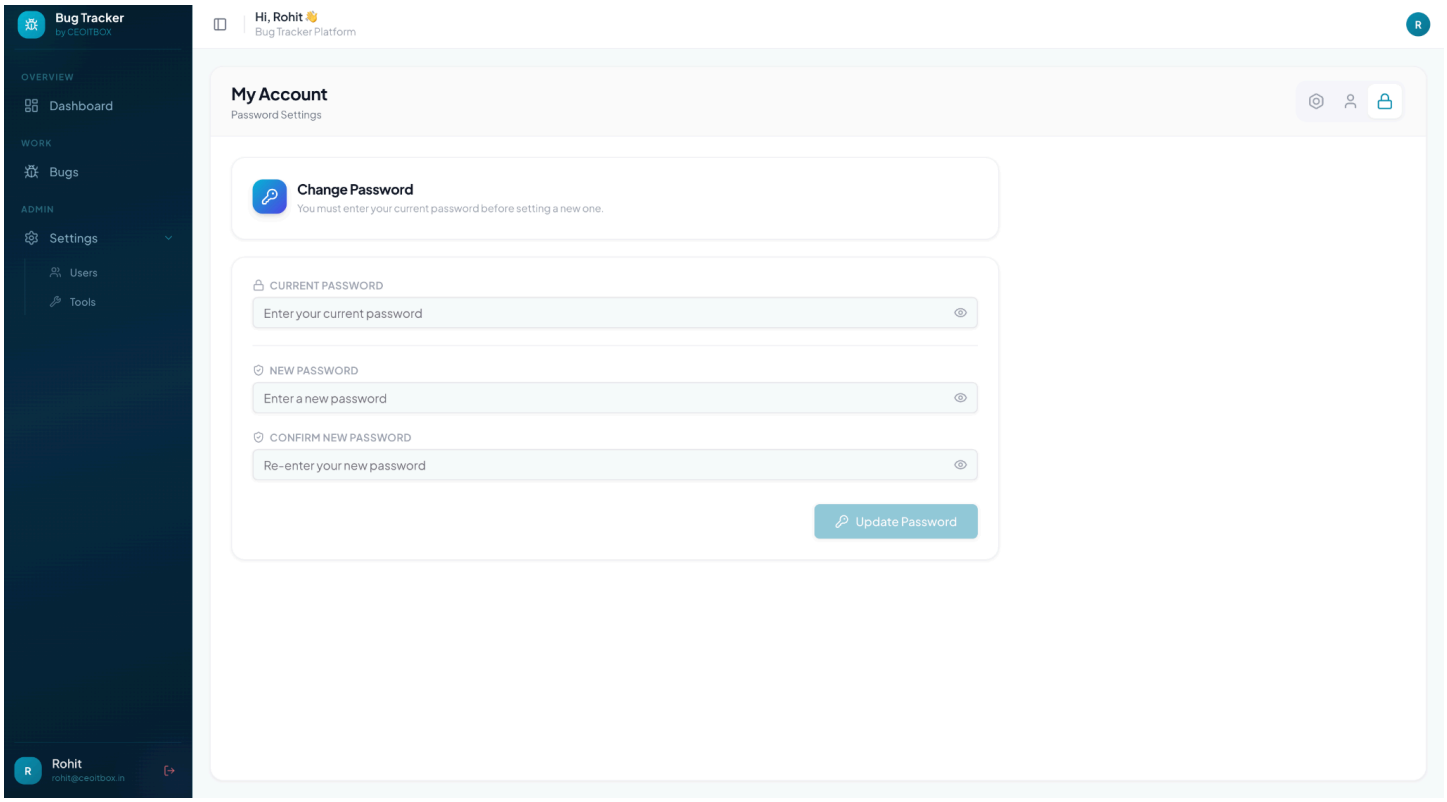
Custom Reporting Parameters: Includes operational settings like the "Daily Report Time," allowing administrators to synchronize automated system updates with their personal schedules.

System Overview (Admin-Only Analytics): Infrastructure Metrics: Features dedicated KPI cards for Total Users, Registered Tools, and the number of active Admins, offering a real-time pulse-check on the platform's scale and management density.

Workforce Stratification: Includes a visual "Role Breakdown" with proportional progress bars. This allows leadership to instantly assess the distribution of technical talent (e.g., 11 Bug Reporters vs. 8 Developers), facilitating better resource planning.

Technical Implementation: This interface aggregates data from the UserController and ToolController. It enforces strict server-side authorization gates to ensure that the "System Overview" metrics are only served to accounts with verified administrative credentials, maintaining absolute data confidentiality.

21. Account Security — Password Management Interface



21. Account Security — Password Management Interface

Overview: The "Password Settings" interface provides users with a highly secure environment for maintaining their account credentials. It is designed around the "Zero-Trust" principle, ensuring that all security updates are performed with maximum verification and data integrity.

Key Features:

Mandatory Verification Logic: Enforces a "Current Password" validation step as a prerequisite for any changes. This prevents unauthorized credential modification in the event of an unattended session, ensuring that only the true account owner can reset their security keys.

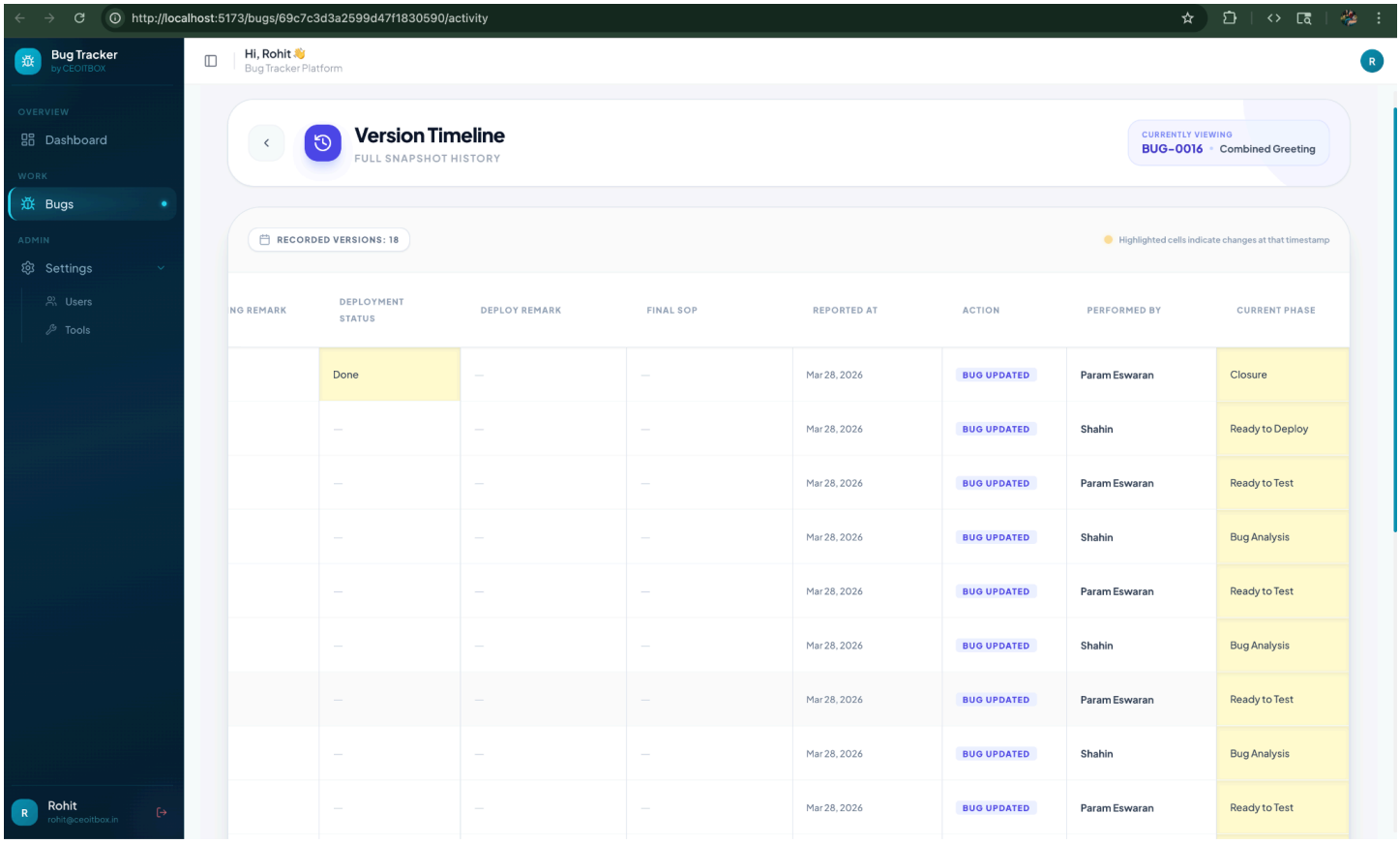
Credential Integrity Controls: Dual-Entry Workflow: Utilizes "New Password" and "Confirm New Password" fields to eliminate typographical errors during the creation of new security credentials.

Enhanced Visibility Toggles: Each field is equipped with an integrated visibility toggle, allowing users to visually confirm complex password strings while maintaining privacy when needed.

Streamlined User Experience: Features a clean, single-column layout that minimizes distractions, focusing the user's attention on the critical task of account protection.

Technical Implementation: This interface communicates with the UserController.changePassword endpoint. On the backend, the system verifies the current password using bcrypt comparison, validates that the new password meets the 8-character complexity requirement, and ensures a perfect match between both new entries before committing the update and invalidating old session tokens for increased security.

22. Bug Logs — Version Timeline & Snapshot History



22. Bug Logs — Version Timeline & Snapshot History

Overview: The "Version Timeline" is a comprehensive, chronological audit engine that provides total transparency for every issue in the platform. It documents every micro-update and phase transition, ensuring a verifiable and immutable history of the bug's resolution journey.

Key Features:

Chronological Snapshot History: Captures a granular record of all system events (e.g., 18 Recorded Versions for BUG-0016), allowing stakeholders to trace the evolution of an issue from its initial report to final closure.

Intelligent Change Detection: Employs a sophisticated visual highlighting system (yellow cells) to pinpoint exactly which technical data points were modified at each timestamp. This allows for rapid forensic analysis of how the bug's state changed during specific updates.

Operational Accountability: Explicitly maps every action to the specific team member ("Performed By") and the system event ("Action"), creating a robust chain of responsibility for every decision made during the lifecycle.

Phase Mapping: Tracks the "Current Phase" for every historical entry, providing a clear visual representation of how the bug navigated through the platform's seven-stage resolution pipeline.

Technical Implementation: This interface is powered by the LogController.GetBugLogs endpoint. On the backend, the system utilizes the captureBugChanges utility to perform deep object comparisons between old and new states. These "delta changes" are persisted in a specialized logs collection, ensuring that a complete forensic record is maintained even if the primary bug document is significantly updated.

11. TESTING TYPES AND TEST CASES

11.1 Manual Testing (Basic Testing)

- **Unit Testing (Manual):** Testing individual screens and buttons manually to ensure they respond correctly (e.g., checking if a "Submit" button becomes disabled after one click).
- **System Testing:** Testing the full flow of the application from a user's perspective—creating a bug, assigning it, and moving it through different status phases.
- **Usability Testing:** Checking the interface on different screen sizes to ensure the design stays consistent and user-friendly.

11.2 Automated Testing (Jest)

- **Logic Testing:** Using Jest to test backend utility functions and math logic (like date formatting or status calculations) to ensure the data is processed correctly without errors.
- **Component Testing:** Using Jest to verify that critical UI components render with the correct information.

11.3 Sample Test Cases

A. Automated Test Cases (Jest)

ID	Test Scenario	Tool	Expected Result
TC-01	Validate Date Formatter Utility	Jest	Converts ISO string to "DD-MM-YYYY" format correctly.
TC-02	Logger Utility Verification	Jest	Ensures system logs are captured in the correct directory.

B. Basic Manual Test Cases

ID	Test Scenario	Method	Expected Result
TC-03	User Login Flow	Manual	User enters email/password and is redirected to Dashboard.
TC-04	Bug Creation Form	Manual	Validation triggers if "Title" is left empty.
TC-05	Dark Mode Toggle	Manual	The entire UI theme switches between light and dark colors.

11.4 Testing Summary

The project followed a simple two-step testing strategy:

1. Manual Verification: Each feature was manually tested by the developer immediately after coding to catch UI bugs.
2. Automated Validation: Critical backend logic was covered by Jest to prevent errors during data processing.

12. SECURITY MECHANISM

12.1 Authentication Security

Password Security:
- Bcrypt hashing algorithm with 10 salt rounds
- Minimum password requirements (8 characters, mixed case, numbers)- No plain text password storage
- Secure password reset mechanism with time-limited tokens

JWT Token Management:
- Secure token generation with secret key
- Token expiration (48 hours for access tokens)
- Token refresh mechanism for extended sessions
- HTTP-only cookies for token storage
- Token validation on every protected route request

12.2 Authorization Security

Role-Based Access Control (RBAC):
- Middleware verification for every protected endpoint
- Four distinct roles: Bug Raiser, Tester, Developer, Administrator
- Role checking before allowing specific actions
- Resource-level permissions
- Phase-specific permissions (only assigned tester can confirm bugs)

12.3 Data Security

Input Validation:
- Server-side validation for all user inputs
- express-validator for comprehensive validation rules
- Type checking and format validation
- Length restrictions on all fields
- Regex patterns for specific formats (email, mobile)

Injection Attack Prevention:
- NoSQL injection prevention through Mongoose schema validation
- Input sanitization for all user-provided data
- Parameterized queries using Mongoose ORM
- HTML entity encoding for displayed user content

12.4 File Upload Security

File Type Validation:
- Whitelist of allowed file types (images, documents)
- MIME type verification
- File extension validation

File Size Restrictions:
- Maximum file size limit (10MB per file)
- Total upload size limits per request
- Prevention of disk space exhaustion attacks

Secure File Storage:
- Unique file naming to prevent overwrites
- Files stored outside web root directory
- Path traversal attack prevention
- Access control on uploaded files

12.5 Communication Security

HTTPS/TLS:
- Secure cookie flags (httpOnly, secure, sameSite)
- HSTS header forcing HTTPS

API Security Headers:
- Content Security Policy (CSP)
- X-Frame-Options preventing clickjacking

12.6 Database Security

MongoDB Security:
- MongoDB authentication enabled
- Database user with limited permissions
- Connection string stored in environment variables

12.7 Application Security

CORS Configuration:
- Restricted origin list for API access
- Only allowed domains can make requests

Rate Limiting:
- Request rate limits to prevent brute force attacks
- Login attempt limiting (5 attempts per 15 minutes)
- API rate limiting per IP address

Error Handling:
- Generic error messages to users (no sensitive details exposed)
- Detailed error logging for debugging (server-side only)
- Stack traces never sent to clients
- Custom error pages

12.8 Logging and Monitoring

Security Event Logging:
- Failed login attempts logged with IP address

Audit Trail:
- User action tracking with timestamps
- Changes to critical data logged

13. FUTURE SCOPE OF THE PROJECT

13.1 Phase 1 Enhancements (6-12 months)

1. Advanced Analytics and Reporting:
- Interactive dashboards with real-time charts
- Trend analysis for bug patterns
- Predictive analytics for resolution time estimation
- Team performance metrics and KPIs
- Custom report generation with export functionality (PDF, Excel)

2. Enhanced Notification System:
- In-app notifications with badge counters
- SMS notifications for critical bugs
- Push notifications for mobile browsers
- Configurable notification rules per user
- Escalation mechanisms for overdue bugs
- Digest emails (daily/weekly summaries)

3. Improved Search and Filtering:- Advanced search with complex boolean queries
- Saved search queries for frequently used filters
- Full-text search across all bug fields
- Search suggestions and autocomplete

13.2 Phase 2 Enhancements (12-24 months)

4. Integration Capabilities:
- Version control system integration (Git, GitHub, GitLab)
- CI/CD pipeline integration (Jenkins, GitLab CI, GitHub Actions)
- Project management tool integration (Jira, Asana, Trello)
- Communication platform integration (Slack, Microsoft Teams)
- RESTful API for third-party integrations
- Webhook support for real-time event notifications

5. Mobile Applications:
- Native iOS and Android applications
- Offline capability for bug reporting
- Push notifications for mobile devices
- Voice-to-text for bug descriptions

6. Artificial Intelligence Features:
- Automatic bug classification using NLP

- Duplicate bug detection using machine learning
- Intelligent assignment recommendations based on expertise
- Predicted resolution time estimation
- Smart notification filtering
- Auto-tagging of bugs based on content analysis
- Sentiment analysis of bug descriptions

13.3 Phase 3 Enhancements (24+ months)

7. Enterprise Features:
- Multi-tenant architecture for multiple organizations
- Single sign-on (SSO) integration (OAuth, SAML)
- Custom workflow designer with drag-and-drop interface
- White-label capabilities for branding
- Advanced compliance reporting (SOC 2, ISO 27001)
- Role hierarchy with delegation capabilities
- SLA (Service Level Agreement) tracking

8. Collaboration Enhancements:
- Real-time chat for bugs with typing indicators
- Video call integration for issue discussion
- Screen sharing for bug demonstration
- Collaborative document editing for analysis
- Knowledge base integration with articles
- Community forums for best practices

9. Advanced Reporting:
- Custom report builder with drag-and-drop
- Scheduled reports via email
- Real-time dashboards with auto-refresh
- Comparison reports (month-over-month, year-over-year)
- Heatmaps for bug distribution
- Burndown charts for sprint planning

10. Automation Features:
- Automated bug assignment based on keywords
- Auto-escalation for overdue bugs
- Automated status updates based on time
- Scheduled bug archival
- Bulk operations on bugs

14. BIBLIOGRAPHY & REFERENCES

Books

5. Node.js Design Patterns (3rd Ed.), Mario Casciaro & Luciano Mammino, Packt Publishing, 2020.
6. Learning React (2nd Ed.), Alex Banks & Eve Porcello, O'Reilly Media, 2020.
7. MongoDB: The Definitive Guide (3rd Ed.), Shannon Bradshaw et al., O'Reilly Media, 2019.
8. UML Distilled (3rd Ed.), Martin Fowler, Addison-Wesley, 2003.

Official Documentation

5. Node.js Official Documentation, https://nodejs.org/en/docs/, Accessed: October 2024.
6. React.js Official Documentation, https://react.dev/, Accessed: October 2024.
7. MongoDB & Mongoose Documentation, https://docs.mongodb.com/, https://mongoosejs.com/, Accessed: October 2024.
8. JWT (JSON Web Tokens) Documentation, https://jwt.io/, Accessed: October 2024.

Research Papers / Journals

2. Bug Tracking Systems: A Systematic Review, Journal of Software Engineering Research and Development, 2019.

Technical Articles / Guidelines

2. RESTful API Design Best Practices, Microsoft Azure Architecture Center, 2021.